PHASE 1 · FOUNDATIONS · VERSION 1.0 · 9 OCTOBER 2026

Deliverance Church Eastern Bypass

Website and admin system design

The complete first set of drawings: UML, flow and block diagrams, data-flow diagrams and the database design, plus the questions the owner must answer and the plan for the next phases. Written for a beginner, to be used by a builder.

125 diagrams | 74 classes in 9 modules | 10 roles | 83 database tables | English + Kiswahili | 51 questions with defaults

Prepared for
Jeff (Muchiri Jefferson Kimotho), Kyfaru, Nairobi
for the church leadership of Deliverance Church Eastern Bypass

Files that belong together
1. This PDF (read and decide)
2. DCEB_Diagrams.drawio (edit and move boxes)
3. DCEB_Prompts.md (hand to the builders)

BEFORE YOU START

How to read this document

This is a long document because the church website is a real system, not a brochure. You do not have to read it in one sitting. Here is the shortest road through it.

If you want to...	Go to	Time
Understand what we are building, in plain words	Part 1 (Start here)	25 min
Check that I understood your brief	Part 2 (What you asked for)	15 min
Answer what I could not decide for you	**Part 3 (Questions with defaults)**	30 min
See the design approach and the "main" class	Part 4 (The approach) and Part 5 (The big picture)	20 min
See the tools and how they connect	Part 6 (Stack and integration)	20 min
Check language, safety and privacy	Parts 7 and 8	15 min
Find mistakes you may make at each stage	Part 9	10 min
Look at one diagram	Part 10 (the atlas, 125 pages, one diagram per page)	as needed
Look up what one table stores	Part 11 (Data dictionary)	as needed
See what comes next	Part 12 (Roadmap and next phases)	5 min

Every diagram page has the same three boxes

HOW TO READ IT
What the picture shows, in order, with a real-life comparison.

MISTAKES TO AVOID
The errors a beginner (or a tired engineer) makes at exactly this step.

WHAT YOU ALREADY DID RIGHT
The parts of your brief that this diagram confirms.

The colours mean people

The same colour is used for the same kind of person in every diagram, so you can follow one role across pages.

Visitor / public · New Member · Super Admin · Staff (branch and content) · Discipler · Treasurer · Department Leader · Translator / Reviewer · Events Coordinator

The three files

File	What it is	Who uses it
DCEB_Diagrams.drawio	All 125 diagrams as one editable draw.io file: one page per diagram, an Index page with clickable chips, connected arrows, layers on B01. Open it at app.diagrams.net or in the draw.io desktop app.	You (to move, add and change); the builders (to check)
DCEB_Design_Phase1.pdf (this file)	Explanations, questions, stack, data dictionary and every diagram as a picture with its notes. Bookmarks on the left follow the parts.	You and the church leaders (to read and decide)
DCEB_Prompts.md	The build prompts: a house brief, the twelve agents (a Chief Officer and eleven specialists), gates, and one prompt per module with design references and two examples each.	Claude Code and any engineer (to build from)

TWO HONEST LIMITS

1. This is the **foundation** phase. The wireframes (screen layouts), the hosting and security diagrams and the delivery plan are the next phases (Part 12). The questions in Part 3 are mostly about things that must be settled *before* those start.

2. Anything about the law (Kenya Data Protection Act 2019, registration, retention times) is a **design default**, marked "verify with a lawyer". I am not a lawyer and I do not claim to be one.

PART 1

Start here: what we are building

If you have never built a website, read this part slowly. Everything else in the document is a more detailed version of these four pages.

> **EXPLAINED LIKE YOU ARE FIVE**
>
> Imagine the church compound on a Sunday morning. There is a **gate with a signboard** (the landing page). There are **notice boards** (events and announcements), a **library** (sermons, bulletins, Bible study), a **reception desk** (contact and forms) and a **collection box** (giving details). Behind a door marked "Staff" are the **offices** (the admin area), and behind the offices is a **filing room** with labelled cabinets (the database).
>
> A website is the same compound, made of information instead of bricks. The visitor walks in through the gate. The staff walk in through the staff door and need **keys**. Different staff have different keys: the treasurer can open the money cabinet; the media team cannot.

1.1 Who is who in the compound

In the compound	On the website	Tool we use	Why this one
The compound itself	The website, running on the internet	Next.js on Vercel	Vercel builds and serves the pages; every change gets a private preview first
The filing room and cabinets	The database: one cabinet (table) per kind of thing, one folder (row) per item	Neon (Postgres) + Drizzle	Neon is the cabinet; Drizzle is the clerk that speaks to it in safe, typed sentences
The staff door and keys	Sign-in and roles	Better Auth + RoleAssignment	Better Auth proves who you are; our own roles say what you may do
The post office and bell	E-mail, SMS and reminders	Resend, Africa's Talking, QStash	Resend sends e-mail, Africa's Talking sends SMS in Kenya, QStash delivers messages later at the right time
A sticky note at reception	Quick counters and limits (views, likes, "too many tries")	Upstash Redis	Very fast. If the note is lost, nothing important is lost: the cabinet still has the truth
The storage shed	Pictures, audio, PDFs, certificates	Cloudflare R2	Big files do not belong in cabinets; the shed hands out short-lived keys
The guard at the gate	Stops robots and floods	Cloudflare Turnstile + rate limits	Robots fill forms with rubbish; limits stop the same person pressing 500 times
Notice-board painter	Look and feel	Tailwind CSS	Fast to style, small result
The interpreter	English and Kiswahili	next-intl + a translation engine + human review	Every public text has two versions; a machine drafts, a person approves
The visitors' book	Who did what (audit) and how people use the site (analytics)	AuditLog table, PostHog (only after consent)	The first is for safety; the second is for learning and must be switched on by the visitor

YOU HAVE SEEN THIS BEFORE

Fechi. You already built this pattern for Fechi: Next.js, Neon, Drizzle, Better Auth, Resend, Africa's Talking, webhooks. The church site uses the same bricks, so you are not starting from zero. The difference is that Fechi sells things and this site mostly *tells* things and *collects people's details carefully*; so privacy matters more than payments here.

Stackable. When you moved Stackable from Supabase to Neon + Better Auth you met the idea of **one login system and your own permission rules on top**. The church site repeats it: Better Auth proves identity (staff only), and the role table (RoleAssignment) decides power, exactly like a school having a principal, a teacher and a bursar.

Upstash study. You mapped Upstash products to projects. Here the map is simple: **Redis** for counters and limits (rebuildable), **QStash** for "do this later" (publish at 6 a.m., send the newsletter in batches, remind a discipler in two days).

1.2 Why we draw before we build

A builder who starts laying bricks without a plan finds, halfway up, that the stairs have no room. Changing a drawing costs minutes; changing a built wall costs weeks. For software the "wall" is the database and the permissions: once real people's phone numbers and decisions are saved in a wrong shape, correcting it is slow and risky. So Phase 1 is only drawings and decisions. No code yet.

There is a second reason that matters for you. You told me you rely on Claude Code to write much of the code. Claude Code is excellent when it has a precise plan and weak when it must guess. These drawings are that plan; the prompt file is how you hand the plan over.

1.3 The ten kinds of drawing, as house plans

Drawing family	House-plan comparison	The question it answers	Codes
Class architecture	Who owns which room	What are the "things" the system remembers, who owns each, what is shared?	B01 to B19
Class tabs	One detailed drawing per room	Exactly what does this one thing store, do, depend on and allow?	B20 to B51
Use case	Who comes to the building and why	What does each kind of person want to do?	U01 to U06
Sequence	The script of a conversation at the desk	Who calls whom, in what order, for one task?	S01 to S14
State machine	The life story of a thing	What stages does it go through and what moves it on?	T01 to T09
Flow and block	The walkways, signs and one-way doors	What is the route from the first click to the finished result?	F01 to F15
Package and route map	The directory board of the compound	Which folder holds what? Which web address exists?	K01, K02
Object diagram	A real form, filled in	Does the design work with real values? Show me one example.	O01, O02
Data flow (DFD)	The plumbing diagram	Where does information come in, where is it kept, who reads it?	D01 to D11
ER (database)	The filing plan of the cabinets	What are the tables, columns, links and rules the database enforces?	E01 to E14

1.4 A good reading order for a beginner

1. **F01** the whole system on one page. Find the three groups: people, the website, outside helpers.
2. **B01** who owns what. Then **B02** (what inherits what) and **B05** (which class is "the main").
3. **U01** what a visitor wants, then **S01** (what happens when someone opens a sermon).
4. **T01** the life story of a piece of content (draft to published).
5. **D01** the context diagram, then **D02** the nine processes and nine stores.
6. **E01** the filing plan, then one tab such as **B26** (Sermon) to see how detailed a class tab is.
7. **O01** a real example (a new member called Grace) to see the whole design working.

1.5 Three words that confuse everybody in this project

The word	Meaning 1	Meaning 2	Meaning 3	How we avoid mixing them
Branch	A church location (Eastern Bypass is one branch)	A git branch (a safe copy of the code to work on)	A Neon database branch (a safe copy of the data for tests)	In diagrams "Branch" with a capital B is always the church. We write "git branch" and "database branch" in full.
Event	A calendar event: youth trip, conference	An analytics event (PostHog "page viewed")	A programming event (something happened inside the code)	The class `Event` is only the calendar one. Analytics words are always "tracked action".
Role	What kind of staff member you are (Treasurer)	Your scope (global, branch, department)	A database role (what the app may do to tables)	RoleAssignment holds role + scope together. Database rights are called "app permissions".

1.6 Glossary: words you will meet

Each entry is in two sentences: a five-year-old version, then the engineer's version.

Word	Like you are five	The engineer's version
Class	A cookie cutter: it says what every sermon looks like.	The blueprint for one kind of thing (Sermon): its fields and what it can do. Each saved sermon is an *object* of that class.
Object / row	One cookie.	One real record, e.g. the sermon of 11 October. O01 and O02 show whole objects.
Attribute / column	A box on the form: Name, Date.	A named piece of data with a type (text, number, date). In a database table it is a column.
Table	A drawer of identical forms.	A set of rows with the same columns. Our 74 classes become tables (plus link tables).
Primary key (id)	A name tag nobody else has.	The column that identifies a row. We use uuid v7, so ids are unique and roughly sorted by time.
Foreign key (FK)	A note saying "see the Branch folder".	A column that holds another row's id and makes the database check the other row exists.
Inheritance	Children get the family nose.	A child class reuses the parent's fields (B02). In Drizzle we do it with shared column helpers.
Enum	A multiple-choice list.	A fixed list of allowed values (status: draft, published…). 27 of them (E12).
Migration	A written instruction to rebuild the shelves.	A versioned file that changes the database structure. Never edited after it has run on a shared database.
ORM (Drizzle)	A translator between you and the filing clerk.	Code that lets TypeScript read and write tables with type checking.
Route	A street address.	A web address handled by a page or handler (K02).
Component	A Lego brick of the screen.	A reusable piece of interface (button, card, tab). It shows things; it does not decide rules.
Server action	A request slip you hand through the window.	A function that runs on the server when a form is sent. It checks input and permission, then calls a service.
Service	The person who knows the rules.	A module with business rules (e.g. publish a sermon). Tested without a browser.
Repository	The person who fetches files.	The only code that talks to the database (Drizzle).
Adapter	A plug adapter for a foreign socket.	One file that wraps one outside tool (mail.ts wraps Resend), so changing tool changes one file (K01).
API / endpoint	A counter where machines queue.	An address meant for programs, not people (e.g. /api/jobs/publish).
Webhook	A doorbell the other company rings.	A message another service sends to us when something happens (Resend: delivered, bounced). We verify its signature.

Word	Like you are five	The engineer's version
Queue / job (QStash)	A to-do tray with a timer.	A scheduled or delayed task QStash delivers to us at the right time, with retries.
Cache	Last Sunday's bulletin kept on the table.	A saved copy of an answer so it is quick next time. Public pages are cached; admin pages never.
Redis	The sticky note.	A very fast key-value memory (Upstash). Holds counters, limits and cache; always rebuildable.
Rate limit	A turnstile: one at a time.	A cap on how often someone may do something, to stop abuse.
Session	A wristband for today.	Proof you signed in, valid for a time. Created by Better Auth.
Role / scope	Your job and your area.	Role = what you may do; scope = where (global, one branch, one department). Together a RoleAssignment.
Permission matrix	A chart of who holds which key.	A table of role against class with letters C R U D P A (B04a, B04b).
Audit log	The visitors' book for staff.	An append-only record of who changed what and when.
Consent	Asking before you take a photo.	A recorded, specific, informed yes from the person, with the wording version.
Personal data (PII)	Anything that points to a real person.	Name, phone, e-mail. In our model marked *priv*.
Sensitive data	Things that could hurt if shown.	Religious belief, health, children's data. Marked *sens*; handled most strictly. A "born again" decision is belief data.
i18n / locale	Speaking in two languages.	Internationalisation: the code supports many languages; locale is one language (en, sw).
hreflang / canonical	Telling Google "this page also exists in Kiswahili".	HTML hints that connect language versions and name the preferred address of a page.
SEO	Making the church easy to find.	Titles, descriptions, sitemaps, speed, structured data so search engines understand the pages.
Presigned URL	A one-time ticket to the shed.	A short-lived address that lets the browser upload straight to R2 without our server carrying the file.
Preview deployment	A rehearsal stage.	A private copy of the site for every change, with its own database branch.
CI / pipeline	A checklist robot.	Automatic checks on every change: types, tests, security scan, speed.
Idempotent	Pressing the lift button twice is the same as once.	Doing it again gives the same result. Essential for jobs and webhooks that may arrive twice.
Minor units	Counting in cents, not in 12.5 dollars.	Money stored as whole numbers (KES 1,500.00 stored as 150000) so nothing is rounded wrongly.
Append-only	Write in ink, never rub out.	Rows are added, never edited or deleted; a mistake is voided with a reason and a new row.
Soft delete	The recycle bin.	Mark as deleted and keep for 30 days before real removal.
Double opt-in	Replying "yes, it is really me".	The person confirms by clicking a link in an e-mail before they are subscribed.
ADR	A signed decision note.	Architecture decision record: the choice, why, and when to revisit it.
Gate	A checkpoint with a pass list.	A named review (G1 to G5) that work must pass before the next layer starts.

PART 2

What you asked for, and where it is answered

I turned your single long brief into **32 numbered requirements** (R-01 to R-32). This table is the proof that nothing was dropped: every requirement points to the diagram pages that answer it and to the questions that still need you. Engineers call this a **traceability matrix**.

> **EXPLAINED LIKE YOU ARE FIVE**
> Think of your brief as a shopping list. This page puts a tick beside each item and writes the aisle where it was found. If an item has no aisle, it was forgotten; if an aisle has no item, it is something I added (I marked those *Proposed*).

#	What you asked	Where it is designed	Status	Questions
The public website				
R-01	Landing page: hero, about, short history, events and programmes, leadership, projects, contact just above the footer	K02, F04, D04	Designed	Q-05, Q-06
R-02	About page: history, vision, leaders, timeline	B33, B24, K02	Designed	Q-07
R-03	Departments with sub-departments, leaders and a join form	B23, B24, B46, F08, B16, E03	Designed	Q-10
R-04	Home-fellowship finder (nearest to me)	B25, S11	Designed	Q-46
R-05	Events with dates, guest ministers, sessions and registration	B34, B35, S07, F10, T06, E05	Designed	Q-36
R-06	Programmes: Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again	B37, B38, S14, F11, E05	Designed	Q-37, Q-38
R-07	Go Ye: outreach, born-again counter, a form to add a convert	B36, B39, S06, F07, T05, O02, E06	Designed	Q-26 to Q-30
R-08	Projects with milestones and a given / remaining bar	B40, B41, S12, F12, T08, E07	Designed	Q-33
R-09	Contact page with giving details	B42, B48, S13	Designed	Q-31, Q-32
R-10	Resources: blogs, bulletins, announcements (Sunday and midweek), Bible study and discipleship, other	B28, B30, B31, T01, E04	Designed	
R-11	Sermons with video, audio, summary, verses, filters, views, comments and likes	B26, B27, B29, S01, S02, S09, T03, E04	Designed	Q-42, Q-43
R-12	English and Kiswahili switch	B08, F04, S03, T02, D09, E11	Designed	Q-13 to Q-19
R-13	Help with translating dynamic content (blogs, sermons, bulletins and so on)	F05, S03, T02, B15, B49	Designed	Q-13, Q-14
R-14	New-member tour of nine steps, ending in a certificate	F06, S04, S05, T04, B44, B45, O01, E08	Designed	Q-20, Q-22 to Q-24
R-15	Modal order (new-member question before the newsletter), a cookie bar, and "No" remembered in localStorage or cookies	F02, T09, S04, B48, E09	Designed	Q-21, Q-25
R-16	Newsletter with a confirmed sign-up	B47, S08, F13, T07, E09	Designed	Q-39
R-17	Comments and likes with moderation	B28, B29, S09, T03	Designed	Q-43
R-18	A search box (my suggestion, not in your brief)	K02	Proposed	Q-44
The admin area				
R-19	An admin area for staff only, signed in with Better Auth	B20, S10, F03, D10, E02	Designed	Q-08, Q-12

#	What you asked	Where it is designed	Status	Questions
R-20	A design for each role, with class territories bordered by role	B01, B04a, B04b, B10 to B19, U01 to U06	Designed	Q-09
R-21	Publishing with a review step and scheduling	T01, F09, S02	Designed	Q-10
R-22	Upload pictures, audio and documents (Cloudflare R2)	B32, F14	Designed	
R-23	Follow-up of new believers by a discipler	B39, F07, T05, B17	Designed	Q-28
R-24	Treasurer's record of gifts that drives the project bars	B42, S12, T08, B18, E07	Designed	Q-31 to Q-35
R-25	Audit log, privacy requests and a security-incident routine	B21, F15, D10, E14	Designed	Q-47, Q-48, Q-51
Technology				
R-26	Next.js, Drizzle, Neon Postgres, Resend, Upstash Redis, QStash, PostHog, Better Auth (admin), Tailwind, Cloudflare R2, Vercel	F01, B09, B49, B50, K01, E10, E12, E13	Designed	Q-49
R-27	Other tools "for the needs ahead"	Part 6	Proposed	
What you asked me to deliver				
R-28	A series of questions about anything I did not understand	Part 3	Done	
R-29	A broad system design: what depends on what, what inherits what, what is "the main", and the synergy between them	B02, B03, B05, B06, B07	Designed	
R-30	Class tabs: each main class broken into its own tab	B20 to B51	Designed	
R-31	UML, flow and block, ER and data-flow diagrams in draw.io, built in phases	all 125 pages; Part 12	Phase 1 done	
R-32	A 50+ page PDF, a prompts file with agents and examples, and one draw.io file that is labelled, categorised, detailed, accurate and movable	the three files	Done	

2.1 What I understood from the videos and the logo

You attached two short logo animations and the logo itself. I looked at both videos frame by frame and at the logo in full.

Item	What it is	What it means for the website
DCEB_INTRO.mp4	10 seconds, 1920×1080, 60 frames a second, about 10 MB. A soft circle appears, becomes the logo, then the words "THE CHURCH OF CHOICE" fade in.	Beautiful, but heavy. 60 fps is not needed on the web; it can be cut to a 2 MB loop at 30 fps.
DCEB_INTRO_2.mp4	15 seconds, 1920×1080, 25 frames a second, about 12.5 MB. Opens on a glassy architectural pattern, then the logo and "DELIVERANCE CHURCH EASTERN BYPASS".	Good for a welcome moment, too heavy for the first screen on a phone. See Q-06.
Logo	A round badge: navy twin triangles forming a mountain or tent, a red cross through the middle, "DELIVERANCE CHURCH" curved on top, "The church of choice" in red, "EASTERN BYPASS" in a gold-to-brown gradient. It sits on a light grey background with a soft shadow.	Navy and red are the brand colours. A clean transparent version is needed (Q-03). The gold in the lettering supports using gold as an accent for the Give button.

MISTAKES TO AVOID

- **Using a 12 MB video as the home page background.** The first impression becomes a spinner on a phone.
- **Putting the logo on the site as a grey-backed picture.** It will look like a sticker on every page. Make a transparent version.
- **Starting a video with sound.** Browsers block it, and it is rude in church.

WHAT YOU ALREADY DID RIGHT

You supplied the logo, the animations and the stack up front. That removes the biggest source of rework: designers and developers guessing the brand.

2.2 Assumptions I made so I could keep going

These are the decisions that are baked into the diagrams. Each is also a question in Part 3, so you can overturn it.

- One site, many possible branches; only Eastern Bypass at launch (Q-01).
- Staff-only accounts. Visitors and new members never sign in (Q-11).
- Giving is information only. The Treasurer records gifts by hand; progress bars come from that record (Q-31).
- English first, Kiswahili second; a machine may draft, a person must approve (Q-13).
- A "No" to the tour or newsletter lives on the visitor's own device and never reaches our server (Q-20, Q-21).
- Likes and views are counted fast in Redis and saved to the database by a scheduled job.
- Belief, children's and phone data are treated as sensitive from day one (Q-47, Q-48).

PART 3

Questions for you, each with a default

You asked for a series of questions about anything I did not understand. There are **51**. For each one I wrote the answer the design already assumes, so you never have to answer from a blank page: **agree, change it, or ask me to explain**.

> **EXPLAINED LIKE YOU ARE FIVE**
> When a builder is not sure whether the door should open inwards or outwards, a good builder does not stop work; they pick the safer one, write it on the plan and ask the owner to confirm. That is what the **default** column is. If you disagree, the cost of changing it is highest for the red ones, so those come first.

Priority	Meaning	Count
Blocking	Answer before Phase 2 starts. A wrong default would change the architecture, the screens or the law we must follow.	10
Before build	Answer before the module that needs it is built. The default is safe in the meantime.	26
Can wait	A safe default exists. Confirm whenever it suits you.	15

The 10 questions that block progress

#	Question	Default I will use	Decides
Q-01	Which branches exist at launch? Is Eastern Bypass the only one?	One live branch (eastern-bypass). The system is built for more, but no branch picker is shown until a second branch is added.	Church leadership
Q-02	Who owns and pays for the domain name, and at which registrar is it?	The church registers it in its own name. DNS is kept at Cloudflare in "DNS only" mode (the site itself stays on Vercel). Jeff gets delegated access.	Church administrator
Q-08	Who are the first one or two Super Admins, and how do they sign in?	Two named people. Sign-in by e-mail with a six-digit code, plus an authenticator app for Super Admin and Treasurer. No shared accounts.	Pastor
Q-09	Are these eight staff roles right, and who holds each one at launch?	Super Admin, Branch Admin, Content Editor, Translator/Reviewer, Department Leader, Discipler, Treasurer, Events Coordinator (B04a, B04b). One person may hold several.	Pastor and church administrator
Q-13	Who writes the Kiswahili, and who approves it before the public sees it?	Machine draft, then a named reviewer approves. Nothing machine-made shows to the public until approved. Please name at least two bilingual reviewers.	Pastor
Q-20	Please confirm the nine steps of the new-member tour and their order.	1 consent, 2 about me, 3 who we are, 4 the leaders, 5 departments, 6 home fellowships, 7 programmes (including New Born Again), 8 projects and giving, 9 feedback and certificate.	Pastor and department leaders
Q-26	Who may add a new believer through the Go Ye form? Anyone, or only people on the outreach team?	The form is public as you asked, but guarded: robot check, rate limit and a consent line. A person is counted at once; a Discipler can mark an entry "not real", which removes it from the counter without deleting history.	Pastor and Discipleship lead
Q-31	Please confirm that giving is "details only" at launch, and give the exact details to display.	Details only (paybill/till, bank, international). No online payment. Any change to these details needs two people: Treasurer proposes, Branch Admin confirms, and an alert e-mail is sent.	Treasurer and Pastor
Q-47	Which legal body is the "data controller", and is it registered with the Office of the Data Protection Commissioner?	The church (its registered name) is the controller; Jeff and Kyfaru are a processor. Registration status to be confirmed with a Kenyan lawyer.	Church leadership and lawyer

#	Question	Default I will use	Decides
Q-50	Who is the one final approver for design and content decisions, and what is the launch target?	One named approver. Launch about twelve weeks after the Phase 2 approval.	Pastor

WHAT YOU ALREADY DID RIGHT

You gave me a long brief with clear ideas for the tour, the pop-up order, the counter and the giving bar. That is why **most** of the 51 questions are about *people and rules* (who approves, who may see, how long we keep) and not about features. The features were already clear.

MISTAKES TO AVOID

- **Answering "whatever you think" to a blocking question.** The default is a suggestion, not your decision. Name a person for every "Who decides".
- **Letting a contractor hold the church's accounts** (domain, hosting, e-mail). See Q-49.
- **Treating the law items as final.** They are design defaults to be checked with a Kenyan lawyer.

PART 3 · GROUP A

A. The church and the brand

Q-01 **Which branches exist at launch? Is Eastern Bypass the only one?** Blocking

Why it matters	The branch is the root of the whole data tree (B05). It decides whether the site shows a branch picker, how URLs look and who may see what.
Default	One live branch (eastern-bypass). The system is built for more, but no branch picker is shown until a second branch is added.
Who decides	Church leadership
See	B05, E03, B13

Q-02 **Who owns and pays for the domain name, and at which registrar is it?** Blocking

Why it matters	The domain carries the website *and* the e-mail (SPF, DKIM, DMARC). If it is in a contractor's name, the church can lose the site when the contractor leaves.
Default	The church registers it in its own name. DNS is kept at Cloudflare in "DNS only" mode (the site itself stays on Vercel). Jeff gets delegated access.
Who decides	Church administrator
See	F01, F13

Q-03 **Is there a clean version of the logo (vector, or transparent background)?** Before build

Why it matters	The supplied logo has a grey background and a soft shadow. A header, a favicon, a certificate and a share picture each need a clean version.
Default	A transparent web version is recreated from the supplied image for review; the church confirms it is faithful before it is used anywhere.
Who decides	Jeff and the church media team
See	B33, B45

Q-04 **Are the colours and fonts approved?** Can wait

Why it matters	Only the navy comes from the logo. Gold for the Give button and the paper-white page colour are proposals.
Default	Navy #000066 and the red of the cross from the logo; gold #B07D1A reserved for "Give"; page colour #FBF9F4; headings Source Serif 4, text Inter.
Who decides	Pastor or media lead
See	Phase 2 style sheet

Q-05 **What is the church's one-line name and tagline for search results and share pictures?** Before build

Why it matters	Google shows about 60 characters of the title. We should choose them, not let the search engine invent them.
Default	"Deliverance Church Eastern Bypass: The church of choice" (the logo says "The church of choice").
Who decides	Pastor or media lead
See	K02

Q-06 **Where should the two intro videos play, or should they play at all?** Can wait

Why it matters	Both are 1080p (10 s and 15 s, 10 MB and 12.5 MB). On a phone with a small data bundle a 12 MB video before the page is a reason to leave.
Default	No splash screen. The 15-second clip becomes a short silent loop (about 2 MB, with a still picture shown first) in the hero or About page. Never autoplay with sound; switched off for visitors who ask for reduced motion.
Who decides	Pastor and media lead
See	F04

Q-07 **Do we have written consent to publish the names and photos of leaders, and never of children?** Before build

Why it matters	Names and photos are personal data. Children are the most protected group.
Default	Adults who gave written consent only. No photo or name of a child without a guardian's written consent.
Who decides	Church leadership
See	B24, B32

PART 3 · GROUP B

B. People, roles and access

Q-08 **Who are the first one or two Super Admins, and how do they sign in?** Blocking

Why it matters	This is the first key to the building. If it is shared or lost, everything else is at risk.
Default	Two named people. Sign-in by e-mail with a six-digit code, plus an authenticator app for Super Admin and Treasurer. No shared accounts.
Who decides	Pastor
See	S10, F03, B20

Q-09 **Are these eight staff roles right, and who holds each one at launch?** Blocking

Why it matters	A role is a set of keys. Too few roles and one person holds every key; too many and nobody knows who to ask.
Default	Super Admin, Branch Admin, Content Editor, Translator/Reviewer, Department Leader, Discipler, Treasurer, Events Coordinator (B04a, B04b). One person may hold several.
Who decides	Pastor and church administrator
See	B04a, B04b, B10 to B19

Q-10 **May a department leader publish their department page without a review?** Before build

Why it matters	Speed against safety. A wrong phone number on a department page reaches the public at once.
Default	Their change goes to "in review" and an Editor approves. A Branch Admin may allow auto-publish for a trusted department.
Who decides	Church administrator
See	B16, T01, F09

Q-11 **Will members ever need their own accounts (certificate library, named comments, group chat)?** Before build

Why it matters	Member accounts bring passwords, recovery, privacy requests and a lot of support work.
Default	No. Staff only. A MemberAccount slot is reserved in the design so it can be added later without changing other tables.
Who decides	Church leadership
See	B21, B20

Q-12 **When a staff member leaves, who removes access and how fast?** Can wait

Why it matters	Most breaches in small organisations are an old account that nobody closed.
Default	Branch Admin ends the RoleAssignment within 24 hours; all sessions are revoked at once; the action is in the audit log.
Who decides	Church administrator
See	B21, D10

PART 3 · GROUP C

C. English and Kiswahili

Q-13 Who writes the Kiswahili, and who approves it before the public sees it? Blocking

Why it matters	A machine draft can be wrong in ways that embarrass a church (wrong tone, wrong sense of a Bible term). A named human must say "this is good".
Default	Machine draft, then a named reviewer approves. Nothing machine-made shows to the public until approved. Please name at least two bilingual reviewers.
Who decides	Pastor
See	S03, T02, F05, B15

Q-14 Which translation engine, and what monthly spending limit? Before build

Why it matters	Engines differ in quality for Kiswahili and in price per word. A limit stops a surprise bill.
Default	The engine sits behind one file (translate.ts), so it can be swapped. The first engine is chosen with a test of 20 church sentences. A monthly cap is set by the church.
Who decides	Jeff with the church administrator
See	B49, F05

Q-15 Who keeps the glossary of church words (Deliverance, born again, tithe, home fellowship, discipleship)? Before build

Why it matters	The same word must be translated the same way everywhere, or the site sounds careless.
Default	The Content Lead drafts it; the pastor approves; the engine is told to follow it.
Who decides	Pastor
See	E11, B15

Q-16 Should the admin screens also be in Kiswahili? Can wait

Why it matters	Doubling the admin text doubles the work and the places for a bug.
Default	English only for staff screens in Phase 1.
Who decides	Church administrator
See	K02

Q-17 Is a third language needed (Kikuyu, Luo, Sign Language)? Can wait

Why it matters	Each language needs a reviewer for ever.
Default	No. The design allows more (the Locale list, E12), so it can be added.
Who decides	Church leadership
See	E12

Q-18 Should sermon audio and video get Kiswahili subtitles? Can wait

Why it matters	Subtitles are a different, bigger job than translating text.
Default	No in Phase 1. The sermon summary, verses and notes are translated; the recording is not.
Who decides	Media team
See	B26

Q-19 What does a visitor see when a page has no Kiswahili yet? Before build

Why it matters	Pages will be published in English first; a broken page or a blank is worse than English.
Default	English text with a visible notice ("This page is not yet in Kiswahili"). Search engines are told the English page is the original; no false language links.
Who decides	Jeff
See	F04, F05, B08

PART 3 · GROUP D

D. The new-member tour, pop-ups and cookies

Q-20 Please confirm the nine steps of the new-member tour and their order. Blocking

Why it matters	The tour is the most personal feature. Its steps decide what is stored and what the certificate means.
Default	1 consent, 2 about me, 3 who we are, 4 the leaders, 5 departments, 6 home fellowships, 7 programmes (including New Born Again), 8 projects and giving, 9 feedback and certificate.
Who decides	Pastor and department leaders
See	F06, S04, T04, U03

Q-21 How long should "No thanks" to the newsletter be remembered? Before build

Why it matters	Remembering for ever is polite; asking again after a while may win a subscriber.
Default	The tour "No" is kept on the device with no re-ask. The newsletter "No thanks" is kept for 90 days. Closing the pop-up with Esc counts as "Not now" (asked again at the next visit, up to three times).
Who decides	Pastor
See	F02, T09

Q-22 What exactly should the certificate say and who signs it? Before build

Why it matters	A certificate is a public-facing, printable object with the church's name on it.
Default	"Certificate of Welcome": the name the member typed, church name, date, serial (DCEB-2026-000123), a verification code, and a pastor's signature image (with his permission). The page /certificate/CODE shows only "valid" and the first name.
Who decides	Pastor
See	B45, S05, O01

Q-23 Who may see what a person types in the tour? Before build

Why it matters	People will share belief and contact details only if they trust where it goes.
Default	Only the Department Leader (for a join request) and the Discipler (if the person asked to be called). The "born again" answer is optional and stays hidden unless follow-up was requested.
Who decides	Pastor
See	B44, B17, B16

Q-24 Should a tour that was stopped be resumable? Can wait

Why it matters	People lose signal or are called away.
Default	Resumable for 30 days through the same device; after 90 days an abandoned journey is anonymised by a scheduled job.
Who decides	Jeff
See	T04, F15

Q-25 On which pages may the pop-ups appear? Before build

Why it matters	A pop-up on top of a giving form or a certificate page is disrespectful.
Default	All public pages except forms, giving details, /certificate and the staff area. Never two at once; the cookie bar first, the new-member question second, the newsletter third.
Who decides	Pastor and media lead
See	F02, T09

PART 3 · GROUP E

E. Go Ye, new believers and follow-up

Q-26 **Who may add a new believer through the Go Ye form? Anyone, or only people on the outreach team?** Blocking

Why it matters	A public form can be filled by anyone: by mistake, by a prankster, or with someone else's phone number. The counter and the call list depend on it.
Default	The form is public as you asked, but guarded: robot check, rate limit and a consent line. A person is counted at once; a Discipler can mark an entry "not real", which removes it from the counter without deleting history.
Who decides	Pastor and Discipleship lead
See	S06, F07, O02, B39

Q-27 **When the team member records a decision, has the person agreed to be contacted?** Before build

Why it matters	A decision for Christ is belief data, which is sensitive under the Data Protection Act 2019.
Default	The form has an explicit "may we call you" line. If the person is not present, only a first name and area are saved and no phone number.
Who decides	Pastor
See	B39, F15

Q-28 **Who is the Discipler for each branch, and how quickly must they respond?** Before build

Why it matters	Care that arrives after a month is not care.
Default	First contact within 48 hours; reminders at day 1, 3 and 7; escalation to the Branch Admin at day 5. The job gives each new case to the Discipler with the fewest open cases.
Who decides	Discipleship lead
See	F07, T05

Q-29 **What does the public born-again counter show?** Can wait

Why it matters	A number on a screen can be read as a promise or as a boast.
Default	The total, plus "this year". Never a name, never a list. The starting value (the baseline before the site existed) is a setting.
Who decides	Pastor
See	B36, E06

Q-30 **Is there a special rule when the person is under 18?** Before build

Why it matters	Minors are protected twice: as people and as belief data.
Default	Record first name and area only; contact through a guardian or the youth leader. This needs checking by a lawyer.
Who decides	Pastor and lawyer
See	B39

PART 3 · GROUP F

F. Giving and projects

Q-31 **Please confirm that giving is "details only" at launch, and give the exact details to display.** Blocking

Why it matters	Showing a wrong paybill or account number sends a congregation's money to a stranger. This is the number-one fraud target on church sites.
Default	Details only (paybill/till, bank, international). No online payment. Any change to these details needs two people: Treasurer proposes, Branch Admin confirms, and an alert e-mail is sent.
Who decides	Treasurer and Pastor
See	B42, S12, F12

Q-32 **How does the Treasurer know which project a gift is for?** Before build

Why it matters	The bar is only true if every shilling is matched to the right project.
Default	The giver writes a short project code in the M-Pesa or bank reference; the Treasurer reads the statement and records the line by hand.
Who decides	Treasurer
See	B42, T08

Q-33 **Should the progress bar show amounts, or only a percentage?** Can wait

Why it matters	Some churches prefer not to publish totals.
Default	Percentage and "raised of target". The church may choose percentage only.
Who decides	Pastor and Treasurer
See	B40, F12

Q-34 **Do givers get a receipt or a thank-you message?** Can wait

Why it matters	Automatic receipts need phone numbers or e-mails, which we do not collect on purpose.
Default	No automatic receipt in Phase 1. A manual thank-you is the church's choice.
Who decides	Treasurer
See	S12

Q-35 **May the ledger record a giver's name?** Can wait

Why it matters	A name next to a gift is private information.
Default	Optional "giver label", default "Anonymous". Never a phone number. Never public unless the giver asked.
Who decides	Treasurer
See	B42

PART 3 · GROUP G

G. Events, programmes and messages

Q-36 Are event registrations always free? Before build

Why it matters	Paid registration needs a payment gateway, refunds and receipts.
Default	Free only. Capacity and a waiting list are optional. Paid registration is a later adapter.
Who decides	Events coordinator
See	B34, B35, S07

Q-37 Is the list of five programmes fixed (Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again)? Before build

Why it matters	If staff can add programmes, the form and cohort rules must be general.
Default	Staff can add programmes. These five are the starting content.
Who decides	Events coordinator
See	B37, B38, F11

Q-38 What do we collect when a parent enrols a child? Before build

Why it matters	Children's data is the most sensitive. Collect less.
Default	Guardian name, phone and consent; the child's first name and age group. No photo. No school name.
Who decides	Children's ministry leader
See	B38, S14

Q-39 Who writes the newsletter, how often, from which address, in which language? Before build

Why it matters	A newsletter that stops after two issues is worse than none.
Default	The Editor writes it, monthly, from an address on the church domain, in the language the subscriber chose.
Who decides	Media lead
See	B47, F13, T07

Q-40 Do we use SMS, and for what? Can wait

Why it matters	Each SMS costs money and reaches a very personal device.
Default	Only reminders to people who asked for them (event, follow-up). Off by default; the church pays per message.
Who decides	Church administrator
See	B50, F01

PART 3 · GROUP H

H. Content and search

Q-41 Who writes the first content, and by what date? Before build

Why it matters	The most common reason a website launches late is not code. It is waiting for text and photos.
Default	The media team supplies English text and photos; content is frozen two weeks before launch.
Who decides	Media lead
See	F09

Q-42 Where are sermons recorded and stored today (YouTube, Facebook, a hard drive)? Can wait

Why it matters	Video is the heaviest part of the site.
Default	Video by privacy-friendly embed (YouTube or Vimeo) or from R2; audio in R2. Decided after counting the current library.
Who decides	Media team
See	B26, F14

Q-43 May visitors comment on sermons and blogs, and who moderates? Before build

Why it matters	Comments invite spam and arguments.
Default	Yes, but each comment is held until a moderator approves it; no links; robot check and rate limit.
Who decides	Media lead
See	S09, T03, B28

Q-44 Is a search box needed? Can wait

Why it matters	It is useful once there are hundreds of sermons.
Default	Yes: a basic search of published titles and summaries.
Who decides	Media lead
See	K02

Q-45 Should page addresses use the same words in both languages? Can wait

Why it matters	Translating addresses helps search in Kiswahili but doubles redirects and mistakes.
Default	Same slug in both: /en/sermons/x and /sw/sermons/x.
Who decides	Jeff
See	F04, K02

Q-46 Do the fellowship leaders have exact coordinates for each home fellowship? Before build

Why it matters	The finder needs numbers, not just an estate name.
Default	Leaders give the estate and a meeting point; the admin places a pin by hand. A visitor's location never leaves their phone.
Who decides	Home fellowship leader
See	S11, B25

PART 3 · GROUP I

I. Law, privacy, money and launch

Q-47 **Which legal body is the "data controller", and is it registered with the Office of the Data Protection Commissioner?** Blocking

Why it matters	The law attaches duties (notices, rights of the person, breach reporting) to one named organisation.
Default	The church (its registered name) is the controller; Jeff and Kyfaru are a processor. Registration status to be confirmed with a Kenyan lawyer.
Who decides	Church leadership and lawyer
See	F15, B48

Q-48 **How long do we keep each kind of personal data?** Before build

Why it matters	Keeping data for ever is a risk; deleting it too early breaks records.
Default	Abandoned tours 90 days; contact messages 12 months; closed follow-up cases 24 months; ledger 7 years. All to be verified with a lawyer.
Who decides	Church leadership and lawyer
See	F15, E14, T04

Q-49 **Who pays the monthly services (Vercel, Neon, Resend, Upstash, R2, PostHog, SMS, translation), and in whose name are the accounts?** Before build

Why it matters	The accounts are the church's property. A contractor's credit card behind a church site is a long-term risk to both.
Default	Accounts in the church's name, billed to the church. A cost estimate comes with the infrastructure diagrams in Phase 2.
Who decides	Church administrator and Jeff
See	F01, K01

Q-50 **Who is the one final approver for design and content decisions, and what is the launch target?** Blocking

Why it matters	A project with five approvers has no approver.
Default	One named approver. Launch about twelve weeks after the Phase 2 approval.
Who decides	Pastor
See	Part 12

Q-51 **What if the website goes down or something goes wrong (an incident)?** Before build

Why it matters	People rely on it on Sundays. The law also has a clock: a breach involving personal data must be reported within 72 hours.
Default	An on-call list of two people; an uptime alert by e-mail and SMS; a written breach checklist (F15). Details in Phase 2.
Who decides	Church administrator and Jeff
See	F15, B09

PART 4

The approach: why, how and what

You asked me to show the style of the design, the approach, and the method, and to answer the why, how and what of your project. This part does that in plain words.

4.1 What this project is

A bilingual (English and Kiswahili) website for one church, with a staff area behind a login, built so that more branches can be added by adding data, not by building another website. It has two kinds of work: **telling** (pages, sermons, events, resources) and **listening carefully** (new members, new believers, requests, gifts). The second kind is smaller but riskier, because it holds personal and belief information. Nearly every design choice below is a consequence of that sentence.

4.2 Why the diagrams are drawn from one model

> **EXPLAINED LIKE YOU ARE FIVE**
>
> Imagine ten people each drawing a map of the same town from memory. The maps will disagree. Now imagine one surveyor measures the town once and each map is printed from those measurements. They cannot disagree.

I wrote the design **once** as data: 74 classes, their fields and rules, who may do what to each (the permission table), the 27 fixed lists, the translation fields and the modules. The class diagrams, the permission matrices, the dependency counts, the ER diagrams and the data dictionary in this PDF are all **printed from that one model**. When I corrected a rule (for example, the Treasurer does not manage projects, only the ledger), every page that mentions it changed together.

> **YOU HAVE SEEN THIS BEFORE**
>
> **Stackable.** In Stackable you met the same problem: a school system has students, fees, events and wallets, and the pricing, the roles and the database must agree. Here the discipline is the same, but it is applied before code. **Zoya Botanicals.** Your Zoya admin design (CRM, inventory, WhatsApp ordering) was an admin-first system; the church admin is a smaller cousin of it.

4.3 How: the method in ten steps

Step	What I did	Where you see it	Beginner comparison
1	Read the brief and wrote it as numbered requirements (R-01 to R-32).	Part 2	A shopping list
2	Found the **nouns** (Sermon, Event, Convert…) and the **verbs** (publish, register, record, void). Nouns became classes.	B20 to B51	Listing the rooms and the jobs
3	Grouped classes into nine modules (folders of code) so one change does not shake the whole building.	B06, K01	Wings of a building
4	Counted who depends on whom to find the **main** classes.	B05	Which pillar holds the roof
5	Found shared shapes and made parents: BaseEntity, Publishable, PersonRequest.	B02	The family tree
6	Drew a border for each role and put each class inside its owner's border.	B01, B10 to B19	Who keeps which room
7	Wrote a permission letter (C R U D P A) for every role against every class.	B04a, B04b	The key chart
8	Checked **behaviour**: who calls whom (sequence), the stages of a thing (state) and the route (flow).	S, T, F	Rehearsing the Sunday
9	Followed the **data**: where it enters, where it rests, who reads it (DFD), then fixed the database shape and rules (ER).	D, E	The plumbing and filing plan
10	Filled in real examples (Grace's tour, a Go Ye decision) to see whether the design really works.	O01, O02	A dry run with real names

4.4 The style: a "modular monolith"

A **monolith** is one program deployed as one unit. **Modular** means it is built from clearly separated folders (modules) that may only talk through small, named doors. It is the opposite of both a tangled single file and a swarm of small services.

Option	What it means	Why not / why
Many small services (microservices)	Dozens of small programs that call each other over the network	**Not chosen.** It solves a problem of a hundred engineers. You are one person; every new service is a new thing to deploy, secure and watch.
A separate website and a separate admin app	Two code bases, two deployments	**Not chosen.** They must share the database, login and rules, so you would copy the same code twice.
WordPress with plugins	A ready-made CMS	**Not chosen.** Fast to start, but the new-member tour, the ledger, the follow-up and the role scopes would become a pile of plugins with their own security history. It also does not fit your stack.
A headless CMS (a hosted editor) plus a front end	Content in a vendor tool, pages in your code	**Not chosen for Phase 1.** Another bill and another login; the church data (converts, fellowships) still needs your database. Can be added later behind the Content module.
One Next.js app with nine modules (chosen)	One deployment, one database, folders with rules about imports	**Chosen.** Cheapest to run, easiest to understand, and one lint rule can forbid the shortcuts that cause tangles (K01).

4.5 The principles every diagram obeys

1. **One-way layers.** Screen asks, action checks, service decides, repository reads and writes, database keeps. Calls go down only (B03).
2. **One adapter per outside tool.** Resend, Africa's Talking, QStash, PostHog, R2, Turnstile, Sentry each live in one file (K01).
3. **Branches are data.** A new branch is a new row, not a new website.
4. **Roles carry a scope.** "Treasurer" alone means nothing; "Treasurer of Eastern Bypass" does.
5. **The server decides.** The browser may hide a button; the server checks again every time (F03).
6. **Consent before collection.** We ask first and record what was agreed and which wording version.
7. **Collect less.** Name, age group and branch are enough to start a tour; contact details only on request.
8. **Money is whole numbers and a pen-written book.** Minor units, append-only, voided never edited (S12).
9. **Sensitive data is marked.** Fields are flagged *priv* or *sens* in the model and never logged or sent to analytics.
10. **Translations are rows, not columns.** One row per language per item, with a status (B08, E11).
11. **Redis can be lost.** Anything in Redis must be rebuildable from the database (E13).
12. **Everything that can be scheduled is a job.** Publish at 6 a.m., send in batches, remind in two days (QStash).

4.6 Decisions, the alternatives and when to revisit

Decision	Alternative considered	Why this one	Revisit when
Translation rows (one per language)	A column per language (title_en, title_sw)	Adding a third language is a new row, not a new column on 25 tables	Never, unless a language is dropped
Money in minor units as bigint	Decimal or float	No rounding errors; the database can check amount > 0	A second currency is needed
Better Auth for staff only	Accounts for everyone	Fewer passwords to protect; smaller attack surface	The church wants member accounts (Q-11)
Likes and views in Redis, flushed by a job	Write to Postgres on every view	A popular sermon would write thousands of rows an hour	Counts are needed in real time to the second
"No" stored on the visitor's device	Store "No" on the server	Nothing to protect, nothing to leak, and honest	Never
Presigned upload straight to R2	Upload through our server	Large files do not hit server time limits or memory	Antivirus scanning is required before storage
Class-table inheritance for OutreachActivity	A separate calendar for outreach	Outreach appears in the main events calendar without extra code	Outreach grows its own very different fields

Decision	Alternative considered	Why this one	Revisit when
Giving as a ledger, not a payment gateway	M-Pesa Daraja or Paystack from day one	Church legal and finance approvals come first; the ledger has a slot for the gateway	The church signs a gateway agreement
Public pages cached, admin never	Cache everything	Fast for visitors; staff always see the truth	Never for admin

PART 5

The big picture: main, inherits, depends

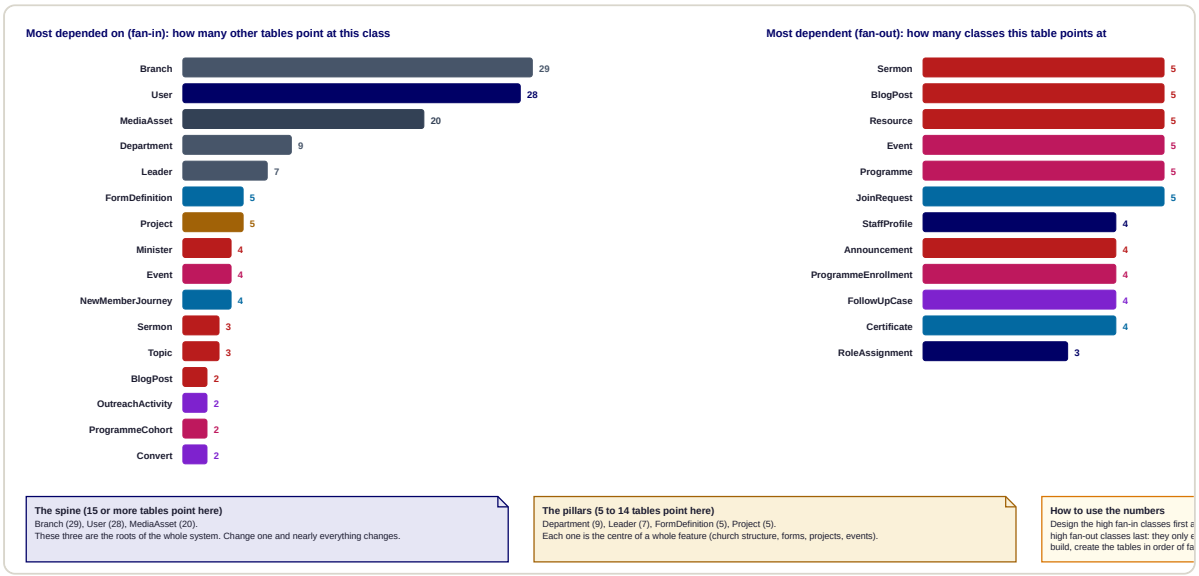
These are the four questions from your brief, answered in plain words with a thumbnail of the page that proves each. All four pages are in the atlas at full size.

5.1 "What is the main?": the three pillars

The honest, measurable answer is: **the class that the most other classes depend on**. I counted, for each class, how many other tables hold a foreign key (a pointer) to it.

Class	Tables pointing at it	Why
Branch	29	The church location: nearly everything belongs to one
User	28	Staff members: who wrote it, who handled it
MediaAsset	20	Pictures, audio, PDFs: almost every page shows one
Department	9	Ministries: leaders, fellowships, forms, requests
Leader	7	People shown on pages: pastors, department heads
FormDefinition	5	Which questions a department or event asks
Project	5	Milestones, updates, gifts, giving channels
Minister	4	Guest preachers across sermons and events

Three classes tower over the rest: **Branch**, **User** and **MediaAsset**. This is why the system is built as one website with branches as data: the Branch is the root of the tree. If you must be careful with one class, be careful with these.



Left: how many tables point at each class. Right: how many classes each class points at. Full size: page B05.

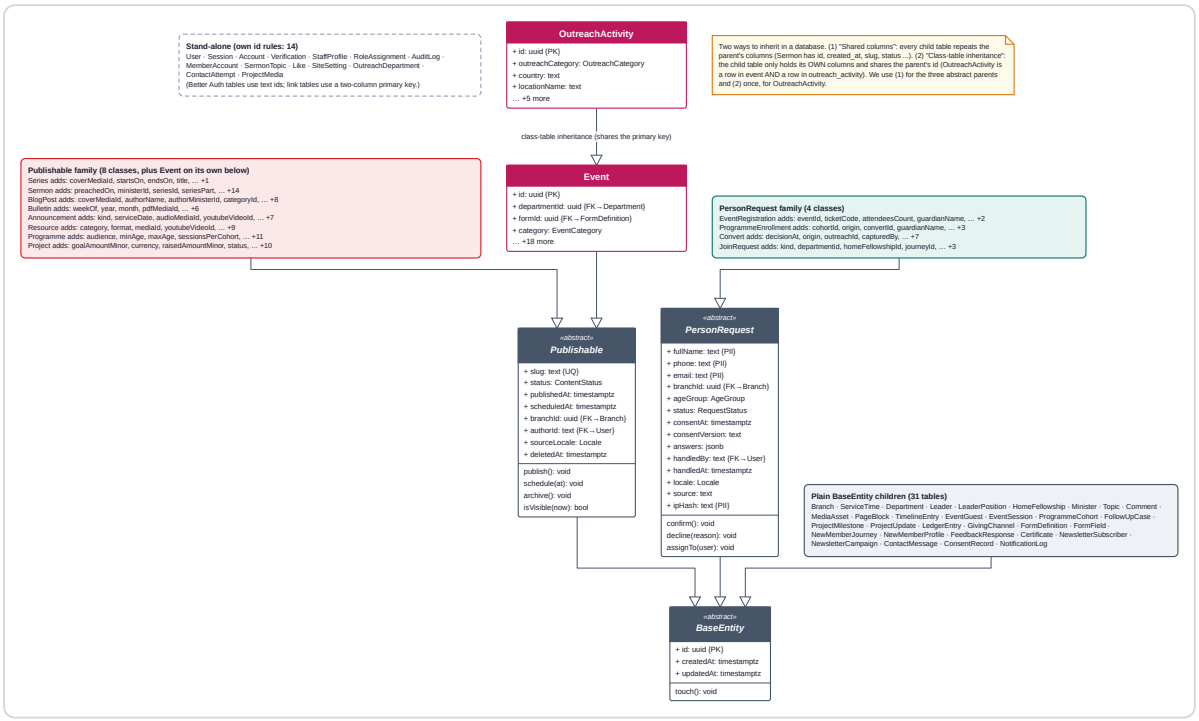
MISTAKES TO AVOID

- **Treating all tables as equal.** A mistake in Branch or User is felt everywhere; a mistake in ContactAttempt is felt on one screen.
- **Copying a name instead of an id** (writing "Eastern Bypass" into many tables). Rename once and half the data is orphaned.

5.2 "What inherits what?"

Inheritance means a child class reuses its parent's fields. There are three families and one special case.

Parent	Children	What they share
BaseEntity	31 classes directly (from Branch to NotificationLog), plus the two parents below	id, createdAt, updatedAt: the birth certificate of every record
Publishable	Series, Sermon, BlogPost, Bulletin, Announcement, Resource, Event, Programme, Project	slug, status (draft to archived), publishedAt, scheduledAt, branch, author, source language, soft delete
PersonRequest	EventRegistration, ProgrammeEnrollment, Convert, JoinRequest	a person asking the church for something: name, phone, e-mail, consent, status, who handled it
Event (class-table)	OutreachActivity	An outreach is an event with a mission: one extra row sharing the same id, so it appears on the events calendar for free



The family tree of the classes. Full size: page B02.

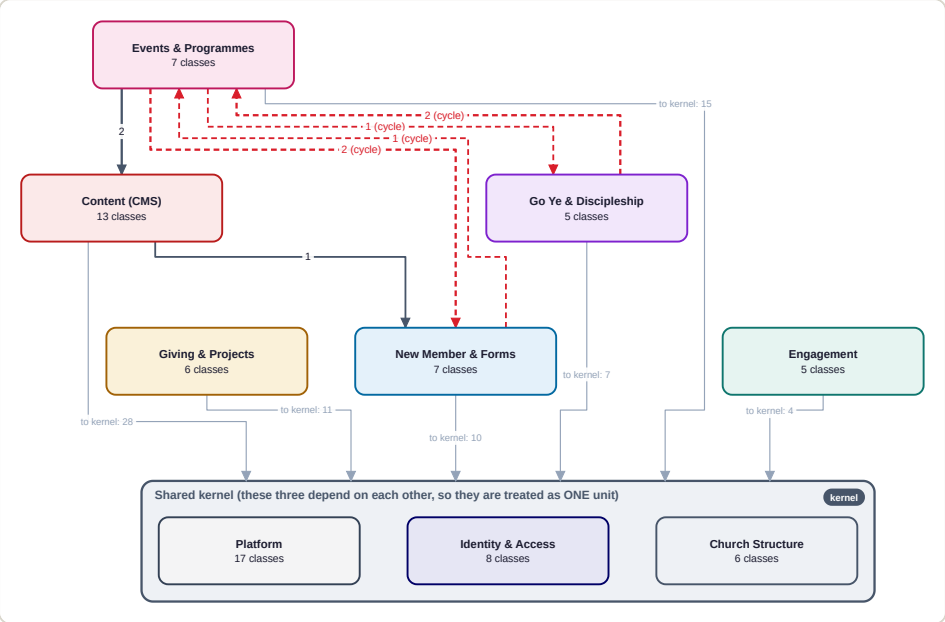
CAREFUL

Drizzle has no built-in inheritance. In code the idea is carried by shared column helpers (a function that returns the id and timestamp columns, spread into each table) and shared TypeScript types. The diagram shows the idea; the helpers are how it is built.

5.3 "What depends on what?"

At class level, the answer is the foreign keys (B05). At module level, it is the nine folders. Three of them (Identity and Access, Church Structure, Platform) point at one another, which is normal for a kernel, so they are drawn as one block. The six feature modules lean on the kernel and hardly on each other.

Module	Classes	What it holds
Identity & Access	8	Better Auth tables, staff profile, role assignment, audit log
Church Structure	6	Branch, service times, departments, leaders, home fellowships
Content (CMS)	13	Sermons, series, ministers, topics, blogs, comments, likes, bulletins, announcements, resources, page blocks, timeline
Events & Programmes	7	Events, guests, sessions, registrations, programmes, cohorts, enrolments
Go Ye & Discipleship	5	Go Ye activities, converts, follow-up cases, contact attempts
Giving & Projects	6	Projects, milestones, updates, the ledger, giving channels
New Member & Forms	7	Forms, the tour journey, profile, feedback, certificate, join requests
Engagement	5	Newsletter, contact messages, consent records, notification log
Platform	17	The three abstract parents, media, settings and the services (authorisation, translation, storage, rate limit, jobs, mail, cache, analytics, moderation, certificates, locator, overlays)



Arrows point from the module that depends to the module it depends on. Full size: page B06.

MISTAKES TO AVOID

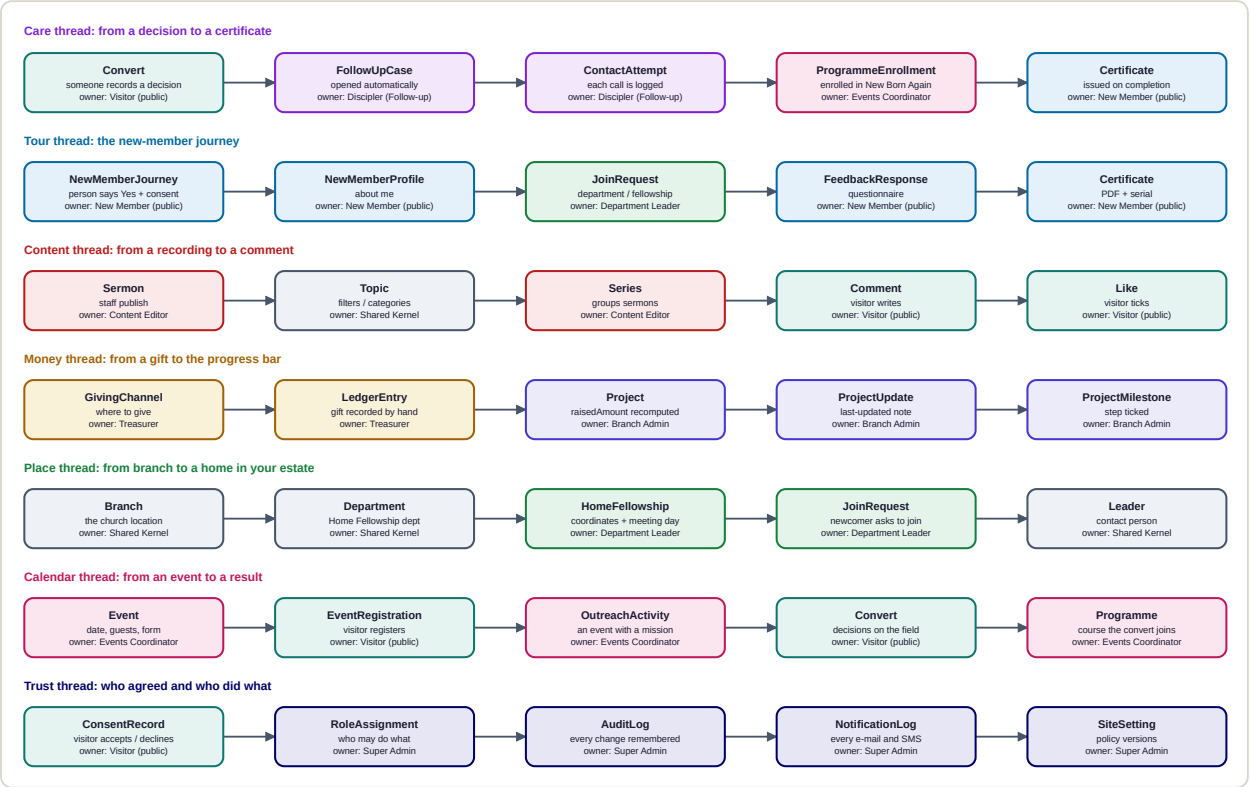
- **Leaving the cycles alone.** Events and New Member point at each other, and Events and Go Ye too. The fix drawn on B06: treat FormDefinition and FormField as part of the kernel, and make the one remaining link (an enrolment pointing at the Convert that caused it) a plain id that the service checks, not a hard foreign key across modules.

5.4 "Find the synergy": seven threads

A design is not only boxes; it is **stories that run through the boxes**. Each thread below crosses several roles, and the places where a story passes from one role to the next are where systems usually break. Those hand-overs are the classes to test first.

Thread	The story	The joint
Care	A decision on the field becomes a Convert, then automatically a FollowUpCase for a Discipler; calls are logged; the person is enrolled in New Born Again; a Certificate follows.	Convert and Certificate

Thread	The story	The joint
Tour	A visitor says Yes and consents; a journey, a profile, a join request, feedback and a certificate are created.	JoinRequest, Certificate
Content	An editor publishes a sermon; topics and series filter it; visitors comment and like.	Sermon, Comment
Money	A giving channel is shown; the Treasurer records a gift; the project bar recomputes in the same transaction.	LedgerEntry, Project
Place	A branch has departments; one is Home Fellowship; a fellowship has coordinates; a newcomer asks to join; a leader answers.	HomeFellowship, JoinRequest
Calendar	An event is created; people register; an outreach is an event with a mission; its decisions become Converts; Converts join programmes.	OutreachActivity, Convert
Trust	Consent is recorded; a role assignment lets a staff member act; the audit log and notification log remember what happened.	ConsentRecord, AuditLog



Seven threads. The colour of each box shows who owns that step. Full size: page B07.

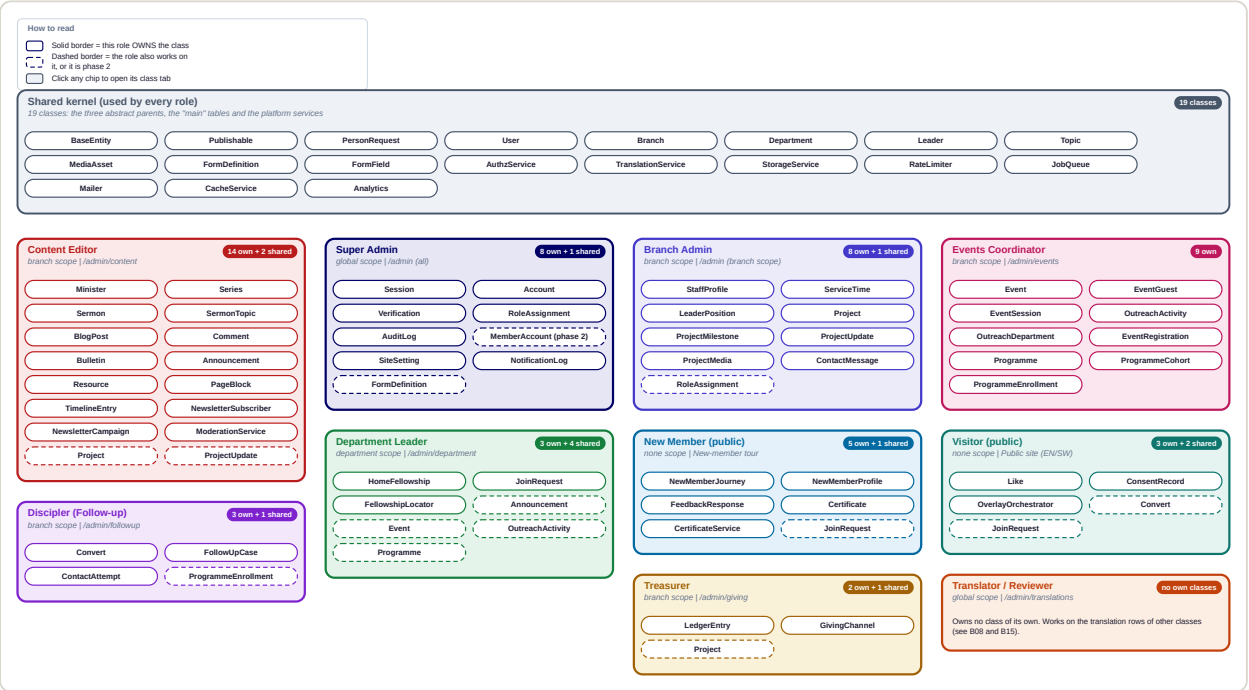
YOU HAVE SEEN THIS BEFORE

Fechi. In Fechi the thread was an order: cart, payment, webhook, stock, SMS. When a webhook failed, you found "orphaned pending orders". The same lesson is in the Care thread: a hand-over that depends on someone remembering will be forgotten, so every arrow between roles is an automatic row, a notification and a reminder.

5.5 One design per role: territories and tabs

Every role has a coloured border in B01 and its own slice page (B10 to B19). The slice shows what the role owns, what it also changes, and what it may only read. The table below is the same idea in words.

Role	Scope	Who it is	Classes it owns
Visitor	none	Anyone on the website who is not logged in.	Like, ConsentRecord, OverlayOrchestrator
New Member	none	A visitor who answered "Yes, give me a tour" in the first modal.	NewMemberJourney, NewMemberProfile, FeedbackResponse, Certificate, CertificateService
Super Admin	global	Head of church IT / Senior Pastor delegate (1-2 people).	Session, Account, Verification, RoleAssignment, AuditLog, MemberAccount, SiteSetting, NotificationLog
Branch Admin	branch	Branch pastor's office / church administrator of ONE branch.	StaffProfile, ServiceTime, LeaderPosition, Project, ProjectMilestone, ProjectUpdate, ProjectMedia, ContactMessage
Content Editor	branch	Media / communications team.	Minister, Series, Sermon, SermonTopic, BlogPost, Comment, Bulletin, Announcement ...
Translator / Reviewer	global	Bilingual member who checks Kiswahili (or English) text.	(owns nothing; uses the shared classes)
Department Leader	department	Head of a department or sub-department (e.g. Youth, Home Fellowship, Missions).	HomeFellowship, JoinRequest, FellowshipLocator
Discipler	branch	Discipleship / follow-up officer.	Convert, FollowUpCase, ContactAttempt
Treasurer	branch	Church finance officer.	LedgerEntry, GivingChannel
Events Coordinator	branch	Events & programmes coordinator.	Event, EventGuest, EventSession, OutreachActivity, OutreachDepartment, EventRegistration, Programme, ProgrammeCohort ...



Class territories: each coloured border is a role. Full size: page B01.

Class tabs

Because a class diagram with 74 boxes is unreadable, each main class (or small family of classes) has its own **tab**, B20 to B51. A tab has the same layout every time so you learn it once: what the class is for in plain words, its fields (with flags for private, sensitive, translatable and cached), its parent and children, who may do what to it, the rules it must keep, the mistakes to avoid, and which diagrams use it. In draw.io each tab is its own page, linked from its chip on B01.

READING A TAB IN FIVE SECONDS

1. Top box: the class name, its module colour and its parent.
2. Middle box: fields. A **lock** flag means personal data, a **red lock** means sensitive.
3. Right: arrows to the classes it points at and the classes that point at it.
4. Bottom: permission letters for each role.
5. Notes: rules, mistakes, and "used by" diagram codes.

PART 6

The stack and how its pieces connect

You named the tools. This part says what each one is for, where it plugs in, what happens when it fails, and which beginner mistake belongs to it. At the end are the extra tools I suggest "for the needs ahead", and what working engineers put around the code.

6.1 The tools you named

Tool	Its one job here	Plugs in at	If it fails	Beginner mistake to avoid
Next.js (App Router) on **Vercel**	Pages, server actions, routes and the first gate (`proxy.ts`)	K01, K02, F04	Site down; a rollback to the previous deployment is one click	Putting secrets in client components; caching admin pages
Neon (Postgres)	The truth: every table, plus point-in-time recovery and a database branch per pull request	E01 to E14, D02	Site cannot save; read-only fallback pages from cache	Using the direct connection string from serverless code. Use the **pooled** one (your Fechi audit flagged missing pooling)
Drizzle + drizzle-kit	Typed tables and migrations	K01 (schema.ts, repo.ts)	Build fails on a type error, which is what you want	Editing a migration after it ran. Add a new one instead
Better Auth	Staff sign-in, sessions, e-mail codes, optional authenticator app	B20, S10, F03	Staff cannot sign in; public site unaffected	Letting the e-mail check be a stub. In Fechi the pre-launch audit found a stubbed email verification. Here it must be real
Resend	Confirmation e-mails, newsletter, notices	S08, F13, B50	E-mails queue and retry; nothing is lost	Sending before SPF, DKIM and DMARC are set; the mail lands in Spam (you fixed this on Fechi)
Upstash Redis	Likes and views counters, rate limits, small caches	E13, B50	Counters pause; limits fail closed for forms, open for reads	Treating Redis as the truth. If it is wiped, the database must rebuild it
QStash	Scheduled and delayed jobs, batches, reminders	F09, F13, F07, K02 (/api/jobs/*)	Retries; a dead-letter list	Leaving a job address open to the world. Always verify the QStash signature
PostHog	Learning how pages are used, only after consent	F02, T09	Analytics gap; no effect on the site	Loading it before the cookie choice; sending names or phones as event data
Cloudflare R2	Pictures, audio, PDFs; a private bucket for certificates	F14, B32	Images fail to load; page text still works	A public bucket with guessable certificate links
Tailwind CSS	Styling with design tokens	Phase 2 style sheet	None at run time	Inventing a new colour on each page. Use the tokens

6.2 The tools I suggest "for the needs ahead"

You asked me to suggest others. Each has a reason tied to a diagram, and an exit if you do not want it.

Suggested tool	Why it is here	Diagram	Replace with
Cloudflare Turnstile	Robot check on the public forms (convert, contact, comments, newsletter), friendlier than image puzzles	S06, S09, S13	hCaptcha or reCAPTCHA
Africa's Talking	SMS reminders to people who asked, in Kenya (you already used it for Fechi)	B50, F07	Another Kenyan SMS provider (one file: sms.ts)
Sentry	Errors with personal data scrubbed, so you hear about a fault before a member does	F01, B09	Vercel observability or Better Stack
next-intl	The English and Kiswahili button labels and the /en and /sw routes	F04	Built-in i18n routing and your own message files

Suggested tool	Why it is here	Diagram	Replace with
Zod + **React Hook Form**	One shared set of input rules used by the browser and the server	B03, S13	Valibot
Tiptap	A safe rich-text editor for blogs and bulletins (output is cleaned before saving)	B28, B30	Lexical or Markdown only
A translation engine	Machine drafts for Kiswahili; a human approves	S03, F05	Any engine behind translate.ts (Q-14)
GitHub Actions	The checklist robot: types, lint, tests, secret scan, a speed check	K01	GitLab CI
Vitest + **Playwright**	Tests of the rules and of whole journeys (the tour, a convert, a ledger void)	S01 to S14	Jest, Cypress
gitleaks	Fails the build if a secret is committed. Your Fechi audit found hardcoded payment credentials in a public repository; this is the guard against a repeat	X-series (Phase 2)	GitHub secret scanning (also turn on)
Lighthouse CI	Speed and accessibility score on every change	Phase 2 (C07)	WebPageTest
Better Stack or **UptimeRobot**	Is the site up? Texts the on-call person	F15	Any monitor
Google Search Console	Shows how Google sees both languages; catches hreflang mistakes	F04	Bing Webmaster Tools

6.3 How the pieces connect: one table to read F01 by

F01 is the whole system on one page. This table is the same picture as sentences. Read "from" to "to" along each row.

From	To	Carried by	How it is trusted	If it breaks
Browser	Vercel (proxy.ts, then the page)	HTTPS	Security headers; language chosen; /admin needs a session	Visitor sees the error page in their language
Page or action	Neon	Drizzle over the pooled connection	Only repositories touch the database; app rights cannot alter tables	Cached public pages keep working
Service	Resend	The Mailer adapter	API key kept in Vercel secrets	Job retries; nothing lost
Resend	Our webhook /api/webhooks/resend	HTTPS POST	Signature checked	Bounce data is late, not wrong
Service	QStash	The JobQueue adapter	Signing key	Task not scheduled: error shown to staff
QStash	/api/jobs/*	HTTPS POST	Signature checked; the job is idempotent	Retries with back-off
Service	Upstash Redis	REST call from the CacheService	Token in secrets	Counters stop; limits fail closed on forms
Browser (staff)	R2	A presigned URL from /api/media/sign	Short-lived, one object, size and type limited	Upload fails; a retry works
Browser	PostHog	Browser script	Loaded only after consent	No analytics; site unaffected
Browser	Turnstile, then our server verifies	Token in the form	Server verifies the token with Cloudflare	Form blocked, honest message
Server	Sentry	SDK	Personal data scrubbed before sending	No error reports

6.4 Environments: three rooms with three sets of keys

Room	For what	Database	Keys	Rule
Local	Writing and trying	A Neon dev branch with **invented** data	.env.local (never committed)	No real people's data on a laptop
Preview	Every pull request gets its own private copy of the site	A Neon branch made for that pull request	Preview keys in Vercel	The reviewer clicks the link and tries the change
Production	The real site	The main Neon branch with backups	Production keys in Vercel; separate from preview	Deploys happen when the gates (G1 to G5) have passed; not on Friday evenings

YOU HAVE SEEN THIS BEFORE

Fechi and Stackable ran on a VPS with Coolify (Hetzner, then Hostinger). This church site is on **Vercel**, which is serverless. Three habits change. (1) **Connections**: every request can open a new database connection, so the pooled connection string is not optional. (2) **Time limits**: a function that runs for minutes is cut off, so long work (newsletter, certificates, translation) becomes a QStash job in small batches. (3) **No long-lived connections**: you used server-sent events for Fechi payment status; on Vercel prefer short polling or a managed push service for anything similar.

6.5 What engineers put around the code, stage by stage

Stage	Tools engineers use	What it protects you from
Plan	draw.io, GitHub Projects or Linear, decision records (ADRs)	Forgetting why a choice was made

Stage	Tools engineers use	What it protects you from
Design	Figma or draw.io wireframes, a token file for colours and spacing	Ten shades of blue
Build	Claude Code, VS Code, pnpm (pin the version), TypeScript strict, ESLint import rules	Shortcuts that break the layers (K01 rule list)
Test	Vitest, Playwright, axe (accessibility), k6 (load)	A form that works on your phone and fails on a cheap one
Secure	gitleaks, Dependabot or Renovate, OWASP ZAP, security headers check	Leaked keys, old libraries, injection
Ship	GitHub Actions, Vercel previews, drizzle-kit migrate on a branch first	A bad migration reaching real data
Run	Sentry, Better Stack, PostHog, Neon backups with a tested restore	Finding out from a member that the site is down
Grow	Search Console, Lighthouse CI, hreflang checks	Slow pages, wrong language in search

HABITS FROM FECHI WORTH KEEPING

- **Pinning nothing.** A pnpm or Node version mismatch caused real deployment pain on Fechi. Pin `packageManager` and the Node version.
- **Creating vendor clients when the file is loaded** (eager instantiation): the build fails when the key is missing on a preview. Create them on first use.
- **Trusting the router cache.** After a staff edit, public pages must be refreshed by tag (`revalidateTag`), or visitors see old content (you met stale router cache on Fechi).

PART 7

English and Kiswahili, including content that staff type

You asked for a switch, and for help translating dynamic content. Those are two different problems, and mixing them is the most common mistake in bilingual sites.

> **EXPLAINED LIKE YOU ARE FIVE**
> There are two kinds of words on the site. **Signs** are the words on buttons and menus ("Give", "Sermons"). They change rarely and are written once by hand, like the signs on the church doors. **Letters** are the words staff write week by week (a sermon summary, a blog, a bulletin). They change all the time, so each one needs its own translation, and a person must read it before it goes on the notice board.

	Signs (interface words)	Letters (content staff write)
Examples	Menu, buttons, error messages, form labels	Sermon title and summary, blog post, bulletin, announcement, department page
Where they live	Two message files in the code: `en.json` and `sw.json`	In the database: one row per language per item (25 translation tables, E11)
Who writes	The Content Lead with the developers, once	The editor writes the original; the engine drafts; a named reviewer approves
Tool	next-intl	TranslationService + the translations queue in /admin
Diagrams	F04	B08, S03, T02, F05, D09, E11

7.1 The life of a translated letter (T02, S03)

1. The editor publishes the English original. The system saves a **fingerprint** of the English text (a hash called sourceHash).
2. A QStash job asks the translation engine for a Kiswahili draft. The draft is saved with the status **machine_draft**. It is **not** shown to the public.
3. The Translator/Reviewer opens the queue, reads the English and the Kiswahili side by side, edits, and presses Approve. The status becomes **approved** and the text is public.
4. If the editor later changes the English, the fingerprint no longer matches. The Kiswahili is flagged **needs_review** so nobody publishes an old translation of a changed text. This is called **drift detection**.
5. If a visitor asks for Kiswahili and there is none yet, they see the English text with a visible notice, and search engines are told the English page is the original.

Status	Visible to the public?	Who moves it on
missing	No (English shown with a notice)	The system creates a draft job
machine_draft	No	A reviewer
needs_review	The previous approved text stays until the reviewer decides (default)	A reviewer
approved	Yes	Only a change to the English can reopen it

7.2 The address of a page (F04)

- Every public page lives under `/en/...` or `/sw/...`. The language is **in the address**, so a link shared on WhatsApp opens in the sender's language.
- The first visit to `/` is sent to one of them: saved choice, else the browser language, else English.
- The language switch keeps you on the same page; it only changes the first part of the address.
- Each page has a **canonical** link (its own preferred address) and **hreflang** links to its other language, **only** when the other language is approved. A missing translation is never advertised to Google.
- The admin area is English only in Phase 1 (Q-16).

7.3 Mistakes to avoid

MISTAKES TO AVOID

- **Publishing the machine draft because "it looks fine".** A single wrong Bible term can embarrass a church. Reviewers are named people.
- **Translating names.** Names of people, places and the church stay as they are.
- **Putting the language only in a cookie.** Google and shared links cannot see cookies.
- **One column per language.** Adding a third language then means editing 25 tables.
- **Translating with different words each time.** The glossary (Q-15) fixes the words for Deliverance, born again, discipleship and so on.

WHAT YOU ALREADY DID RIGHT

You asked for help with dynamic content and for a switch in the same brief. That tells me you already feel the difference between signs and letters. The design simply names it. And because you move between Kiswahili for everyday matters and English for technical ones, you already know that tone changes with the language; the reviewer step is where that care enters the system.

PART 8

Security and privacy: threats and defences

The riskiest data on this site is not money. It is a person's decision to follow Christ together with their phone number. Under the Kenya Data Protection Act 2019 that is **sensitive personal data**. A church website that leaks it can harm real people. Everything below follows from that.

> **LEGAL NOTE**
> I am not a lawyer. Every legal statement in this document (registration with the Office of the Data Protection Commissioner, retention times, deadlines for answering a person's request, the 72-hour breach report) is a **design default to be checked with a Kenyan lawyer** before launch.

8.1 Three kinds of data, three levels of care

Class	Examples	Rule in the model	Never
Public	Sermons, event dates, department pages, the public counter total	Cached and shown to anyone once published and approved	Show a draft
Personal (priv)	Name, phone, e-mail, giver label, comment e-mail, IP hash	Needs a reason and a role; hidden in logs; masked on screen unless needed	Log it; send it to analytics; put it in a URL
Sensitive (sens)	Born-again decision, converts' names and phones, children's data, follow-up notes	Explicit consent text stored with its version; only the Discipler, Branch Admin and Super Admin see it; exports off by default	E-mail it as an attachment; put it in a shared spreadsheet or WhatsApp group

8.2 Twelve threats and the defence drawn for each

#	Threat	Plain-words example	Defence in the design	Diagram
1	Robots and spam on public forms	A script posts 5,000 fake converts or comments	Turnstile robot check, rate limit in Redis, honeypot field, moderation queue	S06, S09, S13
2	Fake or duplicate converts	Two team members enter the same person; a prankster types a stranger's number	Phone normalised and hashed to dedupeKey; a Discipler can mark an entry not real	S06, F07
3	Stolen staff password	Someone guesses a password or phishes the Treasurer	E-mail code; authenticator app for Super Admin and Treasurer; 5 tries then a 15-minute wait; session revocation	S10, F03
4	Staff doing more than their role	An editor opens the converts list	AuthzService.can on **every** action; scope checked against the record's branch or department	B04a, B04b, F03
5	Fraud on the giving details	An account number is quietly changed to a stranger's	Two-person approval and an alert e-mail for any change; audit log	B42, S12
6	Secrets leaking	An API key committed to a public repository (this happened with Fechi's payment credentials)	Keys only in Vercel and GitHub secrets; gitleaks in CI; the prompt files describe secrets by purpose only	F01, K01
7	Fake messages to our job and webhook doors	Someone calls /api/jobs/newsletter directly	Signature check on every /api/jobs/* and /api/webhooks/*; unsigned means 401	K02, S08
8	Guessable files	A certificate PDF at a predictable address	Private bucket; short-lived signed addresses; random keys; verification page shows only "valid" and a first name	S05, F14, B45
9	Script injection in rich text	A blog post contains a hidden script	The editor output is cleaned before saving; strict content security policy	B28, F14

#	Threat	Plain-words example	Defence in the design	Diagram
10	Leaks through logs and analytics	A phone number appears in an error report or a PostHog event	Fields flagged priv and sens; scrubbing in Sentry; PostHog only after consent and without personal fields	F02, B09
11	Old accounts and old libraries	A volunteer left a year ago; a library has a known hole	Role end dates; 24-hour removal rule (Q-12); dependency updates	B21, D10
12	A breach nobody notices	Data copied for weeks	Audit log, alerts, breach checklist with the 72-hour clock	F15

YOU HAVE SEEN THIS BEFORE

Stackable. You audited the custom authentication in Stackable and found 14 weaknesses. That is exactly why this site does **not** write its own login: Better Auth handles passwords, codes and sessions, and we write only the part that is ours (roles and scopes).

8.3 What a person may ask of the church, and how it is answered (F15)

A person may ask what we hold about them, ask for a correction, ask for deletion, or withdraw consent. F15 draws the route: the request arrives (a staff-only form), the identity is checked, the Super Admin finds the records (a search by phone or e-mail hash), exports or erases them, and writes the audit line. A **withdrawal of consent** anonymises a tour profile at once; the certificate serial stays, because it is a record the church issued, but the name can be removed on request. The time limits in the law are marked "verify with a lawyer".

MISTAKES TO AVOID

- **Collecting "just in case".** Every field you do not collect is a field you cannot leak.
- **Keeping a list in WhatsApp "for convenience".** It cannot be audited or deleted.
- **Writing the consent text last.** The wording and its version are part of the data model (consentAt, consentVersion).

PART 9

Mistakes, stage by stage

You asked me to name the mistakes you may make and have made, and what you did correctly, in every stage. I only list what I can see in this project and in the work you have shown me before. I do not invent faults.

Stage	You did right	Easy to get wrong next	A mistake you have met before (and the habit that prevents it)
1. The brief	A long, clear brief with named pages, named programmes, the pop-up order and the tour. Phases requested on your own initiative.	Writing "a form to add a convert" without saying who may use it; "give option" without saying whether money moves (see Q-26, Q-31).	Features without rules. **Habit:** for every feature write who, how often, and what must never happen.
2. Questions	You asked for questions instead of guessing.	Answering "whatever you think" to a blocking question.	Decisions with no owner. **Habit:** name one approver (Q-50).
3. Domain model	You asked "what is the main, what depends on what, what inherits what" before any code.	Giving every class equal importance; copying names instead of ids; a shared kernel that keeps growing.	Modelling after coding. **Habit:** fix B05 and B06 before the first migration.
4. Database	You chose Neon + Drizzle, which you already know.	Floats for money; editing a ran migration; using auto-increment ids; the direct connection string on serverless.	In Fechi the pre-launch audit found **no connection pooling**. **Habit:** pooled string from day one.
5. Sign-in and roles	Better Auth for staff only; roles with scope.	Checking permission only in the browser; one shared admin account; forgetting to close a leaver's access.	Stubbed e-mail verification and permission checks outside try/catch on Fechi's admin. **Habit:** permission check is the **first** line of every action.
6. Content and publishing	Two announcement kinds, one resource list with filters, a sermon with topics and series.	Treating "published" as a yes/no field; showing drafts because one query forgot the filter.	Stale pages after an edit. **Habit:** revalidate by tag after every publish.
7. Language	Asked for dynamic translation help, not just a switch.	Publishing a machine draft; language only in a cookie; translating names.	None known. **Habit:** a named reviewer and a glossary.
8. Forms and people	Asked for consent, a cookie bar and a remembered "No".	Asking for a phone number on step 2 of the tour; importing an old e-mail list into the newsletter.	Forms without limits. **Habit:** robot check + rate limit + consent line on every public form.
9. Giving	A bar that is computed, a ledger, no money moving yet.	Letting anyone edit the account number; editing the project total instead of adding a ledger line.	Webhook handling on Fechi (three payment providers). **Habit:** idempotent handlers and signature checks, ready for the day a gateway arrives.
10. Files and media	R2 with presigned uploads.	A public bucket for certificates; no size limit; storing a URL instead of a key.	Large files through the server. **Habit:** direct upload, private bucket.
11. Jobs and e-mail	QStash for schedules; Resend with a verified domain.	One job that sends 5,000 e-mails; no unsubscribe link; no SPF, DKIM, DMARC.	E-mail landing in spam until DMARC was added on Fechi. **Habit:** set the three records before the first send.
12. Testing	Diagrams give testable rules (T-diagrams are test cases).	Testing only the happy path; no test that a Treasurer cannot read converts.	Late discovery. **Habit:** the permission matrix (B04a, B04b) becomes an automated test.
13. Launch and run	Preview deployments and database branches in the plan.	Launching on a Friday evening; never restoring a backup once.	Config and build surprises at deploy time (versions, SSL, memory). **Habit:** rehearse the release on a preview first.

9.1 Mistakes I made in this design, and corrected

A design is only trustworthy if the person who made it also checks it. These are the real corrections made while building the set.

Where	What was wrong	What I did
Giving data-flow page (D08)	It let the Treasurer manage projects and drew the void line backwards; it disagreed with S12, T08 and the permission table.	Rewrote it: administrators manage projects; the Treasurer works only the ledger; a mistake is voided with a reason.
Module map (B06)	It showed two cycles (Events with New Member, and Events with Go Ye).	Kept them visible, and wrote the fix: FormDefinition and FormField join the kernel; the remaining link is a plain id checked in a service.
Sign-in flow (F03)	A retry arrow made the flow loop in an odd way.	Replaced it with a dashed "goes back to sign-in" step.
Context and level 1 data-flow pages (D01, D02)	Crossing lines, labels on lines, overlapping arrows.	Wrote a dedicated routing step for these pages and re-checked every page by eye.
Pop-up memory (F02)	The text said "we do not ask again" for both questions, but the flow asks the newsletter again after 90 days.	Made the text and the flow agree; the choice is Q-21.

PART 10

The atlas: all 125 diagrams, one per page

From here on, every page is one diagram, drawn at the size it was designed for so you can read the text in it by zooming. Under each picture are three boxes: **how to read it**, **mistakes to avoid** and **what you already did right**. The same 125 diagrams are in **DCEB_Diagrams.drawio**, where every box can be moved and every arrow follows.

Family	Pages	Codes (click to jump)
Class architecture (UML)	20	B01, B02, B03, B04a, B04b, B05, B06, B07, B08, B09, B10, B11, B12, B13, B14, B15, B16, B17, B18, B19
Class tabs (UML detail)	32	B20, B21, B22, B23, B24, B25, B26, B27, B28, B29, B30, B31, B32, B33, B34, B35, B36, B37, B38, B39, B40, B41, B42, B43, B44, B45, B46, B47, B48, B49, B50, B51
UML: use cases	6	U01, U02, U03, U04, U05, U06
UML: sequence	14	S01, S02, S03, S04, S05, S06, S07, S08, S09, S10, S11, S12, S13, S14
UML: state machine	9	T01, T02, T03, T04, T05, T06, T07, T08, T09
UML: component and package	2	K01, K02
UML: object	2	O01, O02
Flow and block diagrams	15	F01, F02, F03, F04, F05, F06, F07, F08, F09, F10, F11, F12, F13, F14, F15
Data flow	11	D01, D02, D03, D04, D05, D06, D07, D08, D09, D10, D11
ER (database)	14	E01, E02, E03, E04, E05, E06, E07, E08, E09, E10, E11, E12, E13, E14

HOW THE CODES WORK

The letter is the family and the number is the page. **B** class architecture and tabs, **U** use case, **S** sequence, **T** state, **F** flow and block, **K** package and route map, **O** object, **D** data flow, **E** database (ER). The prompts file uses the same codes, so "read B26 and S01 first" always means the same two pages.

MISTAKES TO AVOID

- **Reading diagrams like a story.** They are maps. Pick the question first (who owns it? what order? where is it stored?), then pick the family.
- **Zooming out to the whole page and giving up.** Zoom to 100% and read one box at a time.

ATLAS · FAMILY 1

Class architecture (UML)

Who owns what. The first pages to read. They answer your questions about what is the main class, what inherits what, what depends on what, and what ties it all together. Then there is one page per role (B10 to B19) showing that role's territory.

> **EXPLAINED LIKE YOU ARE FIVE**
> Like the master plan of the compound: which team owns which room, which rooms are shared, and which doors a person with a certain key can open.

Code	Title	What it shows
B01	Class territories: who owns which class	Every class has one home role; the border colour tells you who is responsible for it
B02	What inherits what	One parent gives the same columns and the same life-cycle to many children
B03	Layers: who may call whom	Calls travel one way: screen, entry point, service, data access, outside world
B04a	Permission matrix, part 1: content, engagement, structure, events	Class by role: who can create, read, change, publish, approve, export or submit
B04b	Permission matrix, part 2: Go Ye, giving, new member, identity	Class by role: who can create, read, change, publish, approve, export or submit
B05	What is the main class? Counting who depends on whom	Fan-in (how many tables point at a class) and fan-out (how many classes a table points at)
B06	Module dependencies: the kernel and the six feature modules	Arrows point from the module that depends to the module it depends on; numbers count the foreign keys
B07	Synergy: seven threads that tie the classes together	Each row is a story that crosses roles; the border colour of each box shows who owns that step
B08	The translation pattern: one entity, one row per language	How blogs, sermons and every other text can exist in English and Kiswahili
B09	Services: what they wrap, who calls them, what happens when they fail	Each outside tool is reached through one service so it can be replaced or can fail safely
B10	Role slice: Visitor (public)	Visitor (public): the classes it owns, changes, submits and reads
B11	Role slice: New Member (public)	New Member (public): the classes it owns, changes, submits and reads
B12	Role slice: Super Admin	Super Admin: the classes it owns, changes, submits and reads
B13	Role slice: Branch Admin	Branch Admin: the classes it owns, changes, submits and reads
B14	Role slice: Content Editor	Content Editor: the classes it owns, changes, submits and reads
B15	Role slice: Translator / Reviewer	Translator / Reviewer: the classes it owns, changes, submits and reads
B16	Role slice: Department Leader	Department Leader: the classes it owns, changes, submits and reads
B17	Role slice: Discipler (Follow-up)	Discipler (Follow-up): the classes it owns, changes, submits and reads
B18	Role slice: Treasurer	Treasurer: the classes it owns, changes, submits and reads
B19	Role slice: Events Coordinator	Events Coordinator: the classes it owns, changes, submits and reads

B01

CLASS ARCHITECTURE (UML) · PHASE 1

Class territories: who owns which class

Every class has one home role; the border colour tells you who is responsible for it

How to read

- Solid border = this role OWNS the class
- Dashed border = the role also works on it, or it is phase 2
- Click any chip to open its class tab

Shared kernel (used by every role)

19 classes

19 classes: the three abstract parents, the "main" tables and the platform services

BaseEntity | Publishable | PersonRequest | User | Branch | Department | Leader | Topic
MediaAsset | FormDefinition | FormField | AuthzService | TranslationService | StorageService | RateLimiter | JobQueue
Mailer | CacheService | Analytics

Content Editor

14 own + 2 shared

branch scope | /admin/content

Minister | Series
Sermon | SermonTopic
BlogPost | Comment
Bulletin | Announcement
Resource | PageBlock
TimelineEntry | NewsletterSubscriber
NewsletterCampaign | ModerationService
Project | ProjectUpdate

Super Admin

8 own + 1 shared

global scope | /admin (all)

Session | Account
Verification | RoleAssignment
AuditLog | MemberAccount (phase 2)
SiteSetting | NotificationLog
FormDefinition

Branch Admin

8 own + 1 shared

branch scope | /admin (branch scope)

StaffProfile | ServiceTime
LeaderPosition | Project
ProjectMilestone | ProjectUpdate
ProjectMedia | ContactMessage
RoleAssignment

Events Coordinator

9 own

branch scope | /admin/events

Event | EventGuest
EventSession | OutreachActivity
OutreachDepartment | EventRegistration
Programme | ProgrammeCohort
ProgrammeEnrollment

Discipler (Follow-up)

3 own + 1 shared

branch scope | /admin/followup

Convert | FollowUpCase
ContactAttempt | ProgrammeEnrollment

Department Leader

3 own + 4 shared

department scope | /admin/department

HomeFellowship | JoinRequest
FellowshipLocator | Announcement
Event | OutreachActivity
Programme

New Member (public)

5 own + 1 shared

none scope | New-member tour

NewMemberJourney | NewMemberProfile
FeedbackResponse | Certificate
CertificateService | JoinRequest

Visitor (public)

3 own + 2 shared

none scope | Public site (EN/SW)

Like | ConsentRecord
OverlayOrchestrator | Convert
JoinRequest

Treasurer

2 own + 1 shared

branch scope | /admin/giving

LedgerEntry | GivingChannel
Project

Translator / Reviewer

no own classes

global scope | /admin/translations

Owns no class of its own. Works on the translation rows of other classes (see B08 and B15).

HOW TO READ IT

Think of the church building. The kitchen belongs to the kitchen team, the sound desk to the media team, the offering counting room to the treasurer. Everyone may **walk through** the building, but each room has **one team responsible** for it. This page does the same for the 74 classes (the "things" the software remembers).

Every coloured border is a **role**. A class sitting inside a border is **owned** by that role: that role creates the records day to day and answers for them. A **dashed** chip means the role also edits a class owned by someone else (for example the Treasurer also reads and posts to Project, which the Branch Admin owns).

The grey band at the top is the **shared kernel**: classes that almost every role needs (Branch, User, MediaAsset, Department, Leader, Topic, forms) plus the three abstract parents (BaseEntity, Publishable, PersonRequest) and the platform services. Shared does not mean "anyone may change it". It means "many roles read it, so a change here is a change for everybody" (see B05 for how many tables depend on each).

In draw.io you can open **Layers** (Ctrl+Shift+L) and switch roles on and off, so you can look at one role at a time. Every chip is a link: click it to open that class's tab.

MISTAKES TO AVOID

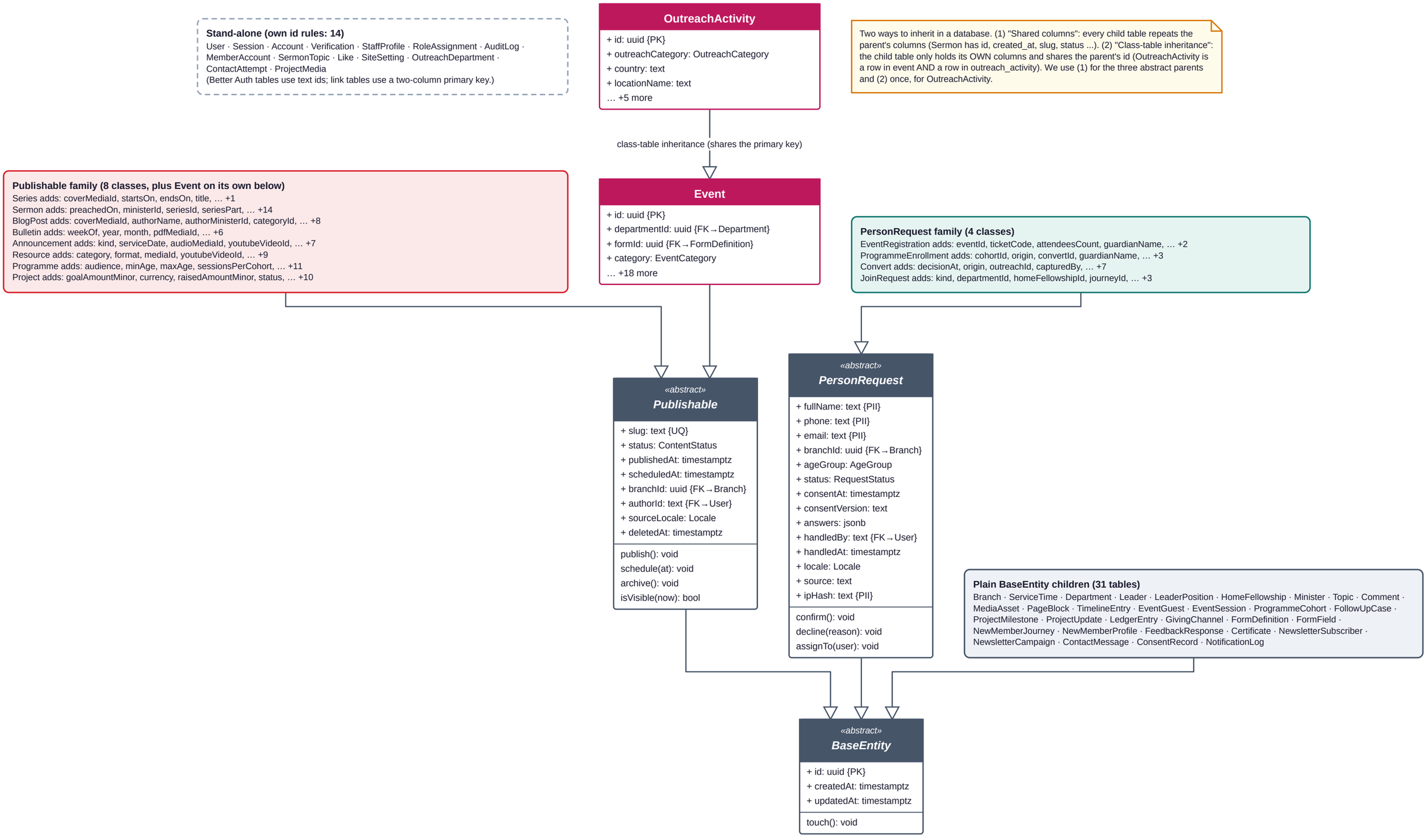
- Letting one role (usually "admin") own everything. When that person leaves, you must remove access to the whole system. Narrow roles can be removed one at a time.
- Confusing **owning** with **the only one allowed to touch**. Ownership says who answers for the data; the permission matrices (B04a, B04b) say who may do what.
- A shared kernel that keeps growing. Every class you add here makes the whole system harder to change. Add to it only when two or more roles truly need the same class.

WHAT YOU ALREADY DID RIGHT

- You wrote your requirements role by role (discipleship receives the new believers; the treasurer-type work is the giving page; departments have their own leaders). That is exactly the territory split on this page, so the design follows your church's real structure and not a programmer's habits.

What inherits what

One parent gives the same columns and the same life-cycle to many children



HOW TO READ IT

Inheritance is how a computer-science family tree works. A parent class holds what all its children share, and each child adds only what makes it different. A real-life version: every person in the church has a name and a date of birth (parent). A pastor *also* has a title; a child in Sunday school *also* has a guardian (children).

BaseEntity is the grandparent: every table has an id, a created-at and an updated-at. Writing those three columns once saves 60 copies and 60 chances for a typo.

Publishable is for anything staff write and the public reads. Because they all inherit it, a sermon, a bulletin and a project all follow the same road: draft, in review, scheduled, published, archived (see T01). Learn the road once and you know it for nine content types.

PersonRequest is "a person knocking on the church door": an event registration, a programme enrolment, a join request and a new convert all carry a name, a phone number, consent and a status. One parent gives them one inbox (T06) and one consent rule.

Be careful with a word: TypeScript and Drizzle do not have "database inheritance" built in. In code the inheritance is made with **shared column helpers** (a function that returns the id, created_at and updated_at columns, spread into each table) and **shared TypeScript types**. The diagram shows the *idea*; the helpers are how the idea is built.

MISTAKES TO AVOID

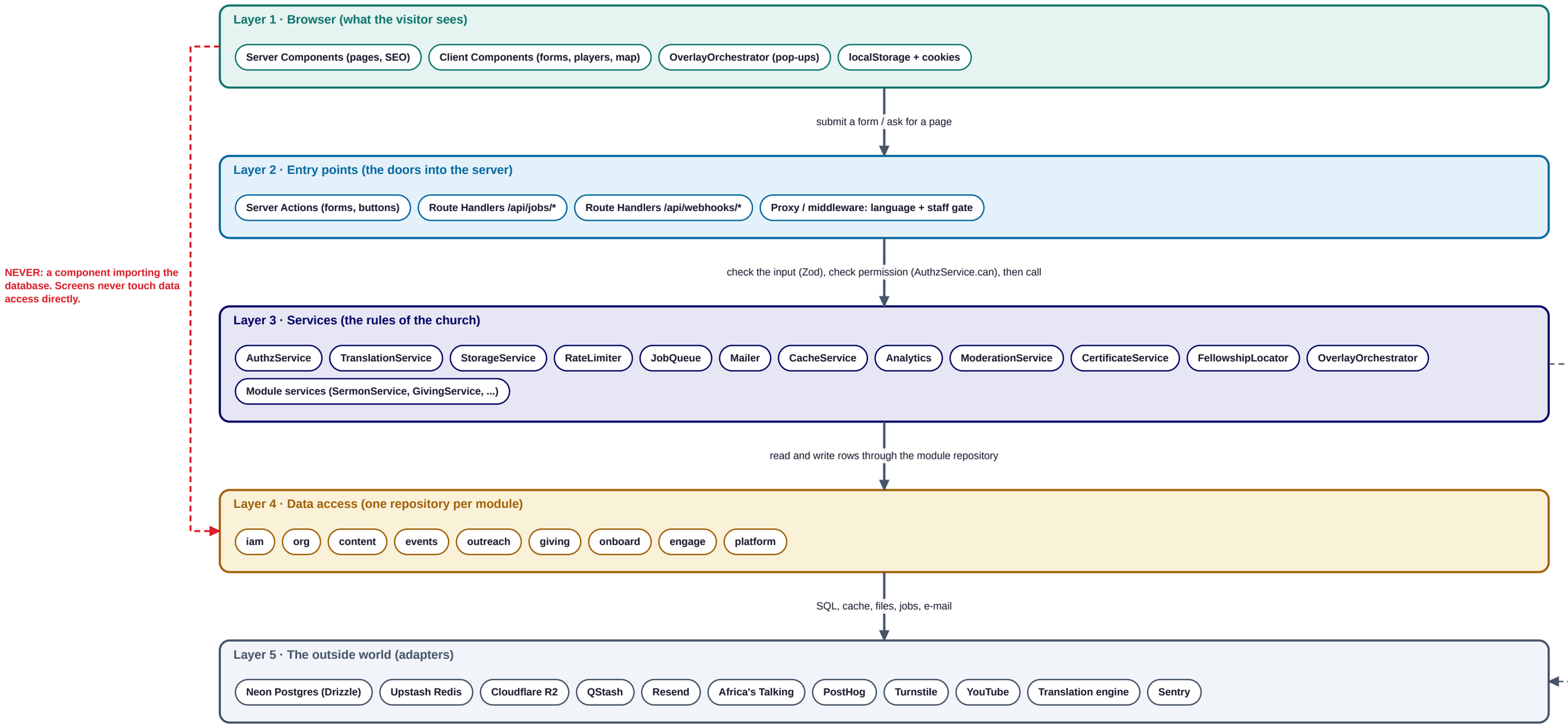
- Using inheritance for everything. Only inherit when children share both data *and* behaviour. Like (a thumbs-up) and ContactAttempt are stand-alone on purpose.
- Making the parent a real table that all children point to (one giant "content" table). It is tempting but forces every query to join. We copy the shared columns into each table instead.
- Forgetting the second kind of inheritance (class-table) exists. OutreachActivity relies on it so that every outreach shows up on the main events calendar without extra code.

WHAT YOU ALREADY DID RIGHT

- You listed that announcements come in two kinds (Sunday and midweek) and that resources share one list with filters. That instinct is the same as the parent-class idea: same shape, different "kind".

Layers: who may call whom

Calls travel one way: screen, entry point, service, data access, outside world



HOW TO READ IT

A website is like a restaurant. The **dining room** (layer 1) is where visitors sit. The **waiters** (layer 2) take the order and bring the food but never cook. The **kitchen rules** (layer 3) say how each dish is made and who may order it. The **pantry shelves** (layer 4) are where ingredients are fetched. The **suppliers** (layer 5) are outside the building.

The golden rule is: **calls only go down one step at a time**. A button never talks to the database. It calls a server action; the action checks the form and the permission, then calls a service; the service asks a repository for rows; the repository talks to Postgres. If you obey this rule you can replace Resend with another e-mail tool by changing **one** file (Mailer), because nobody else calls Resend.

The red dashed arrow on the left is the mistake that beginners and tired engineers make most often: a component that imports the database directly. It works on your laptop and then leaks data or secrets in production.

MISTAKES TO AVOID

- "Quick fix" imports. One shortcut from a page to a repository is copied by the next person, and in six months nobody can tell which rules are enforced. Add a lint rule (eslint-plugin-boundaries or a simple import restriction) so the build fails on a forbidden import.
- Putting business rules inside Server Actions. An action should only check input and permission, then call a service. Rules in services can be tested without a browser.
- Calling vendors from Client Components. Keys would reach the browser. Anything with a secret stays on the server.

WHAT YOU ALREADY DID RIGHT

- You asked for what depends on what. This is the strictest answer: dependencies point **down** only, and B06 shows the same idea between modules.

Permission matrix, part 1: content, engagement, structure, events

Class by role: who can create, read, change, publish, approve, export or submit

Class	Module	Visitor	New member	Super	Branch	Editor	Translator	Dept leader	Discipler	Treasurer	Events
Content											
Sermon	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
Series	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
Minister	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
Topic	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
BlogPost	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
Bulletin	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
Announcement	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	CU	R	R	R
Resource	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
MediaAsset	Platform	–	–	CRUD	CRUD	CRUD	R	CR	R	R	CR
PageBlock	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
TimelineEntry	Content (CMS)	R	R	CRUDPA	CRUDPA	CRUDP	RU	R	R	R	R
Engagement											
Comment	Content (CMS)	S	S	RUDA	RUDA	RUDA	–	–	–	–	–
Like	Content (CMS)	S	S	R	–	–	–	–	–	–	–
NewsletterSubscriber	Engagement	S	S	RUDX	R	RUDX	–	–	–	–	–
NewsletterCampaign	Engagement	–	–	CRUDP	–	CRUDP	–	–	–	–	–
ContactMessage	Engagement	S	–	RUD	RUD	–	–	–	R	R	–
ConsentRecord	Engagement	S	S	R	–	–	–	–	–	–	–
Church structure											
Branch	Church Structure	R	R	CRUD	RU	R	R	R	R	R	R
ServiceTime	Church Structure	R	R	CRUD	CRUD	R	–	R	–	–	R
Department	Church Structure	R	R	CRUDP	CRUDP	RU	RU	RU	R	–	R
Leader	Church Structure	R	R	CRUD	CRUD	RU	RU	R	–	–	–
LeaderPosition	Church Structure	R	R	CRUD	CRUD	–	–	CRU	–	–	–
HomeFellowship	Church Structure	R	R	CRUD	CRUD	–	–	CRUD	–	–	–
Events & programmes											
Event	Events & Programmes	R	R	CRUDP	CRUDP	RU	RU	CU	–	–	CRUDP
EventGuest	Events & Programmes	R	–	CRUD	CRUD	–	–	–	–	–	CRUD
EventSession	Events & Programmes	R	–	CRUD	CRUD	–	–	–	–	–	CRUD
EventRegistration	Events & Programmes	S	S	RUX	RUX	–	–	R	–	–	RUDX
Programme	Events & Programmes	R	R	CRUDP	CRUDP	–	–	RU	R	–	CRUDP
ProgrammeCohort	Events & Programmes	R	R	CRUD	CRUD	–	–	–	RU	–	CRUD
ProgrammeEnrollment	Events & Programmes	S	S	RUX	RUX	–	–	–	CRU	–	RUDX

Letters: C create · R read · U update · D delete or archive · P publish · A approve or moderate · X export · S submit through a public form. A dash means no access at all. Coloured cells are the cells that can change data; grey letters are read-only.

These letters are not decoration. They are loaded into AuthzService, and a test fails the build if a role can do something this table does not allow (that is the "permission-matrix test" of gate G4).

Scope matters too. Branch, Editor, Treasurer, Discipler and Events act only inside their own branch; Department Leader only inside their own department; Super Admin and Translator are global (see RoleAssignment in B20 and B21).

HOW TO READ IT

This is the **door list** of the system. Down the side are the classes; across the top are the roles. Each cell says what that role may do with that class. If you want to know "can a Department Leader delete an event?", find Event, find the Dept leader column and read the letters.

Why write it as a table first? Because a table forces you to decide **every** combination. A role that is not written is a role that has **no access** (the safe default is "no").

A visitor never has a login, yet the visitor column is full of **S** (submit). That is deliberate: a visitor may *put a form in the letterbox* (a comment, a registration, a contact message) but may never read what other people put in.

MISTAKES TO AVOID

- Giving "R" (read) on tables with phone numbers to roles that do not need them. Read access to personal data is a risk by itself: check each R in the people tables (EventRegistration, Convert, JoinRequest).
- Forgetting that export (X) is its own power. A person who can read one record can usually not download all of them. Keep X to a few roles.
- Treating the matrix as documentation. If it is not loaded by the code and tested, it drifts from reality in a month.

WHAT YOU ALREADY DID RIGHT

- You asked that the discipleship department alone receives new believers' names. In this matrix only the Discipler, Branch Admin and Super Admin can read Convert, which is that rule written as data.

Permission matrix, part 2: Go Ye, giving, new member, identity

Class by role: who can create, read, change, publish, approve, export or submit

Class	Module	Visitor	New member	Super	Branch	Editor	Translator	Dept leader	Discipler	Treasurer	Events
Go Ye & discipleship											
OutreachActivity	Go Ye & Discipleship	R	–	**CRUDP**	**CRUDP**	–	–	**CRU**	–	–	**CRUDP**
Convert	Go Ye & Discipleship	**S**	**S**	**RX**	**RX**	–	–	–	**RUX**	–	–
FollowUpCase	Go Ye & Discipleship	–	–	**RU**	**RUD**	–	–	–	**CRUD**	–	–
ContactAttempt	Go Ye & Discipleship	–	–	R	R	–	–	–	**CRU**	–	–
Giving & projects											
Project	Giving & Projects	R	R	**CRUDP**	**CRUDP**	**RU**	**RU**	–	–	R	–
ProjectMilestone	Giving & Projects	R	–	**CRUD**	**CRUD**	**RU**	–	–	–	R	–
ProjectUpdate	Giving & Projects	R	–	**CRUD**	**CRUD**	**CRU**	–	–	–	**CR**	–
LedgerEntry	Giving & Projects	–	–	**RX**	R	–	–	–	–	**CRUX**	–
GivingChannel	Giving & Projects	R	R	**CRUD**	**RA**	–	–	–	–	**CRU**	–
New member & forms											
NewMemberJourney	New Member & Forms	–	**CRU**	R	–	–	–	–	R	–	–
NewMemberProfile	New Member & Forms	–	**CRU**	**RX**	R	–	–	–	R	–	–
FeedbackResponse	New Member & Forms	–	**C**	**RX**	R	R	–	–	–	–	–
Certificate	New Member & Forms	–	**CR**	**RUD**	R	–	–	–	–	–	–
JoinRequest	New Member & Forms	**S**	**S**	**RUX**	**RUX**	–	–	**RUX**	R	–	–
FormDefinition	New Member & Forms	–	–	**CRUDP**	R	**RU**	–	**RU**	–	–	**RU**
FormField	New Member & Forms	–	–	**CRUD**	R	–	–	**RU**	–	–	**RU**
Identity & platform											
User	Identity & Access	–	–	**CRUD**	R	–	–	–	–	–	–
StaffProfile	Identity & Access	–	–	**CRUD**	**CRU**	–	–	–	–	–	–
RoleAssignment	Identity & Access	–	–	**CRUD**	**CRUD**	–	–	–	–	–	–
AuditLog	Identity & Access	–	–	R	R	–	–	–	–	–	–
SiteSetting	Platform	–	–	**CRU**	R	–	–	–	–	–	–
NotificationLog	Engagement	–	–	R	R	–	–	–	–	–	–
Translation (all *_translation rows)	Platform	R	R	**CRUDA**	**CRUA**	**CRU**	**CRUA**	**RU**	–	–	–

Letters: C create · R read · U update · D delete or archive · P publish · A approve or moderate · X export · S submit through a public form. A dash means no access at all. Coloured cells are the cells that can change data; grey letters are read-only.

These letters are not decoration. They are loaded into AuthzService, and a test fails the build if a role can do something this table does not allow (that is the "permission-matrix test" of gate G4).

Scope matters too. Branch, Editor, Treasurer, Discipler and Events act only inside their own branch; Department Leader only inside their own department; Super Admin and Translator are global (see RoleAssignment in B20 and B21).

HOW TO READ IT

This is the **door list** of the system. Down the side are the classes; across the top are the roles. Each cell says what that role may do with that class. If you want to know "can a Department Leader delete an event?", find Event, find the Dept leader column and read the letters.

Why write it as a table first? Because a table forces you to decide **every** combination. A role that is not written is a role that has **no access** (the safe default is "no").

A visitor never has a login, yet the visitor column is full of **S** (submit). That is deliberate: a visitor may *put a form in the letterbox* (a comment, a registration, a contact message) but may never read what other people put in.

MISTAKES TO AVOID

- Giving "R" (read) on tables with phone numbers to roles that do not need them. Read access to personal data is a risk by itself: check each R in the people tables (EventRegistration, Convert, JoinRequest).
- Forgetting that export (X) is its own power. A person who can read one record can usually not download all of them. Keep X to a few roles.
- Treating the matrix as documentation. If it is not loaded by the code and tested, it drifts from reality in a month.

WHAT YOU ALREADY DID RIGHT

- You asked that the discipleship department alone receives new believers' names. In this matrix only the Discipler, Branch Admin and Super Admin can read Convert, which is that rule written as data.

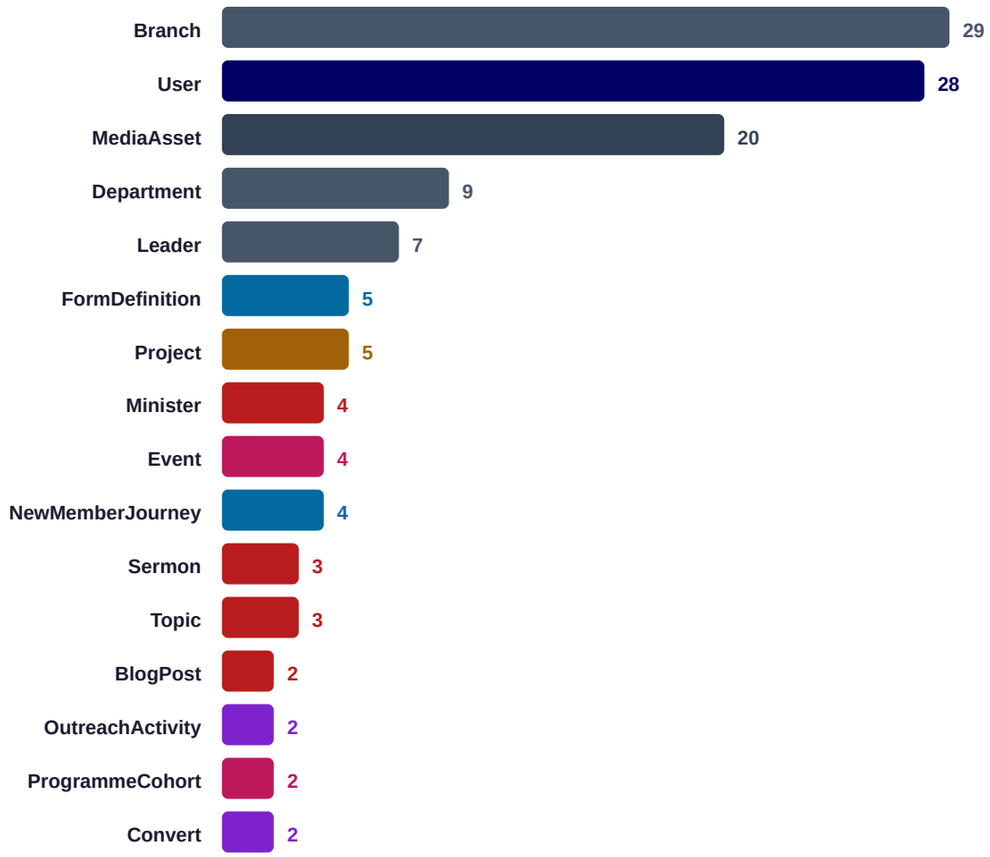
B05

CLASS ARCHITECTURE (UML) · PHASE 1

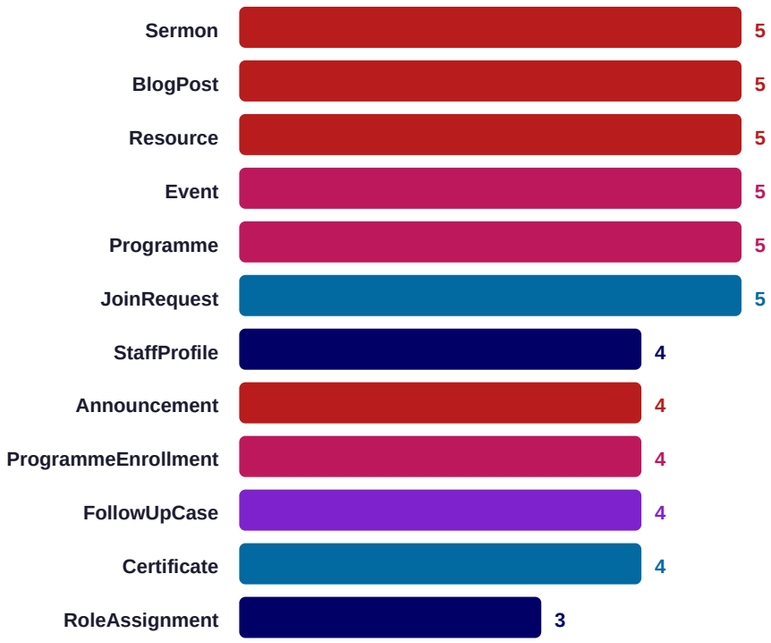
What is the main class? Counting who depends on whom

Fan-in (how many tables point at a class) and fan-out (how many classes a table points at)

Most depended on (fan-in): how many other tables point at this class



Most dependent (fan-out): how many classes this table points at



The spine (15 or more tables point here)
Branch (29), User (28), MediaAsset (20).
These three are the roots of the whole system. Change one and nearly everything changes.

The pillars (5 to 14 tables point here)
Department (9), Leader (7), FormDefinition (5), Project (5).
Each one is the centre of a whole feature (church structure, forms, projects, events).

How to use the numbers
Design the high fan-in classes first a
high fan-out classes last: they only e
build, create the tables in order of fa

HOW TO READ IT

You asked **"what is the main?"**. The honest, measurable answer is: the class that the most other classes depend on. Imagine the church staff list on the notice board. If the head caretaker is away, a handful of people are affected. If the building itself closes, *everyone* is affected. The building is the "main".

Left: for every class we count how many **other tables** hold a foreign key (a pointer) to it. **Branch**, **User** and **MediaAsset** tower over the rest. That is why this system is built as one website with many branches: the branch is the root of the tree, not an afterthought.

Right: the opposite count. A table that points at five other classes (Sermon, BlogPost, Resource, Event, Programme, JoinRequest) is a **busy crossroads**: it is the one most likely to break when something it depends on changes. It is also where most of the screens live.

MISTAKES TO AVOID

- Treating all tables as equal. They are not: a mistake in Branch or User is felt everywhere, a mistake in ContactAttempt is felt in one screen.
- Hiding the dependency by copying a name into many tables (for example the branch name instead of its id). The count drops, the risk rises: rename once and half your data is orphaned.
- Forgetting that a high number is not "bad". It means the class is central and stable. The danger is a central class that changes often.

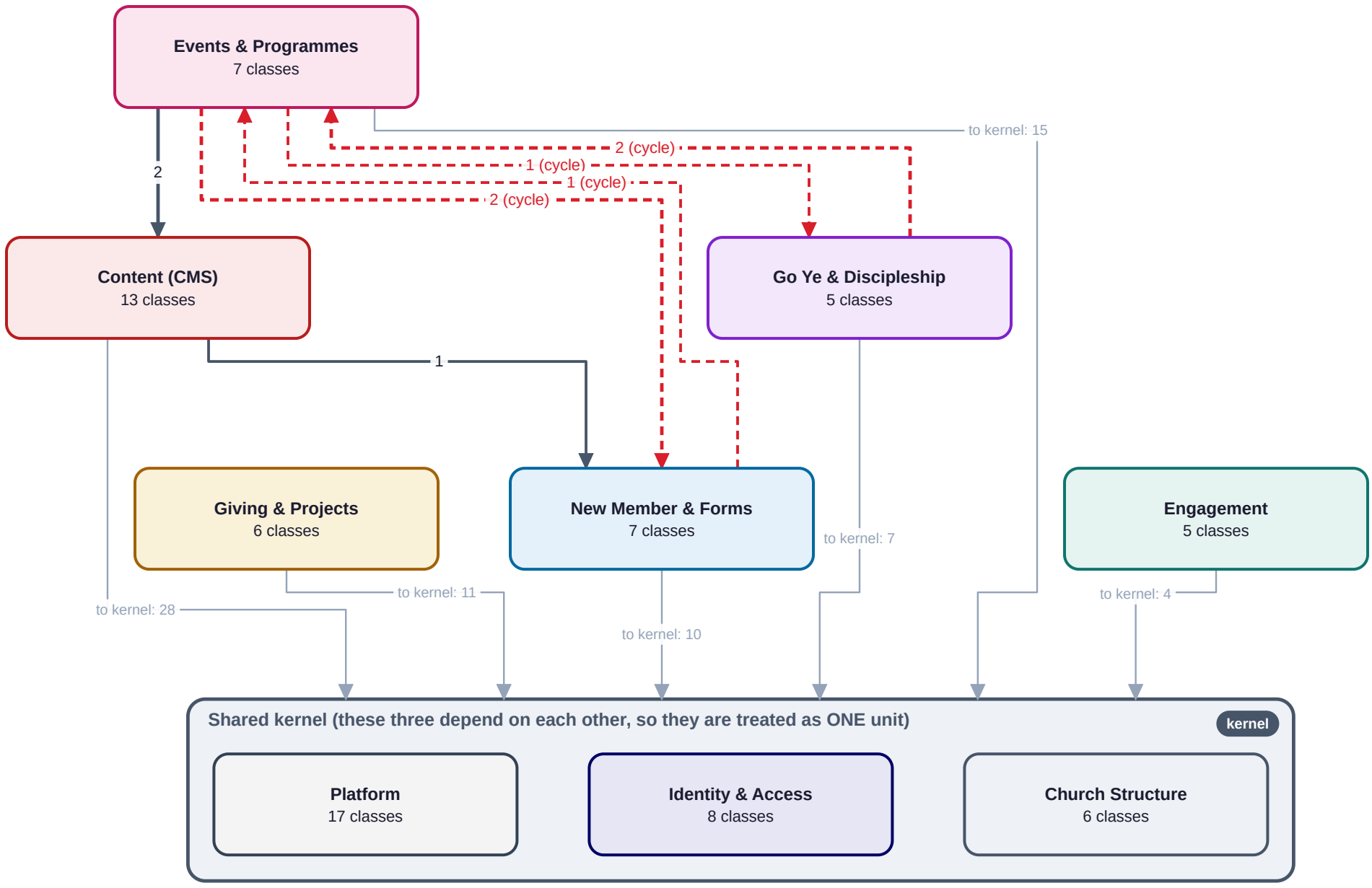
WHAT YOU ALREADY DID RIGHT

- You asked for "a synergy to make it work". The counts show it is real: three classes (Branch, User, MediaAsset) tie all 9 modules together, so the website, the admin area and the branches are one system and not three.

CLASS ARCHITECTURE (UML) · PHASE 1

Module dependencies: the kernel and the six feature modules

Arrows point from the module that depends to the module it depends on; numbers count the foreign keys



HOW TO READ IT

Zoom out one more step. The 74 classes live in **nine modules** (folders of code). This page asks: which module needs which? An arrow from *Events* to *Content* means "Events has a column that points at a Content table" (here: the guest minister, which is shared with sermons).

The three modules Identity & Access, Church Structure and Platform point at each other (a user belongs to a branch, a branch has a media photo, a media file is uploaded by a user). That is normal for a **kernel**, so they are drawn as one grey block. All six feature modules lean on the kernel. Nothing in the kernel may lean on a feature.

Between the six feature modules you can see only a few arrows. That is the sign of a healthy design: a change to Giving does not disturb Content.

Two pairs of feature modules point at *each other*: Events & Programmes ↔ New Member & Forms and Events & Programmes ↔ Go Ye & Discipleship. These are **cycles**. Cycles are what make big programs hard to change, because neither side can be built or tested alone.

MISTAKES TO AVOID

- Leaving the cycles alone. The cheapest fix is to treat **FormDefinition and FormField** as part of the kernel (they are used by Church Structure, Content, Events and New Member); that removes three of the arrows. The remaining link (a programme enrolment can point at the Convert that caused it) should be a plain id reference that the service checks, not a hard foreign key across modules.
- Counting arrows as the only truth. Two foreign keys can be a deeper dependency than ten, depending on how often the code uses them. Use the numbers as a smell test, not a verdict.

WHAT YOU ALREADY DID RIGHT

- You described features (Go Ye, giving, events, sermons) as separate pages that still talk to each other (a convert goes to discipleship, which enrols them in a programme). The design keeps those links but makes them few and visible.

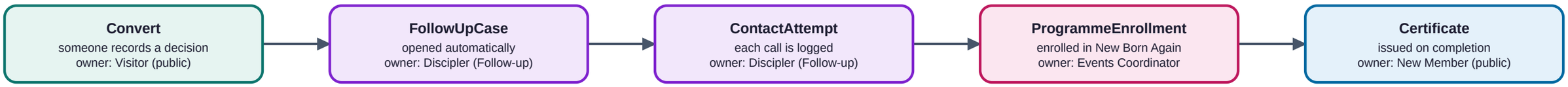
B07

CLASS ARCHITECTURE (UML) · PHASE 1

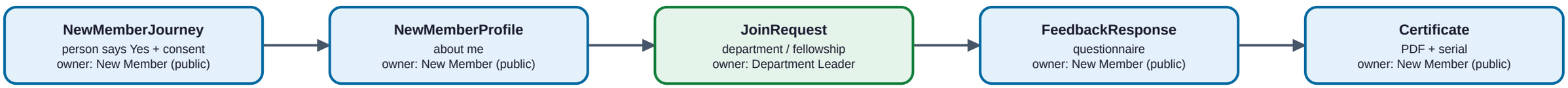
Synergy: seven threads that tie the classes together

Each row is a story that crosses roles; the border colour of each box shows who owns that step

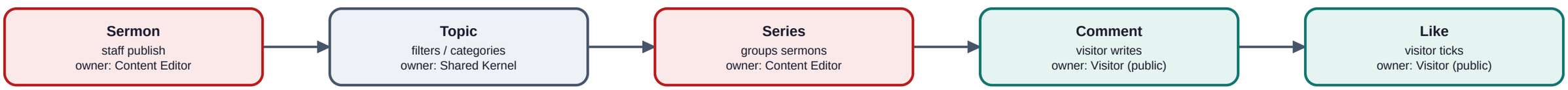
Care thread: from a decision to a certificate



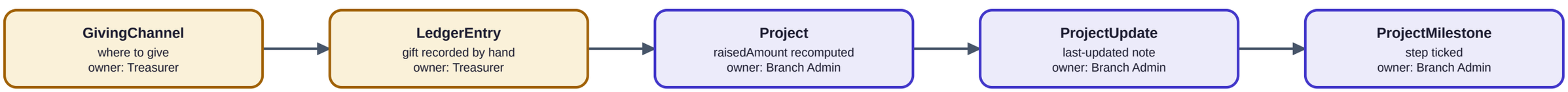
Tour thread: the new-member journey



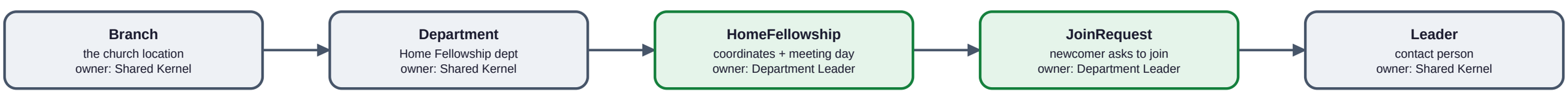
Content thread: from a recording to a comment



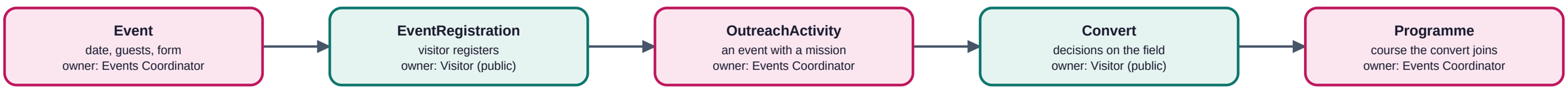
Money thread: from a gift to the progress bar



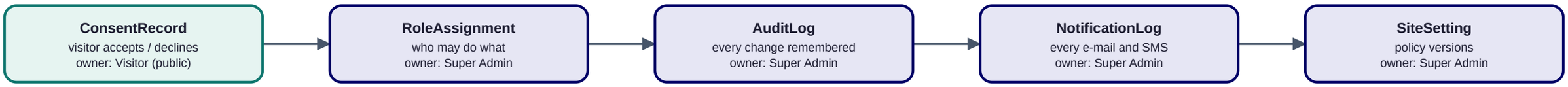
Place thread: from branch to a home in your estate



Calendar thread: from an event to a result



Trust thread: who agreed and who did what



HOW TO READ IT

A good design is not only boxes. It is **stories that run through the boxes**. Here are seven stories of the church. Read each row left to right and notice that the **colour changes** as the story passes from one role to the next. That is where systems usually break: at the hand-over.

Care thread. A visitor on the Go Ye page records a decision (visitor colour). A Convert row appears and, automatically, a FollowUpCase for the Discipler (purple). The Discipler logs calls, enrols the person in the New Born Again programme (the events colour), and later a Certificate exists. Four roles, one person, no copy-and-paste.

Money thread. The Treasurer records a gift (gold). The class that the public sees (Project) is owned by the Branch Admin, but its progress bar is *computed* from the Treasurer's rows, so nobody can type a wrong total.

Trust thread. Every story leaves a footprint: a ConsentRecord when a visitor agrees to something, a RoleAssignment that allows a staff member to act, an AuditLog line when they do. The footprints are what let you answer "who saw this phone number?" later.

This is the "synergy": each thread ends in a class that another thread starts from (the Certificate appears in two threads; Convert in two; JoinRequest in two). They are the **joints** of the system and are the classes to test first.

MISTAKES TO AVOID

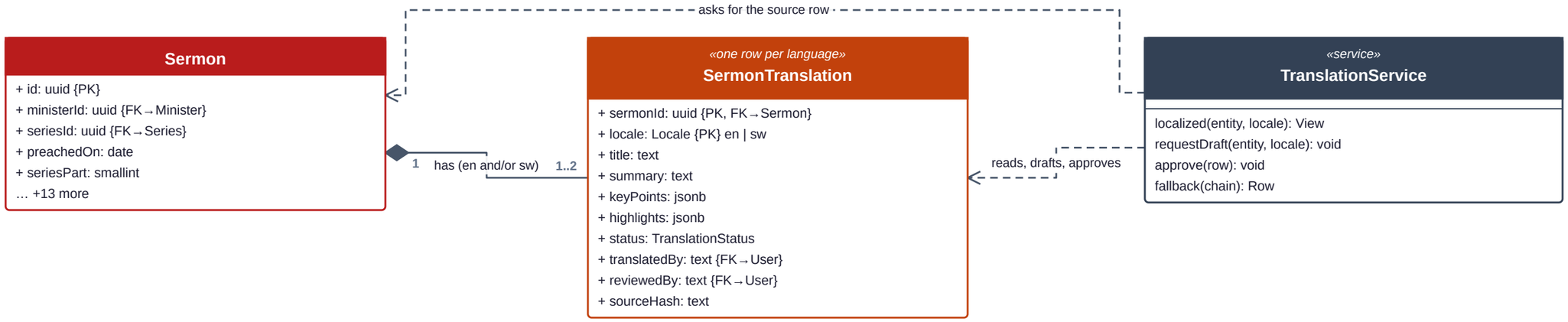
- Building each feature as an island. If the tour does not create a JoinRequest the department leader can see, the new member is "welcomed" and then forgotten.
- Letting a hand-over depend on a person remembering. Every arrow between roles should be an automatic row, a notification and a reminder (QStash), never "please tell the discipler".
- Counting the same person twice. A person in the tour who is also a Convert must produce one record each, linked, and the public counter must count the Convert only once.

WHAT YOU ALREADY DID RIGHT

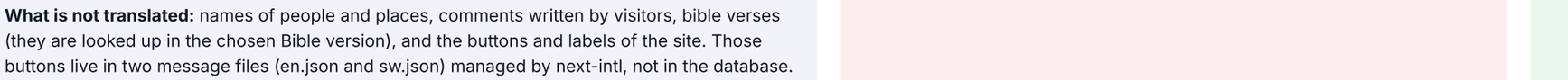
- Your brief already follows these threads: tour to fellowship to feedback to certificate; Go Ye to discipleship to the New Born Again programme; project to giving to progress bar. The design only names the joints.

The translation pattern: one entity, one row per language

How blogs, sermons and every other text can exist in English and Kiswahili



Life of one translation row (see T02)



Only "approved" text is public. If the Kiswahili row is missing or not approv
sees the English row with a small notice: "This page is not yet available in
page is never empty and never shows an unchecked machine translation

The 25 classes that have translated fields (each gets its own *_translation table)

Branch: description · Department: name, summary, whatWeDo, meetingTimes · Leader: bio · LeaderPosition: title, responsibilities · HomeFellowship: notes · Minister: bio · Series: title, description · Topic: name, description · Sermon: title, summary, keyPoints, highlights · BlogPost: title, excerpt, body · Bulletin: title · Announcement: title, body · Resource: title, description · PageBlock: data · TimelineEntry: title, body · Event: title, description, feeInfo · EventSession: title · OutreachActivity: reportSummary · Programme: name, description, scheduleSummary, durationLabel, feeInfo · Project: title, summary, body, impact · ProjectMilestone: title, description · ProjectUpdate: title, body · GivingChannel: label, referenceInstructions · FormDefinition: title, intro, consentText · NewsletterCampaign: subject, body

HOW TO READ IT

You said: *"I do not know how to do the Kiswahili change for all data, including blogs that are written in only one language, and I need help."* This page is the answer in one picture.

Rule 1: never store a sentence twice in the same column. A sermon's title is not "Hope / Tumaini" in one cell. The sermon keeps what is the same in every language (date, minister, YouTube id, counts). Each language's **words** live in their own row of a small side table (SermonTranslation): one row for English, one for Kiswahili.

Rule 2: a blog written only in English is normal. The English row exists and is approved. The Kiswahili row does not exist yet (status "missing"). The page still works: the visitor sees the English text and a notice. When an editor presses "Draft in Kiswahili", an AI engine writes a first draft (status machine_draft). A bilingual reviewer checks it (needs_review) and approves it. Only then does Kiswahili show as the real thing.

Rule 3: when the English changes, the Kiswahili must notice. The row keeps a hash of the source text it was made from (sourceHash). If the English is edited, the hash no longer matches and the Kiswahili row goes back to needs_review. This is how the two languages never silently drift apart.

What is not translated: names of people and places, comments written by visitors, bible verses (they are looked up in the chosen Bible version), and the buttons and labels of the site. Those buttons live in two message files (en.json and sw.json) managed by next-intl, not in the database.

MISTAKES TO AVOID

- Publishing machine translation as final. Church language carries meaning and tone; a mistranslated prayer or scripture is worse than an English page.
- Translating the database by hand into a "title_en / title_sw" pair of columns. It works for two languages and fails the day you add a third. One row per language scales to any number.
- Forgetting search engines. Each language needs its own address (/en/... and /sw/...) with hreflang links, or Google will treat the two as duplicates.

WHAT YOU ALREADY DID RIGHT

- You identified early that blogs are written in one language only. Many projects discover this after launch and have to rebuild the data. Designing the pattern first costs one extra table per class and saves a rewrite.

B09

CLASS ARCHITECTURE (UML) · PHASE 1

Services: what they wrap, who calls them, what happens when they fail

Each outside tool is reached through one service so it can be replaced or can fail safely

Service	Main methods	Wraps (the outside tool)	Classes that call it	When it fails
AuthzService	can, scopesFor, assertCan	Better Auth session + RoleAssignment rows	User, RoleAssignment, LedgerEntry	Deny. Fail closed: if the check cannot run, nobody gets in.
TranslationService	localized, requestDraft, approve	Translation rows + a translation engine (AI draft)	Department, Sermon, Event, Project	Show the other language with a notice. Drafts wait.
StorageService	presignUpload, signedUrl, remove	Cloudflare R2 (presigned uploads, signed downloads)	Sermon, MediaAsset, Certificate	Uploads fail with a clear message. Public pages keep working.
RateLimiter	limit	Upstash Ratelimit (Redis)	Sermon, Comment, EventRegistration, Convert, NewsletterSubscriber	Money and sign-in: fail closed. Read-only counters: fail open.
JobQueue	enqueue, schedule, verify	QStash (delayed and scheduled jobs)	Event, Convert, FollowUpCase, JoinRequest	Retries with back-off; the job log (JobRun) shows what failed.
Mailer	send, syncContact	Resend (e-mail)	EventRegistration, Convert, FollowUpCase, JoinRequest, NewsletterSubscriber	Row stays queued in NotificationLog; retried by a job.
CacheService	tag, revalidate, counter	Next.js cache tags + Upstash Redis counters	Sermon, Project, LedgerEntry	Fall back to the database; pages are slower, not broken.
Analytics	capture, flag	PostHog, only after cookie consent	called by feature code in several modules	Silently skipped. The site never waits for analytics.
ModerationService	score, isHoneypotTripped	Honeypot field, Turnstile, simple word checks	Comment	Comments stay "pending" for a human.
CertificateService	render, nextSerial	PDF renderer + StorageService + running serial number	NewMemberJourney, Certificate	The journey is saved; the certificate is retried by a job.
FellowshipLocator	nearest, search	Distance maths in the browser (device location never leaves the phone)	HomeFellowship	Typed estate search instead of "use my location".
OverlayOrchestrator	next, canShow, dismiss	localStorage + cookies on the visitor's device	NewMemberJourney	Show nothing rather than two pop-ups at once.

HOW TO READ IT

A **service** is a piece of code with no table of its own. Its job is to do one thing for everybody else, like a church caretaker who unlocks doors for every ministry. These twelve services are the only places where the outside world is touched.

Why not call Resend, QStash or R2 straight from the feature code? Three reasons: (1) **one place for the secret**, (2) **one place to log and rate-limit**, (3) **one place to replace** the tool. If you move from Resend to another e-mail service in 2028, only Mailer changes.

The last column is the part many designs forget: **what happens when the tool is down?** A church website must stay readable on a Sunday morning even if the translation engine or the analytics tool is having a bad day.

MISTAKES TO AVOID

- "We will add the fallback later." Later never comes. Decide the failure behaviour now: closed for money and sign-in, open for counters, graceful for translation.
- Letting a service import a feature class. Services sit below features. If TranslationService imports Sermon, you have built a loop.

WHAT YOU ALREADY DID RIGHT

- You chose tools that each do one job (Resend for mail, QStash for scheduled work, Upstash Redis for counters, R2 for files). That makes each service small.

Role slice: Visitor (public)

Visitor (public): the classes it owns, changes, submits and reads

Visitor (public)

no login (public) | scope: none | screens: Public site (EN/SW)

Who: Anyone on the website who is not logged in. Does: Reads content, listens/watches sermons, subscribes, registers for events, sends contact messages, comments, likes.

Owns (3): the classes this role answers for

Like
+ id: uuid {PK}
+ sermonId: uuid {FK→Sermon}
+ blogPostId: uuid {FK→BlogPost}
… +2 more

ConsentRecord
+ id: uuid {PK}
+ visitorId: text {PII}
+ categories: jsonb
… +2 more

«service»
OverlayOrchestrator

Submits through public forms (S): write-only, cannot read back

Comment · EventRegistration · ProgrammeEnrollment · Convert · JoinRequest · NewsletterSubscriber · ContactMessage

Reads only (R)

Sermon · Series · BlogPost · Bulletin · Announcement · Resource · Topic · Minister · PageBlock · TimelineEntry · Branch · ServiceTime · Department · Leader · LeaderPosition · HomeFellowship · Event · EventGuest · EventSession · OutreachActivity · Programme · ProgrammeCohort · Project · ProjectMilestone · ProjectUpdate · GivingChannel · SermonTopic · ProjectMedia

Pages in this set that tell this role's story (click to open)

U01 Visitor: read, listen and explore · U02 Visitor: take part (forms and actions) · S01 Sermon page: view, play, like, count · S09 Comment: submit, filter, moderate · S11 Nearest home fellowship finder · S13 Contact message and prayer request · F02 First visit: cookie bar and the order… · F04 One page request: language, cache and… · F05 Translating dynamic content (blogs, s… · F08 Department page and join request

HOW TO READ IT

Visitor (public). Visitors have no login, so everything they can do is **read** or **submit**. The classes they own are tiny (Like, ConsentRecord and the pop-up traffic light), because a visitor owns nothing; they only leave small marks.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 3, also changes 0, submits to 7, reads 28 classes.

MISTAKES TO AVOID

- Giving visitors an account "to make comments safer". Accounts for visitors mean passwords to protect and data to look after. Honeypot, Turnstile and rate limits do the job without them.
- Letting a visitor read what other visitors submitted (registration lists). Submit-only means write-only.

WHAT YOU ALREADY DID RIGHT

- You kept visitor accounts out of phase 1. Comments, likes and forms work without logins.

B11

CLASS ARCHITECTURE (UML) · PHASE 1

Role slice: New Member (public)

New Member (public): the classes it owns, changes, submits and reads

New Member (public)

no login (public) | scope: none | screens: New-member tour

Who: A visitor who answered "Yes, give me a tour" in the first modal. Does: Walks the 9-step tour, joins fellowships, leaves feedback, receives a certificate.

Owns (5): the classes this role answers for

NewMemberJourney
- + id: uuid {PK}
- + branchId: uuid {FK→Branch}
- + deviceId: text {UQ, PII}
- … +6 more

NewMemberProfile
- + id: uuid {PK}
- + journeyId: uuid {FK→NewMemberJourney, UQ}
- + branchId: uuid {FK→Branch}
- … +14 more

FeedbackResponse
- + id: uuid {PK}
- + journeyId: uuid {FK→NewMemberJourney}
- + branchId: uuid {FK→Branch}
- … +5 more

Certificate
- + id: uuid {PK}
- + journeyId: uuid {FK→NewMemberJourney}
- + enrollmentId: uuid {FK→ProgrammeEnrollment}
- … +8 more

«service»
CertificateService

Submits through public forms (S): write-only, cannot read back

Comment · Like · EventRegistration · ProgrammeEnrollment · Convert · JoinRequest · NewsletterSubscriber · ConsentRecord

Reads only (R)

Sermon · Series · BlogPost · Bulletin · Announcement · Resource · Topic · Minister · PageBlock · TimelineEntry · Branch · ServiceTime · Department · Leader · LeaderPosition · HomeFellowship · Event · Programme · ProgrammeCohort · Project · GivingChannel

Pages in this set that tell this role's story (click to open)

U03 New member: the tour · S04 New-member tour: ask, consent, save, … · S05 Certificate: build, store, auto-downl… · T04 New-member journey states · F06 New-member tour: from "Yes" to the ce… · D07 Level 2: the new-member tour and cert… · O01 Object diagram: one new member's jour…

HOW TO READ IT

New Member (public). A New Member is a visitor who said "Yes, give me a tour". Still no login: the tour is saved against a random id in the browser. They own the journey, their profile, the feedback and the certificate.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 5, also changes 0, submits to 8, reads 21 classes.

MISTAKES TO AVOID

- Collecting everything at once. Only name, age group and branch are required; belief and previous service are optional and sensitive.
- Storing the "No, thanks" on the server. It stays on the visitor's own device (localStorage), so a person who declines leaves no record.

WHAT YOU ALREADY DID RIGHT

- You asked that the answer "No" be remembered on the device and the question never asked again. That is both friendly and privacy-correct.

Role slice: Super Admin

Super Admin: the classes it owns, changes, submits and reads

Super Admin

has a login | scope: global | screens: /admin (all)

Who: Head of church IT / Senior Pastor delegate (1-2 people). Does: Everything, in every branch: roles, settings, forms, audit log, system health.

Owns (8): the classes this role answers for

Session
+ id: text {PK}
+ userId: text {FK→User}
+ impersonatedBy: text {FK→User}
… +6 more

Account
+ id: text {PK}
+ userId: text {FK→User}
+ providerId: text
… +6 more

Verification
+ id: text {PK}
+ identifier: text
+ value: text {PII}
… +3 more

RoleAssignment
+ id: uuid {PK}
+ userId: text {FK→User}
+ branchId: uuid {FK→Branch}
… +6 more

AuditLog
+ id: uuid {PK}
+ actorId: text {FK→User}
+ branchId: uuid {FK→Branch}
… +7 more

«phase 2»
MemberAccount
+ id: uuid {PK}
+ userId: text {FK→User}
+ branchId: uuid {FK→Branch}
… +3 more

SiteSetting
+ key: text {PK}
+ updatedBy: text {FK→User}
+ value: jsonb
… +1 more

NotificationLog
+ id: uuid {PK}
+ channel: NotifyChannel
+ template: text
… +7 more

Super Admin may read every class and change most of them (see B04a and B04b). The list below shows only the classes where the role differs from plain read access.

Also changes (letters = what the role may do on classes owned by someone else)

Sermon CRUDPA | Series CRUDPA | BlogPost CRUDPA | Bulletin CRUDPA | Announcement CRUDPA | Resource CRUDPA | Topic CRUDPA | Minister CRUDPA | PageBlock CRUDPA | TimelineEntry CRUDPA | Comment RUDA | MediaAsset CRUD | Branch CRUD | ServiceTime CRUD | Department CRUDP | Leader CRUD | LeaderPosition CRUD | HomeFellowship CRUD | Event CRUDP | EventGuest CRUD | EventSession CRUD | OutreachActivity CRUDP | EventRegistration RUX | Programme CRUDP | ProgrammeCohort CRUD | ProgrammeEnrollment RUX | Convert RX | FollowUpCase RU | Project CRUDP | ProjectMilestone CRUD | ProjectUpdate CRUD | LedgerEntry RX | GivingChannel CRUD | FormDefinition CRUDP | FormField CRUD | NewMemberProfile RX | FeedbackResponse RX | Certificate RUD | JoinRequest RUX | NewsletterSubscriber RUDX | NewsletterCampaign CRUDP | ContactMessage RUD | User CRUD | StaffProfile CRUD | SermonTopic CRUD | ProjectMedia CRUD | OutreachDepartment CRUD

Reads only (R)

Like | ContactAttempt | NewMemberJourney | ConsentRecord

Pages in this set that tell this role's story (click to open)

U06 Administration: Super Admin, Branch A… | S10 Staff sign-in and the three security … | F03 Staff sign-in and the three gates | F15 Privacy request and security incident | E02 ER: Identity and access | D10 Level 2: identity, access and audit (…

HOW TO READ IT

Super Admin. The Super Admin is the only global role. It owns the identity tables (sessions, accounts, role assignments), the audit log and site settings. One or two people only.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 8, also changes 47, submits to 0, reads 4 classes.

MISTAKES TO AVOID

- Using the Super Admin account for daily editing. Make a normal account for daily work and keep the Super Admin for roles, settings and emergencies.
- Skipping two-factor sign-in. A stolen Super Admin password is a stolen church.

WHAT YOU ALREADY DID RIGHT

- You asked for Better Auth on the admin side. Staff-only accounts keep the attack surface small.

Role slice: Branch Admin

Branch Admin: the classes it owns, changes, submits and reads

Branch Admin

has a login | scope: branch | screens: /admin (branch scope)

Who: Branch pastor's office / church administrator of ONE branch. Does: Runs one branch: branch details, service times, leaders, projects, staff roles inside that branch, contact inbox.

Owns (8): the classes this role answers for

StaffProfile
+ userId: text {PK, FK→User}
+ homeBranchId: uuid {FK→Branch}
+ leaderId: uuid {FK→Leader}
… +6 more

ServiceTime
+ id: uuid {PK}
+ branchId: uuid {FK→Branch}
+ name: text
… +5 more

LeaderPosition
+ id: uuid {PK}
+ leaderId: uuid {FK→Leader}
+ branchId: uuid {FK→Branch}
… +6 more

Project
+ id: uuid {PK}
+ coverMediaId: uuid {FK→MediaAsset}
+ goalAmountMinor: bigint
… +11 more

ProjectMilestone
+ id: uuid {PK}
+ projectId: uuid {FK→Project}
+ mediaId: uuid {FK→MediaAsset}
… +5 more

ProjectUpdate
+ id: uuid {PK}
+ projectId: uuid {FK→Project}
+ postedBy: text {FK→User}
… +4 more

«link table»
ProjectMedia
+ projectId: uuid {PK, FK→Project}
+ sortOrder: int
… +1 more

ContactMessage
+ id: uuid {PK}
+ branchId: uuid {FK→Branch}
+ assignedTo: text {FK→User}
… +8 more

Also changes (letters = what the role may do on classes owned by someone else)

Sermon CRUDPA · Series CRUDPA · BlogPost CRUDPA · Bulletin CRUDPA · Announcement CRUDPA · Resource CRUDPA · Topic CRUDPA · Minister CRUDPA · PageBlock CRUDPA · TimelineEntry CRUDPA · Comment RUDA · MediaAsset CRUD · Branch RU · Department CRUDP · Leader CRUD · HomeFellowship CRUD · Event CRUDP · EventGuest CRUD · EventSession CRUD · OutreachActivity CRUDP · EventRegistration RUX · Programme CRUDP · ProgrammeCohort CRUD · ProgrammeEnrollment RUX · Convert RX · FollowUpCase RUD · GivingChannel RA · JoinRequest RUX · RoleAssignment CRUD · SermonTopic CRUD · OutreachDepartment CRUD

Reads only (R)

ContactAttempt · LedgerEntry · FormDefinition · FormField · NewMemberProfile · FeedbackResponse · Certificate · NewsletterSubscriber · NotificationLog · User · AuditLog · SiteSetting

Pages in this set that tell this role's story (click to open)

U06 Administration: Super Admin, Branch A… · F03 Staff sign-in and the three gates · F12 Giving and the project progress bar · E03 ER: Church structure · E07 ER: Giving and projects · S12 Giving: record a gift, void a mistake…

HOW TO READ IT

Branch Admin. The Branch Admin runs one branch: service times, leaders' positions, projects and milestones, the contact inbox and staff roles inside the branch.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 8, also changes 31, submits to 0, reads 12 classes.

MISTAKES TO AVOID

- Letting a Branch Admin grant roles outside their branch. RoleAssignment checks scope on every grant.
- Editing the project goal quietly. Changes to goalAmount are audited and shown in the project log.

WHAT YOU ALREADY DID RIGHT

- You described branches as locations of the same church; "Branch" as the root of the tree is the right model for that.

Role slice: Content Editor

Content Editor: the classes it owns, changes, submits and reads

Content Editor

has a login | scope: branch | screens: /admin/content

Who: Media / communications team. Does: Sermons, blogs, bulletins, announcements, resources, media library, newsletter, comment moderation.

Owns (14): the classes this role answers for

Minister
+ id: uuid {PK}
+ leaderId: uuid {FK→Leader}
+ photoMediaId: uuid {FK→MediaAsset}
… +6 more

Series
+ id: uuid {PK}
+ coverMediaId: uuid {FK→MediaAsset}
+ startsOn: date
… +2 more

Sermon
+ id: uuid {PK}
+ ministerId: uuid {FK→Minister}
+ seriesId: uuid {FK→Series}
… +15 more

«link table»
SermonTopic
+ sermonId: uuid {PK, FK→Sermon}
… +1 more

BlogPost
+ id: uuid {PK}
+ coverMediaId: uuid {FK→MediaAsset}
+ authorMinisterId: uuid {FK→Minister}
… +9 more

Comment
+ id: uuid {PK}
+ sermonId: uuid {FK→Sermon}
+ blogPostId: uuid {FK→BlogPost}
… +8 more

Bulletin
+ id: uuid {PK}
+ pdfMediaId: uuid {FK→MediaAsset}
+ coverMediaId: uuid {FK→MediaAsset}
… +7 more

Announcement
+ id: uuid {PK}
+ audioMediaId: uuid {FK→MediaAsset}
+ pdfMediaId: uuid {FK→MediaAsset}
… +8 more

Resource
+ id: uuid {PK}
+ mediaId: uuid {FK→MediaAsset}
+ formId: uuid {FK→FormDefinition}
… +10 more

PageBlock
+ id: uuid {PK}
+ mediaId: uuid {FK→MediaAsset}
+ page: text
… +4 more

TimelineEntry
+ id: uuid {PK}
+ mediaId: uuid {FK→MediaAsset}
+ year: smallint
… +2 more

NewsletterSubscriber
+ id: uuid {PK}
+ branchId: uuid {FK→Branch}
+ email: text {UQ, PII}
… +10 more

NewsletterCampaign
+ id: uuid {PK}
+ createdBy: text {FK→User}
+ kind: CampaignKind
… +8 more

«service»
ModerationService

Also changes (letters = what the role may do on classes owned by someone else)

Topic CRUDP | MediaAsset CRUD | Department RU | Leader RU | Event RU | Project RU | ProjectMilestone RU | ProjectUpdate CRU | FormDefinition RU | ProjectMedia CRUD

Reads only (R)

Branch | ServiceTime | FeedbackResponse

Pages in this set that tell this role's story (click to open)

U04 Content team: editor and translator | S02 Staff publishes a sermon (upload, sch… | S03 Translation: AI draft, human review, … | S09 Comment: submit, filter, moderate | T01 Content life-cycle (Publishable) | T02 Translation status | T03 Comment status | F09 Publishing: draft, review, schedule, … | F14 Media upload: from a file to a safe, … | D03 Level 2: manage content and church in…

HOW TO READ IT

Content Editor. The Content Editor is the media and communications team. Sermons, series, ministers, topics, blogs, bulletins, announcements, resources, page blocks, timeline, newsletter and comment moderation. This is the biggest territory.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 14, also changes 10, submits to 0, reads 3 classes.

MISTAKES TO AVOID

- Giving every editor "publish". Use the review step (T01): drafts go to review and a lead editor publishes.
- Uploading files with different naming styles. Name = date, and let the system derive month and year.

WHAT YOU ALREADY DID RIGHT

- You listed bulletins by year and month, announcements by Sunday and midweek, resources by topic and year: all filters that this territory implements.

Role slice: Translator / Reviewer

Translator / Reviewer: the classes it owns, changes, submits and reads

Translator / Reviewer

has a login | scope: global | screens: /admin/translations

Who: Bilingual member who checks Kiswahili (or English) text. Does: Reviews machine drafts, edits translation fields only, approves or rejects.

Owns no class of its own

The Translator works on the *_translation rows of the 25 classes that have translated fields (B08): read the English source, check or edit the Kiswahili, approve or send back.

- Branch: description
- Department: name, summary, whatWeDo, meetingTimes
- Leader: bio
- LeaderPosition: title, responsibilities
- HomeFellowship: notes
- Minister: bio
- Series: title, description
- Topic: name, description
- Sermon: title, summary, keyPoints, highlights
- BlogPost: title, excerpt, body
- Bulletin: title
- Announcement: title, body
- Resource: title, description
- PageBlock: data
- TimelineEntry: title, body
- Event: title, description, feeInfo
- EventSession: title
- OutreachActivity: reportSummary
- Programme: name, description, scheduleSummary, durationLabel, feeInfo
- Project: title, summary, body, impact
- ProjectMilestone: title, description
- ProjectUpdate: title, body
- GivingChannel: label, referenceInstructions
- FormDefinition: title, intro, consentText
- NewsletterCampaign: subject, body

Also changes (letters = what the role may do on classes owned by someone else)

- Sermon RU
- Series RU
- BlogPost RU
- Bulletin RU
- Announcement RU
- Resource RU
- Topic RU
- Minister RU
- PageBlock RU
- TimelineEntry RU
- Department RU
- Leader RU
- Event RU
- Project RU

Reads only (R)

- MediaAsset
- Branch

Pages in this set that tell this role's story (click to open)

- U04 Content team: editor and translator
- S03 Translation: AI draft, human review, ...
- T02 Translation status
- F05 Translating dynamic content (blogs, s...
- B08 The translation pattern: one entity, ...

HOW TO READ IT

Translator / Reviewer. The Translator owns no table. Their whole job is the translation rows: read the English, check the AI draft, approve or send back. Global scope, because language skill is not tied to one branch.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 0, also changes 14, submits to 0, reads 2 classes.

MISTAKES TO AVOID

- Letting the translator also edit the English. They may change translation fields only.
- Treating "approved" as decoration. Public pages show approved text only.

WHAT YOU ALREADY DID RIGHT

- You asked for help with the dual-language data. A role whose only task is that makes it sustainable.

B16

CLASS ARCHITECTURE (UML) · PHASE 1

Role slice: Department Leader

Department Leader: the classes it owns, changes, submits and reads

Department Leader

has a login | scope: department | screens: /admin/department

Who: Head of a department or sub-department (e.g. Youth, Home Fellowship, Missions). Does: Edits ONLY their department page, handles join requests for it, proposes midweek announcements.

Owns (3): the classes this role answers for

HomeFellowship

+ id: uuid {PK}
+ branchId: uuid {FK→Branch}
+ departmentId: uuid {FK→Department}
… +15 more

JoinRequest

+ id: uuid {PK}
+ departmentId: uuid {FK→Department}
+ homeFellowshipId: uuid {FK→HomeFellowship}
… +4 more

«service»
FellowshipLocator

Also changes (letters = what the role may do on classes owned by someone else)

Announcement CU · MediaAsset CR · Department RU · LeaderPosition CRU · Event CU · OutreachActivity CRU · Programme RU · FormDefinition RU · FormField RU

Reads only (R)

Sermon · Series · BlogPost · Bulletin · Resource · Topic · Minister · PageBlock · TimelineEntry · Branch · ServiceTime · Leader · EventRegistration

Pages in this set that tell this role's story (click to open)

U05 Ministry team: department, events and… · S11 Nearest home fellowship finder · T06 Request states (registration, enrolme… · F08 Department page and join request · E03 ER: Church structure

HOW TO READ IT

Department Leader. A Department Leader edits their own department page (and its sub-departments), handles join requests for it, and for the Home Fellowship department maintains fellowship locations. Scope: one department.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 3, also changes 9, submits to 0, reads 13 classes.

MISTAKES TO AVOID

- Letting a leader edit sibling departments. RoleAssignment.departmentId limits the scope; the leader of Youth cannot edit Men.
- Exact home addresses on a public page. Show the estate and a contact person, never the host's door.

WHAT YOU ALREADY DID RIGHT

- You described departments with sub-departments, leaders and what they do. One self-referencing Department table with leader positions covers it.

Role slice: Discipler (Follow-up)

Discipler (Follow-up): the classes it owns, changes, submits and reads

Discipler (Follow-up)

has a login | scope: branch | screens: /admin/followup

Who: Discipleship / follow-up officer. Does: Contacts new converts, logs attempts, enrols them in the New Born Again programme.

Owns (3): the classes this role answers for

Convert
+ id: uuid {PK}
+ outreachId: uuid {FK→OutreachActivity}
+ capturedBy: text {FK→User}
… +8 more

FollowUpCase
+ id: uuid {PK}
+ convertId: uuid {FK→Convert, UQ}
+ branchId: uuid {FK→Branch}
… +8 more

ContactAttempt
+ id: uuid {PK}
+ caseId: uuid {FK→FollowUpCase}
+ byUserId: text {FK→User}
… +4 more

Also changes (letters = what the role may do on classes owned by someone else)

ProgrammeCohort RU | ProgrammeEnrollment CRU

Reads only (R)

Sermon | Series | BlogPost | Bulletin | Announcement | Resource | Topic | Minister | PageBlock | TimelineEntry | MediaAsset | Branch | Department | Programme | NewMemberJourney | NewMemberProfile | JoinRequest | ContactMessage

Pages in this set that tell this role's story (click to open)

U05 Ministry team: department, events and… | S06 Go Ye: record a new believer and star… | T05 Follow-up case states | F07 Go Ye: from a decision to a discipler… | D06 Level 2: Go Ye and follow-up (process… | O02 Object diagram: a Go Ye decision, fol…

HOW TO READ IT

Discipler (Follow-up). The Discipler owns the care of new believers: Convert, FollowUpCase and ContactAttempt. They see names and phone numbers, so the role is narrow and logged.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 3, also changes 2, submits to 0, reads 18 classes.

MISTAKES TO AVOID

- Exporting the converts list to a personal spreadsheet. Exports are logged; the default screens allow none.
- Reaching out by a method the person did not choose. wantsCall and preferredLanguage are respected.

WHAT YOU ALREADY DID RIGHT

- You gave discipleship its own department and its own programme (New Born Again). The model gives it its own role and its own screens.

CLASS ARCHITECTURE (UML) · PHASE 1

Role slice: Treasurer

Treasurer: the classes it owns, changes, submits and reads

Treasurer

has a login | scope: branch | screens: /admin/giving

Who: Church finance officer. Does: Records gifts to the project ledger, voids mistakes, maintains giving details shown on the site.

Owns (2): the classes this role answers for

LedgerEntry

+ id: uuid {PK}
+ projectId: uuid {FK→Project}
+ branchId: uuid {FK→Branch}
… +13 more

GivingChannel

+ id: uuid {PK}
+ branchId: uuid {FK→Branch}
+ projectId: uuid {FK→Project}
… +10 more

Also changes (letters = what the role may do on classes owned by someone else)

ProjectUpdate CR

Reads only (R)

Sermon | Series | BlogPost | Bulletin | Announcement | Resource | Topic | Minister | PageBlock | TimelineEntry | MediaAsset | Branch | Project | ProjectMilestone | ContactMessage

Pages in this set that tell this role's story (click to open)

U06 Administration: Super Admin, Branch A… | S12 Giving: record a gift, void a mistake… | T08 Project and ledger entry states | F12 Giving and the project progress bar | D08 Level 2: giving and projects (process…

HOW TO READ IT

Treasurer. The Treasurer owns the giving details and the ledger. At launch the website does not take money; the Treasurer types in what was received and the progress bars compute from it.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 2, also changes 1, submits to 0, reads 15 classes.

MISTAKES TO AVOID

- Editing a wrong amount. The ledger is append-only: void with a reason and add a new row.
- One person changing a paybill number alone. Changes to GivingChannel need a second person.

WHAT YOU ALREADY DID RIGHT

- You asked for a bar of how much has been given and how much remains. A ledger-based total is the safest way to produce it.

B19

CLASS ARCHITECTURE (UML) · PHASE 1

Role slice: Events Coordinator

Events Coordinator: the classes it owns, changes, submits and reads

Events Coordinator

has a login | scope: branch | screens: /admin/events

Who: Events & programmes coordinator. Does: Creates events, programmes and cohorts, manages registrations and check-in.

Owns (9): the classes this role answers for

Event
+ id: uuid {PK}
+ departmentId: uuid {FK→Department}
+ formId: uuid {FK→FormDefinition}
… +19 more

EventGuest
+ id: uuid {PK}
+ eventId: uuid {FK→Event}
+ ministerId: uuid {FK→Minister}
… +1 more

EventSession
+ id: uuid {PK}
+ eventId: uuid {FK→Event}
+ speakerMinisterId: uuid {FK→Minister}
… +4 more

OutreachActivity
+ id: uuid {PK}
+ outreachCategory: OutreachCategory
+ country: text
… +6 more

«link table»
OutreachDepartment
+ outreachId: uuid {PK, FK→OutreachActivity}
+ role: text
… +1 more

EventRegistration
+ id: uuid {PK}
+ eventId: uuid {FK→Event}
+ ticketCode: text {UQ}
… +3 more

Programme
+ id: uuid {PK}
+ departmentId: uuid {FK→Department}
+ formId: uuid {FK→FormDefinition}
… +12 more

ProgrammeCohort
+ id: uuid {PK}
+ programmeId: uuid {FK→Programme}
+ branchId: uuid {FK→Branch}
… +11 more

ProgrammeEnrollment
+ id: uuid {PK}
+ cohortId: uuid {FK→ProgrammeCohort}
+ convertId: uuid {FK→Convert}
… +4 more

Also changes (letters = what the role may do on classes owned by someone else)

MediaAsset CR | FormDefinition RU | FormField RU

Reads only (R)

Sermon | Series | BlogPost | Bulletin | Announcement | Resource | Topic | Minister | PageBlock | TimelineEntry | Branch | ServiceTime | Department

Pages in this set that tell this role's story (click to open)

U05 Ministry team: department, events and… | S07 Event registration (youth trip, confe… | S14 Programme enrolment (Kairos, CFC, Tee… | T06 Request states (registration, enrolme… | F10 Event: create, register, run the day | F11 Programme and cohort: from a course t… | D05 Level 2: events and programmes (proce…

HOW TO READ IT

Events Coordinator. The Events Coordinator owns events, guests, sessions, programmes, cohorts, registrations and the outreach activity records.

Read the page top to bottom: **who they are**; **what they own** (full class cards, each clickable); **what else they change** with the letters from the permission matrix; **what they may only submit or read**.

Counts for this role: owns 9, also changes 3, submits to 0, reads 13 classes.

MISTAKES TO AVOID

- Opening registration with no capacity or closing date. Every event carries both (or says "unlimited" on purpose).
- Letting minors register without a guardian contact.

WHAT YOU ALREADY DID RIGHT

- You listed events (youth trip, conferences) and programmes (CFC, Kairos, Children 5 Days, Teen 5 Days, New Born Again) separately. Event is the date; Programme is the course; Cohort is one run of it.

ATLAS · FAMILY 2

Class tabs (UML detail)

One detailed drawing per main class. Each tab covers one class or a small family: fields with flags, parents and children, who may do what, the rules to keep and the mistakes to avoid. Use them while building a module.

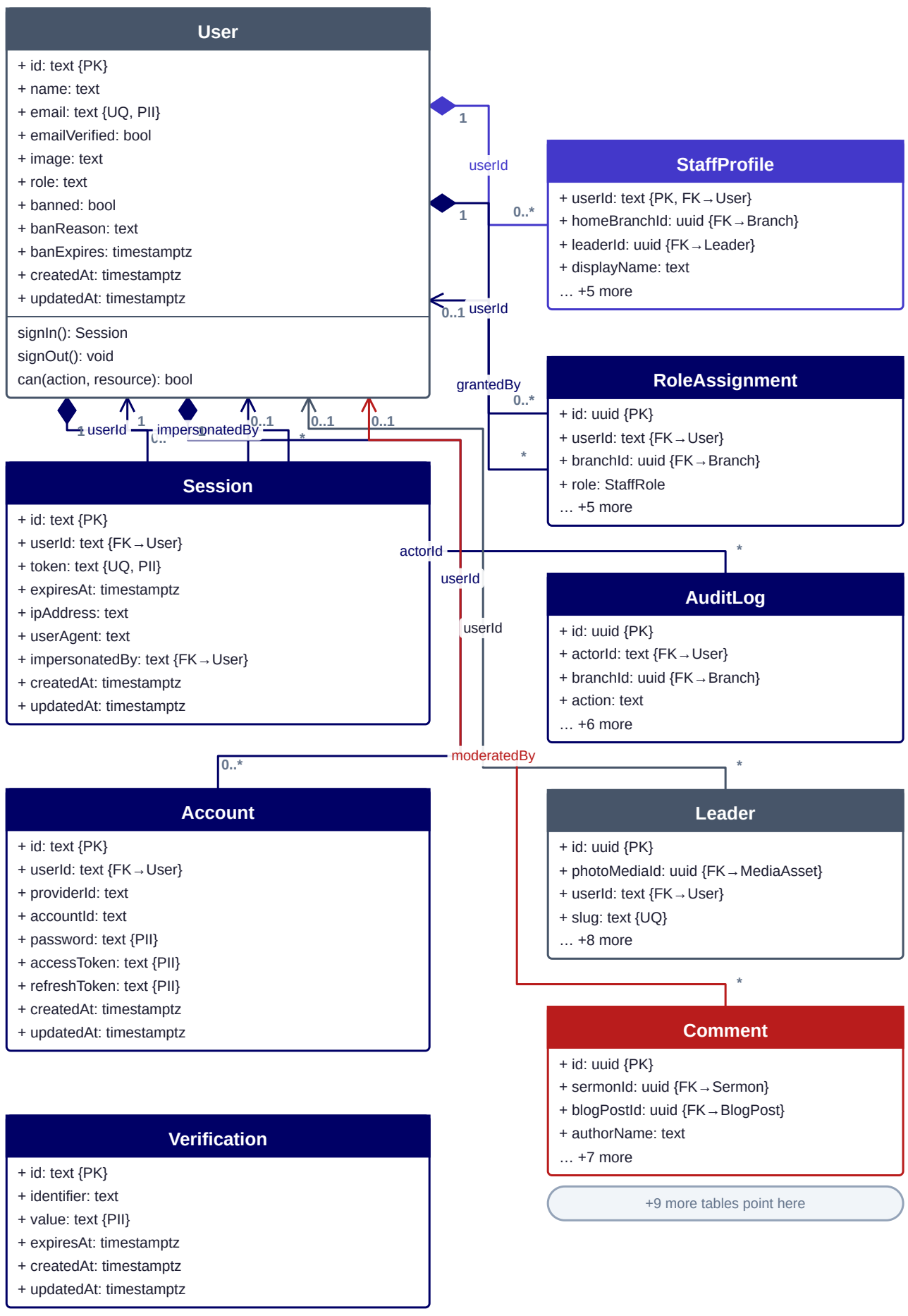
> **EXPLAINED LIKE YOU ARE FIVE**
> Like the detailed drawing of one room: where the sockets are, what the wall is made of, who holds the key.

Code	Title	What it shows
B20	Tab: Better Auth core (User, Session, Account, Verification)	Attributes, parents, children, rules and mistakes
B21	Tab: Staff access (StaffProfile, RoleAssignment, AuditLog)	Attributes, parents, children, rules and mistakes
B22	Tab: Branch and service times	Attributes, parents, children, rules and mistakes
B23	Tab: Department	Attributes, parents, children, rules and mistakes
B24	Tab: Leader and LeaderPosition	Attributes, parents, children, rules and mistakes
B25	Tab: Home fellowship	Attributes, parents, children, rules and mistakes
B26	Tab: Sermon and SermonTopic	Attributes, parents, children, rules and mistakes
B27	Tab: Series, Minister, Topic	Attributes, parents, children, rules and mistakes
B28	Tab: Blog and comments	Attributes, parents, children, rules and mistakes
B29	Tab: Like	Attributes, parents, children, rules and mistakes
B30	Tab: Bulletin and Announcement	Attributes, parents, children, rules and mistakes
B31	Tab: Resource (library)	Attributes, parents, children, rules and mistakes
B32	Tab: MediaAsset	Attributes, parents, children, rules and mistakes
B33	Tab: Page blocks, timeline, site settings	Attributes, parents, children, rules and mistakes
B34	Tab: Event, guests, sessions	Attributes, parents, children, rules and mistakes
B35	Tab: Event registration	Attributes, parents, children, rules and mistakes
B36	Tab: OutreachActivity (Go Ye)	Attributes, parents, children, rules and mistakes
B37	Tab: Programme and cohort	Attributes, parents, children, rules and mistakes
B38	Tab: Programme enrolment	Attributes, parents, children, rules and mistakes
B39	Tab: Go Ye follow-up (Convert, FollowUpCase, ContactAttempt)	Attributes, parents, children, rules and mistakes
B40	Tab: Project and milestones	Attributes, parents, children, rules and mistakes
B41	Tab: Project updates and media	Attributes, parents, children, rules and mistakes
B42	Tab: Ledger and giving channels	Attributes, parents, children, rules and mistakes
B43	Tab: Form builder (FormDefinition, FormField)	Attributes, parents, children, rules and mistakes
B44	Tab: New-member journey and profile	Attributes, parents, children, rules and mistakes
B45	Tab: Feedback and certificate	Attributes, parents, children, rules and mistakes
B46	Tab: JoinRequest	Attributes, parents, children, rules and mistakes
B47	Tab: Newsletter	Attributes, parents, children, rules and mistakes

Code	Title	What it shows
B48	Tab: Contact, consent, notification log	Attributes, parents, children, rules and mistakes
B49	Tab: Platform services (security, language, files)	Attributes, parents, children, rules and mistakes
B50	Tab: Platform services (jobs, mail, cache, analytics)	Attributes, parents, children, rules and mistakes
B51	Tab: Feature services (moderation, overlays)	Attributes, parents, children, rules and mistakes

Tab: Better Auth core (User, Session, Account, Verification)

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

- AuthzService

Who may do what on User (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: CRUD
- Branch: R

Who may do what on Session (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: RD

Who may do what on Account (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: RD

Who may do what on Verification (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: R

User – in plain words

The login account of a staff member.
A staff login. Managed by Better Auth (its "user" table). Only staff have accounts at launch.

Rules the code must keep

- Better Auth owns this table and its id type is text, so every "createdBy / handledBy" column in our own tables is also text.
- Email must be verified before a user can receive any role.
- Banning a user (admin plugin) kills all their sessions immediately.
- Public visitors never get a row here at launch.

Mistakes to avoid

- Adding church columns (phone, branch) straight onto the Better Auth table. Put them in StaffProfile so auth upgrades never fight your schema.
- Using the plugin "role" field as the whole permission system. It only says user/admin; real scoped roles live in RoleAssignment.

Session – in plain words

A "you are logged in" wristband.
One row per signed-in browser. Better Auth creates and expires these.

Rules the code must keep

- Short expiry for admin (e.g. 7 days sliding) and revoke-all on password change.
- Cookie is httpOnly, secure, sameSite=lax.
- Created and destroyed by Better Auth only. Application code never inserts here.
- Cookie is httpOnly + secure + sameSite=lax; a session lasts 7 days and refreshes on use.

Mistakes to avoid

- Writing your own session code next to Better Auth. Two systems = two bugs.

Account – in plain words

The key that opens the login.
Stores HOW a user proves who they are (password hash, or Google later).

Rules the code must keep

- One user may have several accounts (email+password now, Google later). Better Auth owns this table.
- Password hash never leaves the server and is never logged.

Mistakes to avoid

- Storing plain passwords or logging login requests with bodies.

Verification – in plain words

One-time codes that expire.
Short-lived codes for email verification and password reset.

Rules the code must keep

- Short-lived tokens for e-mail verification and password reset (minutes, single use).
- A cron/QStash job deletes expired rows daily.

Mistakes to avoid

- Letting tokens live for days. Short life = small damage if an email is forwarded.

How this group fits together

Better Auth owns these four tables. DCEB adds nothing to them except the "role" link in the next tab, so upgrades stay painless.

HOW TO READ IT

User (Identity & Access, owned by **Shared Kernel**). The login account of a staff member. A staff login. Managed by Better Auth (its "user" table). Only staff have accounts at launch.

Session (Identity & Access, owned by **Super Admin**). A "you are logged in" wristband. One row per signed-in browser. Better Auth creates and expires these.

Account (Identity & Access, owned by **Super Admin**). The key that opens the login. Stores HOW a user proves who they are (password hash, or Google later).

Verification (Identity & Access, owned by **Super Admin**). One-time codes that expire. Short-lived codes for email verification and password reset.

Right column = children. These tables point back at User, Session, Account, Verification: StaffProfile, RoleAssignment, AuditLog, Leader, Comment and 9 more.

Better Auth owns these four tables. DCEB adds nothing to them except the "role" link in the next tab, so upgrades stay painless.

MISTAKES TO AVOID

- Adding church columns (phone, branch) straight onto the Better Auth table. Put them in StaffProfile so auth upgrades never fight your schema.
- Using the plugin "role" field as the whole permission system. It only says user/admin; real scoped roles live in RoleAssignment.
- Writing your own session code next to Better Auth. Two systems = two bugs.
- Storing plain passwords or logging login requests with bodies.
- Letting tokens live for days. Short life = small damage if an email is forwarded.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-19** An admin area for staff only, signed in with Better Auth
- **R-30** Class tabs: each main class broken into its own tab

B21

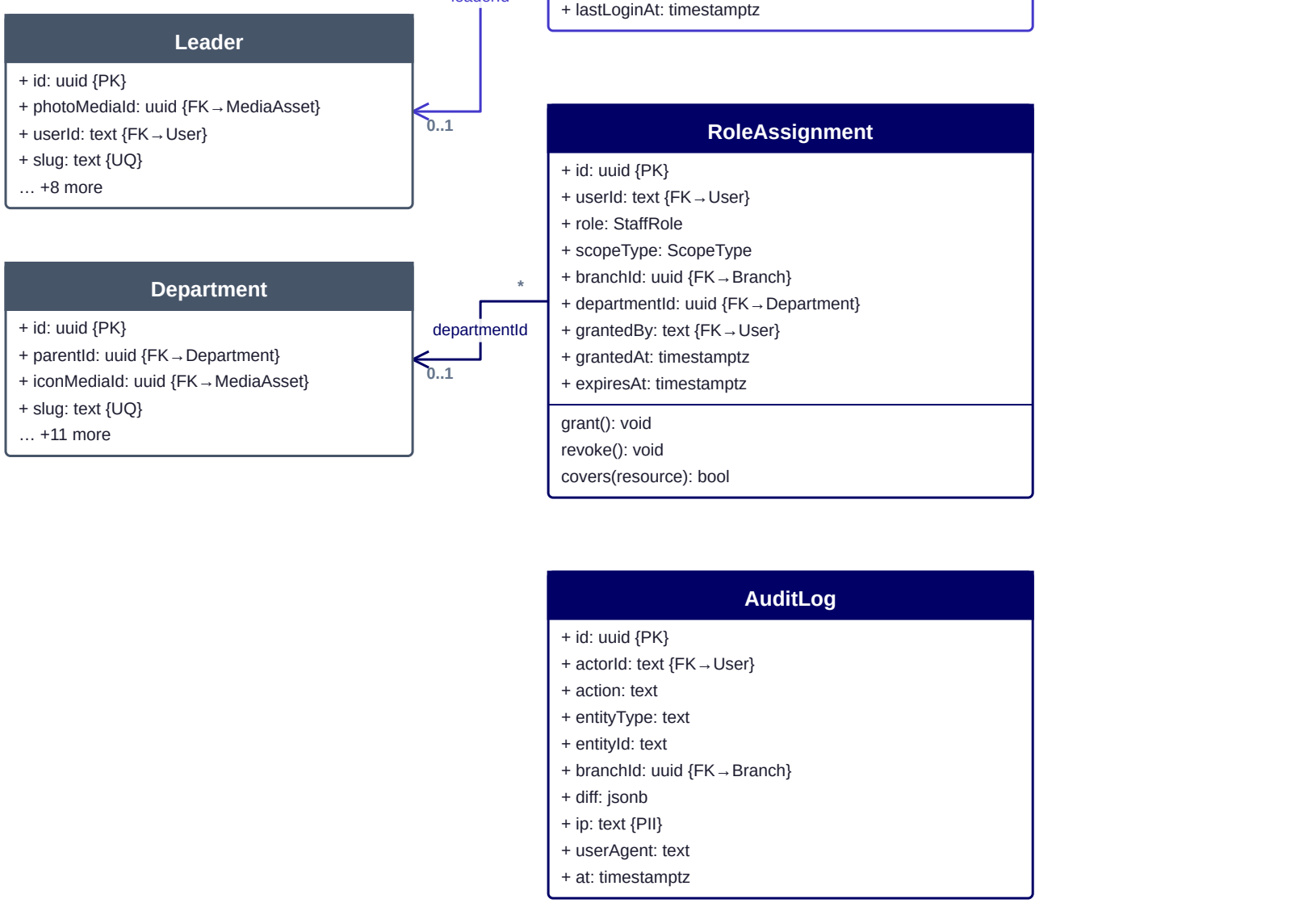
CLASS TABS (UML DETAIL) · PHASE 1

Tab: Staff access (StaffProfile, RoleAssignment, AuditLog)

Attributes, parents, children, rules and mistakes

Shared-kernel references (every role uses these, so no arrows are drawn)

- User ← userId
- Branch ← homeBranchId
- MediaAsset ← avatarMediaId
- Branch ← branchId
- User ← grantedBy
- User ← actorId



Depends on (platform code this class calls)

- AuthzService

Who may do what on StaffProfile (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: CRUD
- Branch: CRU

Who may do what on RoleAssignment (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: CRUD
- Branch: CRUD

Who may do what on AuditLog (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: R
- Branch: R

StaffProfile – in plain words

The staff member's name-badge details.
Church-specific facts about a staff member, kept apart from the auth table.

Rules the code must keep

- 1:1 with User.
- leaderId links a staff member to their public leader profile when they have one.
- One profile per staff User: display name, phone, photo, bio shown on leader pages (only if showPublic = true).
- Deactivating a staff member sets active=false; the User row stays so the audit trail still has a name.

Mistakes to avoid

- Showing staff phone numbers publicly by default.

RoleAssignment – in plain words

A sticker that says "Grace may edit sermons at Eastern Bypass". "Grace is a Content Editor for Eastern Bypass." One row = one role in one scope. A person can hold several.

Rules the code must keep

- scopeType = global | branch | department; branchId / departmentId must match the scope (CHECK constraint).
- Unique on (user, role, scope, branch, department) so nobody gets the same role twice.
- Branch Admins can only grant roles inside their own branch; only Super Admin grants global roles.
- Every grant/revoke writes an AuditLog row.

Mistakes to avoid

- Putting a single "role" column on the user. A department leader who also edits blogs needs two assignments.
- Forgetting expiresAt for temporary helpers (e.g. conference volunteers).

AuditLog – in plain words

The church's security-camera footage for the database.
A tamper-evident diary: who did what, to which record, when. Essential for money and personal data.

Rules the code must keep

- Append-only: no UPDATE or DELETE grants for the app role.
- Written for: grants, publishes, deletes, ledger entries and voids, exports, PII views.
- Kept 24 months; older rows archived to R2.

Mistakes to avoid

- Logging only creates. Deletes, exports and "viewed phone number" are the ones you need when something goes wrong.

How this group fits together

A User is just a login. StaffProfile says who the person is; RoleAssignment says what they may do and WHERE (global, branch or department); AuditLog remembers every change.

HOW TO READ IT

StaffProfile (Identity & Access, owned by **Branch Admin**). The staff member's name-badge details. Church-specific facts about a staff member, kept apart from the auth table.

RoleAssignment (Identity & Access, owned by **Super Admin**). A sticker that says "Grace may edit sermons at Eastern Bypass". "Grace is a Content Editor for Eastern Bypass." One row = one role in one scope. A person can hold several.

AuditLog (Identity & Access, owned by **Super Admin**). The church's security-camera footage for the database. A tamper-evident diary: who did what, to which record, when. Essential for money and personal data.

Left column = parents. These are the classes StaffProfile, RoleAssignment, AuditLog points at. StaffProfile / RoleAssignment / AuditLog cannot exist without them: Leader, Department.

A User is just a login. StaffProfile says who the person is; RoleAssignment says what they may do and WHERE (global, branch or department); AuditLog remembers every change.

MISTAKES TO AVOID

- Showing staff phone numbers publicly by default.
- Putting a single "role" column on the user. A department leader who also edits blogs needs two assignments.
- Forgetting expiresAt for temporary helpers (e.g. conference volunteers).
- Logging only creates. Deletes, exports and "viewed phone number" are the ones you need when something goes wrong.

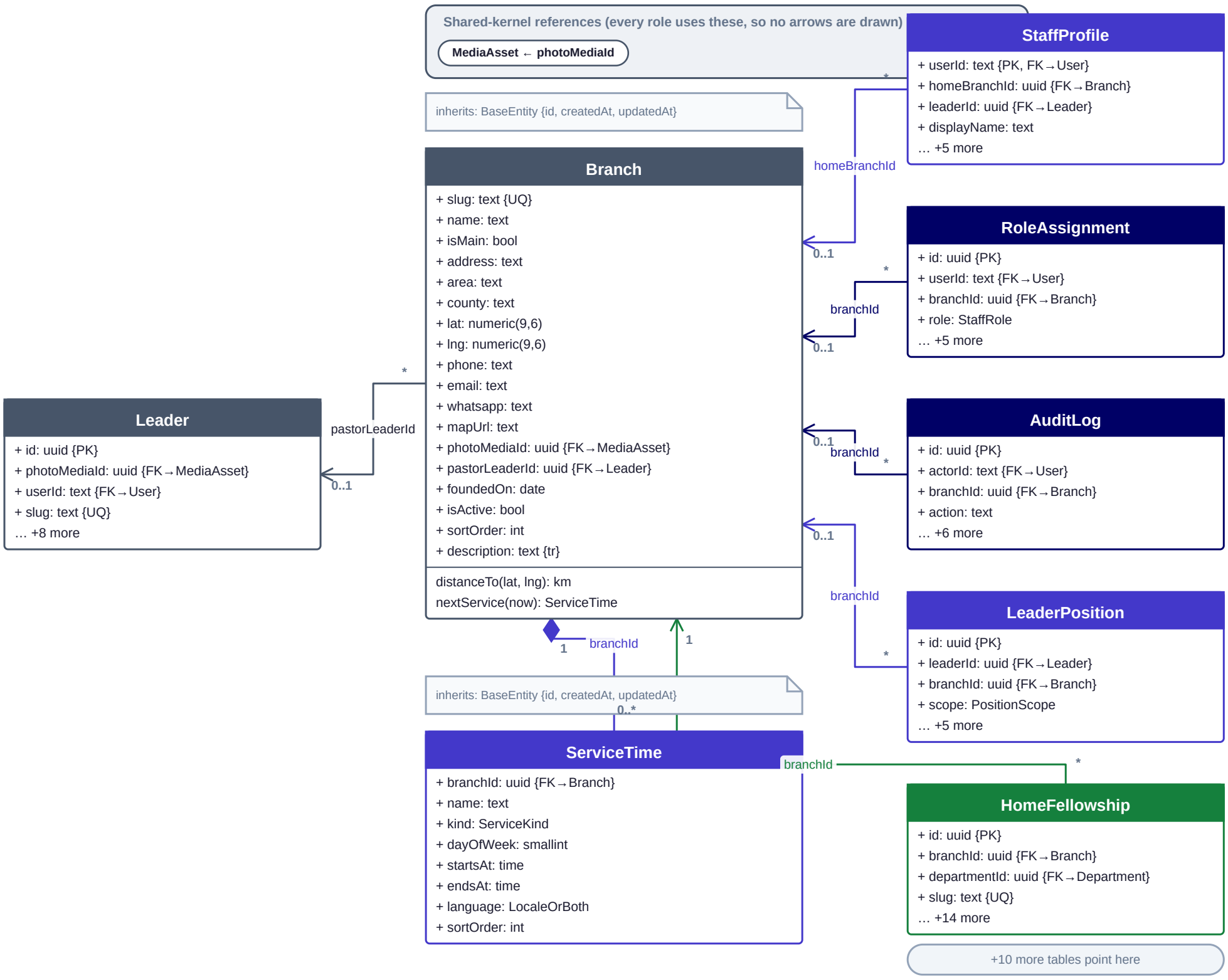
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-25** Audit log, privacy requests and a security-incident routine
- **R-30** Class tabs: each main class broken into its own tab

Tab: Branch and service times

Attributes, parents, children, rules and mistakes



Who may do what on Branch (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · New member: R · **Super: CRUD** · **Branch: RU** · Editor: R · Translator: R · Dept leader: R · Discipler: R · Treasurer: R · Events: R

Who may do what on ServiceTime (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · New member: R · **Super: CRUD** · **Branch: CRUD** · Editor: R · Dept leader: R · Events: R

Branch – in plain words

One church building and its community.
A physical church location. Almost every other table carries a branchId, so Branch is the most-referenced class in the system.

Rules the code must keep

- Exactly one branch has isMain = true (partial unique index).
- Never hard-delete a branch; set isActive = false so history keeps its meaning.
- Branch scope drives who may edit what (RoleAssignment.branchId).

Mistakes to avoid

- Storing the branch NAME in other tables instead of its id. Rename once and half your data is orphaned.

ServiceTime – in plain words

The timetable on the church door.
When each branch meets (Sunday services, midweek prayer, youth night).

Rules the code must keep

- Times are stored as local Nairobi time (no date), displayed with the viewer's language.
- Many per branch (Sunday 1st/2nd service, Wednesday midweek, Friday prayer).
- weekday 0-6 + local time + timezone Africa/Nairobi; the site never stores "9am" as text.

Mistakes to avoid

- Typing service times as free text on the page: they go stale in 3 months. Keep them as data.

How this group fits together

Branch is the most depended-on class in the system: 29 tables carry a branchId. That is how one website serves many churches.

HOW TO READ IT

Branch (Church Structure, owned by **Shared Kernel**). One church building and its community. A physical church location. Almost every other table carries a branchId, so Branch is the most-referenced class in the system.

ServiceTime (Church Structure, owned by **Branch Admin**). The timetable on the church door. When each branch meets (Sunday services, midweek prayer, youth night).

Left column = parents. These are the classes Branch, ServiceTime points at. Branch / ServiceTime cannot exist without them: Leader.

Right column = children. These tables point back at Branch, ServiceTime: StaffProfile, RoleAssignment, AuditLog, LeaderPosition, HomeFellowship and 10 more.

Branch is the most depended-on class in the system: 29 tables carry a branchId. That is how one website serves many churches.

MISTAKES TO AVOID

- Storing the branch NAME in other tables instead of its id. Rename once and half your data is orphaned.
- Typing service times as free text on the page: they go stale in 3 months. Keep them as data.

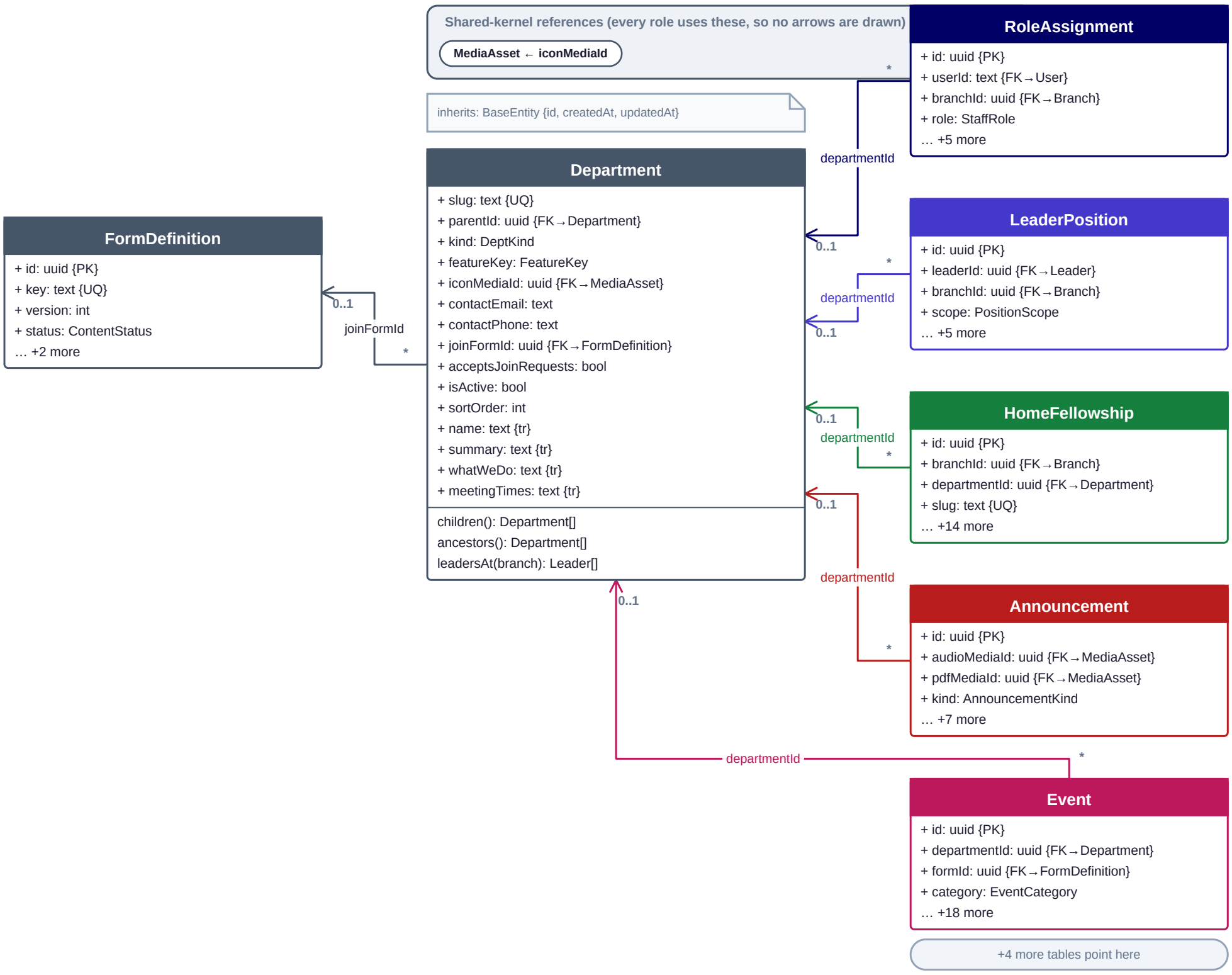
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-30** Class tabs: each main class broken into its own tab

Tab: Department

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

- TranslationService

Who may do what on Department (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDP** | **Branch: CRUDP** | **Editor: RU** | **Translator: RU** | **Dept leader: RU** | Discipler: R | Events: R

Department – in plain words

A team in the church. A team can have smaller teams inside it. Ministries (Youth, Men, Women, Missions...) and their sub-departments. It points at itself (parentId) to build a tree of any depth.

Rules the code must keep

- parentId must not create a loop (checked in code and by a recursive CTE test).
- featureKey wires special behaviour: home_fellowship turns on the fellowship finder, missions powers the Go Ye "join the team" link, discipleship receives converts.
- joinFormId decides which follow-up questions the new-member tour asks about this department.
- Leaders belong to a department through LeaderPosition, not through a column here, so each branch can have different leaders.

Mistakes to avoid

- Making "SubDepartment" a separate table. A department with a parent IS a sub-department; one table, one query.
- Hard-coding "if department == Missions" in the UI instead of using featureKey.

How this group fits together

A department can have sub-departments (parentId points at another Department). Youth → Teens; Missions → Go Ye team.

HOW TO READ IT

Department (Church Structure, owned by **Shared Kernel**). A team in the church. A team can have smaller teams inside it. Ministries (Youth, Men, Women, Missions...) and their sub-departments. It points at itself (parentId) to build a tree of any depth.

Left column = parents. These are the classes Department points at. Department cannot exist without them: FormDefinition.

Right column = children. These tables point back at Department: RoleAssignment, LeaderPosition, HomeFellowship, Announcement, Event and 4 more.

A department can have sub-departments (parentId points at another Department). Youth → Teens; Missions → Go Ye team.

MISTAKES TO AVOID

- Making "SubDepartment" a separate table. A department with a parent IS a sub-department; one table, one query.
- Hard-coding "if department == Missions" in the UI instead of using featureKey.

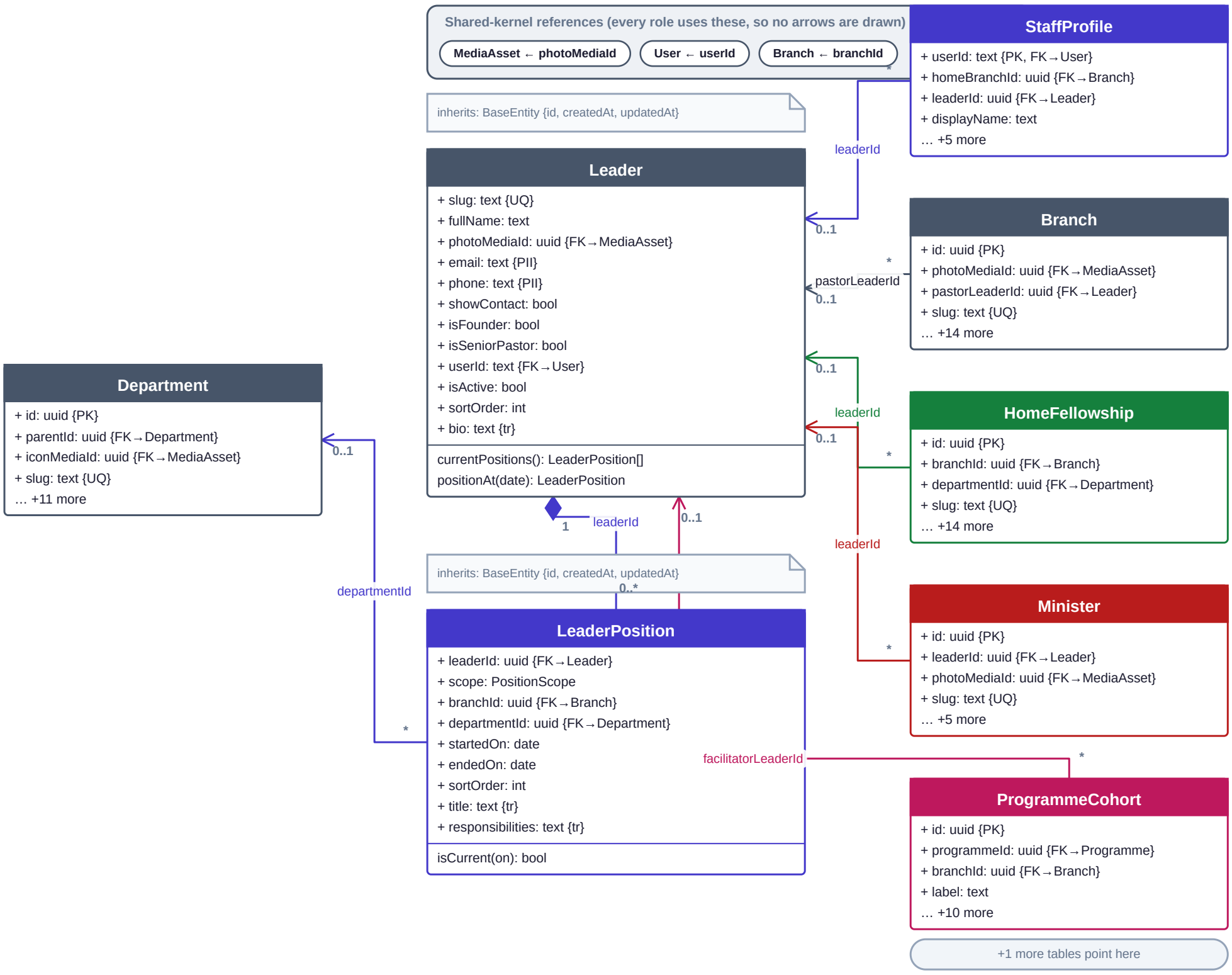
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-03** Departments with sub-departments, leaders and a join form
- **R-30** Class tabs: each main class broken into its own tab

Tab: Leader and LeaderPosition

Attributes, parents, children, rules and mistakes



Who may do what on Leader (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUD** | **Branch: CRUD** | **Editor: RU** | **Translator: RU** | Dept leader: R

Who may do what on LeaderPosition (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUD** | **Branch: CRUD** | **Dept leader: CRU**

Leader – in plain words
A leader's public profile card.
A person shown publicly as a pastor, elder or leader. NOT a login (that is User) and NOT a sermon speaker record (that is Minister).

Rules the code must keep
• Email and phone are private unless showContact = true.
• userId is optional: most leaders never log in.

Mistakes to avoid
• Merging Leader, Minister and User into one "person" table. They overlap but have different privacy and life-cycles.

LeaderPosition – in plain words
A job on a leader's CV, with a start date.
"When they became what, and what they hold." One leader has many positions over time; history is kept, not overwritten.

Rules the code must keep
• endedOn null means current.
• A position is scoped to church, one branch, or one department (CHECK).
• Never edit a past position to record a promotion; end it and add a new one.

Mistakes to avoid
• A single "title" text column on Leader. You lose the history and the "since when".

How this group fits together
A Leader is a person. A LeaderPosition is "this person holds this title in this department from this date to that date", so history ("when they became what") is kept.

HOW TO READ IT

Leader (Church Structure, owned by **Shared Kernel**). A leader's public profile card. A person shown publicly as a pastor, elder or leader. NOT a login (that is User) and NOT a sermon speaker record (that is Minister).

LeaderPosition (Church Structure, owned by **Branch Admin**). A job on a leader's CV, with a start date. "When they became what, and what they hold." One leader has many positions over time; history is kept, not overwritten.

Left column = parents. These are the classes Leader, LeaderPosition points at. Leader / LeaderPosition cannot exist without them: Department.

Right column = children. These tables point back at Leader, LeaderPosition: StaffProfile, Branch, HomeFellowship, Minister, ProgrammeCohort and 1 more.

A Leader is a person. A LeaderPosition is "this person holds this title in this department from this date to that date", so history ("when they became what") is kept.

MISTAKES TO AVOID

- Merging Leader, Minister and User into one "person" table. They overlap but have different privacy and life-cycles.
- A single "title" text column on Leader. You lose the history and the "since when".

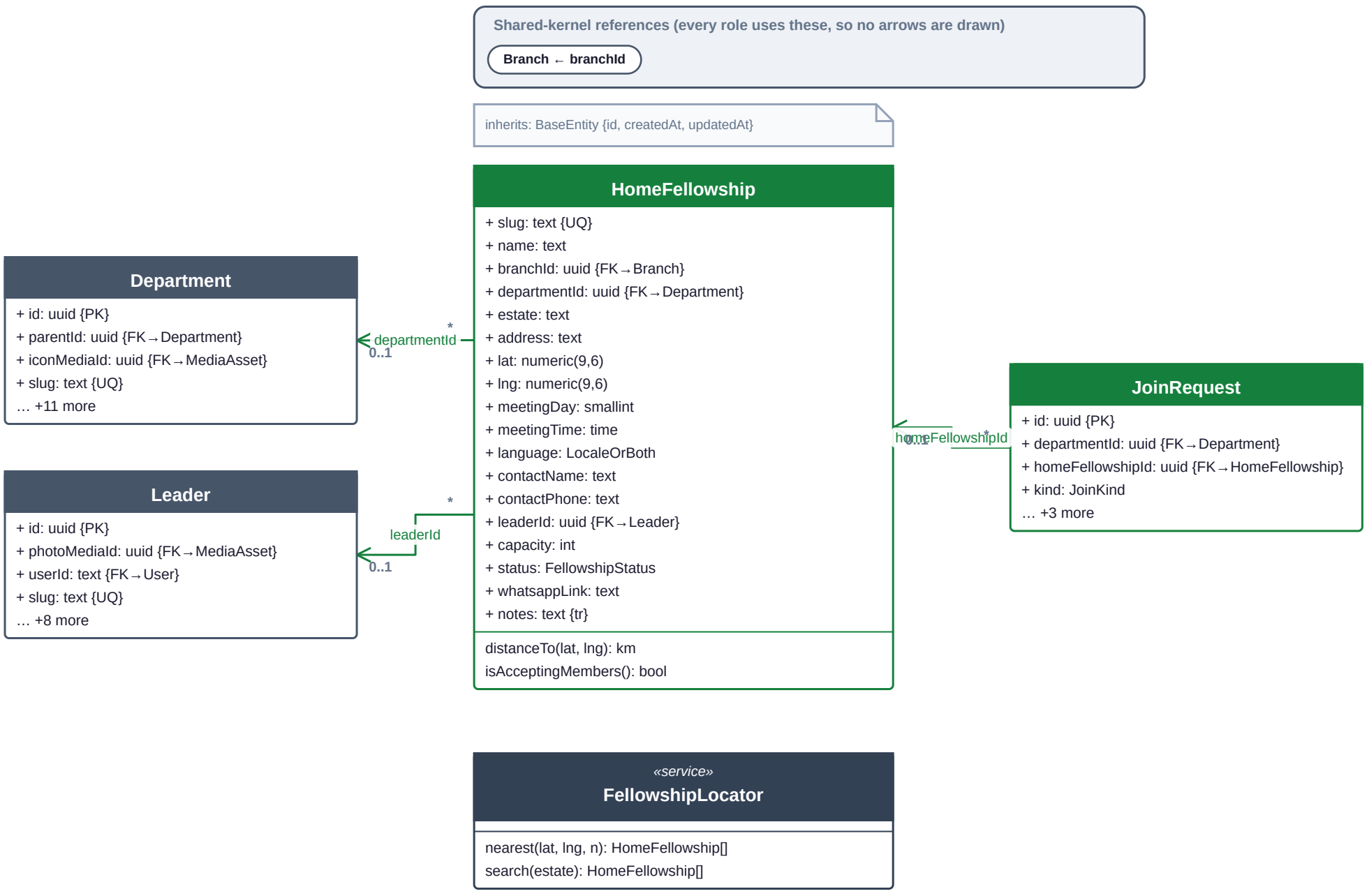
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-02** About page: history, vision, leaders, timeline
- **R-03** Departments with sub-departments, leaders and a join form
- **R-30** Class tabs: each main class broken into its own tab

Tab: Home fellowship

Attributes, parents, children, rules and mistakes



Called by (classes that depend on this service)

HomeFellowship

Who may do what on HomeFellowship (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUD** | **Branch: CRUD** | **Dept leader: CRUD**

HomeFellowship – in plain words

A small church that meets in somebody's sitting room.
One small group that meets in a home. Its coordinates power the "find my nearest fellowship" feature.

Rules the code must keep

- lat/lng required: no coordinates, no "nearest" result.
- Public contact is the fellowship contact, never the host's private number unless they agreed.
- status = full or paused removes it from "nearest" results but keeps the page.

Mistakes to avoid

- Searching by typing the estate name only. Typos break it; coordinates + a typed fallback is robust.

FellowshipLocator – in plain words

The "which fellowship is nearest me?" calculator.
Distance maths for the closest home fellowship.

Rules the code must keep

- Primary: browser geolocation (asks permission). Fallback: typed estate/area matched against HomeFellowship.areaText.
- Distance = Haversine on lat/lng; returns the 3 nearest with their meeting day and leader contact.

Mistakes to avoid

- Silently using the IP address for location. Ask permission or let them type.

How this group fits together

The nearest-fellowship feature = HomeFellowship rows with latitude/longitude + FellowshipLocator code that measures distance.

HOW TO READ IT

HomeFellowship (Church Structure, owned by **Department Leader**). A small church that meets in somebody's sitting room. One small group that meets in a home. Its coordinates power the "find my nearest fellowship" feature.

FellowshipLocator (Platform, owned by **Department Leader**). The "which fellowship is nearest me?" calculator. Distance maths for the closest home fellowship.

Left column = parents. These are the classes HomeFellowship, FellowshipLocator points at. HomeFellowship / FellowshipLocator cannot exist without them: Department, Leader.

Right column = children. These tables point back at HomeFellowship, FellowshipLocator: JoinRequest.

The nearest-fellowship feature = HomeFellowship rows with latitude/longitude + FellowshipLocator code that measures distance.

MISTAKES TO AVOID

- Searching by typing the estate name only. Typos break it; coordinates + a typed fallback is robust.
- Silently using the IP address for location. Ask permission or let them type.

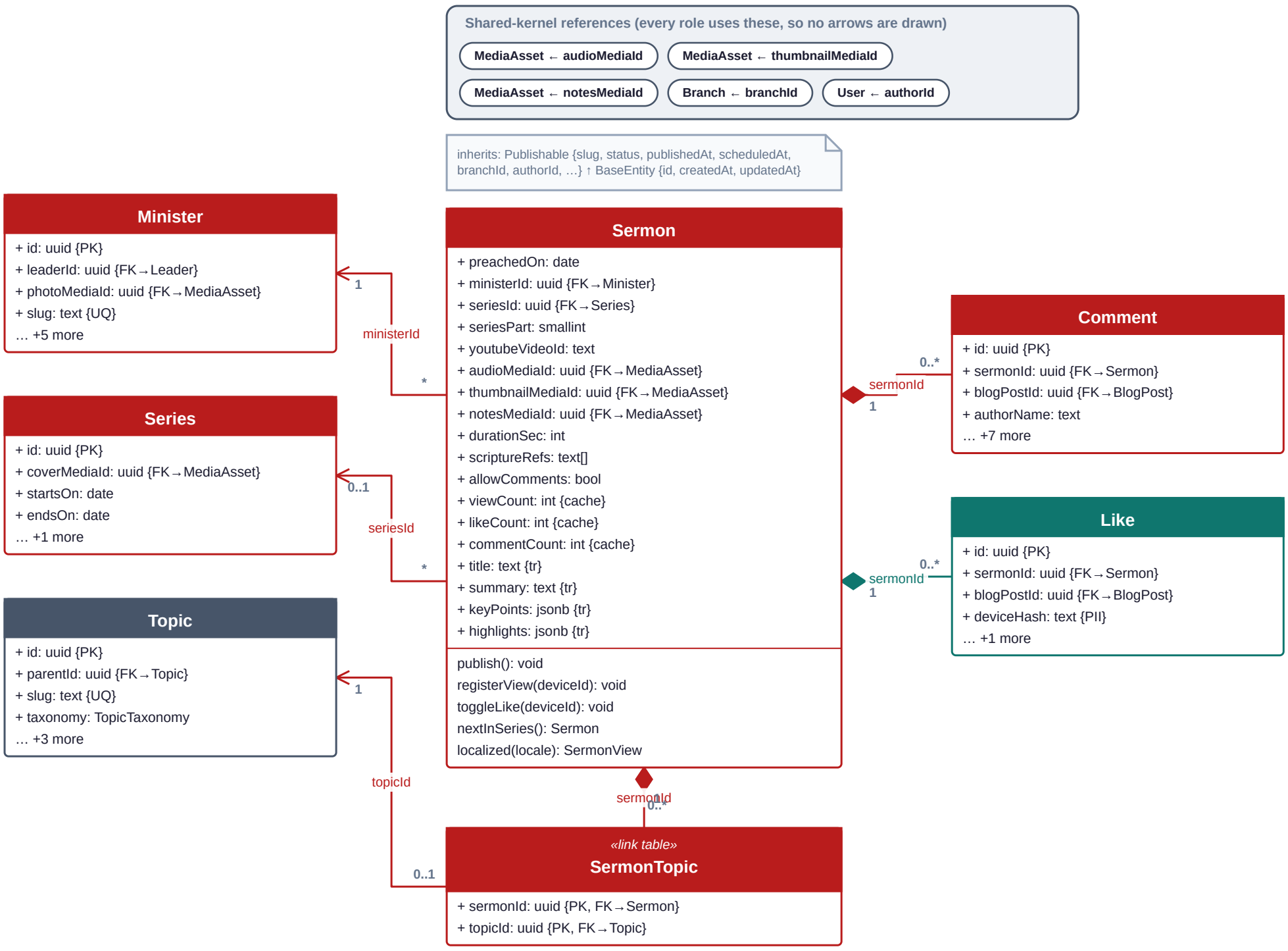
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-04** Home-fellowship finder (nearest to me)
- **R-30** Class tabs: each main class broken into its own tab

Tab: Sermon and SermonTopic

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

StorageService · TranslationService · CacheService · RateLimiter

Who may do what on Sermon (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · New member: R · **Super: CRUDPA** · **Branch: CRUDPA** · **Editor: CRUDP** · **Translator: RU** · Dept leader: R · Discipler: R · Treasurer: R · Events: R

Who may do what on SermonTopic (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · **Super: CRUD** · **Branch: CRUD** · **Editor: CRUD**

Sermon – in plain words

One recorded message with everything people want to know about it. The heart of the website: video (YouTube), audio (R2), summary, verses, key points, highlights, views, likes, comments.

Rules the code must keep

- A sermon needs a YouTube id OR an audio file OR both; never neither (CHECK).
- preachedOn is the sermon date; publishedAt is when it appeared on the site.
- branchId = branch it was uploaded from, shown on the card.
- viewCount/likeCount are CACHES of Redis counters flushed every few minutes by QStash; the truth for likes is the Like table.
- Filters: series, month, theme (Topic), time, minister, branch.

Mistakes to avoid

- Letting the browser bump viewCount directly in Postgres on every play. Count in Redis, flush in batches.
- Embedding the YouTube player on page load. Show a thumbnail and load the iframe on click (faster on mobile data).

SermonTopic – in plain words

The string tying a sermon to its topics.
Many-to-many link: a sermon can have several themes and a theme many sermons.

Rules the code must keep

- Pure link table: (sermonId, topicId) is the primary key, so the same topic cannot be attached twice.
- Deleting a Sermon removes its links (cascade); deleting a Topic that is in use is refused.

Mistakes to avoid

- Using a comma-separated "topics" text column. You cannot filter or count it.

How this group fits together

A sermon points to ONE minister and ONE series, and to MANY topics through the SermonTopic link table.

HOW TO READ IT

Sermon (Content (CMS), owned by **Content Editor**). One recorded message with everything people want to know about it. The heart of the website: video (YouTube), audio (R2), summary, verses, key points, highlights, views, likes, comments.

SermonTopic (Content (CMS), owned by **Content Editor**). The string tying a sermon to its topics. Many-to-many link: a sermon can have several themes and a theme many sermons.

Left column = parents. These are the classes Sermon, SermonTopic points at. Sermon / SermonTopic cannot exist without them: Minister, Series, Topic.

Right column = children. These tables point back at Sermon, SermonTopic: Comment, Like.

A sermon points to ONE minister and ONE series, and to MANY topics through the SermonTopic link table.

MISTAKES TO AVOID

- Letting the browser bump viewCount directly in Postgres on every play. Count in Redis, flush in batches.
- Embedding the YouTube player on page load. Show a thumbnail and load the iframe on click (faster on mobile data).
- Using a comma-separated "topics" text column. You cannot filter or count it.

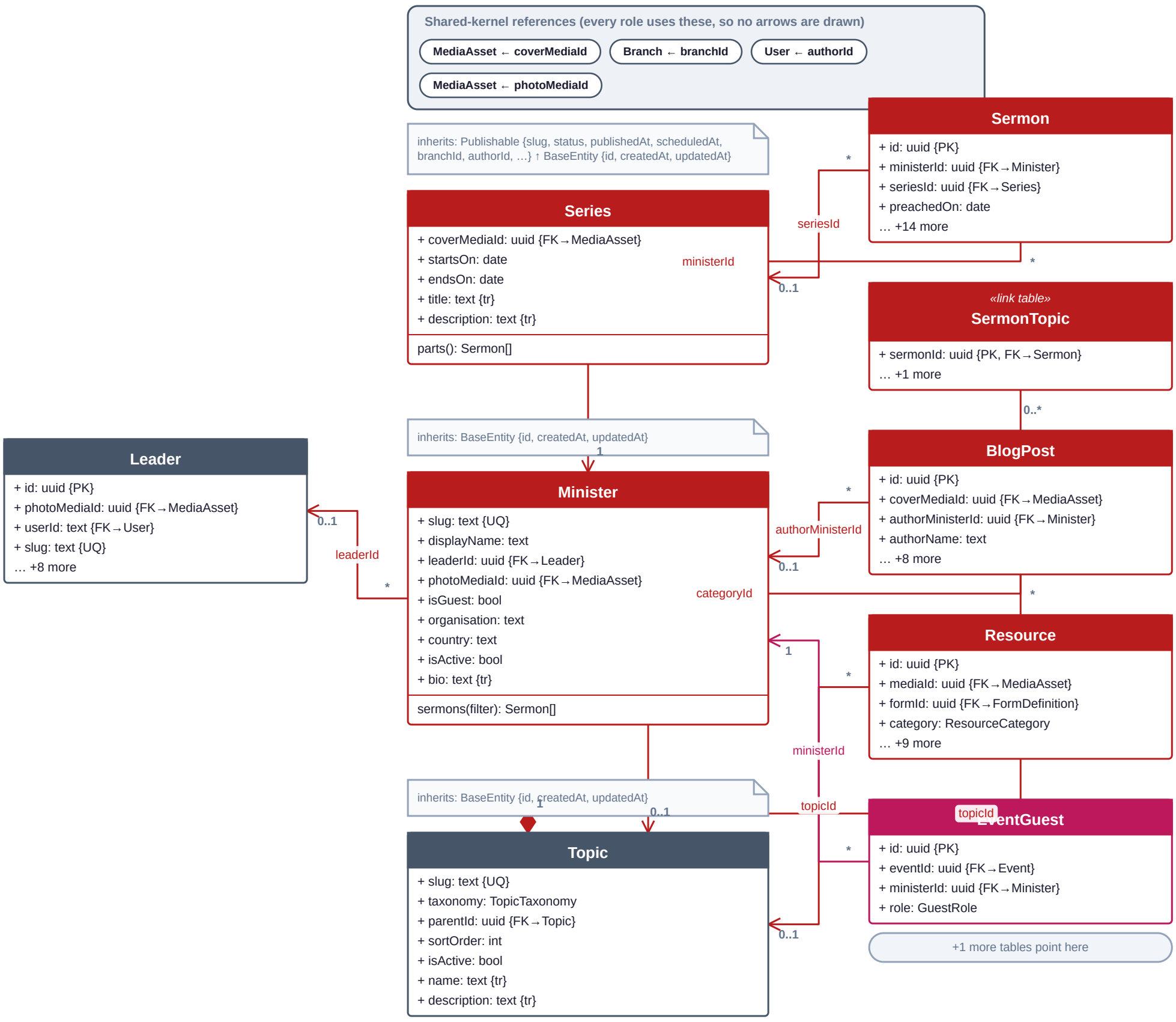
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-11** Sermons with video, audio, summary, verses, filters, views, comments and likes
- **R-30** Class tabs: each main class broken into its own tab

Tab: Series, Minister, Topic

Attributes, parents, children, rules and mistakes



Who may do what on Series (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDPA** | **Branch: CRUDPA** | **Editor: CRUDP** | **Translator: RU** | Dept leader: R | Discipler: R | Treasurer: R | Events: R

Who may do what on Minister (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDPA** | **Branch: CRUDPA** | **Editor: CRUDP** | **Translator: RU** | Dept leader: R | Discipler: R | Treasurer: R | Events: R

Who may do what on Topic (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDPA** | **Branch: CRUDPA** | **Editor: CRUDP** | **Translator: RU** | Dept leader: R | Discipler: R | Treasurer: R | Events: R

Series – in plain words
A playlist of sermons.
A run of connected messages (e.g. "Faith that Moves").

Rules the code must keep
• Sermon.seriesPart orders the sermons inside a series.
• A series has a start and end date; sermons attach to it with seriesPart (1, 2, 3...).
• Series with no sermons is hidden from filters.

Mistakes to avoid
• Creating a series per sermon. A series is a run of 2 or more.

Minister – in plain words
A preacher who can have many sermons.
The speaker. Clicking a minister's name filters every sermon by that person, so Minister must be its own record.

Rules the code must keep
• leaderId is optional because guest ministers are not church leaders.
• Merge duplicates ("Pr. John" vs "Pastor John Kamau") before launch, never after.

Mistakes to avoid
• Free-text speaker names on sermons. You cannot filter or link what you did not model.

Topic – in plain words
A label that can be stuck on many things.
The reusable "category dictionary". Resource topics, sermon themes and blog categories are all Topics, so a topic typed once can be reused forever.

Rules the code must keep
• taxonomy separates theme / resource_topic / blog_category so lists stay tidy.
• Editors pick from existing topics first; creating a new one is one click but needs a Swahili name before publish.

Mistakes to avoid
• Typing categories as free text on every upload ("Prayer", "prayer", "Prayers"). That is why you wanted a topic table, and you were right.

How this group fits together
These three are the "filters" of the Sermons page: clicking a minister, a series or a theme narrows the list.

HOW TO READ IT

Series (Content (CMS), owned by **Content Editor**). A playlist of sermons. A run of connected messages (e.g. "Faith that Moves").

Minister (Content (CMS), owned by **Content Editor**). A preacher who can have many sermons. The speaker. Clicking a minister's name filters every sermon by that person, so Minister must be its own record.

Topic (Content (CMS), owned by **Shared Kernel**). A label that can be stuck on many things. The reusable "category dictionary". Resource topics, sermon themes and blog categories are all Topics, so a topic typed once can be reused forever.

Left column = parents. These are the classes Series, Minister, Topic points at. Series / Minister / Topic cannot exist without them: Leader.

Right column = children. These tables point back at Series, Minister, Topic: Sermon, SermonTopic, BlogPost, Resource, EventGuest and 1 more.

These three are the "filters" of the Sermons page: clicking a minister, a series or a theme narrows the list.

MISTAKES TO AVOID

- Creating a series per sermon. A series is a run of 2 or more.
- Free-text speaker names on sermons. You cannot filter or link what you did not model.
- Typing categories as free text on every upload ("Prayer", "prayer", "Prayers"). That is why you wanted a topic table, and you were right.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

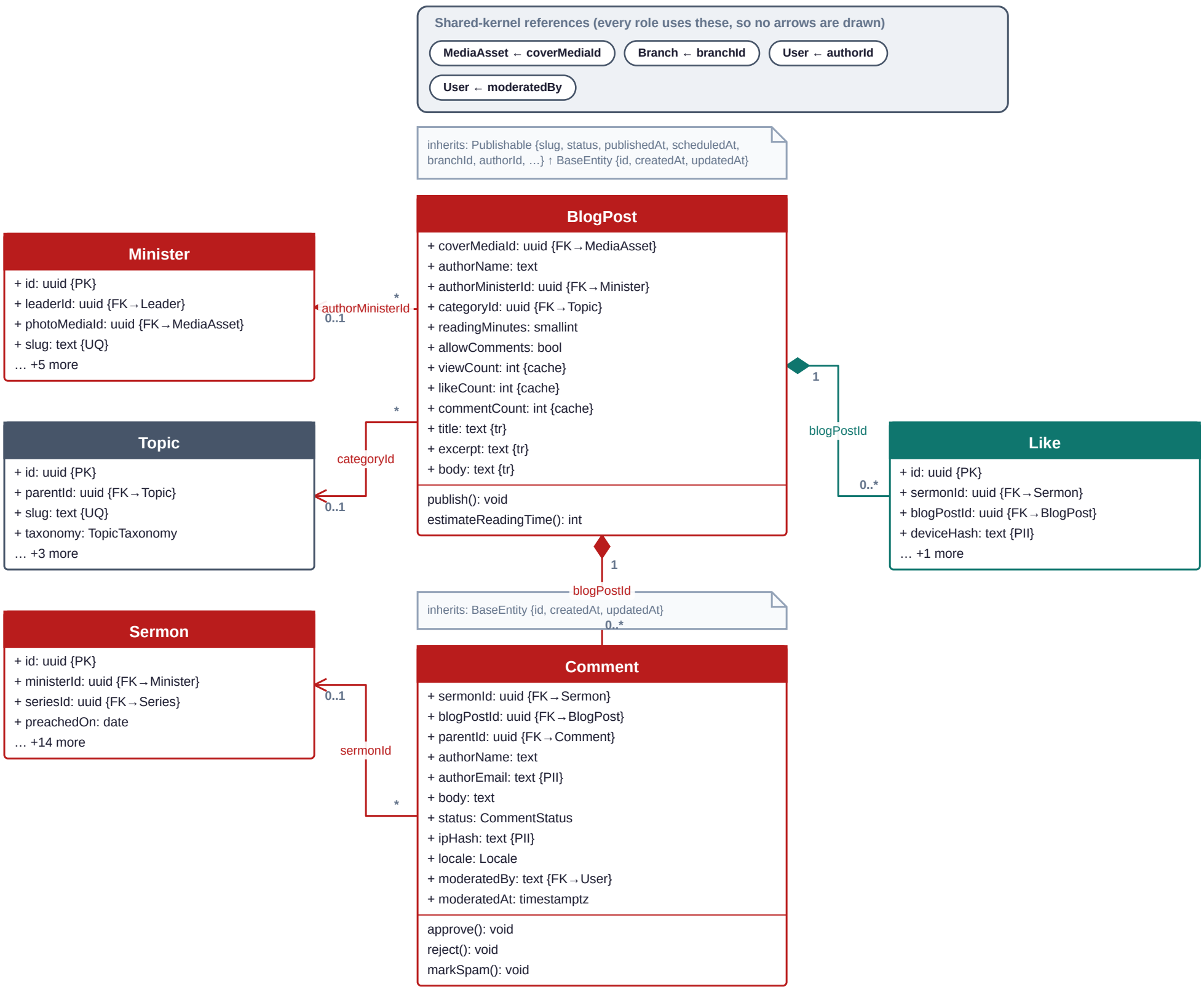
- **R-11** Sermons with video, audio, summary, verses, filters, views, comments and likes
- **R-30** Class tabs: each main class broken into its own tab

B28

CLASS TABS (UML DETAIL) · PHASE 1

Tab: Blog and comments

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

RateLimiter · ModerationService

Who may do what on BlogPost (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · New member: R · **Super: CRUDPA** · **Branch: CRUDPA** · **Editor: CRUDP** · **Translator: RU** · Dept leader: R · Discipler: R · Treasurer: R · Events: R

Who may do what on Comment (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: S · New member: S · **Super: RUDA** · **Branch: RUDA** · **Editor: RUDA**

BlogPost – in plain words

A written article.
Written articles. Blogs are written in ONE language first; a Translation row adds the other.

Rules the code must keep

- body is rich text (Tiptap JSON or MDX) stored in the translation row.
- Reading time is computed on save, not on every view.
- Translation is per field (title, excerpt, body) in blog_post_translation; the English row is the source of truth for slug.
- body is stored as Tiptap JSON, rendered to HTML on the server and sanitised.
- allowComments can be switched off per post.

Mistakes to avoid

- Pasting raw HTML from Word. It brings hidden styles and scripts; use the editor and sanitise.

Comment – in plain words

A message left under a sermon.
A visitor's comment under a sermon or blog. Starts as "pending" until a moderator approves.

Rules the code must keep

- Exactly one of sermonId / blogPostId is set (CHECK).
- Honeypot field + Turnstile + Upstash rate limit before the row is created.
- Emails are never shown publicly.
- Replies one level deep (parentId) to keep the page readable.

Mistakes to avoid

- Publishing comments instantly. The first spam wave will land on a church page.

How this group fits together

Comments hang under either a sermon or a blog post, never both.

HOW TO READ IT

BlogPost (Content (CMS), owned by **Content Editor**). A written article. Written articles. Blogs are written in ONE language first; a Translation row adds the other.

Comment (Content (CMS), owned by **Content Editor**). A message left under a sermon. A visitor's comment under a sermon or blog. Starts as "pending" until a moderator approves.

Left column = parents. These are the classes BlogPost, Comment points at. BlogPost / Comment cannot exist without them: Minister, Topic, Sermon.

Right column = children. These tables point back at BlogPost, Comment: Like.

Comments hang under either a sermon or a blog post, never both.

MISTAKES TO AVOID

- Pasting raw HTML from Word. It brings hidden styles and scripts; use the editor and sanitise.
- Publishing comments instantly. The first spam wave will land on a church page.

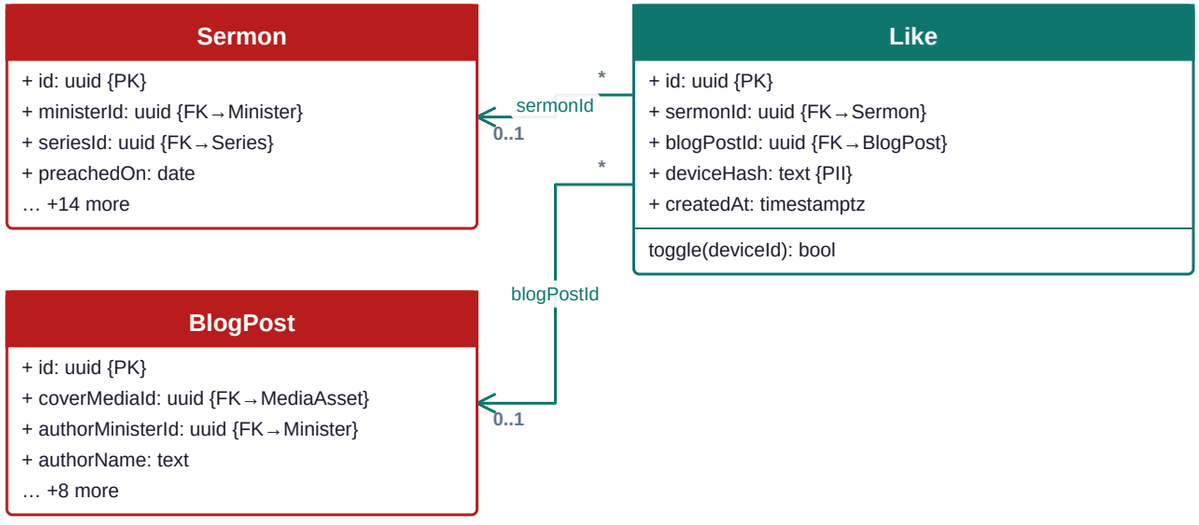
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-10** Resources: blogs, bulletins, announcements (Sunday and midweek), Bible study and discipleship, other
- **R-17** Comments and likes with moderation
- **R-30** Class tabs: each main class broken into its own tab

Tab: Like

Attributes, parents, children, rules and mistakes



Who may do what on Like (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: S | New member: S | Super: R

Like – in plain words
A tick from one phone.
One thumbs-up per device per item. No login needed.

Rules the code must keep
- Unique (sermonId, deviceHash) and (blogPostId, deviceHash).
- deviceHash = HMAC of a random browser id; never the IP.
- Fast path is a Redis SET; the table is the durable truth.

Mistakes to avoid
- Counting likes with a plain number. People refresh and like 40 times.

How this group fits together
One like per device per item. The count shown on the page is a fast Redis counter; this table is the proof.

HOW TO READ IT

Like (Content (CMS), owned by **Visitor (public)**). A tick from one phone. One thumbs-up per device per item. No login needed.

Left column = parents. These are the classes Like points at. Like cannot exist without them: Sermon, BlogPost.

One like per device per item. The count shown on the page is a fast Redis counter; this table is the proof.

MISTAKES TO AVOID

- Counting likes with a plain number. People refresh and like 40 times.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-11** Sermons with video, audio, summary, verses, filters, views, comments and likes
- **R-17** Comments and likes with moderation
- **R-30** Class tabs: each main class broken into its own tab

Tab: Bulletin and Announcement

Attributes, parents, children, rules and mistakes

Shared-kernel references (every role uses these, so no arrows are drawn)

- MediaAsset ← pdfMediaId
- MediaAsset ← coverMediaId
- Branch ← branchId
- User ← authorId
- MediaAsset ← audioMediaId

inherits: Publishable {slug, status, publishedAt, scheduledAt, branchId, authorId, …} ↑ BaseEntity {id, createdAt, updatedAt}

Bulletin

+ weekOf: date
+ year: smallint
+ month: smallint
+ pdfMediaId: uuid {FK→MediaAsset}
+ coverMediaId: uuid {FK→MediaAsset}
+ pageCount: smallint
+ downloadCount: int {cache}
+ readCount: int {cache}
+ extractedText: text
+ title: text {tr}

download(): Url
readInline(): Url

Department

+ id: uuid {PK}
+ parentId: uuid {FK→Department}
+ iconMediaId: uuid {FK→MediaAsset}
+ slug: text {UQ}
… +11 more

0..1 — departmentId — *

inherits: Publishable {slug, status, publishedAt, scheduledAt, branchId, authorId, …} ↑ BaseEntity {id, createdAt, updatedAt}

Announcement

+ kind: AnnouncementKind
+ serviceDate: date
+ audioMediaId: uuid {FK→MediaAsset}
+ youtubeVideoId: text
+ pdfMediaId: uuid {FK→MediaAsset}
+ eventAt: timestamptz
+ venue: text
+ departmentId: uuid {FK→Department}
+ expiresAt: timestamptz
+ title: text {tr}
+ body: text {tr}

isExpired(now): bool

Who may do what on Bulletin (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDPA** | **Branch: CRUDPA** | **Editor: CRUDP** | **Translator: RU** | Dept leader: R | Discipler: R | Treasurer: R | Events: R

Who may do what on Announcement (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDPA** | **Branch: CRUDPA** | **Editor: CRUDP** | **Translator: RU** | **Dept leader: CU** | Discipler: R | Treasurer: R | Events: R

Bulletin – in plain words

This week's church newspaper.
The Sunday bulletin PDF. All years, all months; read in the browser or download.

Rules the code must keep

• Unique (branchId, weekOf).
• year and month are derived from weekOf and stored for fast filtering.
• A QStash job renders page 1 as a cover image and extracts text for search.

Mistakes to avoid

• Uploading every bulletin as a differently named file. Name = date, and let the system derive the rest.

Announcement – in plain words

A notice read out in church or sent during the week.
Two kinds in one table: SUNDAY announcements (audio + readable + video, one per Sunday) and MIDWEEK ones (Friday youth fellowship, men's retreat...).

Rules the code must keep

• kind = sunday requires serviceDate; kind = midweek requires eventAt (CHECK).
• One sunday announcement per (branch, serviceDate).
• Midweek announcements auto-hide after expiresAt.
• Department leaders may draft midweek announcements; an Editor approves.

Mistakes to avoid

• Two separate tables for sunday and midweek. They share 90% of their columns; use one table with a kind column (single-table inheritance).

How this group fits together

Bulletins are weekly PDFs by year/month. Announcements share ONE table with a "kind" column: sunday (audio+text+video) or midweek (text or PDF).

HOW TO READ IT

Bulletin (Content (CMS), owned by **Content Editor**). This week's church newspaper. The Sunday bulletin PDF. All years, all months; read in the browser or download.

Announcement (Content (CMS), owned by **Content Editor**). A notice read out in church or sent during the week. Two kinds in one table: SUNDAY announcements (audio + readable + video, one per Sunday) and MIDWEEK ones (Friday youth fellowship, men's retreat...).

Left column = parents. These are the classes Bulletin, Announcement points at. Bulletin / Announcement cannot exist without them: Department.

Bulletins are weekly PDFs by year/month. Announcements share ONE table with a "kind" column: sunday (audio+text+video) or midweek (text or PDF).

MISTAKES TO AVOID

- Uploading every bulletin as a differently named file. Name = date, and let the system derive the rest.
- Two separate tables for sunday and midweek. They share 90% of their columns; use one table with a kind column (single-table inheritance).

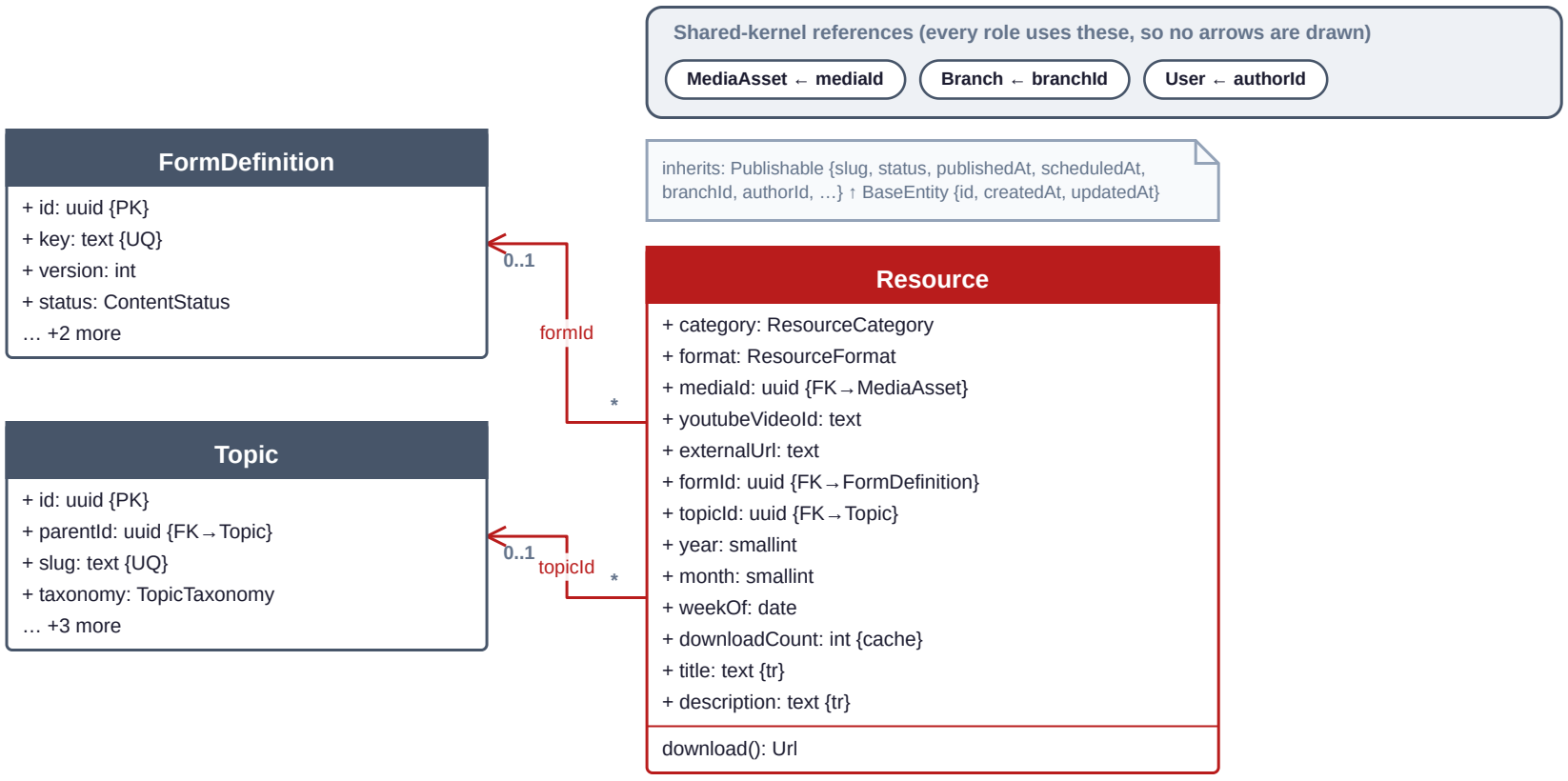
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-10** Resources: blogs, bulletins, announcements (Sunday and midweek), Bible study and discipleship, other
- **R-30** Class tabs: each main class broken into its own tab

Tab: Resource (library)

Attributes, parents, children, rules and mistakes



Who may do what on Resource (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDPA** | **Branch: CRUDPA** | **Editor: CRUDP** | **Translator: RU** | Dept leader: R | Discipler: R | Treasurer: R | Events: R

Resource – in plain words
A downloadable or watchable learning item.
Discipleship material, Bible study notes, mission-week PDFs, weekly prayer points, registration forms: one table, filtered by category, topic, year and format.

Rules the code must keep
• Main topic = Topic row; year/month for the filters you described.
• format = form means formId is set and the page renders the registration form inline.

Mistakes to avoid
• One table per resource page (discipleship, bible_study, other...). Add a category value instead.

How this group fits together
Bible study, discipleship PDFs/videos, forms, mission-week PDFs, weekly prayer points: one table, a category and a year/type filter.

HOW TO READ IT

Resource (Content (CMS), owned by **Content Editor**). A downloadable or watchable learning item. Discipleship material, Bible study notes, mission-week PDFs, weekly prayer points, registration forms: one table, filtered by category, topic, year and format.

Left column = parents. These are the classes Resource points at. Resource cannot exist without them: FormDefinition, Topic.

Bible study, discipleship PDFs/videos, forms, mission-week PDFs, weekly prayer points: one table, a category and a year/type filter.

MISTAKES TO AVOID

- One table per resource page (discipleship, bible_study, other...). Add a category value instead.

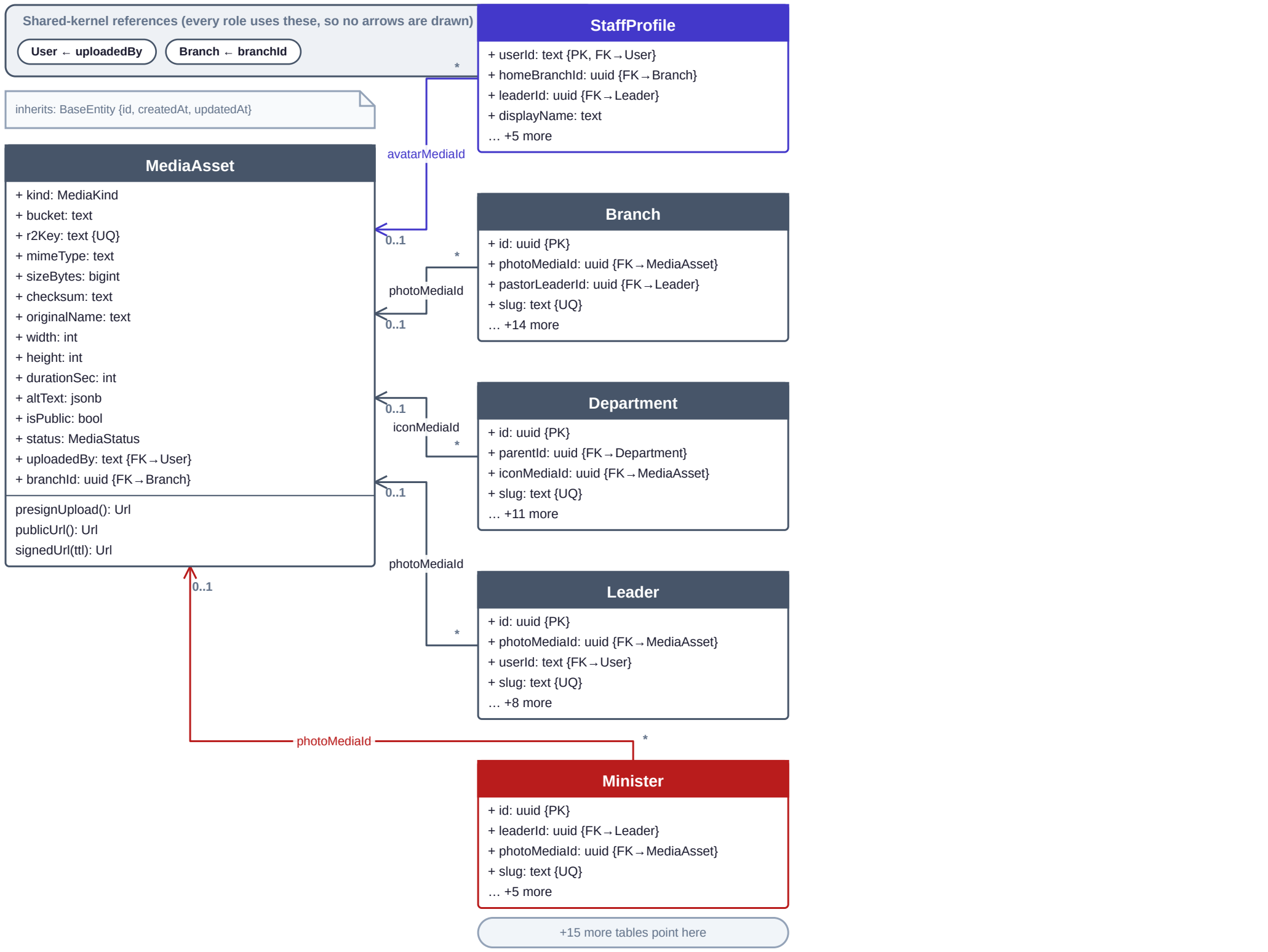
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-10** Resources: blogs, bulletins, announcements (Sunday and midweek), Bible study and discipleship, other
- **R-30** Class tabs: each main class broken into its own tab

Tab: MediaAsset

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

StorageService

Who may do what on MediaAsset (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Super: CRUD | Branch: CRUD | Editor: CRUD | Translator: R | Dept leader: CR | Discipler: R | Treasurer: R | Events: CR

MediaAsset – in plain words

The library card for one file.
Every file (image, audio, PDF, certificate) is one row pointing to a Cloudflare R2 object. Other tables store the id, never the URL.

Rules the code must keep

- Browser uploads straight to R2 with a presigned URL; the app only stores the row.
- isPublic = false for certificates and anything with personal data; served with short-lived signed URLs.
- checksum prevents duplicate uploads.
- status goes uploading -> ready (after a verification job) or failed.

Mistakes to avoid

- Saving the full URL. If you ever change CDN or bucket, every row breaks. Save the key.

How this group fits together

Every file (photo, PDF, audio, video link) is registered once here. Other tables only store its id.

HOW TO READ IT

MediaAsset (Platform, owned by **Shared Kernel**). The library card for one file. Every file (image, audio, PDF, certificate) is one row pointing to a Cloudflare R2 object. Other tables store the id, never the URL.

Right column = children. These tables point back at MediaAsset: StaffProfile, Branch, Department, Leader, Minister and 15 more.

Every file (photo, PDF, audio, video link) is registered once here. Other tables only store its id.

MISTAKES TO AVOID

- Saving the full URL. If you ever change CDN or bucket, every row breaks. Save the key.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-22** Upload pictures, audio and documents (Cloudflare R2)
- **R-30** Class tabs: each main class broken into its own tab

B33

CLASS TABS (UML DETAIL) · PHASE 1

Tab: Page blocks, timeline, site settings

Attributes, parents, children, rules and mistakes

Shared-kernel references (every role uses these, so no arrows are drawn)

MediaAsset ← mediaId | User ← updatedBy

inherits: BaseEntity {id, createdAt, updatedAt}

PageBlock

+ page: text
+ key: text
+ type: BlockType
+ mediaId: uuid {FK→MediaAsset}
+ sortOrder: int
+ isActive: bool
+ data: jsonb {tr}

inherits: BaseEntity {id, createdAt, updatedAt}

TimelineEntry

+ year: smallint
+ mediaId: uuid {FK→MediaAsset}
+ sortOrder: int
+ title: text {tr}
+ body: text {tr}

SiteSetting

+ key: text {PK}
+ value: jsonb
+ updatedBy: text {FK→User}
+ updatedAt: timestamptz

Who may do what on PageBlock (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDPA** | **Branch: CRUDPA** | **Editor: CRUDP** | **Translator: RU** | Dept leader: R | Discipler: R | Treasurer: R | Events: R

Who may do what on TimelineEntry (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDPA** | **Branch: CRUDPA** | **Editor: CRUDP** | **Translator: RU** | Dept leader: R | Discipler: R | Treasurer: R | Events: R

Who may do what on SiteSetting (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Super: CRU | Branch: R

PageBlock – in plain words
A piece of the home page you can edit.
Editable landing-page sections (hero, about blurb, short history) so staff can change words without a developer.

Rules the code must keep
• Unique (page, key).
• Only fixed block types are allowed; this is a mini-CMS, not a page builder.
• A block = (page, key) such as home/hero_title; the code asks for the key, the CMS decides the words.
• If a block is missing the code falls back to the message file so the page never shows an empty hole.

Mistakes to avoid
• Making every sentence on the site editable. Editable text = more admin work and more typos. Make only what changes.

TimelineEntry – in plain words
One dot on the church history line.
The church history timeline used on About and in the new-member tour.

Rules the code must keep
• Used by "short history" on Home and the full timeline on About.
• year + sortOrder decide the order; text comes from timeline_entry_translation.

Mistakes to avoid
• Storing the year inside the translated text; you cannot sort by it.

SiteSetting – in plain words
The control-panel switches.
Small global facts: born-again baseline, social links, policy versions, feature flags.

Rules the code must keep
• Key/value config editable by Super Admin: church name, social links, default meta tags, bank/M-Pesa display flags.
• Secrets (API keys) NEVER live here. They live in Vercel environment variables.

Mistakes to avoid
• Putting API secrets in this table because "it is easy to edit".

How this group fits together
The small editable bits of the public pages: hero text, history timeline entries and site-wide settings.

HOW TO READ IT

PageBlock (Content (CMS), owned by **Content Editor**). A piece of the home page you can edit. Editable landing-page sections (hero, about blurb, short history) so staff can change words without a developer.

TimelineEntry (Content (CMS), owned by **Content Editor**). One dot on the church history line. The church history timeline used on About and in the new-member tour.

SiteSetting (Platform, owned by **Super Admin**). The control-panel switches. Small global facts: born-again baseline, social links, policy versions, feature flags.

The small editable bits of the public pages: hero text, history timeline entries and site-wide settings.

MISTAKES TO AVOID

- Making every sentence on the site editable. Editable text = more admin work and more typos. Make only what changes.
- Storing the year inside the translated text; you cannot sort by it.
- Putting API secrets in this table because "it is easy to edit".

WHAT YOU ALREADY DID RIGHT

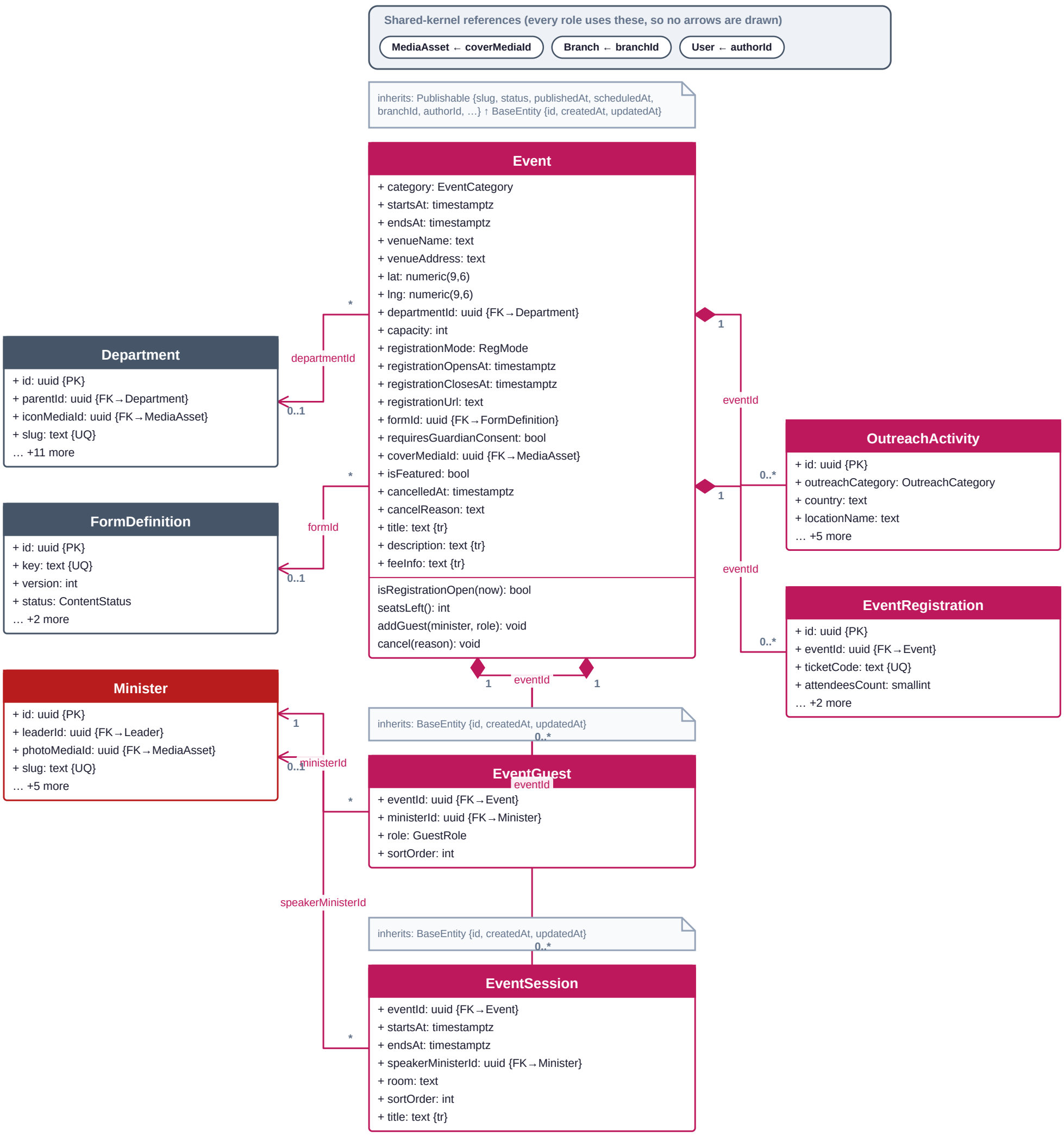
This page serves your own requirements:

- **R-02** About page: history, vision, leaders, timeline
- **R-30** Class tabs: each main class broken into its own tab

CLASS TABS (UML DETAIL) · PHASE 1

Tab: Event, guests, sessions

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

TranslationService · JobQueue

Who may do what on Event (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · New member: R · Super: CRUDP · Branch: CRUDP · Editor: RU · Translator: RU · Dept leader: CU · Events: CRUDP

Who may do what on EventGuest (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · Super: CRUD · Branch: CRUD · Events: CRUD

Who may do what on EventSession (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · Super: CRUD · Branch: CRUD · Events: CRUD

Event – in plain words

A date on the church calendar.
Anything with a date: youth conference, marriage conference, worship conference, youth trip, mission week.

Rules the code must keep

- registrationMode: none | form (our form) | external (link) | walk_in.
- seatsLeft = capacity - confirmed attendees (computed, not stored).
- Guest ministers come from EventGuest, so the same Minister page works for sermons and events.
- Fees are INFORMATION only (giving details); no payment at launch.

Mistakes to avoid

- A separate table for every kind of event. category + optional subtype (OutreachActivity) is enough.

EventGuest – in plain words

Speaker list for an event.
Which ministers speak at which event.

Rules the code must keep

- A guest minister attached to an event; may be an internal Minister or an external name + photo.
- Shown on the event card as "Guest: Pastor ...".

Mistakes to avoid

- Making guest ministers free text on Event: you lose "all events with this minister".

EventSession – in plain words

One slot in the conference timetable.
The day-by-day programme of a multi-day conference.

Rules the code must keep

- One event can have many sessions (Day 1 morning, Day 1 evening).
- startsAt/endsAt are timestamptz; the page renders them in Africa/Nairobi.

Mistakes to avoid

- Storing local time without timezone; a phone abroad will show the wrong hour.

How this group fits together

An Event is anything with a date. Guests and sessions hang under it.

HOW TO READ IT

Event (Events & Programmes, owned by **Events Coordinator**). A date on the church calendar. Anything with a date: youth conference, marriage conference, worship conference, youth trip, mission week.

EventGuest (Events & Programmes, owned by **Events Coordinator**). Speaker list for an event. Which ministers speak at which event.

EventSession (Events & Programmes, owned by **Events Coordinator**). One slot in the conference timetable. The day-by-day programme of a multi-day conference.

Left column = parents. These are the classes Event, EventGuest, EventSession points at. Event / EventGuest / EventSession cannot exist without them: Department, FormDefinition, Minister.

Right column = children. These tables point back at Event, EventGuest, EventSession: OutreachActivity, EventRegistration.

An Event is anything with a date. Guests and sessions hang under it.

MISTAKES TO AVOID

- A separate table for every kind of event. category + optional subtype (OutreachActivity) is enough.
- Making guest ministers free text on Event: you lose "all events with this minister".
- Storing local time without timezone; a phone abroad will show the wrong hour.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-05** Events with dates, guest ministers, sessions and registration
- **R-30** Class tabs: each main class broken into its own tab

B35

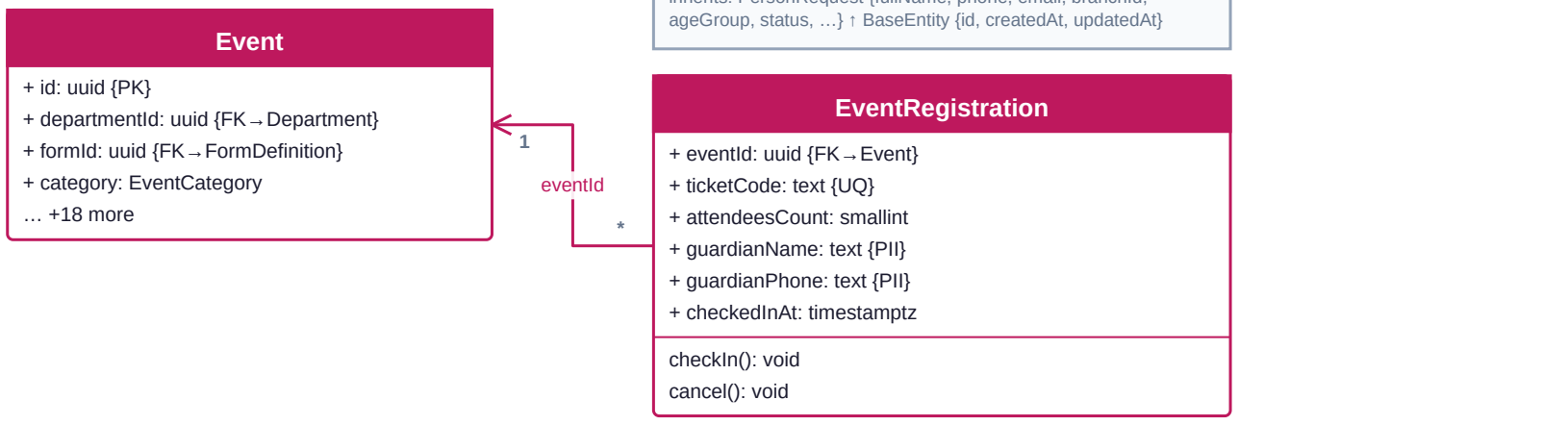
CLASS TABS (UML DETAIL) · PHASE 1

Tab: Event registration

Attributes, parents, children, rules and mistakes

Shared-kernel references (every role uses these, so no arrows are drawn)

Branch ← branchId | User ← handledBy



Depends on (platform code this class calls)

Mailer | RateLimiter

Who may do what on EventRegistration (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: S | New member: S | Super: RUX | Branch: RUX | Dept leader: R | Events: RUDX

EventRegistration – in plain words

A seat reserved in someone's name.
Someone registering for an event. A "PersonRequest" with an event attached.

Rules the code must keep

- Unique (eventId, phone) stops duplicate sign-ups.
- If event.requiresGuardianConsent and ageGroup is under 18, guardian fields are required.
- Confirmation by email (Resend) and optional SMS; ticketCode is shown and emailed.

Mistakes to avoid

- Letting minors register without a guardian contact.

How this group fits together

A visitor registers; the Events Coordinator manages the list. The class extends PersonRequest, so name, phone, consent and status are inherited.

HOW TO READ IT

EventRegistration (Events & Programmes, owned by **Events Coordinator**). A seat reserved in someone's name. Someone registering for an event. A "PersonRequest" with an event attached.

Left column = parents. These are the classes EventRegistration points at. EventRegistration cannot exist without them: Event.

A visitor registers; the Events Coordinator manages the list. The class extends PersonRequest, so name, phone, consent and status are inherited.

MISTAKES TO AVOID

- Letting minors register without a guardian contact.

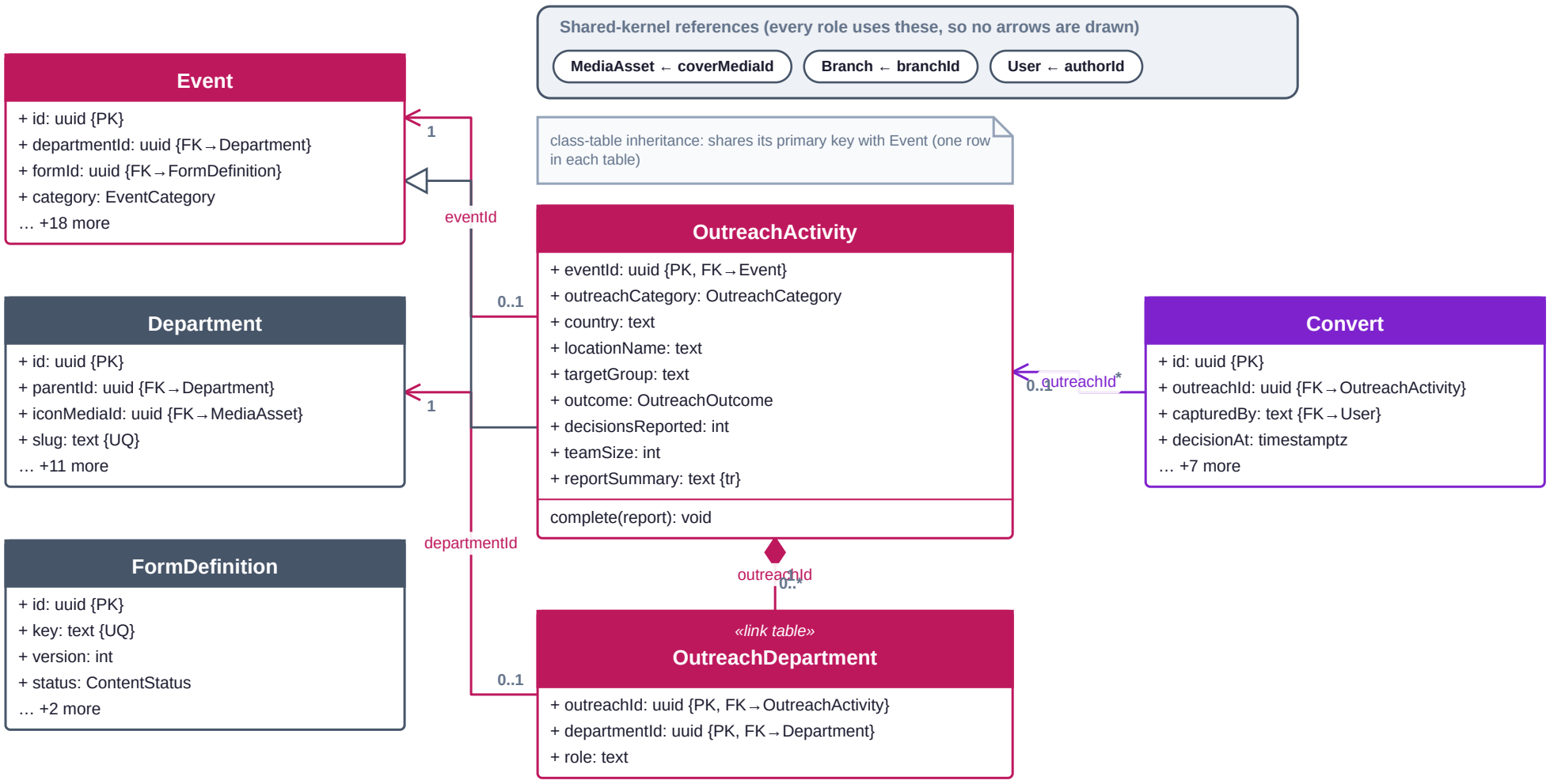
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-05** Events with dates, guest ministers, sessions and registration
- **R-30** Class tabs: each main class broken into its own tab

Tab: OutreachActivity (Go Ye)

Attributes, parents, children, rules and mistakes



Who may do what on OutreachActivity (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · **Super: CRUDP** · **Branch: CRUDP** · **Dept leader: CRU** · **Events: CRUDP**

Who may do what on OutreachDepartment (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Super: CRUD · **Branch: CRUD** · **Events: CRUD**

OutreachActivity – in plain words

An event with a mission and a result.
An Event that is also a Go Ye outreach (school, unreached community, international mission, mission week). Inherits everything from Event and adds results.

Rules the code must keep

- Shares its primary key with Event (class-table inheritance), so it appears on the main calendar for free.
- decisionsReported is what the team says; the counter on the website uses Convert records only.

Mistakes to avoid

- None recorded beyond the shared mistakes on page B03.

OutreachDepartment – in plain words

Which teams helped.
Which departments take part in an outreach.

Rules the code must keep

- Link table: which departments take part in an outreach (Youth, Missions, Children...).
- Used to build the "Departments involved" list on the Go Ye page.

Mistakes to avoid

- Duplicating department names as text.

How this group fits together

An outreach IS an event (class-table inheritance), plus mission results and the departments involved.

HOW TO READ IT

OutreachActivity (Go Ye & Discipleship, owned by **Events Coordinator**). An event with a mission and a result. An Event that is also a Go Ye outreach (school, unreached community, international mission, mission week). Inherits everything from Event and adds results.

OutreachDepartment (Go Ye & Discipleship, owned by **Events Coordinator**). Which teams helped. Which departments take part in an outreach.

Left column = parents. These are the classes OutreachActivity, OutreachDepartment points at. OutreachActivity / OutreachDepartment cannot exist without them: Event, Department, FormDefinition.

Right column = children. These tables point back at OutreachActivity, OutreachDepartment: Convert.

An outreach IS an event (class-table inheritance), plus mission results and the departments involved.

MISTAKES TO AVOID

- Duplicating department names as text.

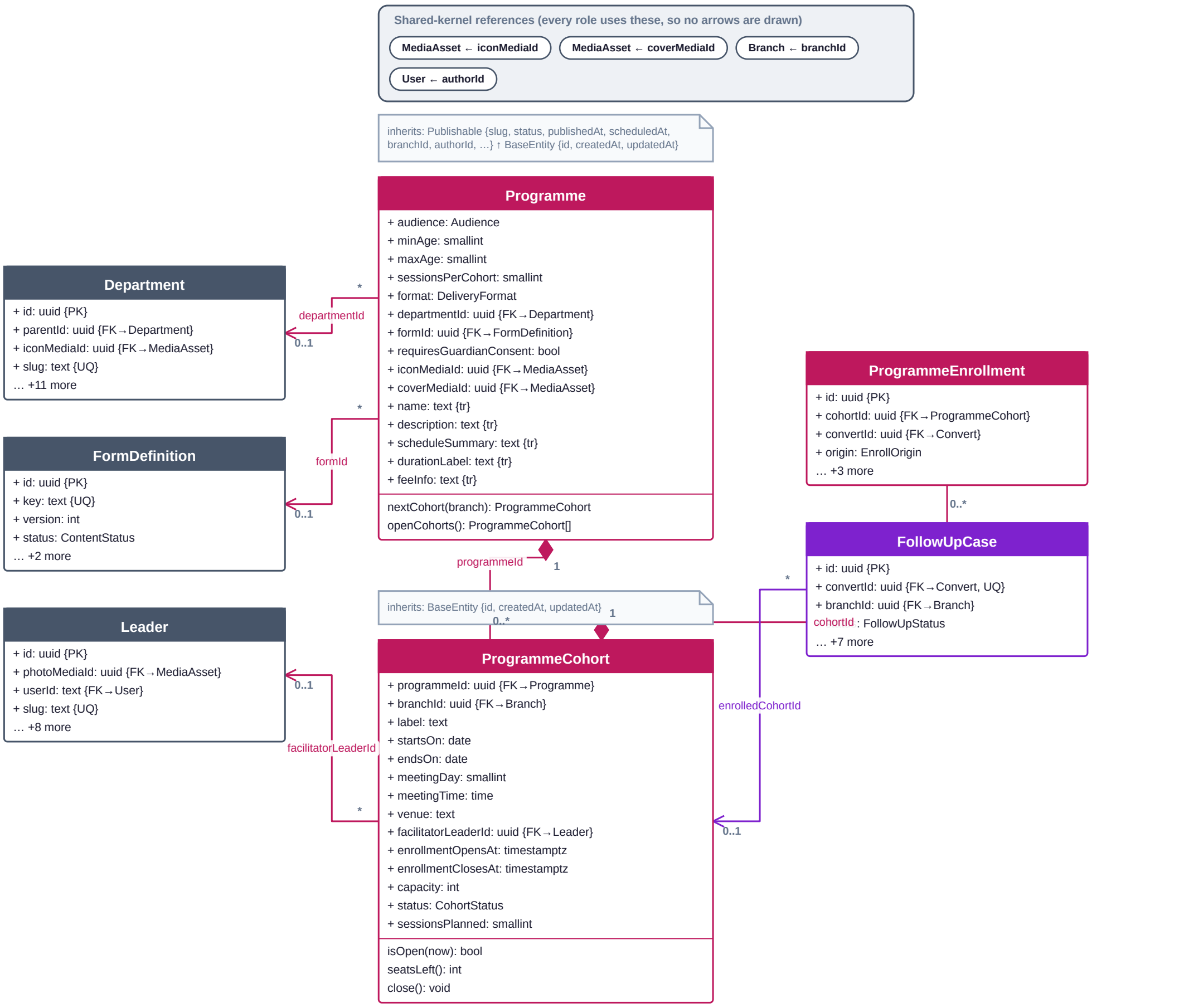
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-07** Go Ye: outreach, born-again counter, a form to add a convert
- **R-30** Class tabs: each main class broken into its own tab

Tab: Programme and cohort

Attributes, parents, children, rules and mistakes



Who may do what on Programme (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUDP** | **Branch: CRUDP** | **Dept leader: RU** | Discipler: R | **Events: CRUDP**

Who may do what on ProgrammeCohort (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | New member: R | **Super: CRUD** | **Branch: CRUD** | **Discipler: RU** | **Events: CRUD**

Programme – in plain words
The recipe. A cohort is one time you cook it.
A repeating course: Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again. Says WHAT it is and HOW OFTEN.

Rules the code must keep
• A programme has many cohorts over the years.
• A programme is the course (Kairos); a ProgrammeCohort is one run of it (Kairos Jan-Mar 2027).
• Frequency text ("every Tuesday 6pm") is translatable.

Mistakes to avoid
• Mixing the course and the run. "Kairos 2027 Term 1" is a cohort, not a programme.

ProgrammeCohort – in plain words
One class intake.
One run of a programme at one branch: "CFC - Eastern Bypass - Oct 2026". Enrolment attaches to the cohort, not the programme.

Rules the code must keep
• Visitors enrol in the NEXT OPEN cohort; the programme page shows it automatically.
• status flow: planned -> open -> full/running -> completed (or cancelled).

Mistakes to avoid
• None recorded beyond the shared mistakes on page B03.

How this group fits together
Programme = the course. Cohort = one run of it with dates and capacity. People enrol in a cohort.

HOW TO READ IT

Programme (Events & Programmes, owned by **Events Coordinator**). The recipe. A cohort is one time you cook it. A repeating course: Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again. Says WHAT it is and HOW OFTEN.

ProgrammeCohort (Events & Programmes, owned by **Events Coordinator**). One class intake. One run of a programme at one branch: "CFC - Eastern Bypass - Oct 2026". Enrolment attaches to the cohort, not the programme.

Left column = parents. These are the classes Programme, ProgrammeCohort points at. Programme / ProgrammeCohort cannot exist without them: Department, FormDefinition, Leader.

Right column = children. These tables point back at Programme, ProgrammeCohort: ProgrammeEnrollment, FollowUpCase.

Programme = the course. Cohort = one run of it with dates and capacity. People enrol in a cohort.

MISTAKES TO AVOID

- Mixing the course and the run. "Kairos 2027 Term 1" is a cohort, not a programme.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

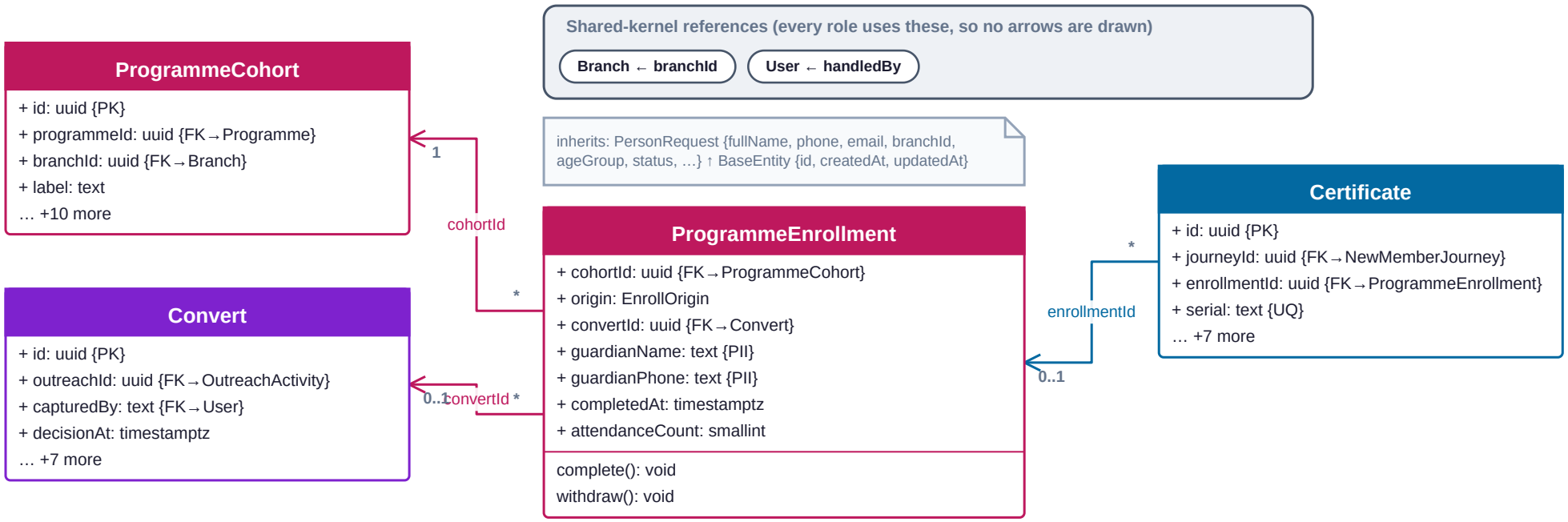
- **R-06** Programmes: Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again
- **R-30** Class tabs: each main class broken into its own tab

B38

CLASS TABS (UML DETAIL) · PHASE 1

Tab: Programme enrolment

Attributes, parents, children, rules and mistakes



Who may do what on ProgrammeEnrollment (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: S | New member: S | **Super: RUX** | **Branch: RUX** | **Discipler: CRU** | **Events: RUDX**

ProgrammeEnrollment – in plain words

A name on the class register.
A person joining a cohort. The same class serves the web form, the Go Ye hand-off and walk-ins.

Rules the code must keep

• origin = go_ye or new_member links back to the Convert or journey that caused it.
• Under-18 programmes require guardian fields.
• One person per cohort (unique on cohortId + contact).
• Status flow: requested -> confirmed -> attending -> completed | dropped.
• No certificate column here: a Certificate points back to the enrollment, not the other way round (breaks a circular dependency).

Mistakes to avoid

• Letting the same phone number enrol twice: it inflates numbers and wastes a place.

How this group fits together

The link between a person and a cohort. A Convert can be enrolled in New Born Again from the follow-up screen.

HOW TO READ IT

ProgrammeEnrollment (Events & Programmes, owned by **Events Coordinator**). A name on the class register. A person joining a cohort. The same class serves the web form, the Go Ye hand-off and walk-ins.

Left column = parents. These are the classes ProgrammeEnrollment points at. ProgrammeEnrollment cannot exist without them: ProgrammeCohort, Convert.

Right column = children. These tables point back at ProgrammeEnrollment: Certificate.

The link between a person and a cohort. A Convert can be enrolled in New Born Again from the follow-up screen.

MISTAKES TO AVOID

- Letting the same phone number enrol twice: it inflates numbers and wastes a place.

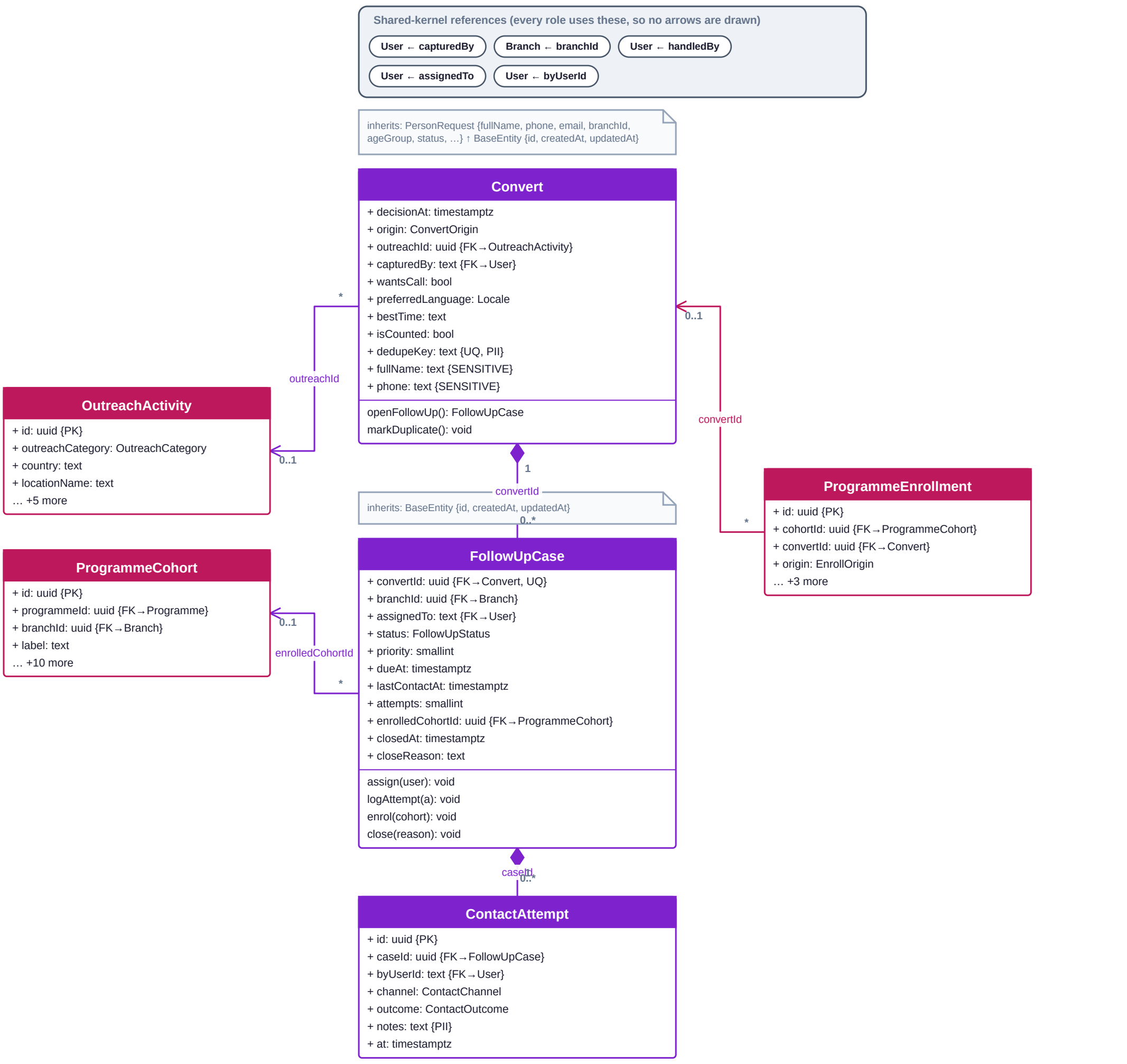
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-06** Programmes: Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again
- **R-30** Class tabs: each main class broken into its own tab

Tab: Go Ye follow-up (Convert, FollowUpCase, ContactAttempt)

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

RateLimiter · JobQueue · Mailer

Who may do what on Convert (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: S · New member: S · Super: RX · Branch: RX · Discipler: RUX

Who may do what on FollowUpCase (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Super: RU · Branch: RUD · Discipler: CRUD

Who may do what on ContactAttempt (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Super: R · Branch: R · Discipler: CRU

Convert – in plain words

A person who just said yes to Jesus and wants someone to call.
A decision to follow Christ, recorded with a name and phone so the discipleship team can reach out. This is the data behind the "born again" counter.

Rules the code must keep

- Belief is SENSITIVE personal data in Kenya: explicit consent text must be shown and stored (consentAt + consentVersion).
- dedupeKey = hash(normalised phone) stops double counting.
- The public counter = count(isCounted) + SiteSetting baseline.
- Visible only to Discipler(s) of that branch, Branch Admin and Super Admin.
- A FollowUpCase is opened automatically.

Mistakes to avoid

- Counting every "I am already born again" tick from the new-member tour. That is a profile fact, not a new decision.
- Showing the converts list to every editor. Least privilege: only disciplers.

FollowUpCase – in plain words

The folder a discipler keeps for each new believer.
The care file for one convert: who is responsible, how many attempts, where they are on the journey.

Rules the code must keep

- Target: first contact within 48 hours (dueAt); QStash reminds the discipler and escalates to the Branch Admin if overdue.
- 1:1 with Convert.
- Closing needs a reason (enrolled, completed, declined, unreachable).
- Created automatically when a Convert row is created; assigned to a Discipler of that branch.
- Status: new -> contacting -> connected -> in_programme -> closed. A case can reopen.
- Reminder jobs run through QStash (day 1, day 3, day 7).

Mistakes to avoid

- Assigning one Discipler to everything: when they are away, nobody calls the new believer.

ContactAttempt – in plain words

A line in the care diary.
Each call, SMS or visit, with the result.

Rules the code must keep

- Notes are private to disciplers and above; never exported by default.
- Append-only log: channel (call, SMS, WhatsApp, visit), outcome, note, who, when.
- Only the assigned Discipler, Branch Admin and Super Admin can read notes.

Mistakes to avoid

- Editing past notes. A log you can rewrite is not a log.

How this group fits together

Three steps: someone is recorded (Convert), a case is opened for a Discipler (FollowUpCase), each call is logged (ContactAttempt).

HOW TO READ IT

Convert (Go Ye & Discipleship, owned by **Discipler (Follow-up)**). A person who just said yes to Jesus and wants someone to call. A decision to follow Christ, recorded with a name and phone so the discipleship team can reach out. This is the data behind the "born again" counter.

FollowUpCase (Go Ye & Discipleship, owned by **Discipler (Follow-up)**). The folder a discipler keeps for each new believer. The care file for one convert: who is responsible, how many attempts, where they are on the journey.

ContactAttempt (Go Ye & Discipleship, owned by **Discipler (Follow-up)**). A line in the care diary. Each call, SMS or visit, with the result.

Left column = parents. These are the classes Convert, FollowUpCase, ContactAttempt points at. Convert / FollowUpCase / ContactAttempt cannot exist without them: OutreachActivity, ProgrammeCohort.

Right column = children. These tables point back at Convert, FollowUpCase, ContactAttempt: ProgrammeEnrollment.

Three steps: someone is recorded (Convert), a case is opened for a Discipler (FollowUpCase), each call is logged (ContactAttempt).

MISTAKES TO AVOID

- Counting every "I am already born again" tick from the new-member tour. That is a profile fact, not a new decision.
- Showing the converts list to every editor. Least privilege: only disciplers.
- Assigning one Discipler to everything: when they are away, nobody calls the new believer.
- Editing past notes. A log you can rewrite is not a log.

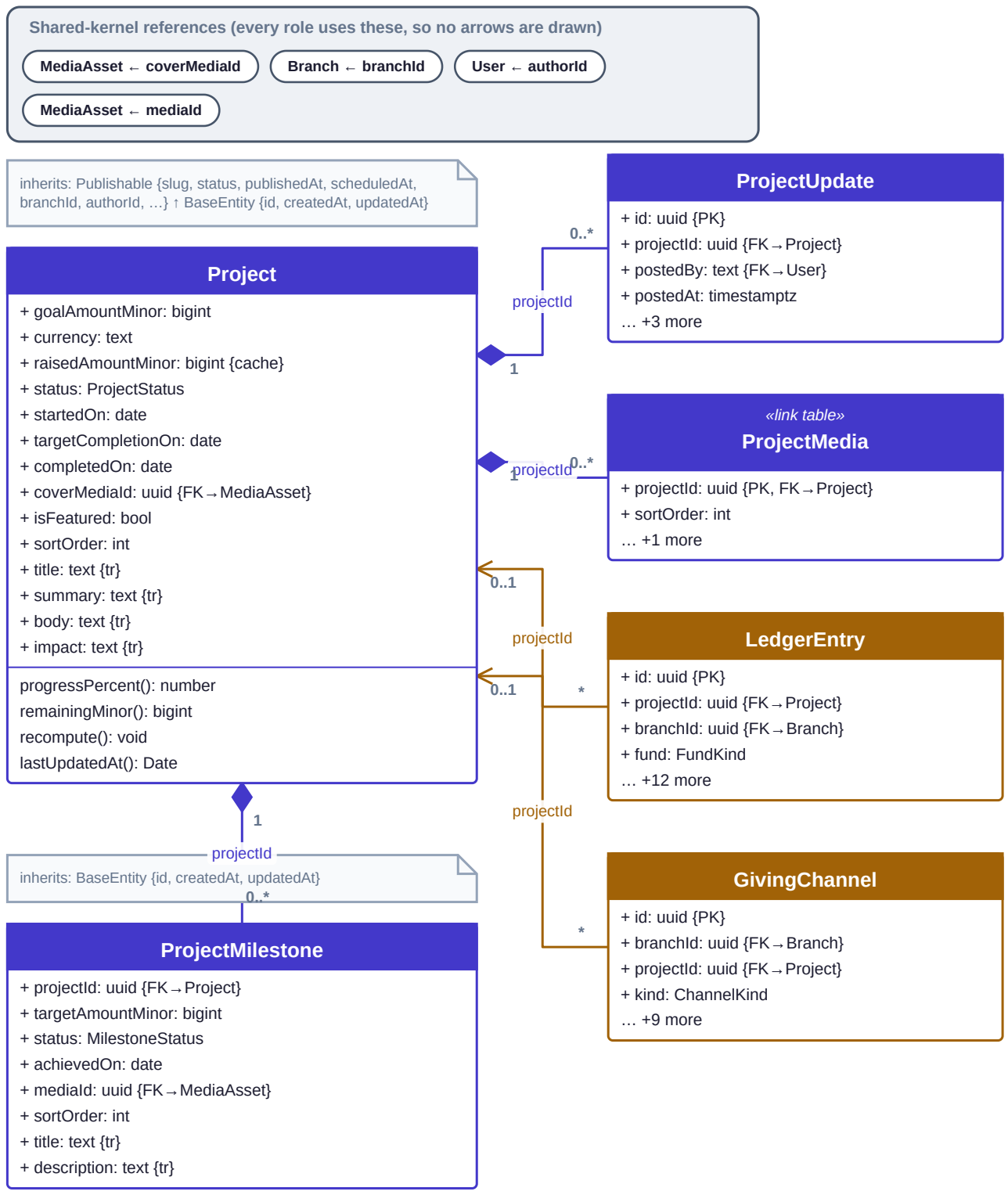
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-07** Go Ye: outreach, born-again counter, a form to add a convert
- **R-23** Follow-up of new believers by a discipler
- **R-30** Class tabs: each main class broken into its own tab

Tab: Project and milestones

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

CacheService · TranslationService

Who may do what on Project (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · New member: R · **Super: CRUDP** · **Branch: CRUDP** · **Editor: RU** · **Translator: RU** · Treasurer: R

Who may do what on ProjectMilestone (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R · **Super: CRUD** · **Branch: CRUD** · **Editor: RU** · Treasurer: R

Project – in plain words

A big thing the church is building, with a thermometer.
A church building or mission project with a goal, milestones, impact story and a progress bar.

Rules the code must keep

• Money is stored in the smallest unit (cents) as bigint: KES 1,500.00 = 150000. Never floats.
• raisedAmountMinor is a CACHE = SUM(ledger where status = recorded); recompute() runs inside the same transaction as every ledger write.
• Percent is capped at 100 for display; "remaining" never goes below 0.
• "Last updated" = newest of ProjectUpdate.postedAt and LedgerEntry.recordedAt.

Mistakes to avoid

• Using decimals/floats for money. 0.1 + 0.2 is not 0.3 in a computer.
• Letting editors change goalAmount silently. It is audited and shown in the update log.

ProjectMilestone – in plain words

A step on the staircase.
Stages of a project (land bought, foundation, roof...). Completed ones are ticked on the project page.

Rules the code must keep

• Ordered steps with a target amount or date; "done" is set by Branch Admin.
• Percent complete = done milestones / all, shown beside the money bar.

Mistakes to avoid

• Calculating "percent complete" from money only. A project can be 80% funded and 10% built.

How this group fits together

A project has a goal amount and milestones. The progress bar compares raised vs goal; milestones show build progress.

HOW TO READ IT

Project (Giving & Projects, owned by **Branch Admin**). A big thing the church is building, with a thermometer. A church building or mission project with a goal, milestones, impact story and a progress bar.

ProjectMilestone (Giving & Projects, owned by **Branch Admin**). A step on the staircase. Stages of a project (land bought, foundation, roof...). Completed ones are ticked on the project page.

Right column = children. These tables point back at Project, ProjectMilestone: ProjectUpdate, ProjectMedia, LedgerEntry, GivingChannel.

A project has a goal amount and milestones. The progress bar compares raised vs goal; milestones show build progress.

MISTAKES TO AVOID

- Using decimals/floats for money. 0.1 + 0.2 is not 0.3 in a computer.
- Letting editors change goalAmount silently. It is audited and shown in the update log.
- Calculating "percent complete" from money only. A project can be 80% funded and 10% built.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

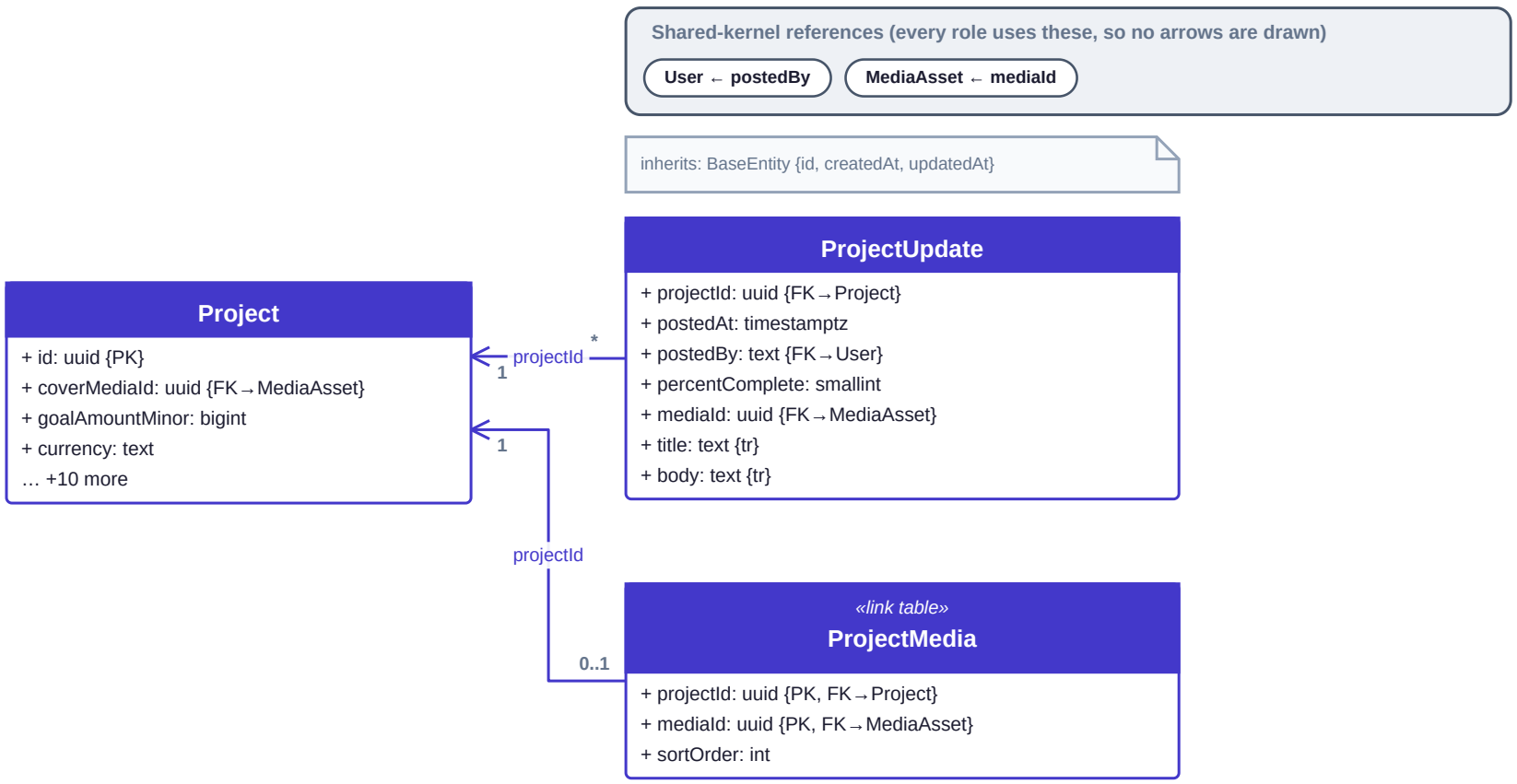
- **R-08** Projects with milestones and a given / remaining bar
- **R-30** Class tabs: each main class broken into its own tab

B41

CLASS TABS (UML DETAIL) · PHASE 1

Tab: Project updates and media

Attributes, parents, children, rules and mistakes



Who may do what on ProjectUpdate (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | **Super: CRUD** | **Branch: CRUD** | **Editor: CRU** | **Treasurer: CR**

Who may do what on ProjectMedia (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: R | **Super: CRUD** | **Branch: CRUD** | **Editor: CRUD**

ProjectUpdate – in plain words
A progress note with a date.
The "latest news" log at the bottom of a project page; also gives the "last updated" date.

Rules the code must keep
• A dated post ("Roofing completed") with optional photos; its createdAt feeds the "last updated" label.
• Public. Written by Branch Admin or Content Editor.

Mistakes to avoid
• Forgetting to update: a stale "last updated" date kills donor trust faster than no page at all.

ProjectMedia – in plain words
Photos for the project page.
Photo gallery of a project.

Rules the code must keep
• Link table between Project and MediaAsset with sortOrder and caption (translatable).

Mistakes to avoid
• Uploading full 6MB camera photos. Resize on upload (StorageService).

How this group fits together
Updates give the "last updated" date and the story; media gives photos.

HOW TO READ IT

ProjectUpdate (Giving & Projects, owned by **Branch Admin**). A progress note with a date. The "latest news" log at the bottom of a project page; also gives the "last updated" date.

ProjectMedia (Giving & Projects, owned by **Branch Admin**). Photos for the project page. Photo gallery of a project.

Left column = parents. These are the classes ProjectUpdate, ProjectMedia points at. ProjectUpdate / ProjectMedia cannot exist without them: Project.

Updates give the "last updated" date and the story; media gives photos.

MISTAKES TO AVOID

- Forgetting to update: a stale "last updated" date kills donor trust faster than no page at all.
- Uploading full 6MB camera photos. Resize on upload (StorageService).

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

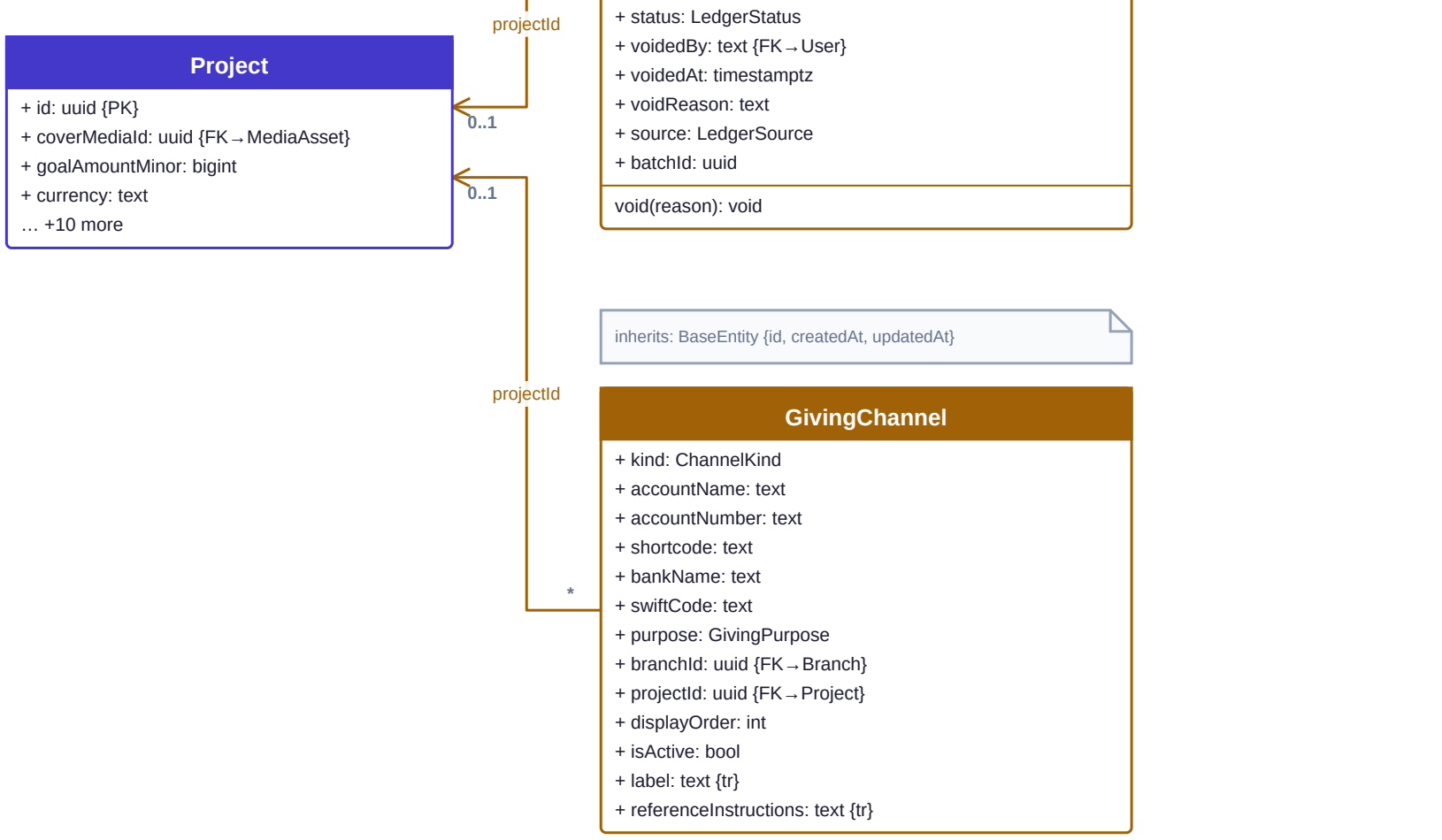
- **R-08** Projects with milestones and a given / remaining bar
- **R-30** Class tabs: each main class broken into its own tab

Tab: Ledger and giving channels

Attributes, parents, children, rules and mistakes

Shared-kernel references (every role uses these, so no arrows are drawn)

- Branch ← branchId
- User ← recordedBy
- User ← voidedBy



Depends on (platform code this class calls)

- AuthzService
- CacheService

Who may do what on LedgerEntry (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: RX
- Branch: R
- Treasurer: CRUX

Who may do what on GivingChannel (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Visitor: R
- New member: R
- Super: CRUD
- Branch: RA
- Treasurer: CRU

LedgerEntry – in plain words

A line in the church's cash book.
One gift recorded by the treasurer. The ONLY source of truth for the progress bars.

Rules the code must keep

- Append-only: mistakes are VOIDED with a reason, never edited or deleted.
- amountMinor > 0 (CHECK).
- donorLabel defaults to "Anonymous"; the church sees only what it needs.
- source = manual today; "gateway" is the reserved slot for M-Pesa/Paystack later, so no schema change is needed.
- Every insert and void writes AuditLog and recomputes Project.raisedAmountMinor.

Mistakes to avoid

- Editing the total on the project instead of adding a ledger row. Then nobody can explain the number.

GivingChannel – in plain words

The church's payment addresses.
The "how to give" details shown on the contact page and on each project: paybill, till, bank, international.

Rules the code must keep

- Changing a paybill number or bank account is a high-risk edit: two-person approval (Treasurer proposes, Branch Admin confirms) and an email alert.
- projectId set = shown on that project page too.

Mistakes to avoid

- Letting any editor change account numbers. This is the #1 church-website fraud target.

How this group fits together

GivingChannel = where to give (M-Pesa Paybill, bank). LedgerEntry = the treasurer's record of money received for a project; append-only.

HOW TO READ IT

LedgerEntry (Giving & Projects, owned by **Treasurer**). A line in the church's cash book. One gift recorded by the treasurer. The ONLY source of truth for the progress bars.

GivingChannel (Giving & Projects, owned by **Treasurer**). The church's payment addresses. The "how to give" details shown on the contact page and on each project: paybill, till, bank, international.

Left column = parents. These are the classes LedgerEntry, GivingChannel points at. LedgerEntry / GivingChannel cannot exist without them: Project.

GivingChannel = where to give (M-Pesa Paybill, bank). LedgerEntry = the treasurer's record of money received for a project; append-only.

MISTAKES TO AVOID

- Editing the total on the project instead of adding a ledger row. Then nobody can explain the number.
- Letting any editor change account numbers. This is the #1 church-website fraud target.

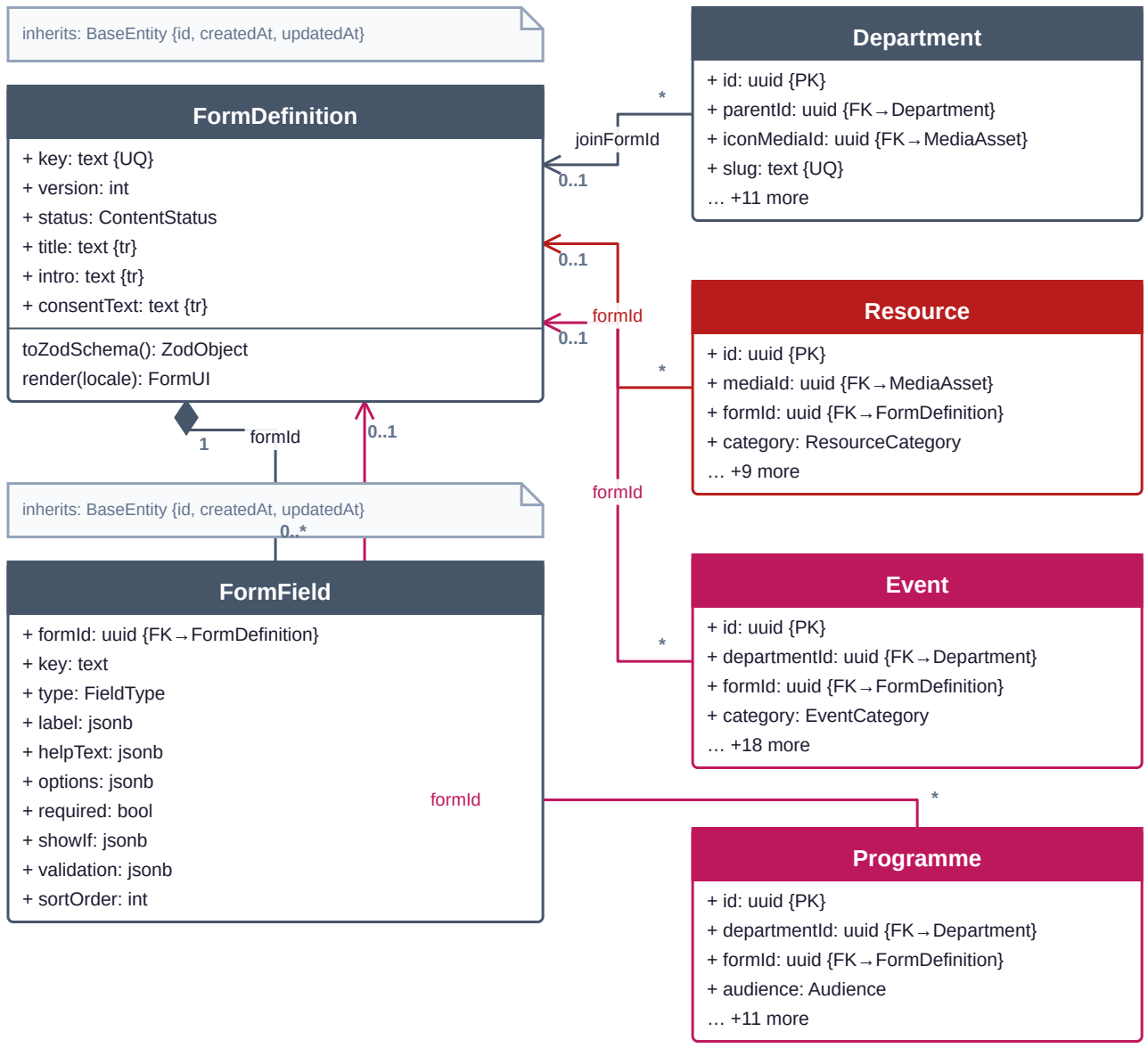
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-09** Contact page with giving details
- **R-24** Treasurer's record of gifts that drives the project bars
- **R-30** Class tabs: each main class broken into its own tab

Tab: Form builder (FormDefinition, FormField)

Attributes, parents, children, rules and mistakes



Who may do what on FormDefinition (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Super: CRUDP | Branch: R | Editor: RU | Dept leader: RU | Events: RU

Who may do what on FormField (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Super: CRUD | Branch: R | Dept leader: RU | Events: RU

FormDefinition – in plain words

A blank form that staff can design.
A small form builder. Event registrations, programme enrolment, department follow-up questions and mission applications all use it.

Rules the code must keep

- A published form is immutable: edits create version + 1, old submissions keep their version.
- Fields support conditional visibility (showIf).
- Describes a configurable form (new-member tour questions, programme enrolment questions).
- Version number increases on every publish; old answers keep the version they were collected under.

Mistakes to avoid

- Editing a published form in place: old answers no longer match the questions. Always publish a new version.

FormField – in plain words

One question on the form.
One question on a form. Labels and options are small JSON objects with an English and a Kiswahili text.

Rules the code must keep

- Field types: text, textarea, select, radio, checkbox, date, phone, email, consent.
- Labels and help texts are JSONB per locale: {"en": "...", "sw": "..."}.
- Conditional rule "showIf" is stored as a small JSON expression, evaluated by one shared function on server and client.

Mistakes to avoid

- Storing the label as the answer key. Use the field id, so renaming a label does not orphan answers.

How this group fits together

The new-member tour questions and the programme enrolment questions are data, so the church can change them without a programmer.

HOW TO READ IT

FormDefinition (New Member & Forms, owned by **Shared Kernel**). A blank form that staff can design. A small form builder. Event registrations, programme enrolment, department follow-up questions and mission applications all use it.

FormField (New Member & Forms, owned by **Shared Kernel**). One question on the form. One question on a form. Labels and options are small JSON objects with an English and a Kiswahili text.

Right column = children. These tables point back at FormDefinition, FormField: Department, Resource, Event, Programme.

The new-member tour questions and the programme enrolment questions are data, so the church can change them without a programmer.

MISTAKES TO AVOID

- Editing a published form in place: old answers no longer match the questions. Always publish a new version.
- Storing the label as the answer key. Use the field id, so renaming a label does not orphan answers.

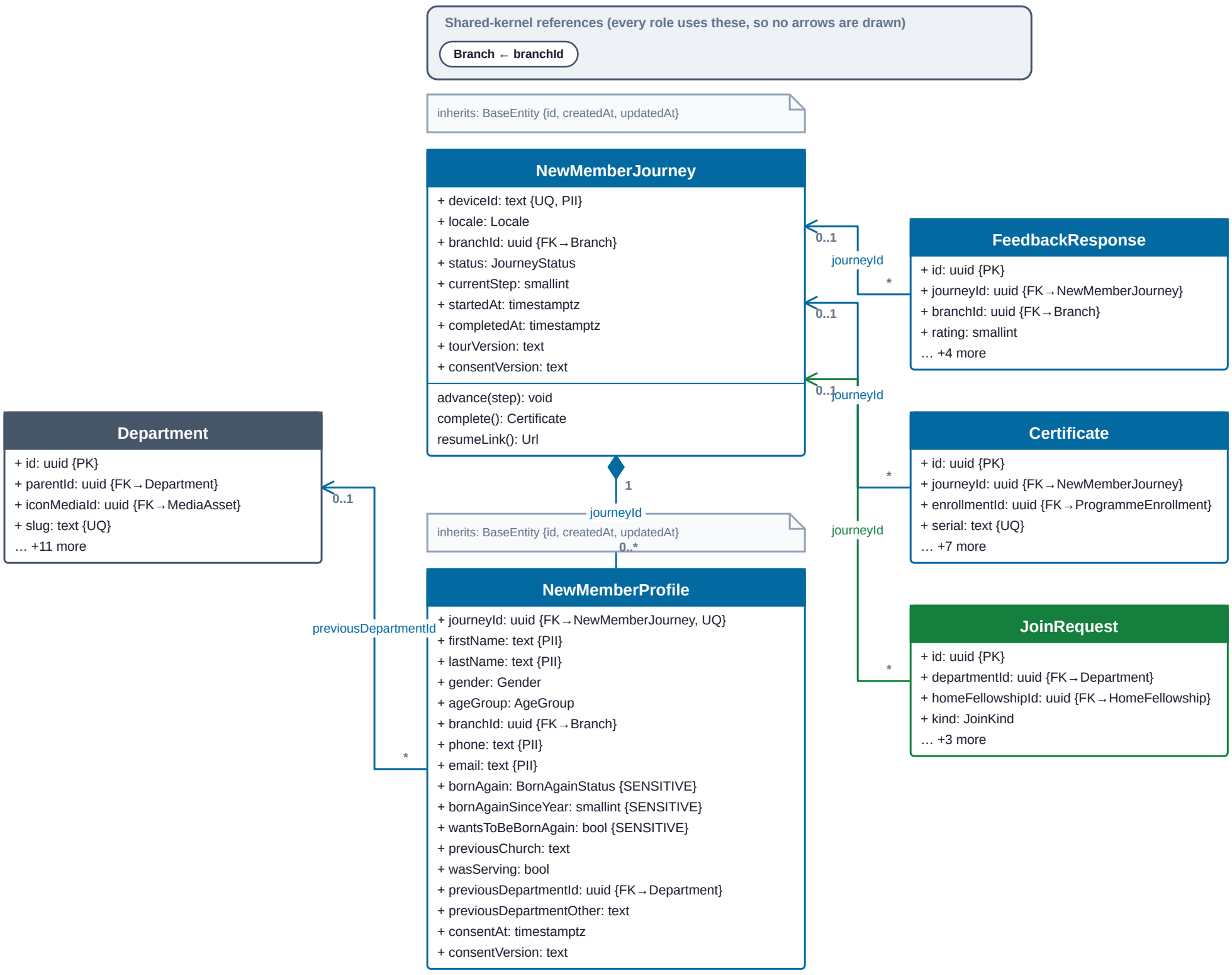
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-30** Class tabs: each main class broken into its own tab

Tab: New-member journey and profile

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

- OverlayOrchestrator
- CertificateService

Who may do what on NewMemberJourney (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- New member: CRU
- Super: R
- Discipler: R

Who may do what on NewMemberProfile (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- New member: CRU
- Super: RX
- Branch: R
- Discipler: R

NewMemberJourney – in plain words

A bookmark that remembers where the newcomer is in the tour.
The server-side memory of one person's tour, created ONLY after they say Yes and give consent. "No" is remembered on their device only.

Rules the code must keep

- No server record for "No, thanks": it lives in localStorage (+ cookie if cookies accepted).
- deviceId is a random id from the browser, stored hashed; progress survives a refresh.
- Abandoned journeys older than 90 days are anonymised.
- Created ONLY after the visitor says Yes and accepts consent. "No" never reaches the server.
- Stores progress as currentStep + completedSteps so a refresh resumes where they left.
- Anonymous token in an httpOnly cookie links the browser to the journey until contact details are given.

Mistakes to avoid

- Saving data before consent. A "No" must leave no trace on the server.
- Asking all nine steps at once on one long page: people leave. One short step at a time.

NewMemberProfile – in plain words

The newcomer's "about me" card.
What the newcomer tells us about themselves in the tour.

Rules the code must keep

- born-again fields are sensitive belief data: collected only with explicit consent and shown only to Disciplers.
- If wantsToBeBornAgain is true, a Convert + FollowUpCase are created in the same transaction.

Mistakes to avoid

- Making every field mandatory. Only name, age group and branch are required; everything else is optional.

How this group fits together

Journey = where the person is in the tour. Profile = what they told us about themselves.

HOW TO READ IT

NewMemberJourney (New Member & Forms, owned by **New Member (public)**). A bookmark that remembers where the newcomer is in the tour. The server-side memory of one person's tour, created ONLY after they say Yes and give consent. "No" is remembered on their device only.

NewMemberProfile (New Member & Forms, owned by **New Member (public)**). The newcomer's "about me" card. What the newcomer tells us about themselves in the tour.

Left column = parents. These are the classes NewMemberJourney, NewMemberProfile points at. NewMemberJourney / NewMemberProfile cannot exist without them: Department.

Right column = children. These tables point back at NewMemberJourney, NewMemberProfile: FeedbackResponse, Certificate, JoinRequest.

Journey = where the person is in the tour. Profile = what they told us about themselves.

MISTAKES TO AVOID

- Saving data before consent. A "No" must leave no trace on the server.
- Asking all nine steps at once on one long page: people leave. One short step at a time.
- Making every field mandatory. Only name, age group and branch are required; everything else is optional.

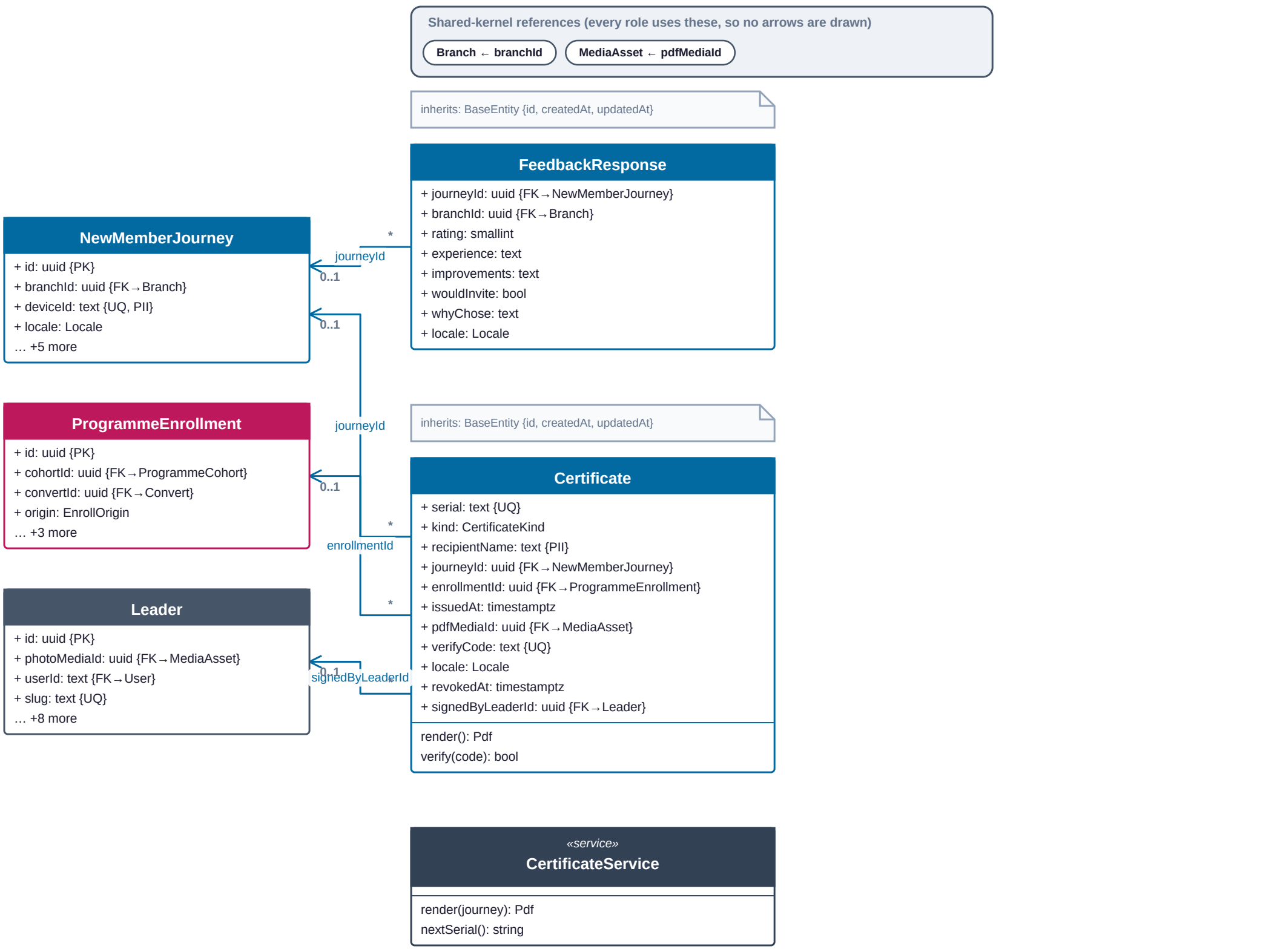
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-14** New-member tour of nine steps, ending in a certificate
- **R-30** Class tabs: each main class broken into its own tab

Tab: Feedback and certificate

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

StorageService

Called by (classes that depend on this service)

NewMemberJourney | Certificate

Who may do what on FeedbackResponse (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

New member: C | Super: RX | Branch: R | Editor: R

Who may do what on Certificate (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

New member: CR | Super: RUD | Branch: R

FeedbackResponse – in plain words

The suggestion box.
The end-of-tour questionnaire: what they experienced, what to improve, would they invite a friend and why DCEB.

Rules the code must keep

- One per completed journey; answers in JSONB keyed by FormField id.
- Read by Super Admin, Branch Admin and Content Editor for improvement only.

Mistakes to avoid

- Showing feedback with names to the whole team. Show it grouped, names only to the Branch Admin.

Certificate – in plain words

A printable "well done" paper with a serial number.
The "Completion certificate" generated when the tour ends (later also programme completion).

Rules the code must keep

- serial like DCEB-2026-000123 (year and a running number); verifyCode lets anyone check authenticity at /verify/CODE.
- PDF is stored PRIVATE in R2 and downloaded with a short-lived signed URL.
- Rendered once by a QStash job, then cached.
- Generated once from a PDF template, stored in a PRIVATE R2 bucket and served by a short-lived signed URL.
- serial number is unique and printed on the PDF so it can be verified.
- Name on the certificate is exactly what the member typed (they confirm spelling before the PDF is made).

Mistakes to avoid

- Storing certificates in a public bucket with a guessable URL.
- Rendering the PDF in the browser only: a phone may fail; build it on the server.

CertificateService – in plain words

The certificate printer: fills the template with the name and saves the PDF safely.
Makes the completion PDF.

Rules the code must keep

- Builds the PDF (react-pdf or pdf-lib) from the template, uploads to R2, stores Certificate row, returns a signed URL.
- Idempotent: calling twice for the same journey returns the same certificate.

Mistakes to avoid

- Generating the PDF in a request that can time out. Queue it and poll.

How this group fits together

At the end of the tour the answers are saved and a certificate PDF is made and downloaded.

HOW TO READ IT

FeedbackResponse (New Member & Forms, owned by **New Member (public)**). The suggestion box. The end-of-tour questionnaire: what they experienced, what to improve, would they invite a friend and why DCEB.

Certificate (New Member & Forms, owned by **New Member (public)**). A printable "well done" paper with a serial number. The "Completion certificate" generated when the tour ends (later also programme completion).

CertificateService (Platform, owned by **New Member (public)**). The certificate printer: fills the template with the name and saves the PDF safely. Makes the completion PDF.

Left column = parents. These are the classes FeedbackResponse, Certificate, CertificateService points at. FeedbackResponse / Certificate / CertificateService cannot exist without them: NewMemberJourney, ProgrammeEnrollment, Leader.

At the end of the tour the answers are saved and a certificate PDF is made and downloaded.

MISTAKES TO AVOID

- Showing feedback with names to the whole team. Show it grouped, names only to the Branch Admin.
- Storing certificates in a public bucket with a guessable URL.
- Rendering the PDF in the browser only: a phone may fail; build it on the server.
- Generating the PDF in a request that can time out. Queue it and poll.

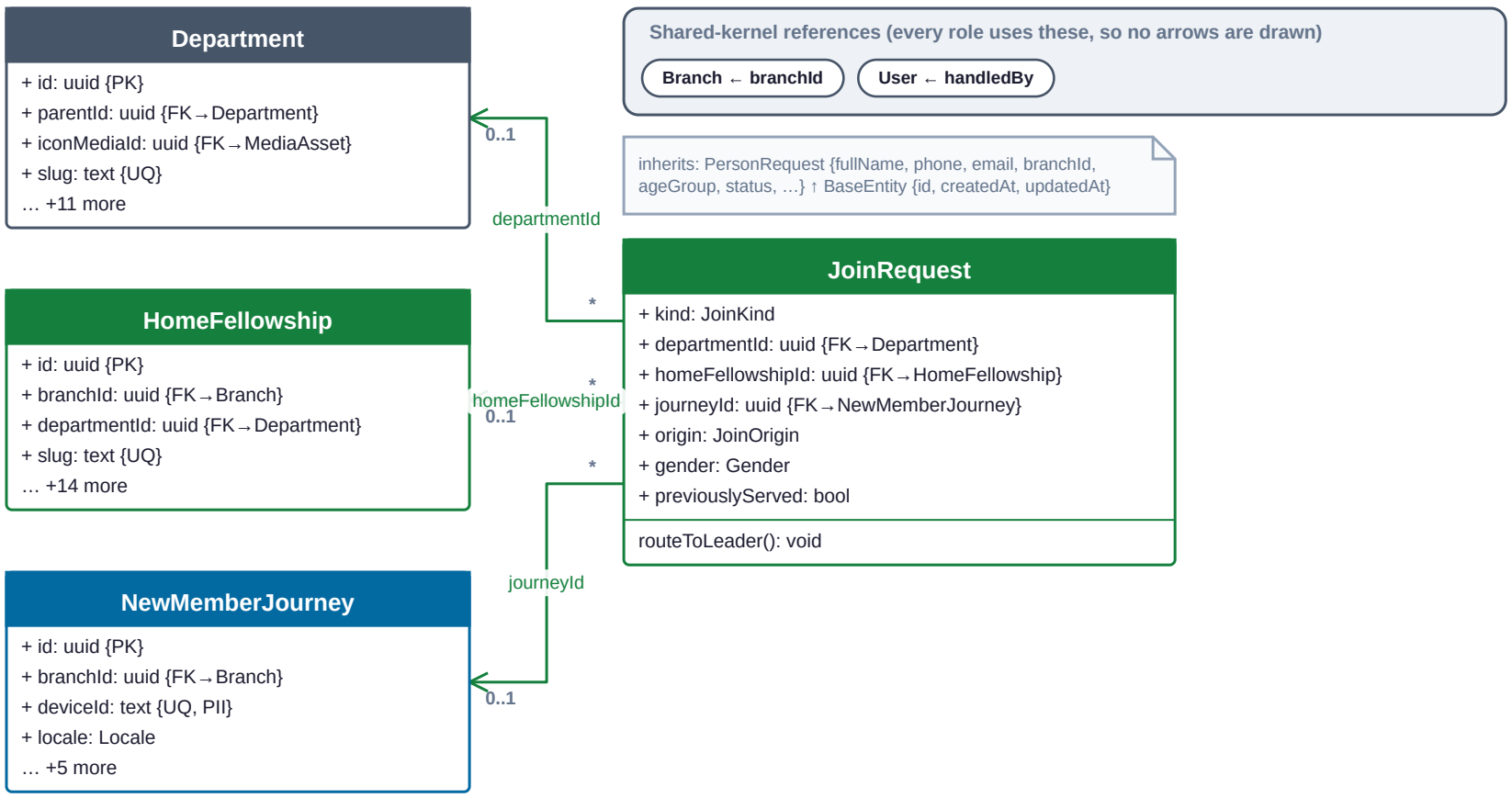
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-14** New-member tour of nine steps, ending in a certificate
- **R-30** Class tabs: each main class broken into its own tab

Tab: JoinRequest

Attributes, parents, children, rules and mistakes



Depends on (platform code this class calls)

Mailer · JobQueue

Who may do what on JoinRequest (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

Visitor: S · New member: S · **Super: RUX** · **Branch: RUX** · **Dept leader: RUX** · Discipler: R

JoinRequest – in plain words

A hand raised: "count me in".
"I want to join the Youth fellowship / this home fellowship / the missions team." One class for every kind of join.

Rules the code must keep

- kind = department | home_fellowship | mission_team | fellowship_group; exactly one target column is set (CHECK).
- Notifies the matching Department Leader or fellowship contact by email/SMS.
- Men / Women / Youth suggestions use gender + ageGroup from the tour.
- Types: department, fellowship, home_fellowship, mission_team.
- Routed to the Department Leader of that department, or the Branch Admin when no leader is set.

Mistakes to avoid

- Sending every request to the Super Admin. Route to the right leader or nobody will answer.

How this group fits together

One table for "I want to join a department / fellowship / home fellowship / mission team".

HOW TO READ IT

JoinRequest (New Member & Forms, owned by **Department Leader**). A hand raised: "count me in". "I want to join the Youth fellowship / this home fellowship / the missions team." One class for every kind of join.

Left column = parents. These are the classes JoinRequest points at. JoinRequest cannot exist without them: Department, HomeFellowship, NewMemberJourney.

One table for "I want to join a department / fellowship / home fellowship / mission team".

MISTAKES TO AVOID

- Sending every request to the Super Admin. Route to the right leader or nobody will answer.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-03** Departments with sub-departments, leaders and a join form
- **R-30** Class tabs: each main class broken into its own tab

Tab: Newsletter

Attributes, parents, children, rules and mistakes

Shared-kernel references (every role uses these, so no arrows are drawn)

- Branch ← branchId
- User ← createdBy

inherits: BaseEntity {id, createdAt, updatedAt}

NewsletterSubscriber

+ email: text {UQ, PII}
+ name: text {PII}
+ locale: Locale
+ branchId: uuid {FK→Branch}
+ status: SubscriberStatus
+ source: SubSource
+ confirmTokenHash: text {PII}
+ confirmedAt: timestamptz
+ consentAt: timestamptz
+ consentVersion: text
+ unsubscribedAt: timestamptz
+ resendContactId: text
+ topics: text[]

confirm(token): void
unsubscribe(): void

inherits: BaseEntity {id, createdAt, updatedAt}

NewsletterCampaign

+ kind: CampaignKind
+ status: CampaignStatus
+ audienceLocale: Locale
+ scheduledAt: timestamptz
+ sentAt: timestamptz
+ recipientCount: int
+ resendBroadcastId: text
+ createdBy: text {FK→User}
+ stats: jsonb
+ subject: text {tr}
+ body: text {tr}

Depends on (platform code this class calls)

- Mailer
- RateLimiter

Who may do what on NewsletterSubscriber (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Visitor: S
- New member: S
- **Super: RUDX**
- Branch: R
- **Editor: RUDX**

Who may do what on NewsletterCampaign (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- **Super: CRUDP**
- **Editor: CRUDP**

NewsletterSubscriber – in plain words

A name on the mailing list.
Someone who gave their email in the popup or footer.

Rules the code must keep

• Double opt-in: status stays pending until they click the email link.
• Every email carries a one-click unsubscribe; bounces and complaints from Resend update status via webhook.
• Never email minors flagged in a form.
• Double opt-in: status pending until the link in the confirmation e-mail is clicked.
• Every e-mail carries a one-click unsubscribe link (token).
• Subscribers are per branch or "all branches".

Mistakes to avoid

• Adding people from the contact form to the newsletter without asking. It is illegal under the Kenya Data Protection Act consent rule and gets your domain blocked.

NewsletterCampaign – in plain words

One newsletter issue.
One email sent to the list (weekly digest or special message).

Rules the code must keep

• A campaign is a draft -> scheduled -> sending -> sent record; sending is done in batches by QStash so Resend limits are respected.
• Test-send to yourself is mandatory before "schedule".

Mistakes to avoid

• Sending 2000 emails in one loop inside a request: it times out half way. Use batches.

How this group fits together

Subscribers confirm by e-mail (double opt-in). Campaigns are sent by Resend in batches.

HOW TO READ IT

NewsletterSubscriber (Engagement, owned by **Content Editor**). A name on the mailing list. Someone who gave their email in the popup or footer.

NewsletterCampaign (Engagement, owned by **Content Editor**). One newsletter issue. One email sent to the list (weekly digest or special message).

Subscribers confirm by e-mail (double opt-in). Campaigns are sent by Resend in batches.

MISTAKES TO AVOID

- Adding people from the contact form to the newsletter without asking. It is illegal under the Kenya Data Protection Act consent rule and gets your domain blocked.
- Sending 2000 emails in one loop inside a request: it times out half way. Use batches.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-16** Newsletter with a confirmed sign-up
- **R-30** Class tabs: each main class broken into its own tab

Tab: Contact, consent, notification log

Attributes, parents, children, rules and mistakes

Shared-kernel references (every role uses these, so no arrows are drawn)

- **Branch ← branchId**
- **User ← assignedTo**

inherits: BaseEntity {id, createdAt, updatedAt}

ContactMessage

+ name: text {PII}
+ email: text {PII}
+ phone: text {PII}
+ topic: ContactTopic
+ branchId: uuid {FK→Branch}
+ message: text
+ status: MessageStatus
+ assignedTo: text {FK→User}
+ ipHash: text {PII}
+ repliedAt: timestamptz
+ locale: Locale

inherits: BaseEntity {id, createdAt, updatedAt}

ConsentRecord

+ visitorId: text {PII}
+ categories: jsonb
+ policyVersion: text
+ source: text
+ userAgentHash: text {PII}

inherits: BaseEntity {id, createdAt, updatedAt}

NotificationLog

+ channel: NotifyChannel
+ template: text
+ toMasked: text
+ status: NotifyStatus
+ providerMessageId: text
+ error: text
+ relatedType: text
+ relatedId: text
+ attempts: smallint
+ sentAt: timestamptz

Who may do what on ContactMessage (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Visitor: S
- **Super: RUD**
- **Branch: RUD**
- Discipler: R
- Treasurer: R

Who may do what on ConsentRecord (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Visitor: S
- New member: S
- Super: R

Who may do what on NotificationLog (C create, R read, U update, D delete, P publish, A approve, X export, S submit)

- Super: R
- Branch: R

ContactMessage – in plain words
A letter in the church mailbox.
The contact form submissions.

Rules the code must keep
• Prayer and counselling topics route to a pastor, giving questions to the Treasurer, "visit us" to the branch admin.
• Public form (name, e-mail/phone, topic, message). Rate-limited + Turnstile.
• Routed to the Branch Admin inbox by chosen branch and topic; a copy goes to e-mail.

Mistakes to avoid
• Only e-mailing the message and not saving it: if the e-mail bounces the message is gone.

ConsentRecord – in plain words
A receipt that says "yes, you may use cookies".
Proof that a visitor accepted (or declined) cookie categories on a given policy version.

Rules the code must keep
• Necessary cookies need no consent; analytics (PostHog) only after consent.
• Re-ask when policyVersion changes.
• Stores WHAT was agreed (cookies analytics, newsletter, new-member data), WHEN, which policy version and the anonymised IP hash.
• Withdrawing consent adds a new row; old rows are never edited.

Mistakes to avoid
• Overwriting the old consent row when someone changes their mind. You lose the proof.

NotificationLog – in plain words
The post-office receipt book.
Every email/SMS the system tries to send, with the result. The first place to look when "I never got the email".

Rules the code must keep
• One row per e-mail/SMS attempt: template, to (hashed), provider id, status, error.
• Used to answer "did the member get the certificate e-mail?".

Mistakes to avoid
• Logging the whole e-mail body with personal data. Log template + id only.

How this group fits together
The three "paper trail" classes: what visitors wrote, what they agreed to, and what the system sent back.

HOW TO READ IT

ContactMessage (Engagement, owned by **Branch Admin**). A letter in the church mailbox. The contact form submissions.

ConsentRecord (Engagement, owned by **Visitor (public)**). A receipt that says "yes, you may use cookies". Proof that a visitor accepted (or declined) cookie categories on a given policy version.

NotificationLog (Engagement, owned by **Super Admin**). The post-office receipt book. Every email/SMS the system tries to send, with the result. The first place to look when "I never got the email".

The three "paper trail" classes: what visitors wrote, what they agreed to, and what the system sent back.

MISTAKES TO AVOID

- Only e-mailing the message and not saving it: if the e-mail bounces the message is gone.
- Overwriting the old consent row when someone changes their mind. You lose the proof.
- Logging the whole e-mail body with personal data. Log template + id only.

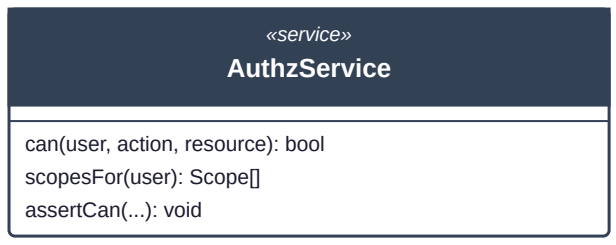
WHAT YOU ALREADY DID RIGHT

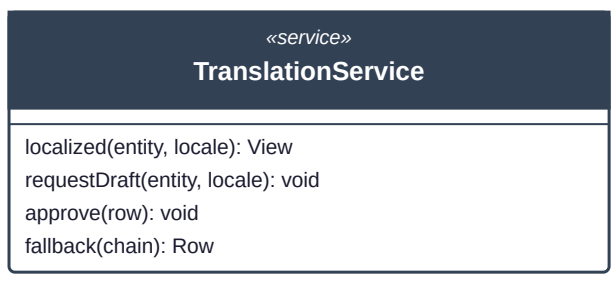
This page serves your own requirements:

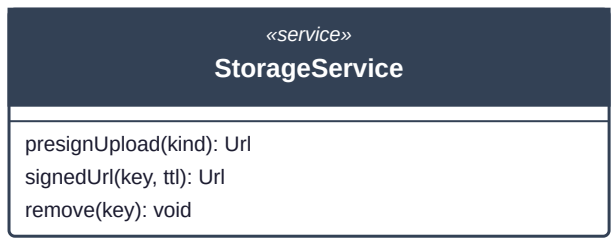
- **R-09** Contact page with giving details
- **R-15** Modal order (new-member question before the newsletter), a cookie bar, and "No" remembered in localStorage or cookies
- **R-30** Class tabs: each main class broken into its own tab

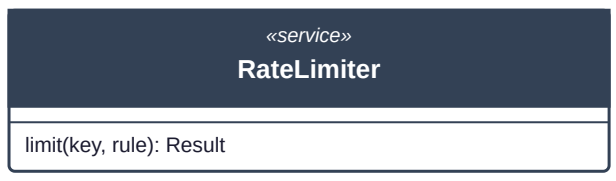
Tab: Platform services (security, language, files)

Attributes, parents, children, rules and mistakes









Called by (classes that depend on this service)

User · RoleAssignment · LedgerEntry · Department · Sermon · Event · Project · MediaAsset · Certificate · Comment · EventRegistration · Convert · NewsletterSubscriber

AuthzService – in plain words
The security guard at the door of every action: "are you allowed to do this?"
One function every server action calls before touching data.

Rules the code must keep
• Every Server Action and Route Handler starts with assertCan(...). No exception.
• Decisions use RoleAssignment (role + scope) loaded once per request and cached for that request only.

Mistakes to avoid
• Checking permissions only in the UI (hiding buttons). The server must check too.

TranslationService – in plain words
The language helper: finds the Kiswahili or English words and says so when one is missing.
Reads the right language, falls back safely, and drives the AI-draft to human-review flow.

Rules the code must keep
• Fallback chain: requested locale -> other locale -> marked "not yet translated" badge.
• Machine drafts are stored as status machine_draft and never shown publicly until approved (or flagged "auto-translated" if Super Admin allows).

Mistakes to avoid
• Translating on every page view. Translate once, store, approve.

StorageService – in plain words
The delivery person for photos, PDFs and audio: takes them to the cloud store and brings back a safe link.
Only class allowed to talk to Cloudflare R2.

Rules the code must keep
• Uploads go browser -> R2 by presigned URL (the file never passes through Vercel).
• Files are checked for type, size and (images) re-encoded to strip hidden data.

Mistakes to avoid
• Trusting the file extension. Check the real type.

RateLimiter – in plain words
The queue marshal: lets people in at a sensible speed so robots cannot flood the forms.
Upstash Ratelimit wrapper: per IP, per device, per form.

Rules the code must keep
• Sliding-window limits per IP-hash and per route in Upstash Redis.
• Limits: comment 5/10min, newsletter 3/hour, contact 3/hour, converts 20/hour per staff device.

Mistakes to avoid
• One global limit for the whole site. A single noisy visitor blocks everyone.

How this group fits together
Four services that nearly every module calls. They are code, not tables.

HOW TO READ IT

AuthzService (Platform, owned by **Shared Kernel**). The security guard at the door of every action: "are you allowed to do this?" One function every server action calls before touching data.

TranslationService (Platform, owned by **Shared Kernel**). The language helper: finds the Kiswahili or English words and says so when one is missing. Reads the right language, falls back safely, and drives the AI-draft to human-review flow.

StorageService (Platform, owned by **Shared Kernel**). The delivery person for photos, PDFs and audio: takes them to the cloud store and brings back a safe link. Only class allowed to talk to Cloudflare R2.

RateLimiter (Platform, owned by **Shared Kernel**). The queue marshal: lets people in at a sensible speed so robots cannot flood the forms. Upstash Ratelimit wrapper: per IP, per device, per form.

Four services that nearly every module calls. They are code, not tables.

MISTAKES TO AVOID

- Checking permissions only in the UI (hiding buttons). The server must check too.
- Translating on every page view. Translate once, store, approve.
- Trusting the file extension. Check the real type.
- One global limit for the whole site. A single noisy visitor blocks everyone.

WHAT YOU ALREADY DID RIGHT

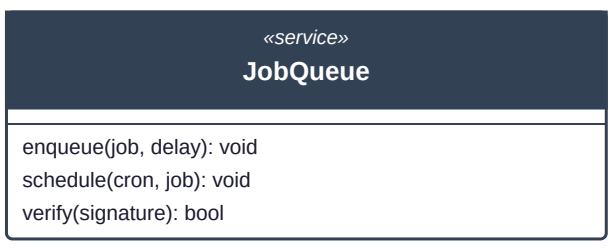
This page serves your own requirements:

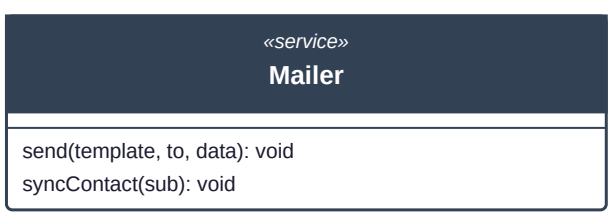
- **R-13** Help with translating dynamic content (blogs, sermons, bulletins and so on)
- **R-26** Next.js, Drizzle, Neon Postgres, Resend, Upstash Redis, QStash, PostHog, Better Auth (admin), Tailwind, Cloudflare R2, Vercel
- **R-30** Class tabs: each main class broken into its own tab

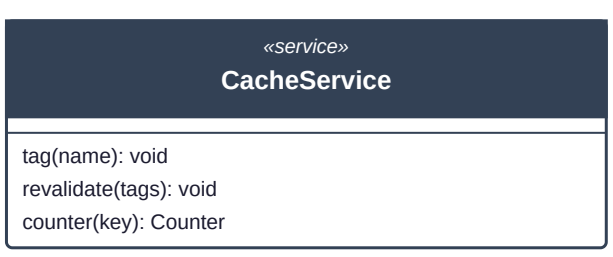
CLASS TABS (UML DETAIL) · PHASE 1

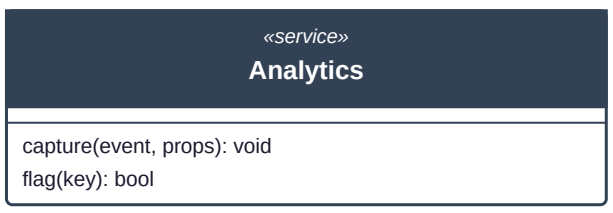
Tab: Platform services (jobs, mail, cache, analytics)

Attributes, parents, children, rules and mistakes









Called by (classes that depend on this service)

Event · Convert · FollowUpCase · JoinRequest · EventRegistration · NewsletterSubscriber · Sermon · Project · LedgerEntry

JobQueue – in plain words
The to-do list for slow jobs; a worker does them in the background and tries again if they fail.
QStash wrapper for emails, reminders, digests, counter flush.

Rules the code must keep
• All slow or retryable work (emails, counters flush, translation drafts) goes through QStash with signed requests.
• Jobs are idempotent: running twice must not double-send.

Mistakes to avoid
• Running jobs that are not idempotent. A retry then sends the same email twice.

Mailer – in plain words
The post office: sends the e-mails and writes down what was sent.
Resend wrapper; every send is logged in NotificationLog.

Rules the code must keep
• Templates are React Email components, one per message, bilingual.
• Sending domain has SPF, DKIM and DMARC set in DNS before the first campaign.

Mistakes to avoid
• Sending from a no-reply address with no SPF/DKIM: it lands in spam.

CacheService – in plain words
The memory shelf: keeps recent answers close so pages load fast.
Next.js cache tags + Upstash Redis counters.

Rules the code must keep
• Next.js cache tags for pages, Redis for counters and shared lookups.
• After a publish, call revalidateTag(tag) so the public sees the change within seconds.

Mistakes to avoid
• Caching pages that show a logged-in user's data. Only cache public pages.

Analytics – in plain words
The people counter: only counts visitors who said yes to counting.
PostHog wrapper that respects cookie consent.

Rules the code must keep
• PostHog loads ONLY after the visitor accepts analytics cookies.
• Event names follow the taxonomy page (object_action, snake_case).

Mistakes to avoid
• Loading PostHog before consent. Under Kenyan and EU rules that is a breach.

How this group fits together
Four more services. Each wraps one outside tool (QStash, Resend, Upstash/Next cache, PostHog) so the tool can be swapped without touching the modules.

HOW TO READ IT

JobQueue (Platform, owned by **Shared Kernel**). The to-do list for slow jobs; a worker does them in the background and tries again if they fail. QStash wrapper for emails, reminders, digests, counter flush.

Mailer (Platform, owned by **Shared Kernel**). The post office: sends the e-mails and writes down what was sent. Resend wrapper; every send is logged in NotificationLog.

CacheService (Platform, owned by **Shared Kernel**). The memory shelf: keeps recent answers close so pages load fast. Next.js cache tags + Upstash Redis counters.

Analytics (Platform, owned by **Shared Kernel**). The people counter: only counts visitors who said yes to counting. PostHog wrapper that respects cookie consent.

Four more services. Each wraps one outside tool (QStash, Resend, Upstash/Next cache, PostHog) so the tool can be swapped without touching the modules.

MISTAKES TO AVOID

- Running jobs that are not idempotent. A retry then sends the same email twice.
- Sending from a no-reply address with no SPF/DKIM: it lands in spam.
- Caching pages that show a logged-in user's data. Only cache public pages.
- Loading PostHog before consent. Under Kenyan and EU rules that is a breach.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

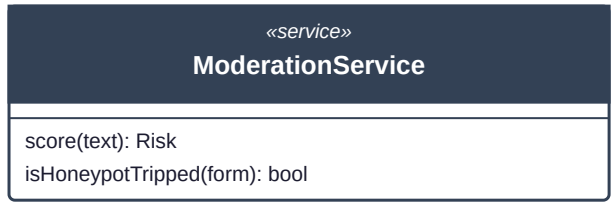
- **R-26** Next.js, Drizzle, Neon Postgres, Resend, Upstash Redis, QStash, PostHog, Better Auth (admin), Tailwind, Cloudflare R2, Vercel
- **R-30** Class tabs: each main class broken into its own tab

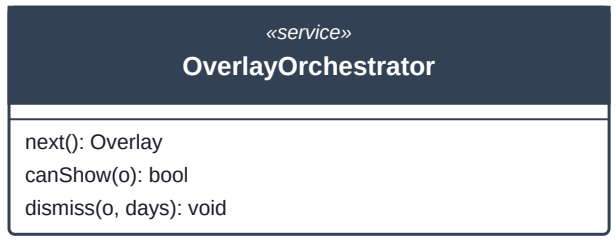
B51

CLASS TABS (UML DETAIL) · PHASE 1

Tab: Feature services (moderation, overlays)

Attributes, parents, children, rules and mistakes





Called by (classes that depend on this service)

Comment | NewMemberJourney

ModerationService – in plain words
The doorkeeper for comments and messages: checks for spam before they reach a human.
First-line spam checks for comments and contact messages.

Rules the code must keep
• Checks: honeypot, link count, banned words list (EN + SW), duplicate body, rate limit result.
• A suspicious item becomes status "pending", never silently dropped.

Mistakes to avoid
• Relying only on word lists. Add rate limits and the human review queue.

OverlayOrchestrator – in plain words
The traffic light for pop-ups so only one shows at a time.
Client-side traffic cop: cookie bar, new-member modal, newsletter modal never fight.

Rules the code must keep
• Order of overlays: 1 cookie bar (bottom), 2 new-member modal, 3 newsletter modal (after About section).
• Only ONE modal at a time; dismissals are remembered (localStorage, and cookie if accepted).

Mistakes to avoid
• Letting three popups open together on a phone. The visitor leaves.

How this group fits together
Two small services owned by features: comment/contact moderation and the pop-up traffic light.

HOW TO READ IT

ModerationService (Platform, owned by **Content Editor**). The doorkeeper for comments and messages: checks for spam before they reach a human. First-line spam checks for comments and contact messages.

OverlayOrchestrator (Platform, owned by **Visitor (public)**). The traffic light for pop-ups so only one shows at a time. Client-side traffic cop: cookie bar, new-member modal, newsletter modal never fight.

Two small services owned by features: comment/contact moderation and the pop-up traffic light.

MISTAKES TO AVOID

- Relying only on word lists. Add rate limits and the human review queue.
- Letting three popups open together on a phone. The visitor leaves.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-30** Class tabs: each main class broken into its own tab

ATLAS · FAMILY 3

UML: use cases

What each person wants. Six pages: what the visitor wants to read and do, what a new member does in the tour, what the content team, the ministry team and the administrators do.

> **EXPLAINED LIKE YOU ARE FIVE**
> Like a list at the gate: who comes to the compound and why.

Code	Title	What it shows
U01	Visitor: read, listen and explore	Everything an anonymous person can do on the public site (no forms yet)
U02	Visitor: take part (forms and actions)	Every form a visitor can send, and the automatic safety steps behind them
U03	New member: the tour	Yes/No question, nine short steps, feedback and the certificate
U04	Content team: editor and translator	Publishing, moderating and translating, and what each of the two roles may touch
U05	Ministry team: department, events and follow-up	Three roles that look after people: Department Leader, Events Coordinator, Discipler
U06	Administration: Super Admin, Branch Admin and Treasurer	Who can change structure, projects, money, roles and the whole system

U01

UML: USE CASES · PHASE 1

Visitor: read, listen and explore

Everything an anonymous person can do on the public site (no forms yet)



How to read this page

- A stick figure is a PERSON or an outside SERVICE that uses the system.
- An oval is one thing they can do (a use case).
- A grey frame groups ovals. A line from an actor to a frame means the actor takes part in EVERY oval inside it. A line to one oval means only that oval.
- A dashed purple arrow means "this oval always uses that one" (include) or "may add this step" (extend).
- The dashed outline is the edge of our website; anything outside it is not ours.

HOW TO READ IT

A **use case** is just a sentence that starts with a verb: "Watch a sermon video". The stick figure is the **actor**, the person who does it. The dashed outline is the **system boundary**: everything inside is what we build.

This first page is the **read-only side** of the church website. The visitor needs no account and no password. That one decision keeps the site fast, safe and cheap: nothing to protect, nothing to reset.

The outside services on the right are **secondary actors**. YouTube hosts the video, a map service draws the branch map, a search engine sends visitors who type "Deliverance Church Eastern Bypass". They are not ours, so they sit outside the dashed line, joined only to the one oval they help with.

MISTAKES TO AVOID

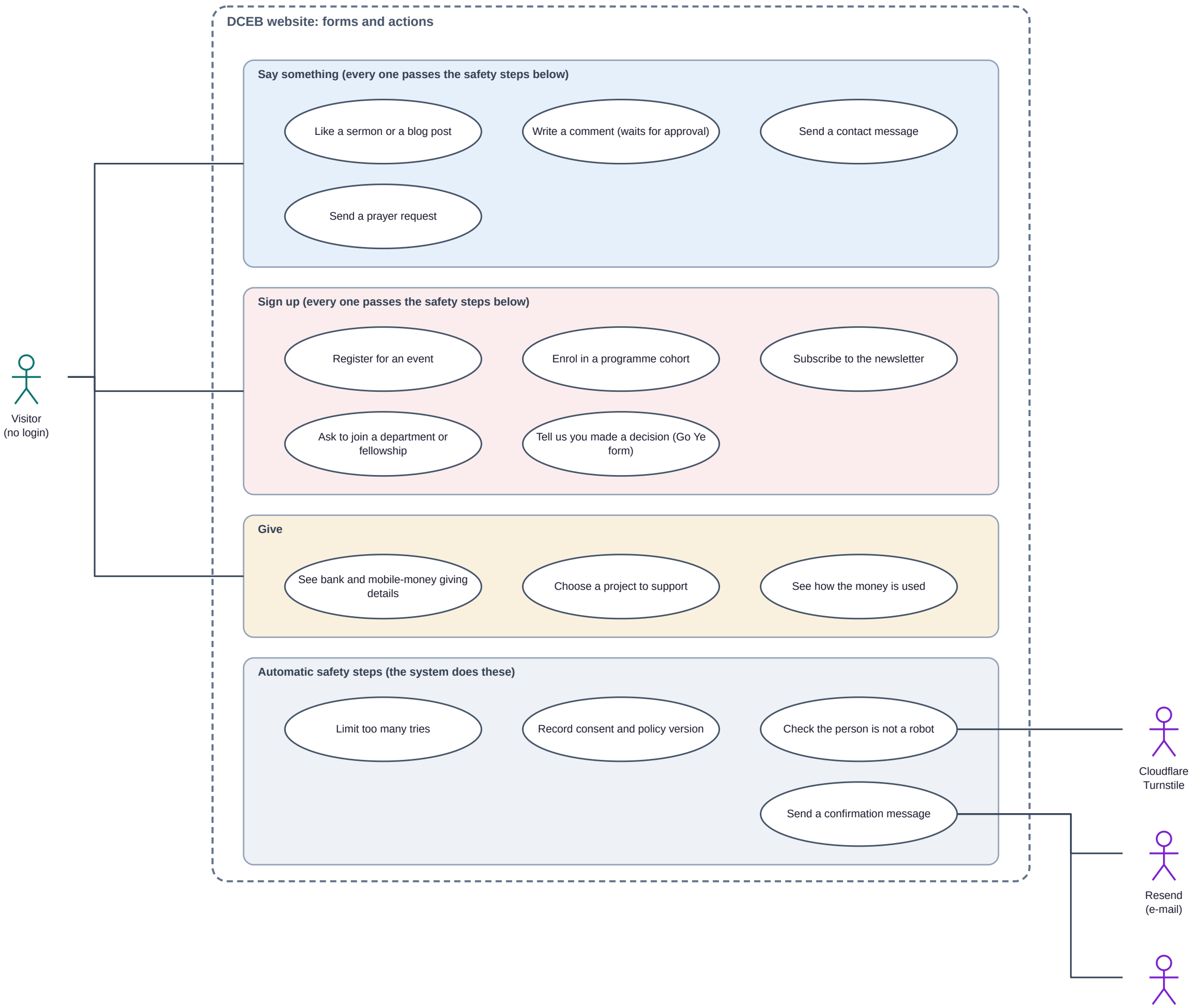
- Hiding the language switch in a menu. Both languages must be one tap away on every page (it is a use case of its own: switch language).
- Loading the YouTube player on page open. It should load after the tap (see S01), otherwise a phone on a data bundle pays for video the person did not choose.
- Forgetting that search engines are also "users". Titles, descriptions, the language alternates (hreflang) and a sitemap are requirements, not decoration.

WHAT YOU ALREADY DID RIGHT

- You listed every public page of the site up front (landing, about, departments, events, programmes, Go Ye, projects, resources, sermons, contact). Each one is a use case on this page, so nothing was left to guess.

Visitor: take part (forms and actions)

Every form a visitor can send, and the automatic safety steps behind them



How to read this page

- A stick figure is a PERSON or an outside SERVICE that uses the system.
- An oval is one thing they can do (a use case).
- A grey frame groups ovals. A line from an actor to a frame means the actor takes part in EVERY oval inside it. A line to one oval means only that oval.
- A dashed purple arrow means "this oval always uses that one" (include) or "may add this step" (extend).
- The dashed outline is the edge of our website; anything outside it is not ours.

HOW TO READ IT

Reading is free. **Doing** is where risk starts: every form is a door that a robot or an angry person can also use. So the forms in "Say something" and "Sign up" all **pass through** the safety steps in the bottom frame (limit tries, record consent, check it is not a robot, send a confirmation). The system does those steps by itself; no person clicks them.

Notice the visitor is still not logged in. A visitor can **write** (send a form) but cannot **read back** what others sent: nobody sees the list of event registrations except staff. We call this "write-only for the public"; in the permission matrix (B04) it appears as an **S** (submit).

Giving is on this page, but only as **information**: bank and mobile-money details and which project to support. Money itself moves outside the website in phase 1. That is why no payment company is shown on the right.

MISTAKES TO AVOID

- Treating "Ask to join a department" as a quick e-mail. It creates a JoinRequest row that a Department Leader must answer. If nobody owns that inbox, requests rot (see T06).
- Showing other people's e-mail or phone numbers beside comments. Public pages show a first name or chosen display name only.
- Forgetting consent. Every form that stores personal data records a ConsentRecord (what, when, which policy version).

WHAT YOU ALREADY DID RIGHT

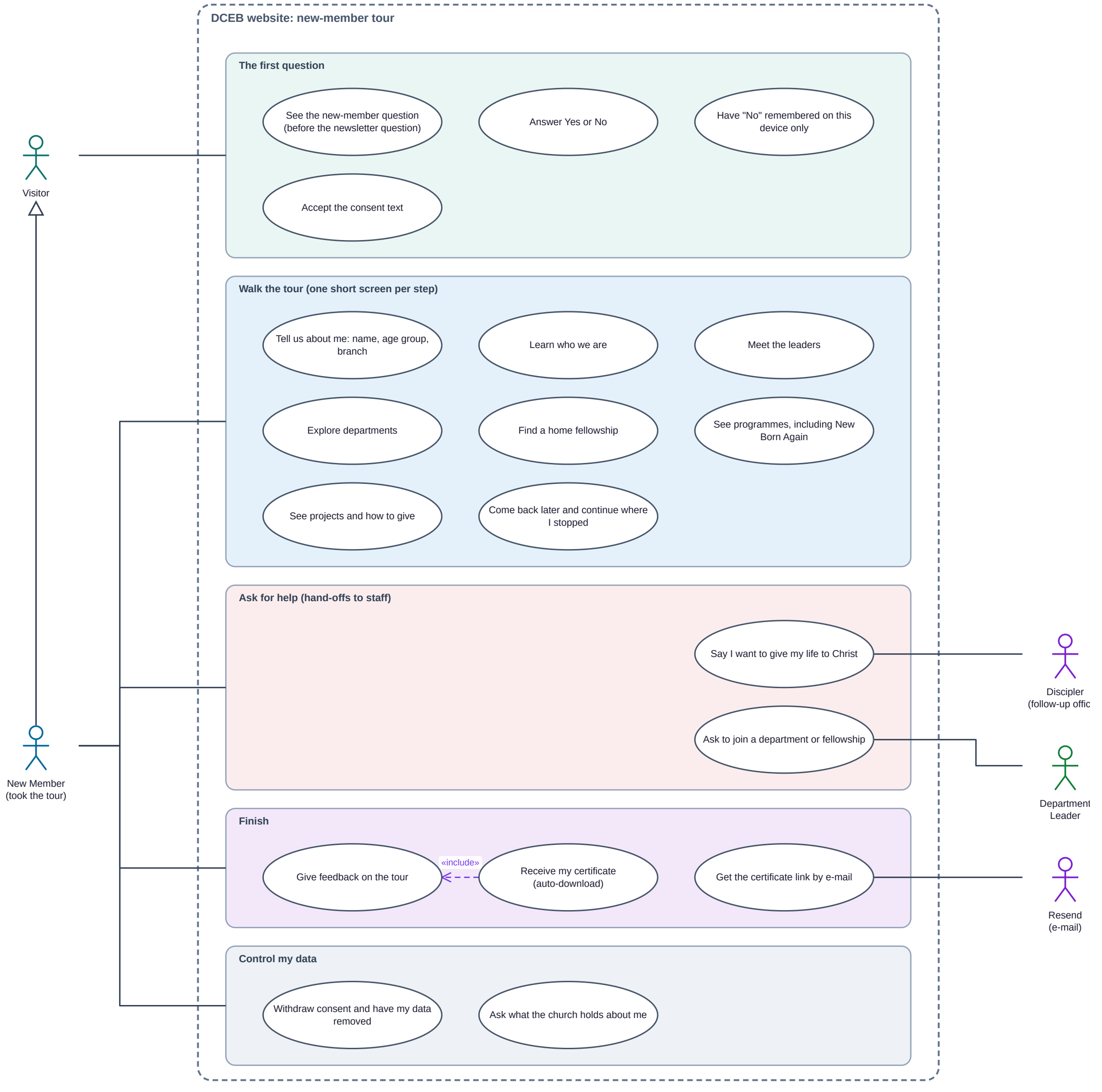
- You asked for contact and giving details on the site and for forms such as Go Ye. Both exist here, with safety built in, and none needs a visitor account.

U03

UML: USE CASES · PHASE 1

New member: the tour

Yes/No question, nine short steps, feedback and the certificate



How to read this page

- A stick figure is a PERSON or an outside SERVICE that uses the system.
- An oval is one thing they can do (a use case).
- A grey frame groups ovals. A line from an actor to a frame means the actor takes part in EVERY oval inside it. A line to one oval means only that oval.
- A dashed purple arrow means "this oval always uses that one" (include) or "may add this step" (extend).
- The dashed outline is the edge of our website; anything outside it is not ours.

HOW TO READ IT

A **New Member** is not a different person. It is the same **Visitor** who pressed **Yes**. The hollow-headed arrow from New Member to Visitor is **generalisation**: "a New Member is a kind of Visitor, so they can do everything a visitor can, plus the tour".

The first frame belongs to the visitor because the question is shown to **everyone**. Only after **Yes + consent** does the person become a New Member and the server hears about them (see S04). "No" stays in the browser (localStorage), so a person who declines leaves no record.

Two staff roles appear on the right because the tour **hands work to people**: a decision for Christ goes to a Discipler (Go Ye follow-up, F07); a request to join a department goes to that Department Leader (F08). The tour is not a dead end; it is the front door of the church's care system.

MISTAKES TO AVOID

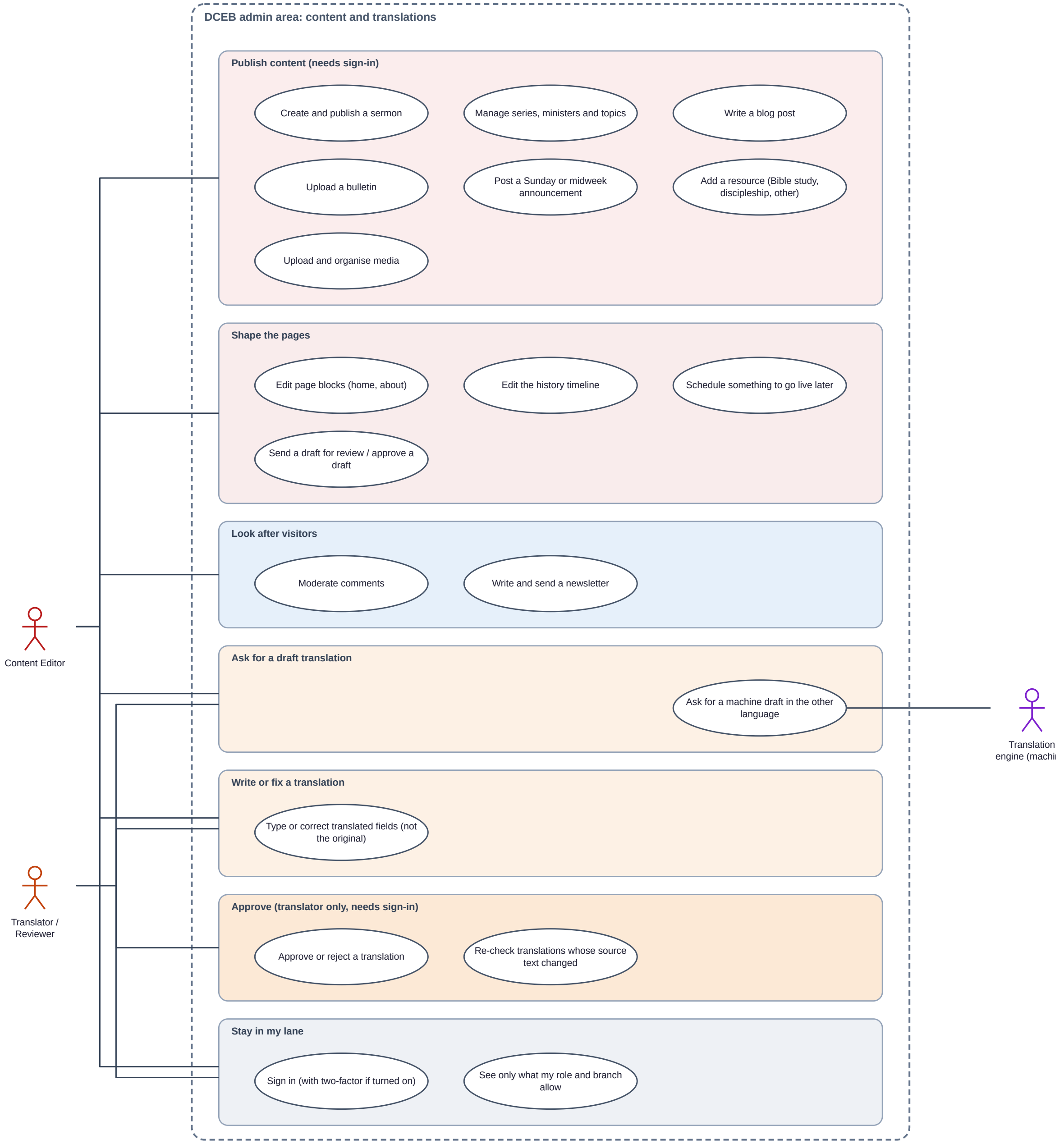
- Treating the first modal as just a pop-up. It is the point where consent starts. Without the consent step you would hold belief data (sensitive under the Data Protection Act, 2019) without permission.
- Making the tour mandatory. It must stay optional and skippable; a person must reach any page without it.
- Nine steps on one page. One idea per screen, with progress dots, so a phone user is not overwhelmed. The proposed nine steps are on F06 and need your confirmation (see the questions document).

WHAT YOU ALREADY DID RIGHT

- You said the answer "No" should be remembered so the modal does not nag. That is both friendlier and more private than storing it on a server.
- You asked for a certificate at the end. It gives the person a reason to finish, and it gave us a clean "completed" state (T04).

Content team: editor and translator

Publishing, moderating and translating, and what each of the two roles may touch



How to read this page

- A stick figure is a PERSON or an outside SERVICE that uses the system.
- An oval is one thing they can do (a use case).
- A grey frame groups ovals. A line from an actor to a frame means the actor takes part in EVERY oval inside it. A line to one oval means only that oval.
- A dashed purple arrow means "this oval always uses that one" (include) or "may add this step" (extend).
- The dashed outline is the edge of our website; anything outside it is not ours.

HOW TO READ IT

Two roles share this page because they work on **the same rows**: the editor writes the original, the translator checks the other language. They meet in the three **translation frames**.

The editor can **ask** for a machine draft and can type or correct translated text, but only the translator can **approve** it (see the permission matrix, letter A). This is the answer to your question about *"blogs written in only one language"*: nothing is blocked. The post goes live in its own language, and the other language appears when someone approves it (pattern on B08).

Everything on this page needs a **sign-in**. Signing in proves who you are; the role decides what you may do, and "See only what my role and branch allow" is checked on every action (S10).

MISTAKES TO AVOID

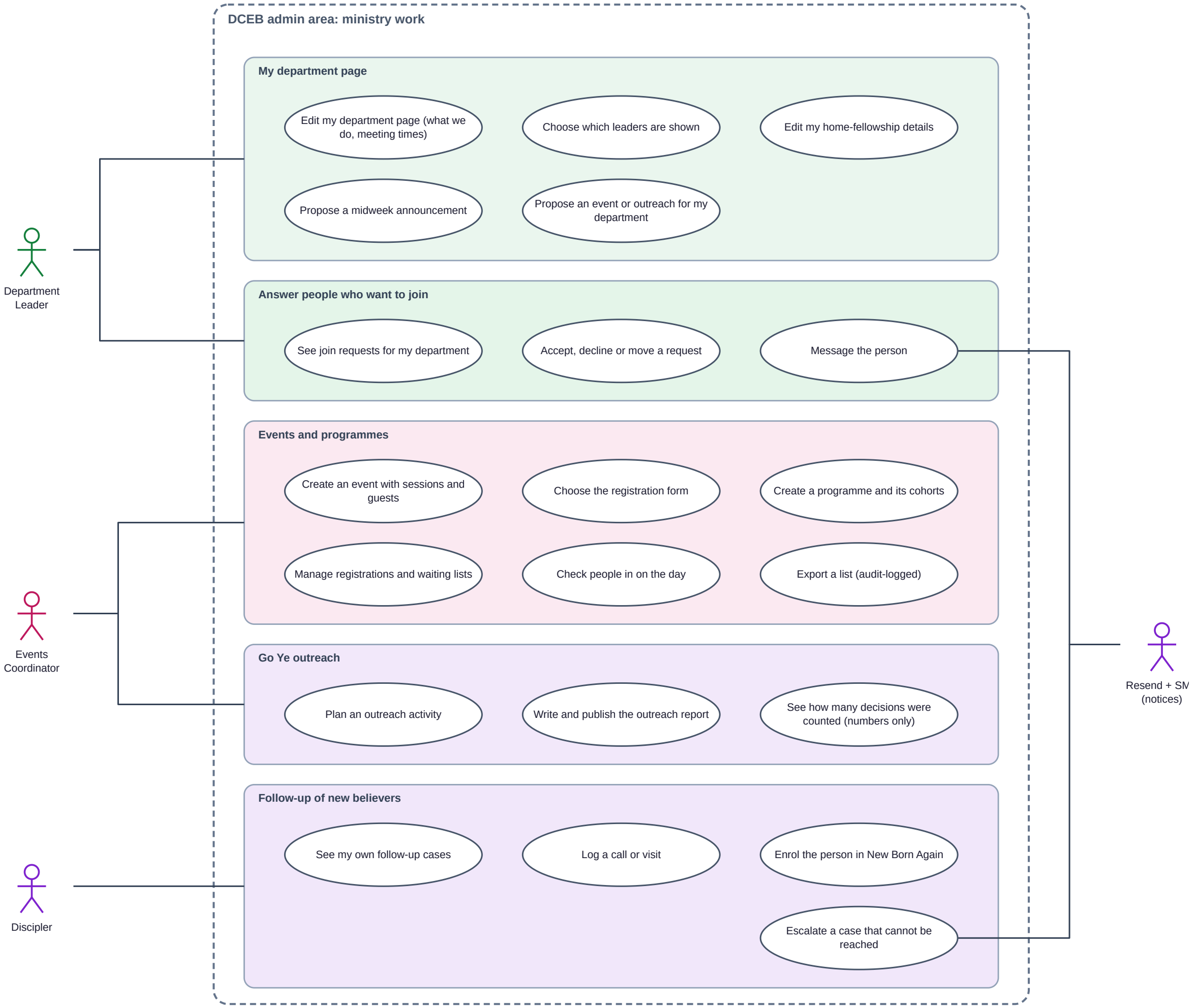
- Letting the editor also approve translations "to save time". It turns the safety net into a rubber stamp. Keep the roles separate even if the same person holds both on a quiet week.
- Publishing without the review step because it is Sunday morning. T01 allows an Editor to publish directly; decide who may skip review and write it down.
- Forgetting that scheduled items need the background job (QStash) to run; if the job fails, "scheduled" stays scheduled. The admin list must show overdue items.

WHAT YOU ALREADY DID RIGHT

- You asked about translating dynamic content such as blogs. Making Translator its own role (global scope, no tables of its own) is the right answer for a church where language skill is not tied to a branch.

Ministry team: department, events and follow-up

Three roles that look after people: Department Leader, Events Coordinator, Discipler



How to read this page

- A stick figure is a PERSON or an outside SERVICE that uses the system.
- An oval is one thing they can do (a use case).
- A grey frame groups ovals. A line from an actor to a frame means the actor takes part in EVERY oval inside it. A line to one oval means only that oval.
- A dashed purple arrow means "this oval always uses that one" (include) or "may add this step" (extend).
- The dashed outline is the edge of our website; anything outside it is not ours.

HOW TO READ IT

These three roles do not publish sermons; they run **people work**. A Department Leader looks after one department. An Events Coordinator runs events, programmes and the Go Ye outreach. A Discipler follows up new believers.

Each role sees **only their own slice**: a Department Leader sees only join requests for their department, a Discipler only the cases assigned to them. In the class diagrams this is the **scope** (department / branch). The use cases here are the human version of those rules.

Go Ye is split on purpose. The Events Coordinator owns the **outreach activity** (plan it, report it). The names and phone numbers of people who decided are submitted through the Go Ye form and are visible **only to Disciplers**, the Branch Admin and the Super Admin. The coordinator sees counts, not names.

MISTAKES TO AVOID

- One person doing all three jobs with one account. A user can hold several roles, but each action is still checked against the role that allows it, so the audit log tells the truth.
- Exporting people lists to a personal phone or WhatsApp. Exports are audit-logged for a reason; a list should leave the system only when it has to.
- Leaving a follow-up case open forever. T05 has a "closed with a reason" state; use it so the dashboard can show real numbers.

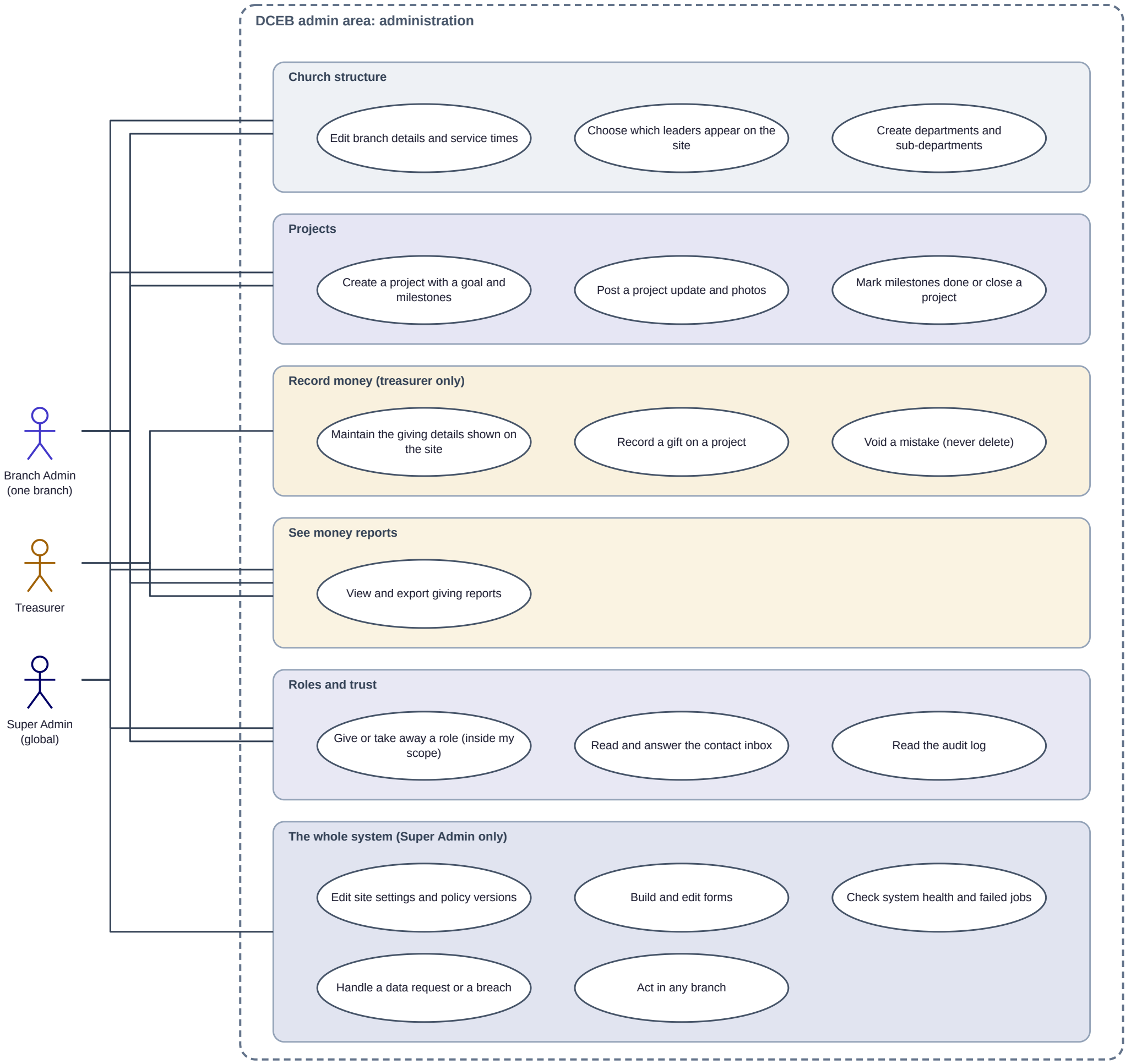
WHAT YOU ALREADY DID RIGHT

- You asked for a Go Ye page with a counter and a form to add a convert. This diagram shows who feeds that form and who acts on it, so the "born again so far" number is something staff can stand behind.

UML: USE CASES · PHASE 1

Administration: Super Admin, Branch Admin and Treasurer

Who can change structure, projects, money, roles and the whole system



How to read this page

- A stick figure is a PERSON or an outside SERVICE that uses the system.
- An oval is one thing they can do (a use case).
- A grey frame groups ovals. A line from an actor to a frame means the actor takes part in EVERY oval inside it. A line to one oval means only that oval.
- A dashed purple arrow means "this oval always uses that one" (include) or "may add this step" (extend).
- The dashed outline is the edge of our website; anything outside it is not ours.

HOW TO READ IT

This page is where **power** lives, so it is also where **limits** matter most. The Super Admin is global (any branch). The Branch Admin has most of the same buttons but only for **one branch**. The Treasurer touches **only money**.

Look at the two money frames. Only the **Treasurer** can record or void a gift, and nobody can edit or delete one: every change makes a new row. That is an **append-only ledger** (S12): the church can always show how the progress bar got to its number. Super Admin and Branch Admin may **read** the reports, but cannot change a figure.

The bottom frame is joined to the Super Admin only. Settings, the form builder, health checks, data requests and "act in any branch" are not for branch staff.

MISTAKES TO AVOID

- Giving the Super Admin account to many people. Two people at most, each with two-factor sign-in. Everyone else gets a narrower role.
- Letting a Branch Admin give out roles in another branch. RoleAssignment is checked on scope at every grant (B21, S10).
- Treating the giving bar as a promise. It shows what the Treasurer recorded, with a "last updated" date, not live bank data.

WHAT YOU ALREADY DID RIGHT

- You asked for a project giving bar. Because gifts are recorded in an append-only ledger and the total is recomputed from it, the bar can never silently drift from the records.

ATLAS · FAMILY 4

UML: sequence

Who calls whom, in order. Fourteen conversations between the browser, the server, the services and the outside tools. Read them top to bottom; time runs downward.

> **EXPLAINED LIKE YOU ARE FIVE**
> Like the script of a play: who speaks, who answers, what happens if the answer is no.

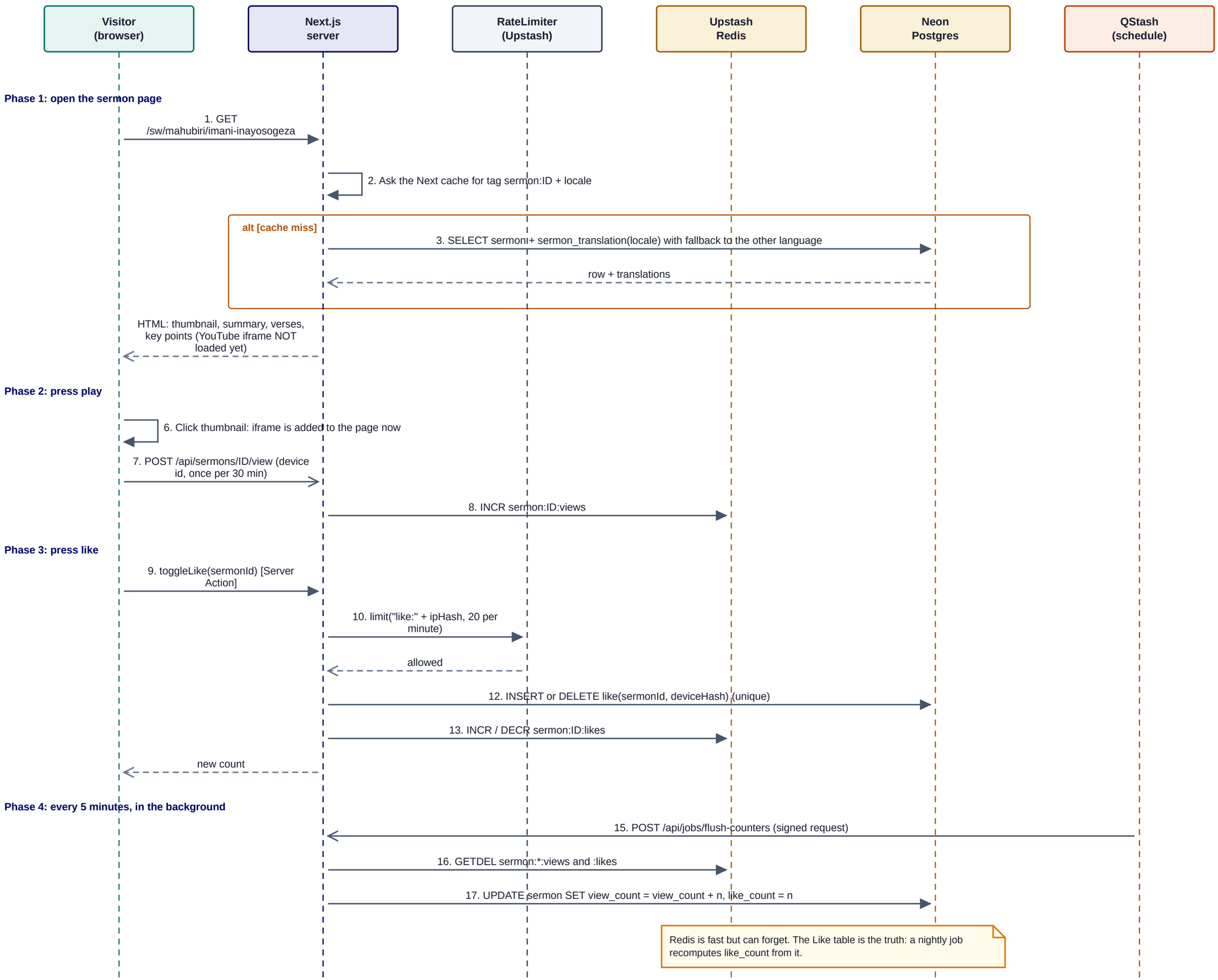
Code	Title	What it shows
S01	Sermon page: view, play, like, count	Why counters live in Redis and are flushed to Postgres
S02	Staff publishes a sermon (upload, schedule, go live)	Direct-to-R2 upload, scheduled publish, cache refresh
S03	Translation: AI draft, human review, publish	How Kiswahili (or English) text is produced and checked
S04	New-member tour: ask, consent, save, finish	The "No is remembered locally" rule and the first server contact
S05	Certificate: build, store, auto-download	Private bucket and signed links
S06	Go Ye: record a new believer and start follow-up	From a name and phone number to a discipler's call list
S07	Event registration (youth trip, conference)	Rate limit, capacity check, confirmation
S08	Newsletter: double opt-in and batch sending	Confirm by e-mail, send in batches, unsubscribe in one click
S09	Comment: submit, filter, moderate	Spam defence in layers, then a human
S10	Staff sign-in and the three security gates	Gate 1 session, gate 2 route, gate 3 action + scope
S11	Nearest home fellowship finder	Geolocation first, typed estate as the fallback
S12	Giving: record a gift, void a mistake, update the bar	Append-only ledger and a cached total
S13	Contact message and prayer request	Save first, notify second
S14	Programme enrolment (Kairos, CFC, Teen 5 Days, ...)	Enrol in the NEXT cohort and track attendance

S01

UML: SEQUENCE · PHASE 1

Sermon page: view, play, like, count

Why counters live in Redis and are flushed to Postgres



HOW TO READ IT

Think of Redis as a **tally sheet on the wall** and Postgres as the **official register**. Every play adds a tick on the wall sheet (very fast, no queue). Every five minutes a helper (QStash) tells the server to copy the ticks into the register.

The YouTube player is loaded only after the visitor taps the thumbnail. A page with an embedded player downloads about 1 MB of YouTube code before anything else; on a Safaricom data bundle that matters.

Likes are written to **both**: the Like table (unique on sermonId + deviceHash, so double-likes are impossible) and the Redis counter (so the heart updates instantly).

MISTAKES TO AVOID

- Never write view counts straight to Postgres on every play: 500 people watching live = 500 writes per second.
- Do not trust the device hash as identity. It only prevents casual double-likes.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

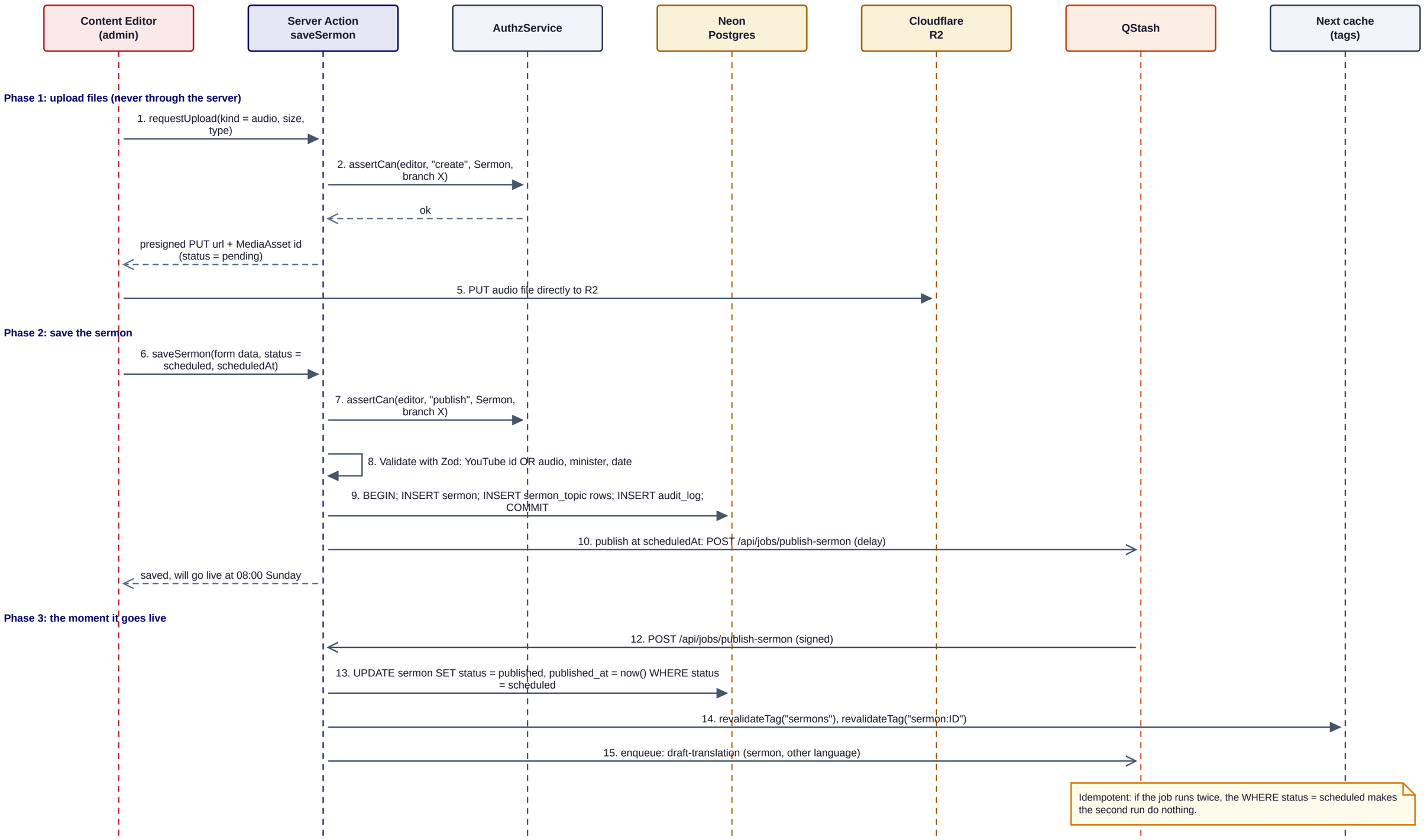
- **R-11** Sermons with video, audio, summary, verses, filters, views, comments and likes

S02

UML: SEQUENCE · PHASE 1

Staff publishes a sermon (upload, schedule, go live)

Direct-to-R2 upload, scheduled publish, cache refresh



HOW TO READ IT

Large files go **straight from the editor's browser to Cloudflare R2** using a one-time permission slip (presigned URL). Vercel functions have size and time limits; a 60 MB sermon audio would fail if it passed through them.

Publishing is a status change, not a copy. The scheduled job simply flips **scheduled** to **published** and tells the cache "the sermons list changed, refresh it".

Every save also writes an **AuditLog** row inside the same database transaction, so we can always answer "who changed this and when".

MISTAKES TO AVOID

- Do not mark the MediaAsset as ready until R2 confirms the file exists (head-object check) or you will publish a broken audio player.
- Make the publish job idempotent. QStash retries on failure.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

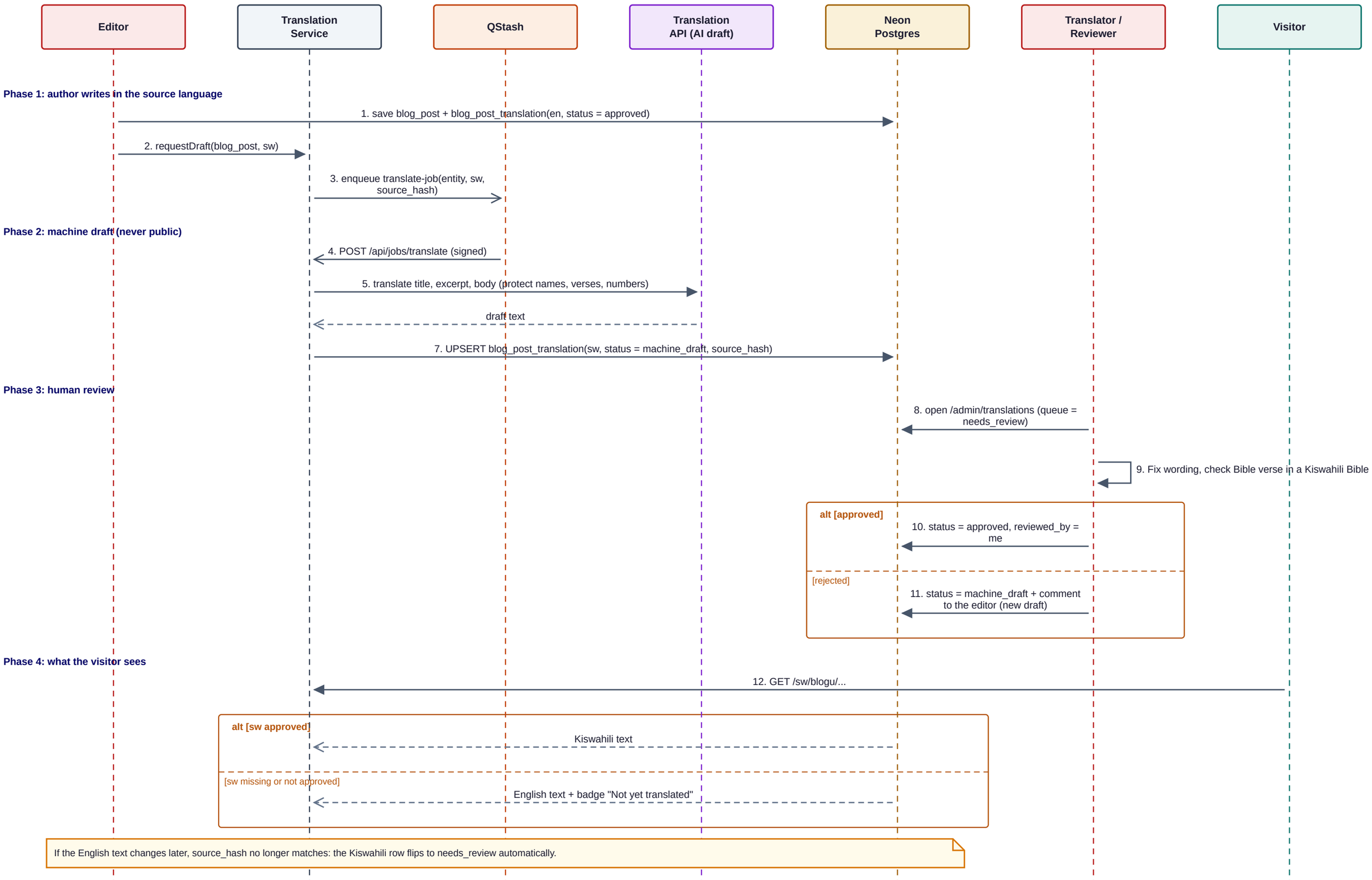
- **R-11** Sermons with video, audio, summary, verses, filters, views, comments and likes
- **R-21** Publishing with a review step and scheduling

S03

UML: SEQUENCE · PHASE 1

Translation: AI draft, human review, publish

How Kiswahili (or English) text is produced and checked



HOW TO READ IT

A machine can translate fast but it can be wrong about church words ("fellowship", "born again", "tithe"). So the machine only writes a **draft**. A bilingual person must press Approve before the public sees it.

The **source_hash** is a fingerprint of the English text. If someone edits the English later, the fingerprint changes and the Kiswahili version is marked **needs review** so the two languages never silently disagree.

While a translation is missing, the visitor still sees the other language with a small badge. A page is never empty.

MISTAKES TO AVOID

- Do not run the translator on every page view: translate once, store, review.
- Keep a glossary (Bible book names, church terms) and pass it to the translation call so key words stay consistent.

WHAT YOU ALREADY DID RIGHT

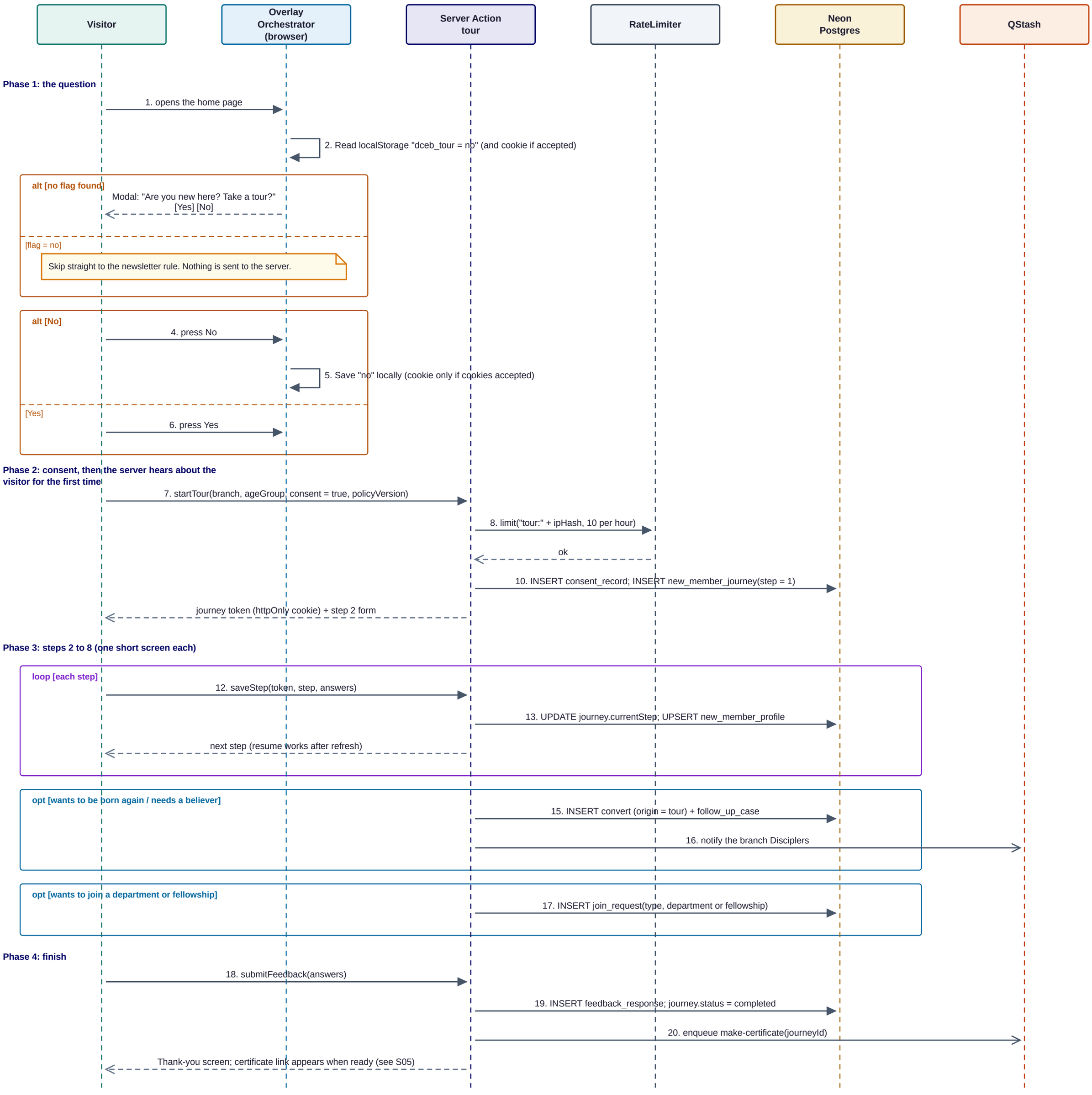
- You asked for help translating dynamic content such as blogs. This design answers it without paying for a translation on every visit.

S04

UML: SEQUENCE · PHASE 1

New-member tour: ask, consent, save, finish

The "No is remembered locally" rule and the first server contact



HOW TO READ IT

The most important rule: **"No" never touches the server**. The browser remembers it (localStorage; a cookie too if the visitor accepted cookies). This respects people who just want to read a sermon, and it keeps data we have no reason to hold.

The first time the server hears about a visitor is when they press Yes **and** accept the consent text. That consent is stored (what, when, which version of the policy) as a ConsentRecord.

The tour saves after every step. A phone that loses signal in the middle of step 5 resumes at step 5.

MISTAKES TO AVOID

- Do not create the journey before consent. Belief ("born again") is sensitive personal data under the Kenya Data Protection Act 2019.
- Do not ask nine questions on one page. One short screen per step.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

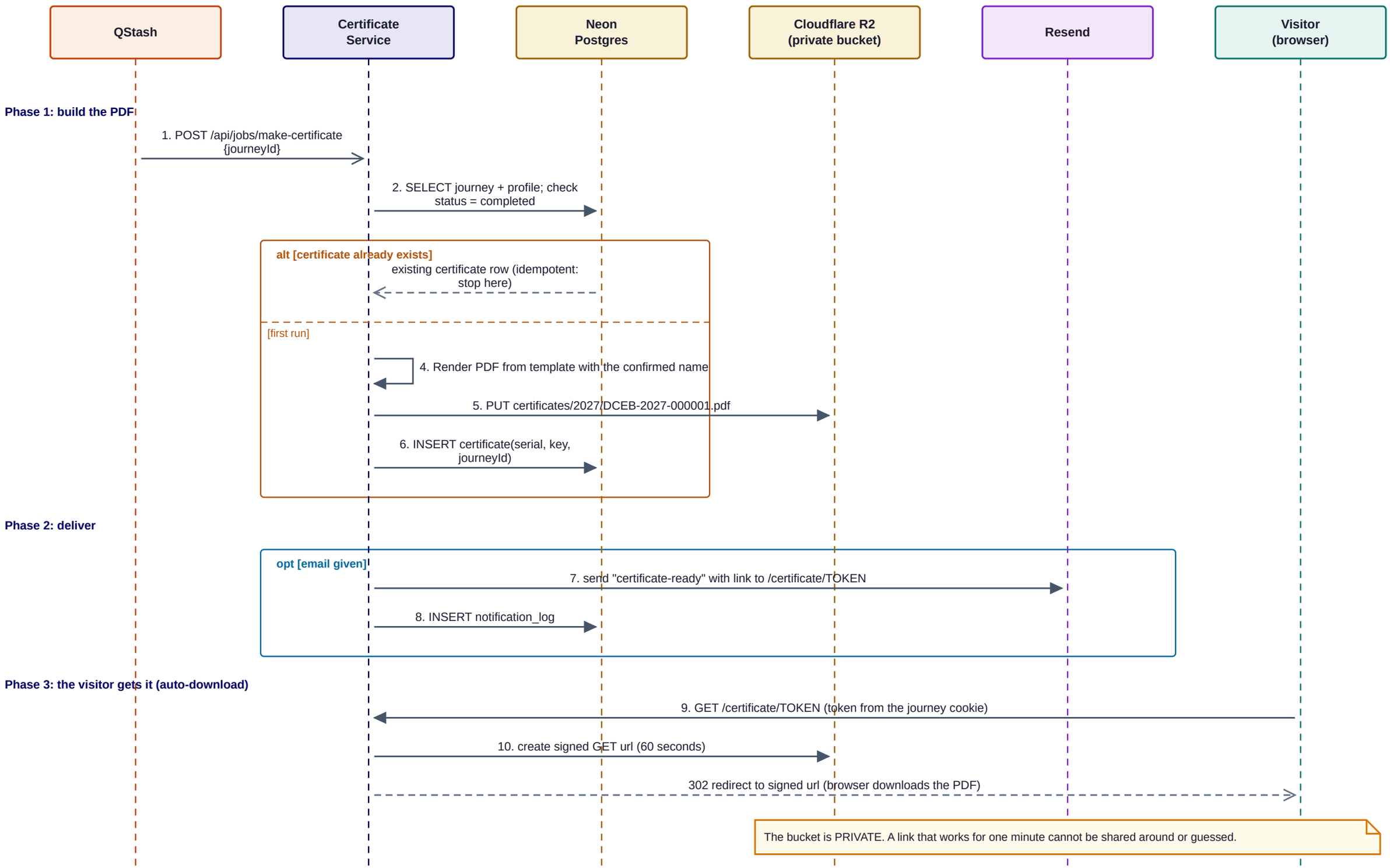
- **R-14** New-member tour of nine steps, ending in a certificate
- **R-15** Modal order (new-member question before the newsletter), a cookie bar, and "No" remembered in localStorage or cookies

S05

UML: SEQUENCE · PHASE 1

Certificate: build, store, auto-download

Private bucket and signed links



HOW TO READ IT

The certificate is a PDF with the member's name and the church seal. It is built **once**, stored in a **private** bucket, and downloaded through a link that expires after 60 seconds.

If the job runs twice (QStash retries), the second run finds the existing certificate and stops. That property is called **idempotency**.

MISTAKES TO AVOID

- Make the visitor confirm the spelling of the name before generation. A misspelled certificate is a bad first gift.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

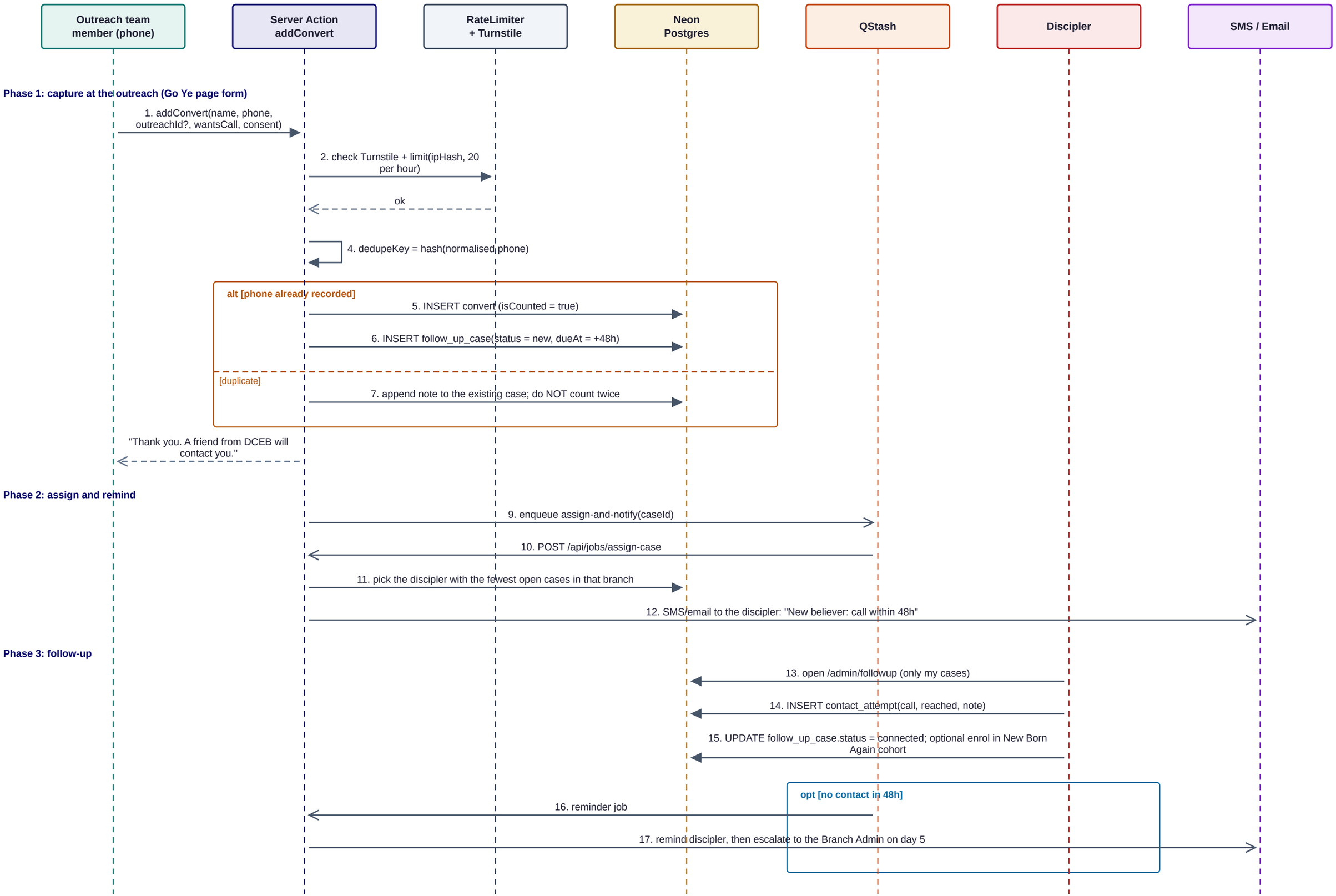
- **R-14** New-member tour of nine steps, ending in a certificate

S06

UML: SEQUENCE · PHASE 1

Go Ye: record a new believer and start follow-up

From a name and phone number to a discipler's call list



HOW TO READ IT

This is the heart of the Go Ye page. A name and a phone number become a **Convert**, which opens a **FollowUpCase** assigned to one discipler. The website counter "Born again so far" counts only unique, counted converts.

Phone numbers are normalised (+2547...) and hashed to a **dedupeKey** so the same person cannot be counted twice when two team members both record them.

MISTAKES TO AVOID

- Phone numbers and "decision for Christ" are sensitive. The list is visible only to Disciplers of that branch, the Branch Admin and the Super Admin.
- Never send the converts list in e-mail attachments.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

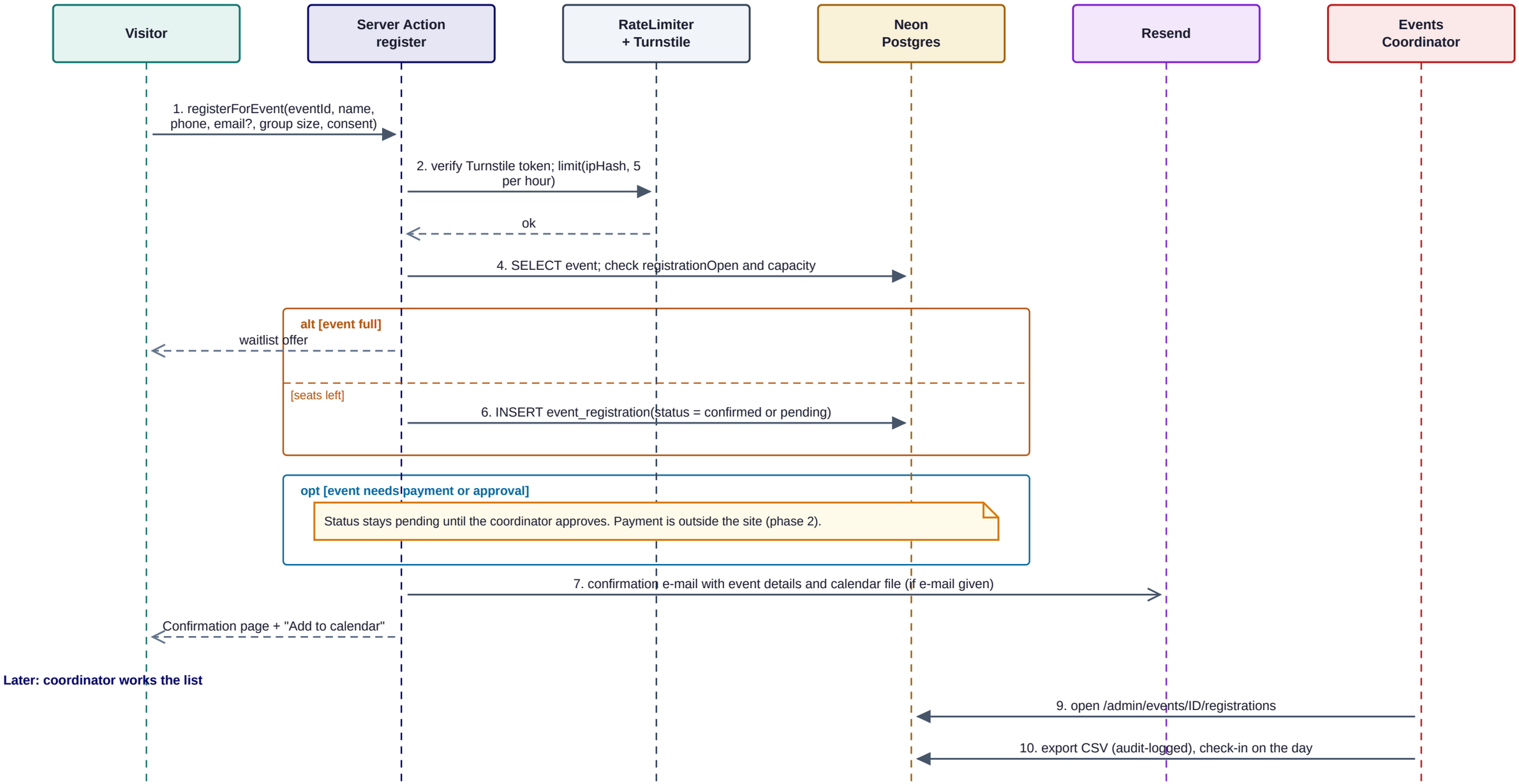
- **R-07** Go Ye: outreach, born-again counter, a form to add a convert

S07

UML: SEQUENCE · PHASE 1

Event registration (youth trip, conference)

Rate limit, capacity check, confirmation



HOW TO READ IT

Registration is a **PersonRequest**: name, phone, optional email, consent, status. The same inherited columns serve events, programmes, join requests and converts.

Capacity is checked **inside the same database transaction** that inserts the row, so two people cannot take the last seat at the same moment.

MISTAKES TO AVOID

- Do not check capacity in the browser or in a separate query. Do it in one transaction (or with a unique counter row).
- Exporting CSVs of personal data must be an audited action.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

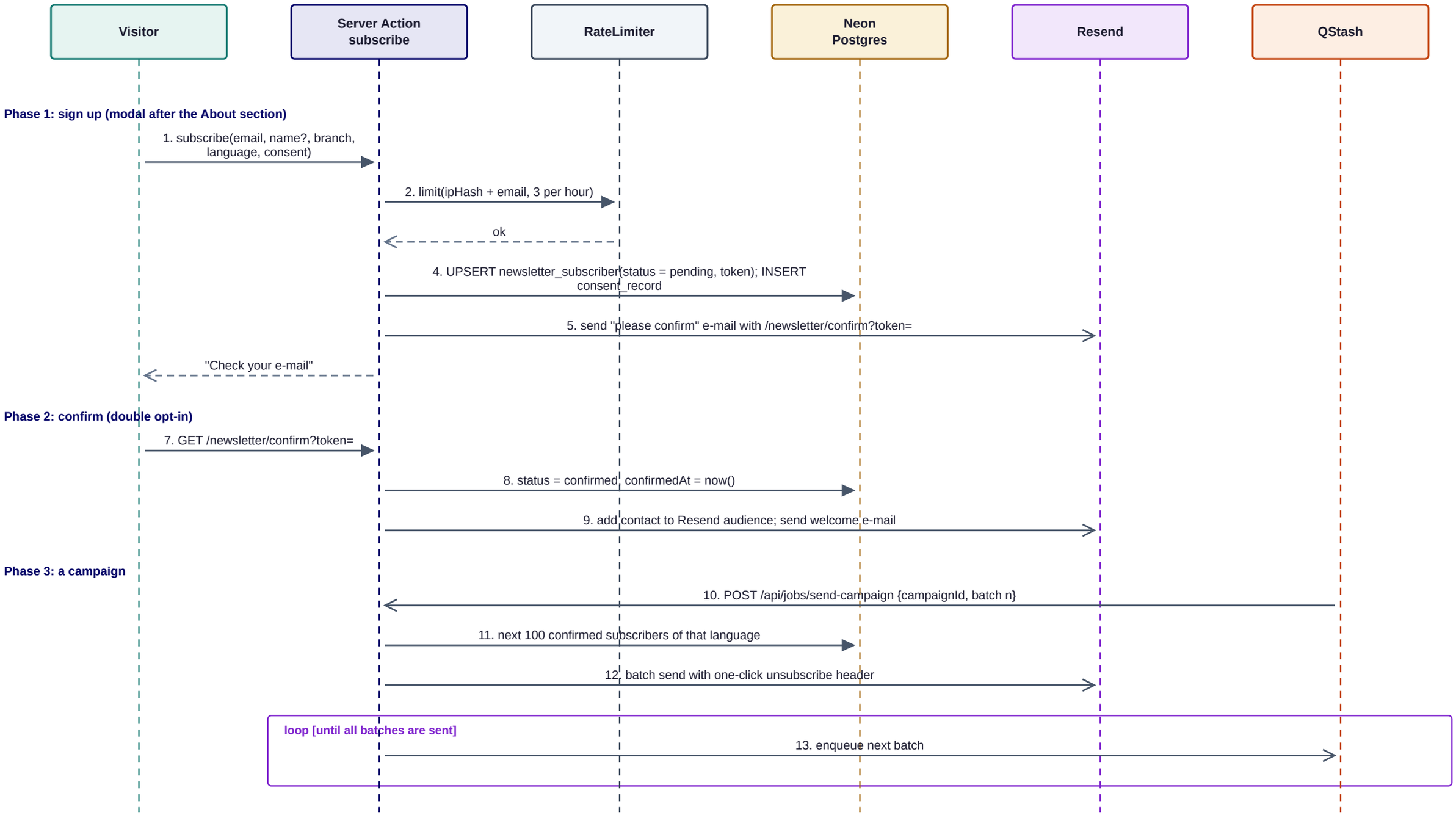
- **R-05** Events with dates, guest ministers, sessions and registration

S08

UML: SEQUENCE · PHASE 1

Newsletter: double opt-in and batch sending

Confirm by e-mail, send in batches, unsubscribe in one click



HOW TO READ IT

Double opt-in means: type your e-mail, then click the link we send to prove it is really yours. It stops people from signing up someone else, and keeps Resend from flagging us as a spammer.

Campaigns are sent in **batches** by QStash so no single request runs for minutes.

MISTAKES TO AVOID

- Every e-mail must carry a working unsubscribe link. Gmail and Yahoo now require it for bulk senders.
- Set SPF, DKIM and DMARC records for the sending domain before the first campaign.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

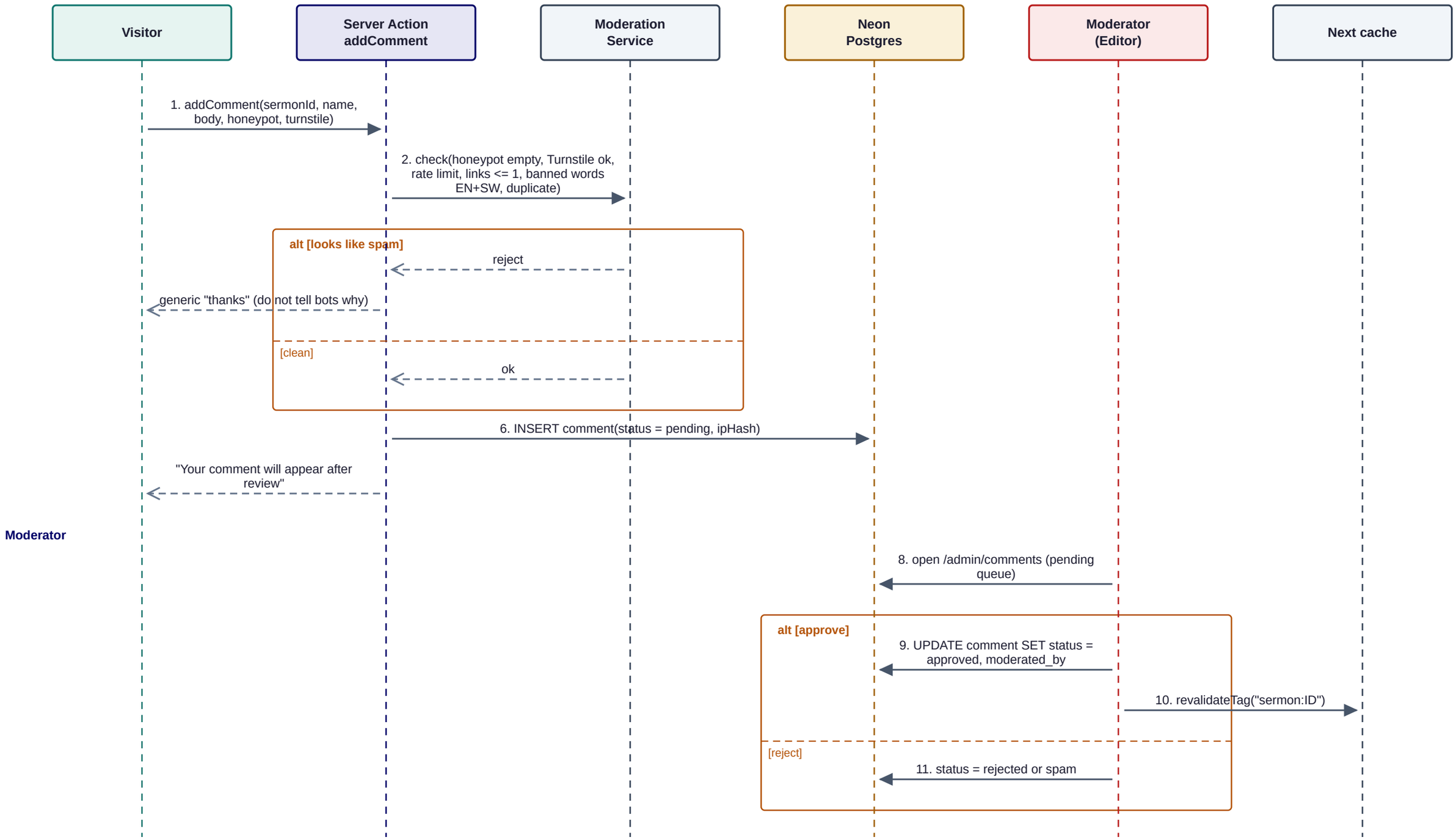
- **R-16** Newsletter with a confirmed sign-up

S09

UML: SEQUENCE · PHASE 1

Comment: submit, filter, moderate

Spam defence in layers, then a human



HOW TO READ IT

Spam defence is a stack: a hidden **honeypot** field (bots fill it, people do not), **Turnstile** (a friendlier CAPTCHA), a **rate limit**, a quick content check, and finally a **human** who approves.

A rejected bot is told "thanks" like everyone else. If we explain why it failed, the bot author learns what to change.

MISTAKES TO AVOID

- Publishing comments instantly. The first spam wave will land on a church page.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

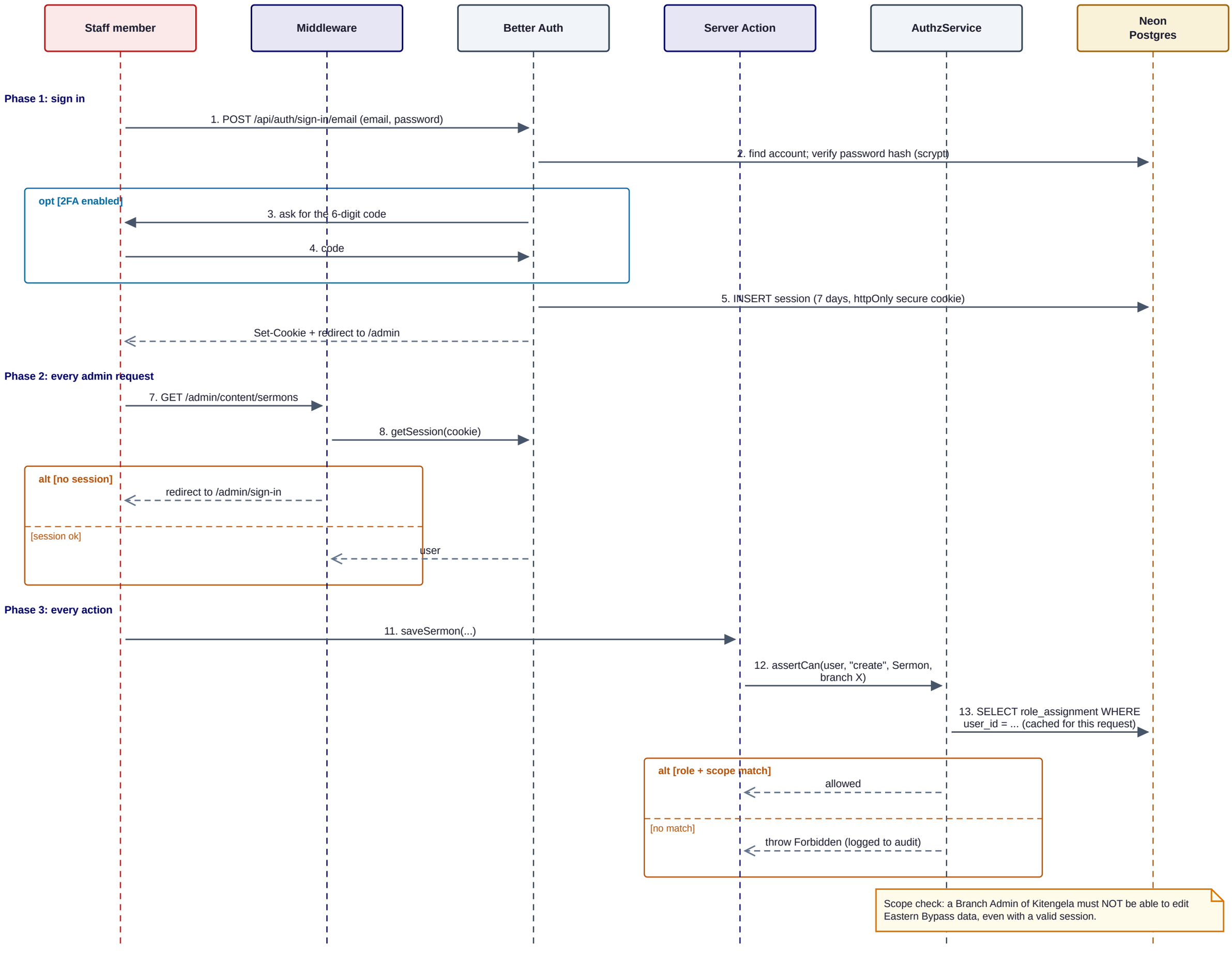
- **R-11** Sermons with video, audio, summary, verses, filters, views, comments and likes
- **R-17** Comments and likes with moderation

S10

UML: SEQUENCE · PHASE 1

Staff sign-in and the three security gates

Gate 1 session, gate 2 route, gate 3 action + scope



HOW TO READ IT

Think of a building with three doors. **Door 1**: do you have a valid badge (session)? **Door 2**: is this corridor for you (middleware on /admin)? **Door 3**: may you do this thing, in this branch (AuthzService)?

Hiding a button in the page is **not** security. Anyone can call the server action directly, so Door 3 must live on the server, next to the data.

MISTAKES TO AVOID

- Checking only the role name ("editor") and forgetting the scope (which branch). This is the most common authorisation bug.
- Put sign-in behind a rate limit; otherwise people can try thousands of passwords.

WHAT YOU ALREADY DID RIGHT

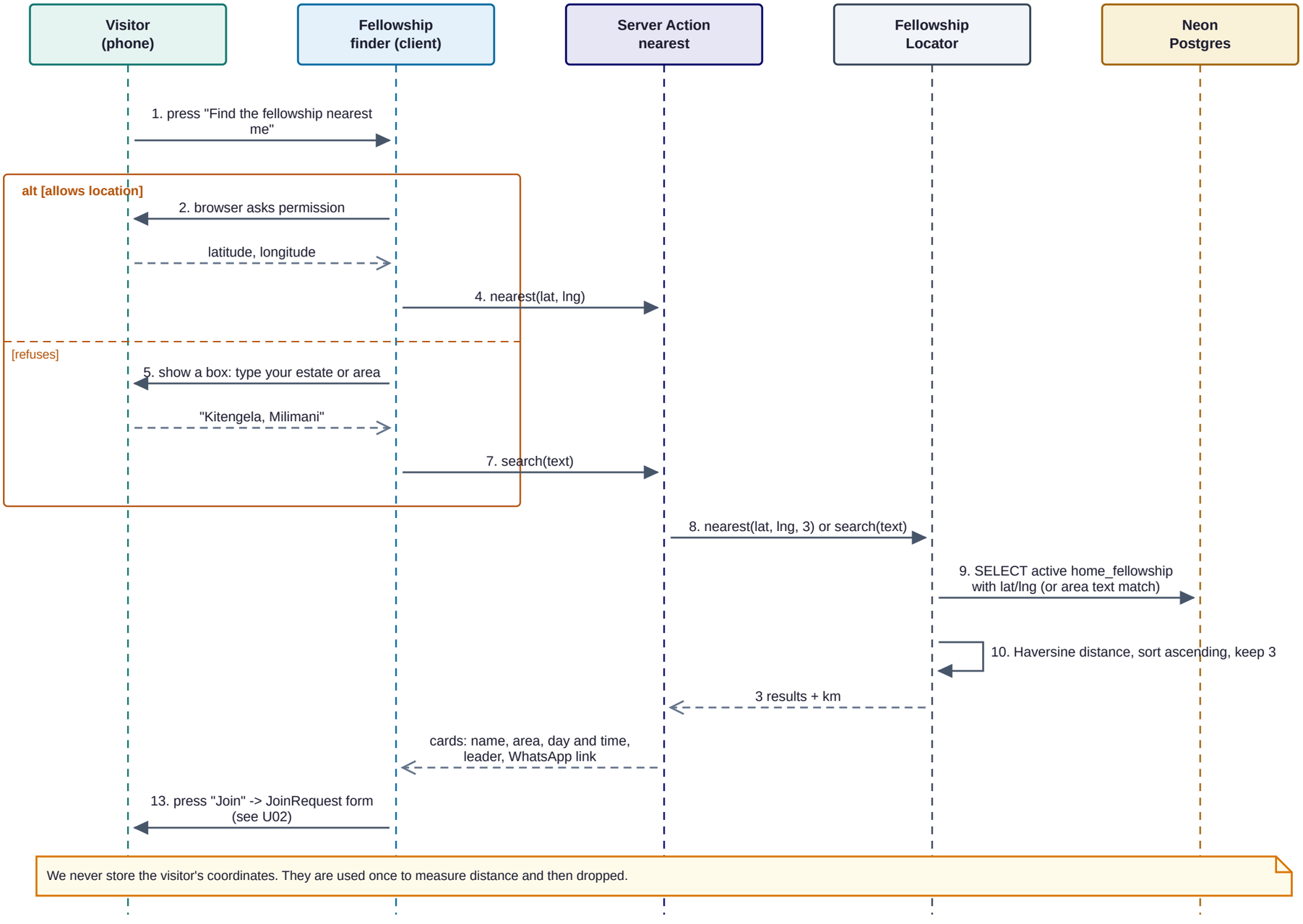
- You chose Better Auth for the admin side only. That keeps the public site fast and simple, and the attack surface small.

S11

UML: SEQUENCE · PHASE 1

Nearest home fellowship finder

Geolocation first, typed estate as the fallback



HOW TO READ IT

Two ways in: **(1)** the phone says where it is (the browser asks permission first), or **(2)** the visitor types their estate. Both end in the same list of three nearest fellowships.

Distance is measured with the **Haversine formula** (straight-line distance on a ball). For a city the size of Nairobi this is accurate enough; no map service is needed to rank results.

MISTAKES TO AVOID

- Do not guess the location from the IP address: on mobile data in Kenya the IP often points to the operator's gateway, not the visitor.
- Fellowship leaders must keep their pin location accurate (a yearly reminder job helps).

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

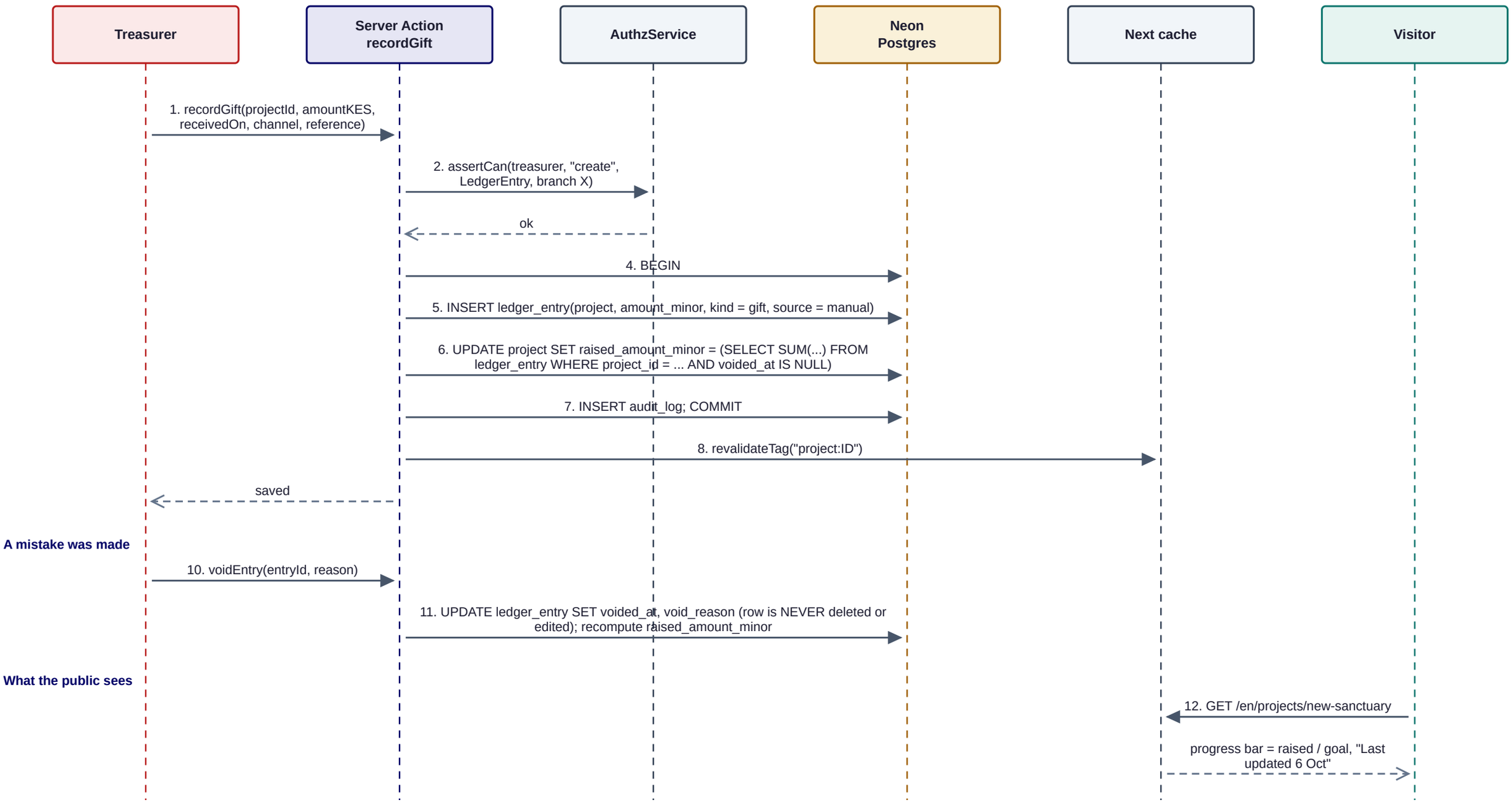
- **R-04** Home-fellowship finder (nearest to me)

S12

UML: SEQUENCE · PHASE 1

Giving: record a gift, void a mistake, update the bar

Append-only ledger and a cached total



HOW TO READ IT

The site does **not** take money. The treasurer records what the bank or M-Pesa statement shows. The progress bar is real because it is the sum of those entries.

A ledger is like a cash book written in pen: you never rub out. A wrong entry is **voided** with a reason and stays visible to auditors. The total on the project row is a **cache**, recomputed in the same transaction, so it can never drift.

MISTAKES TO AVOID

- Do not store money as a decimal or float. Use whole shillings in "minor units" (cents) as bigint.
- Do not let two people edit the same ledger row. Appending avoids edit conflicts.

WHAT YOU ALREADY DID RIGHT

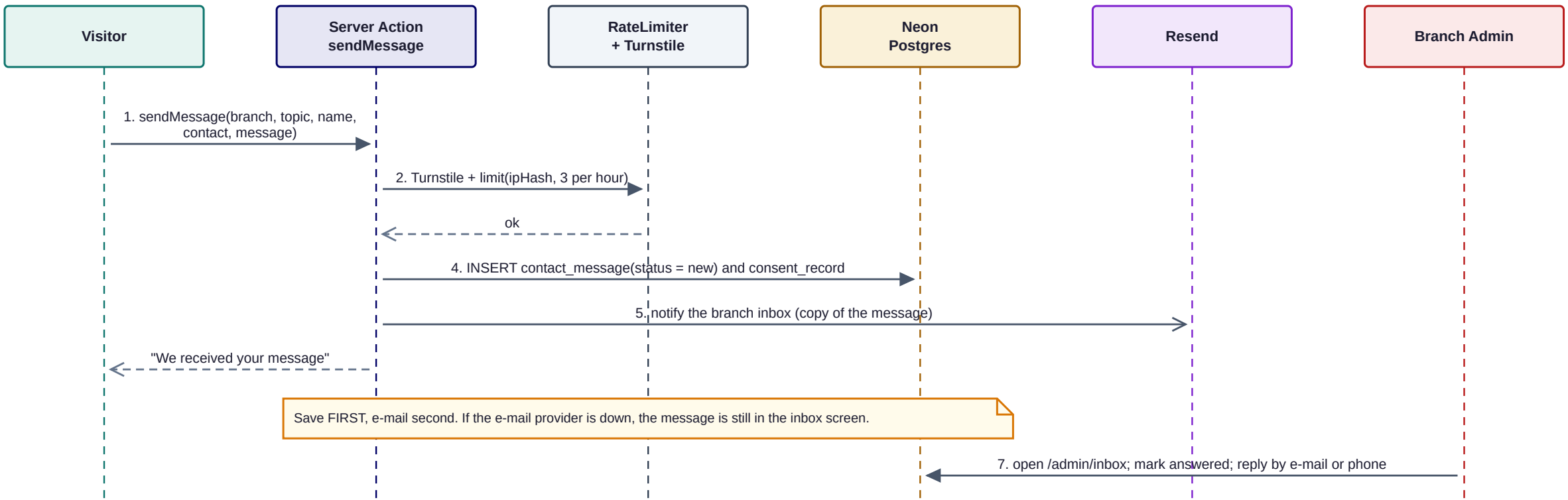
- You chose "details only" for giving now. That removes payment-card compliance work (PCI) from phase 1 and still gives donors an honest progress bar.

S13

UML: SEQUENCE · PHASE 1

Contact message and prayer request

Save first, notify second



HOW TO READ IT

The contact form is the simplest public form, and it shows the standard pattern for all of them: **protect, validate, save, then notify**.

MISTAKES TO AVOID

- Only e-mailing the message and not saving it: when the e-mail bounces the message is lost.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

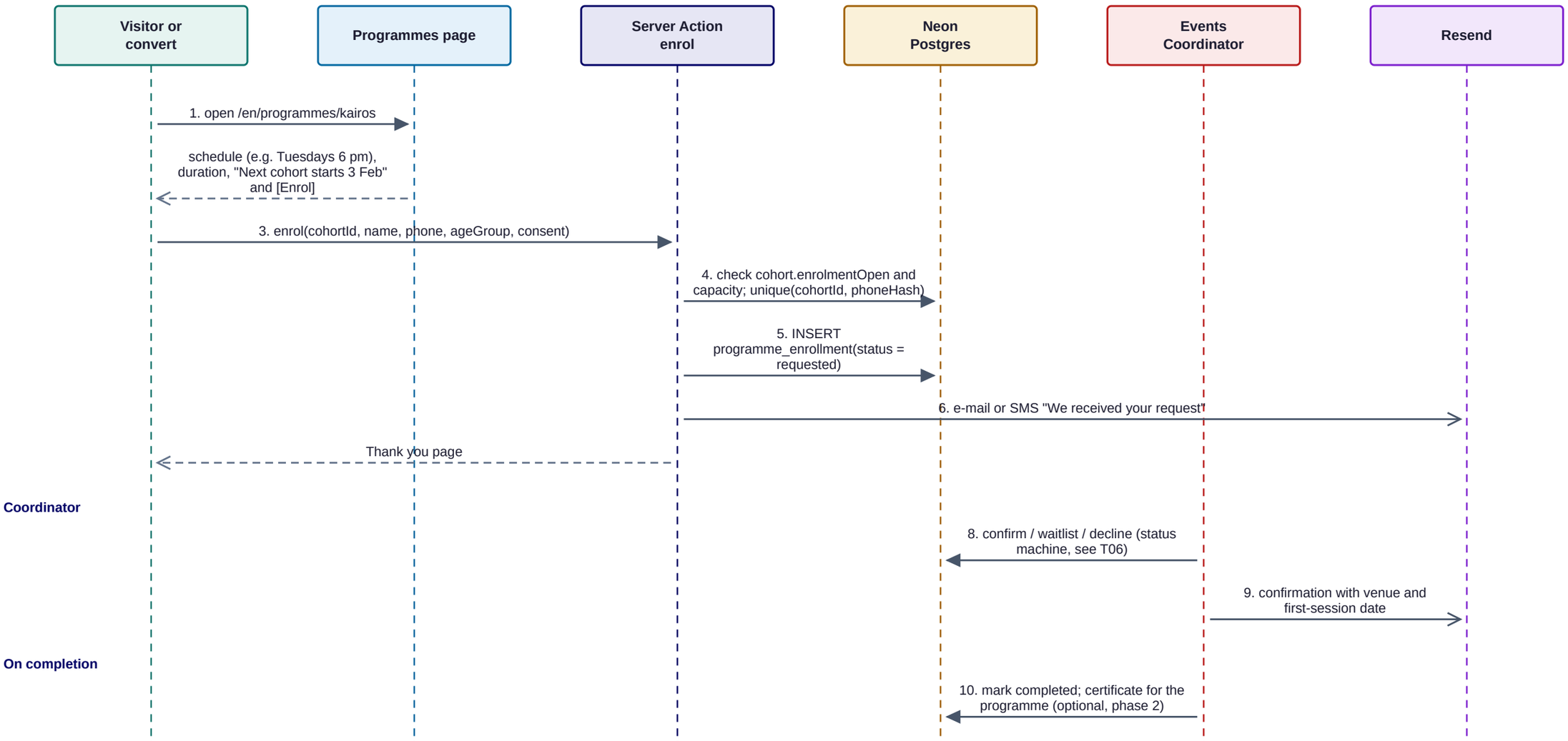
- **R-09** Contact page with giving details

S14

UML: SEQUENCE · PHASE 1

Programme enrolment (Kairos, CFC, Teen 5 Days, ...)

Enrol in the NEXT cohort and track attendance



HOW TO READ IT

A **Programme** is the course; a **ProgrammeCohort** is one run with dates. The page always shows the next open cohort, so the church never edits the page text when dates change.

MISTAKES TO AVOID

- Mixing the course and the run in one table. You will not be able to say "Kairos 2027 Term 1 had 42 people".

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-06** Programmes: Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again

ATLAS · FAMILY 5

UML: state machine

The life story of a thing. Nine life stories: content from draft to archive, translations, comments, the tour journey, follow-up cases, requests, newsletters, projects and ledger lines, and the pop-up manager.

> **EXPLAINED LIKE YOU ARE FIVE**
> Like the stages of a child growing up: each stage has a name and something that moves them on.

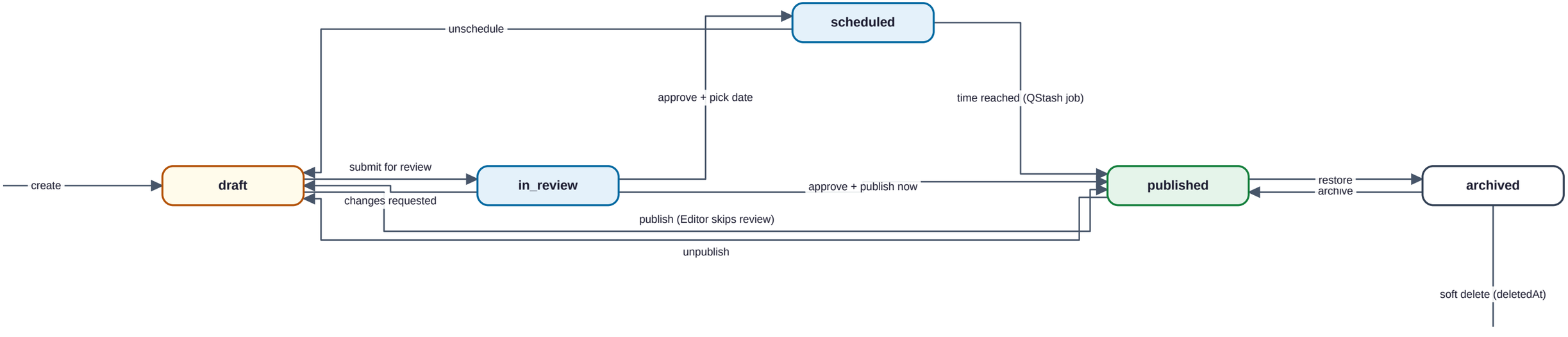
Code	Title	What it shows
T01	Content life-cycle (Publishable)	Draft to published to archived for 9 content classes
T02	Translation status	Why a changed English text sends Kiswahili back to review
T03	Comment status	Every comment starts as pending
T04	New-member journey states	From "No" to a downloaded certificate, with data-deletion paths
T05	Follow-up case states	What a discipler sees for each new believer
T06	Request states (registration, enrolment, join request)	One status machine for everything that extends PersonRequest
T07	Newsletter subscriber and campaign	Two small state machines on one page
T08	Project and ledger entry states	The project life and the (tiny) life of a ledger entry
T09	Overlay orchestrator (client)	Cookie bar, new-member modal, newsletter modal: one at a time, in order

T01

UML: STATE MACHINE · PHASE 1

Content life-cycle (Publishable)

Draft to published to archived for 9 content classes



HOW TO READ IT

Every publishable thing (sermon, blog, bulletin, announcement, resource, project, programme, event, outreach) follows **the same road**. That is the whole point of the Publishable base class: learn the road once.

Only **published** rows whose publishedAt is in the past are visible to the public. Archived rows stay reachable by direct link for 6 months (so shared links do not 404) and are hidden from lists.

"Delete" never removes the row. It sets deletedAt. A Super Admin script can purge after the retention period.

MISTAKES TO AVOID

- Adding a status without adding the transition. Write the allowed arrows in ONE function (canMove(from, to)) and test it.
- A scheduled item whose time is in the past because the job failed: the admin list must show an "overdue" warning.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

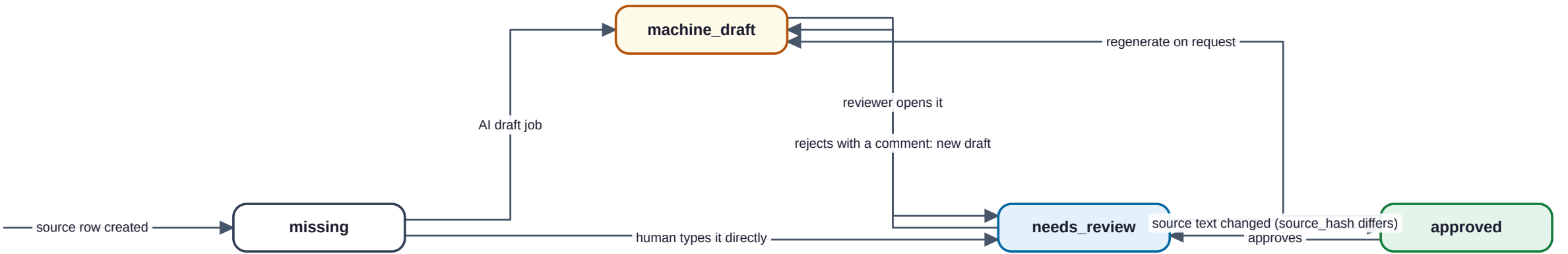
- **R-10** Resources: blogs, bulletins, announcements (Sunday and midweek), Bible study and discipleship, other
- **R-21** Publishing with a review step and scheduling

T02

UML: STATE MACHINE · PHASE 1

Translation status

Why a changed English text sends Kiswahili back to review



HOW TO READ IT

The status lives on each row of every `*_translation` table. The public site shows only **approved** text; everything else falls back to the other language with a badge.

The arrow **approved to needs_review** is the safety net. The English text has a fingerprint (source_hash). If it changes, the Kiswahili row was written for an older text, so it must be checked again.

MISTAKES TO AVOID

- Letting a machine_draft go public "just this once". It will happen again.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

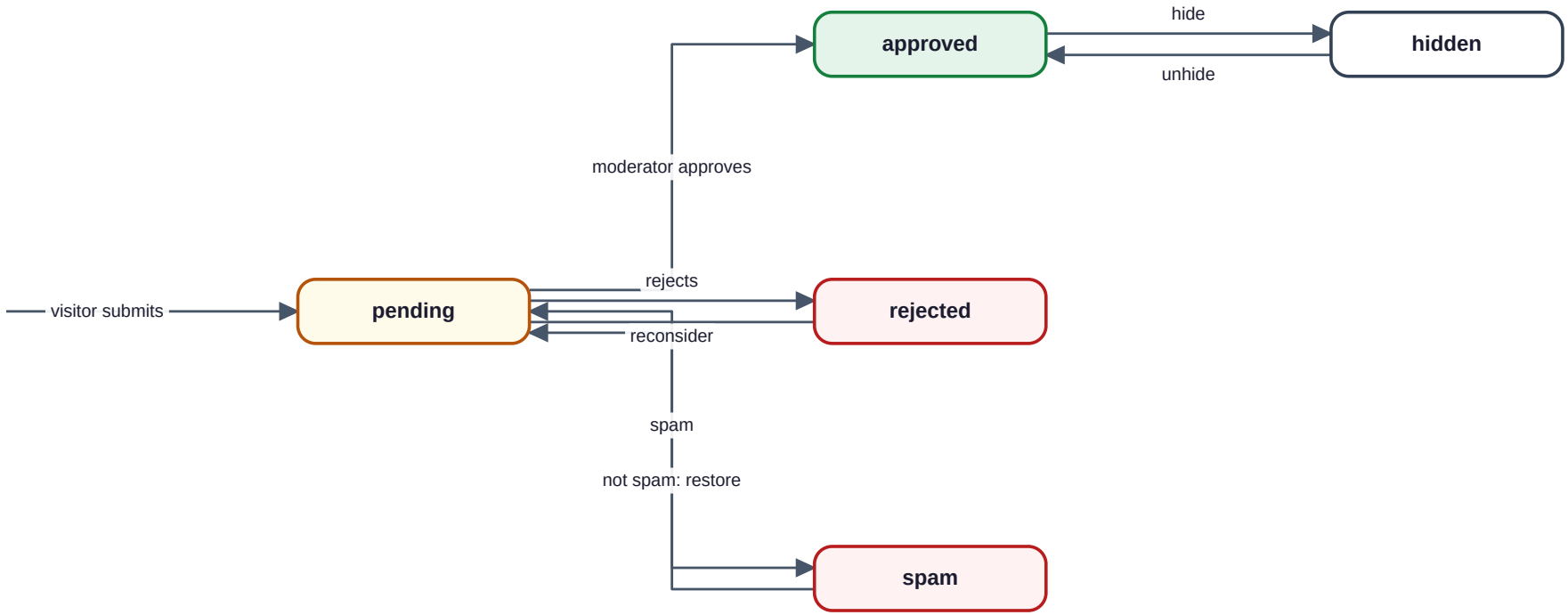
- **R-12** English and Kiswahili switch
- **R-13** Help with translating dynamic content (blogs, sermons, bulletins and so on)

T03

UML: STATE MACHINE · PHASE 1

Comment status

Every comment starts as pending



HOW TO READ IT

A comment is public only in the **approved** state. Everything else is invisible to visitors but kept so moderators can reverse mistakes and learn what spam looks like.

MISTAKES TO AVOID

- Deleting spam rows: keep them 30 days. The patterns teach you what to block.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

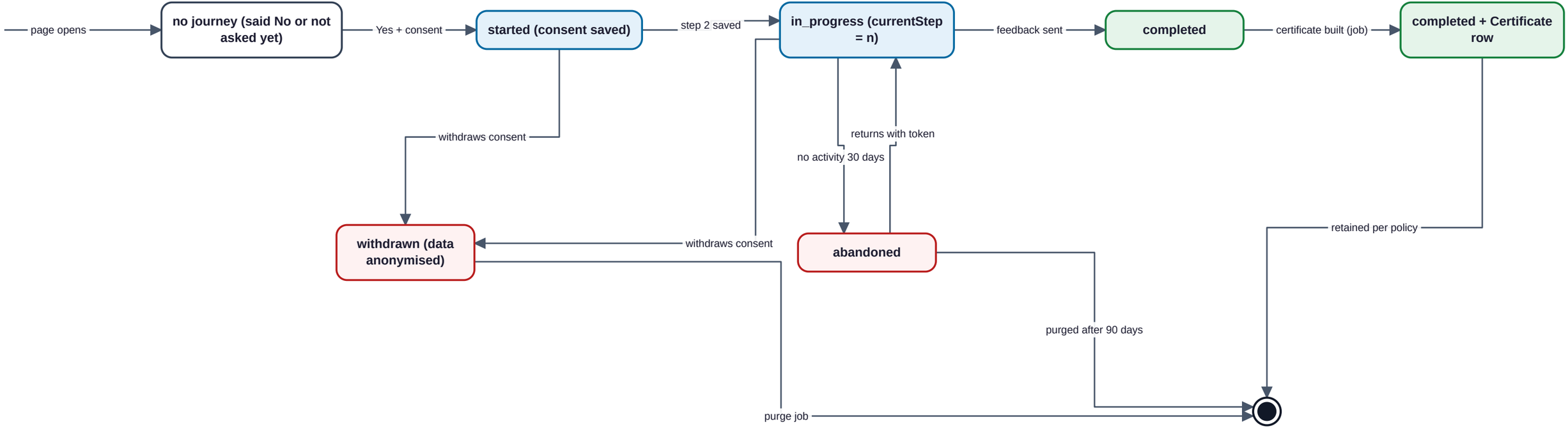
- **R-11** Sermons with video, audio, summary, verses, filters, views, comments and likes
- **R-17** Comments and likes with moderation

T04

UML: STATE MACHINE · PHASE 1

New-member journey states

From "No" to a downloaded certificate, with data-deletion paths



HOW TO READ IT

The first state is special: **no journey exists**. A visitor who answers No stays here forever and costs us nothing.

Two exits exist for privacy: the person can **withdraw consent** (their profile is anonymised), and an **abandoned** journey is deleted after 90 days by a scheduled job.

MISTAKES TO AVOID

- Keeping half-finished journeys forever. Data you do not need is a liability under the Data Protection Act.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

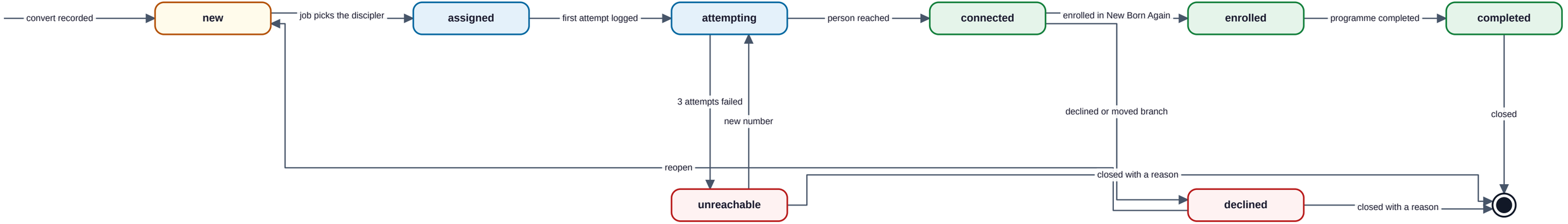
- **R-14** New-member tour of nine steps, ending in a certificate

T05

UML: STATE MACHINE · PHASE 1

Follow-up case states

What a discipler sees for each new believer



HOW TO READ IT

A case always has an owner (the assigned discipler) and a due date. **Overdue** is not a state; it is a warning computed from dueAt, so a case can be both "attempting" and overdue.

There is no "closed" status. A case is closed when **closedAt** and a **closeReason** are filled in, and the status then says how it ended: completed, declined or unreachable.

MISTAKES TO AVOID

- Closing a case without a reason. Reasons are how you learn why people drift away.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

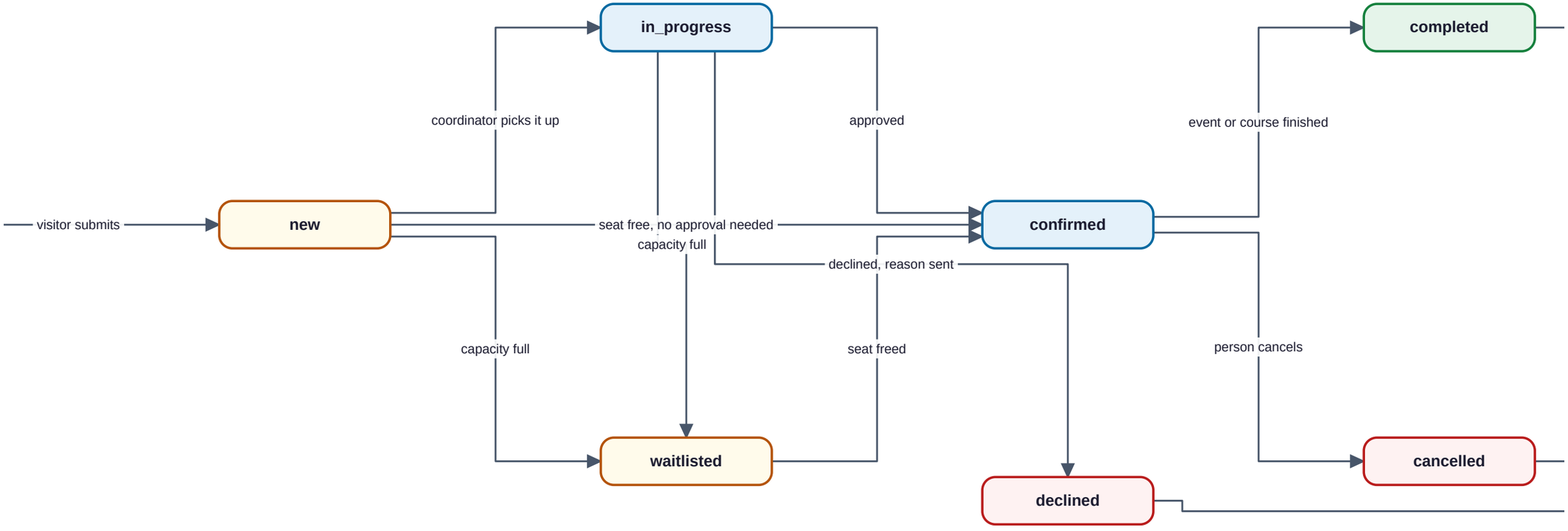
- **R-07** Go Ye: outreach, born-again counter, a form to add a convert
- **R-23** Follow-up of new believers by a discipler

T06

UML: STATE MACHINE · PHASE 1

Request states (registration, enrolment, join request)

One status machine for everything that extends PersonRequest



HOW TO READ IT

Because EventRegistration, ProgrammeEnrollment and JoinRequest all extend PersonRequest, they share **one status machine** (the RequestStatus list on E12). A join request usually ends at confirmed or declined and never uses completed.

Whether a person actually came is not a status. Check-in on the day sets **checkedInAt** on an event registration, and the attendance count on an enrolment; the status stays confirmed until the event or course is finished.

MISTAKES TO AVOID

- Giving each request table its own status words ("pending", "new", "open"). Reports then need three different queries.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-05** Events with dates, guest ministers, sessions and registration

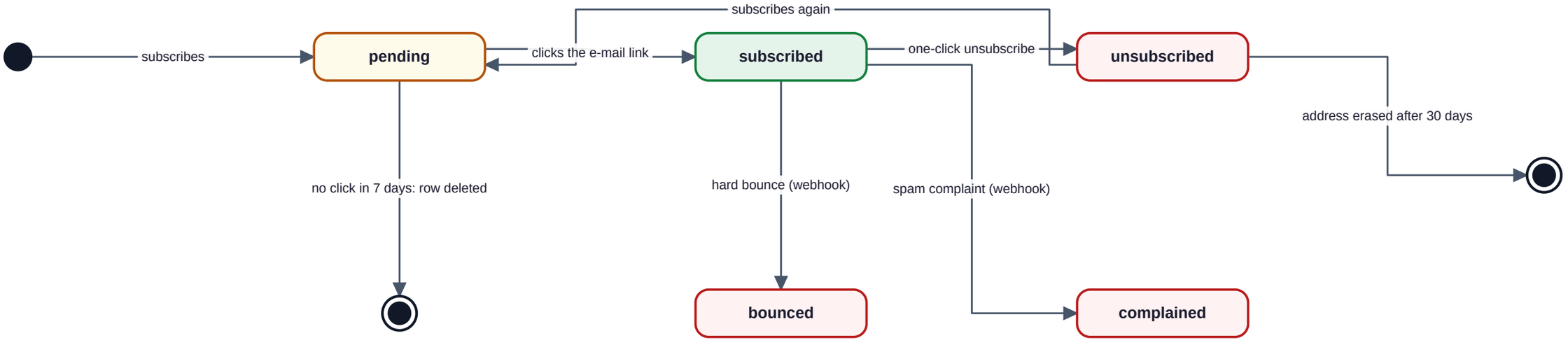
T07

UML: STATE MACHINE · PHASE 1

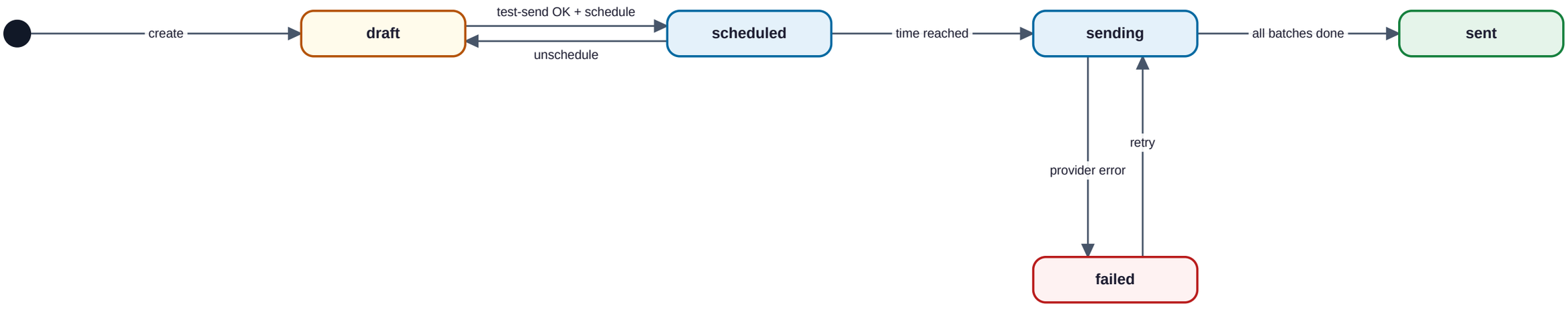
Newsletter subscriber and campaign

Two small state machines on one page

Subscriber



Campaign



HOW TO READ IT

Subscriber (top): only **subscribed** addresses (those that clicked the link) receive campaigns. A pending address that is never confirmed is deleted after 7 days by a job. Bounced and complained addresses are never mailed again. **Campaign** (bottom): a draft becomes scheduled, then sending in batches, then sent; a failure can be retried.

MISTAKES TO AVOID

- Sending to "pending" addresses. They have not proven the e-mail is theirs.
- Mailing an address that complained. A complaint is the strongest signal an e-mail provider reads; one repeat can damage the sending reputation for everyone.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-16** Newsletter with a confirmed sign-up

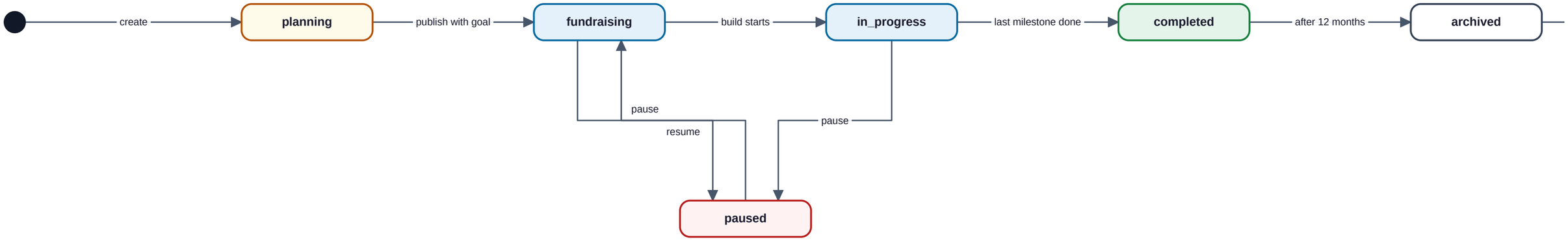
T08

UML: STATE MACHINE · PHASE 1

Project and ledger entry states

The project life and the (tiny) life of a ledger entry

Project



LedgerEntry



HOW TO READ IT

A **Project** moves forward through planning, fundraising and building. Money and build progress are separate: a project can be 90% funded and 20% built.

A **LedgerEntry** (bottom) has only two states: **recorded** and **voided**. There is no edit and no delete. That is deliberate.

MISTAKES TO AVOID

- A "completed" project with an open fundraising bar. Completing should close giving or show "fully funded".

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

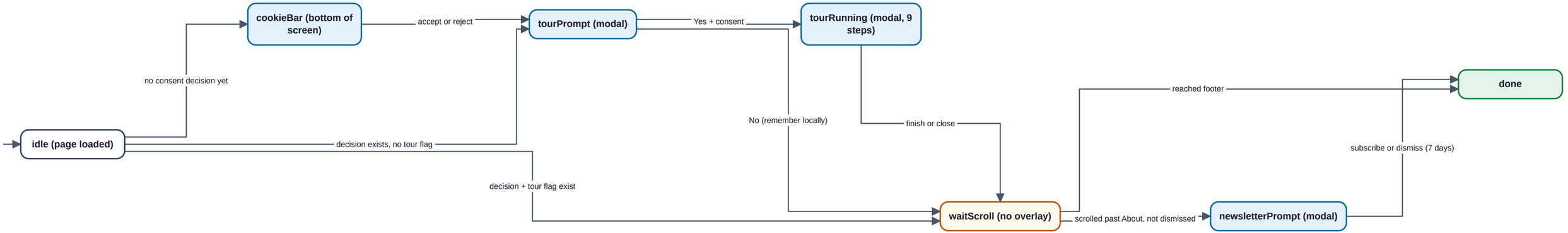
- **R-08** Projects with milestones and a given / remaining bar
- **R-24** Treasurer's record of gifts that drives the project bars

T09

UML: STATE MACHINE · PHASE 1

Overlay orchestrator (client)

Cookie bar, new-member modal, newsletter modal: one at a time, in order



HOW TO READ IT

Three things want the visitor's attention: the **cookie bar**, the **new-member question**, and the **newsletter modal**. If they all appear together on a phone, the visitor leaves.

The orchestrator is a tiny client-side state machine. It decides **who goes next**, remembers decisions in localStorage (and a cookie if cookies were accepted), and never lets two modals overlap.

The cookie bar sits at the bottom and does not block reading, so it can show first without hurting the experience; the modals wait for it.

MISTAKES TO AVOID

- Showing the newsletter modal on the first second of the first visit. Wait until they have scrolled past About.
- Forgetting a "never ask again" memory. A dismissed popup that returns on every page is the fastest way to lose a visitor.

WHAT YOU ALREADY DID RIGHT

- You asked for this order yourself (cookie bar, new-member question, then newsletter after About). It is the right order: consent first, then help, then ask.

ATLAS · FAMILY 6

UML: component and package

Folders and addresses. Where code lives in the repository (K01) and every web address on the site with who may open it (K02).

> **EXPLAINED LIKE YOU ARE FIVE**
> Like the directory board at the gate: this wing holds this, that door is for staff.

Code	Title	What it shows
K01	Package diagram: the code folders and who may import whom	Where every class lives in the repository and which direction imports are allowed
K02	Route map: every address in the website	Public pages, staff pages and the machine doors, with who may open each one

K01

UML: COMPONENT AND PACKAGE · PHASE 1

Package diagram: the code folders and who may import whom

Where every class lives in the repository and which direction imports are allowed

src/ : the application

app/ Routes only: they ask, they do not decide

- **[locale]/(public)/** — *every public page, English and Kiswahili*
- **admin/** — *staff area (English), behind Better Auth*
- **api/jobs/* api/webhooks/*** — *QStash and Resend call these; signature checked*
- **api/auth/[...all]** — *Better Auth endpoints (staff only)*
- **proxy.ts** — *runs first: language redirect, staff gate, headers*
- **sitemap.ts robots.ts opengraph** — *what search engines read*

renders

components/ Pieces of screen: no rules, no database

- **ui/** — *buttons, cards, tabs*
- **forms/** — *fields, Zod messages*
- **overlays/** — *cookie bar, tour, newsletter*
- **admin/** — *tables, editors*

forms and buttons call actions

modules/<area>/ One folder per business area (nine areas)

Example: modules/content/ (every area has the same four files)

actions.ts — *'use server': check input (Zod), call service*
→ **service.ts** — *the rules: permission, status, events*
→ **repo.ts** — *database questions only (Drizzle)*
→ **schema.ts** — *table definitions for this area*

- **iam** — *8 classes*
- **org** — *6 classes*
- **content** — *13 classes*
- **events** — *7 classes*
- **outreach** — *5 classes*
- **giving** — *6 classes*
- **onboard** — *7 classes*
- **engage** — *5 classes*
- **platform (kernel)** — *17 classes: services, media, settings*

only service.ts and repo.ts reach out

lib/ One adapter per outside tool: only services import these

- **auth.ts** — *Better Auth*
- **db.ts** — *Drizzle + Neon*
- **redis.ts** — *Upstash Redis*
- **jobs.ts** — *QStash*
- **mail.ts** — *Resend*
- **sms.ts** — *Africa's Talking*
- **storage.ts** — *Cloudflare R2*
- **analytics.ts** — *PostHog*
- **antispam.ts** — *Turnstile*
- **translate.ts** — *translation engine*
- **monitor.ts** — *Sentry*
- **env.ts** — *checked settings (Zod)*

Beside src/ (not application code)

- **drizzle/** — *SQL migrations made by drizzle-kit*
- **messages/** — *en.json and sw.json: interface words*
- **tests/** — *unit, integration (Neon branch), end-to-end*
- **scripts/** — *seed data, imports, one-off fixes*
- **.github/workflows/** — *CI: types, lint, tests, secret scan, preview*
- **docs/** — *this design set and the decisions*
- **next.config.ts drizzle.config.ts** — *build and database settings*
- **.env.example** — *names of settings, never the secrets*
- **eslint boundaries** — *enforces the rules in the box below*

Five import rules (checked by the linter)

1. app/ and components/ may import a module only through its index.ts or actions.ts, never repo.ts.
2. Only service.ts uses a lib/ adapter. Only repo.ts uses db.ts.
3. A module may import another module's index.ts. Never its repo.ts or schema.ts.
4. lib/ imports nothing from modules/ or app/.
5. iam, org and platform form one kernel (they know each other). Events with onboard and with outreach talk only through service calls and ids (see B03).

HOW TO READ IT

This page is the **map of the building** as the developers will see it: a folder tree. The big grey rectangle `src/` is the application. Inside, four layers sit one above the other, and arrows only go **downwards**: a route (`app/`) asks, a piece of screen (`components/`) shows, a module decides, a `lib/` adapter talks to an outside company.

Every business area under `modules/` has the **same four files**: `actions.ts` (the door that forms knock on), `service.ts` (the rules), `repo.ts` (the questions to the database) and `schema.ts` (the tables). A new volunteer who learns one module can read all nine. This is the "layer rule" of B03 written as folders.

Why `lib/` has one file per tool: if Resend is replaced by another mail company, only `mail.ts` changes. The same idea is drawn as the service table on B09, and the data stores behind each adapter on F01.

MISTAKES TO AVOID

- Calling the database from a page "just to make it quick". Pages that talk to the database directly become impossible to test and to secure.
- Putting rules in `actions.ts` . An action is a thin door: check the shape of the input, call the service, return the result. Rules in the door get copied into the next door.
- Letting two modules read each other's tables. When the table changes, both modules break. Ask the other module's service instead.
- Importing a vendor package (Resend, PostHog) anywhere except its adapter in `lib/` . The next developer will not know it exists until it fails.

WHAT YOU ALREADY DID RIGHT

- You asked for the stack to be named and for integration to be planned: every tool in your list has a home (Better Auth in auth.ts, Drizzle and Neon in db.ts, Resend in mail.ts, Upstash in redis.ts, QStash in jobs.ts, PostHog in analytics.ts, R2 in storage.ts), and the extra tools I suggested sit beside them.

Route map: every address in the website

Public pages, staff pages and the machine doors, with who may open each one

proxy.ts runs before every request
1) adds /en or /sw if missing 2) sends /admin to sign-in if no session 3) adds security headers

adds /en or /sw

/[locale]/ : public pages (locale = en or sw)

- **/** — *Home: hero, service times, next event, latest sermon, born-again counter*
- **/about** — *History, vision, leaders, timeline*
- **/departments and /departments/[slug]** — *List; one department with its join form*
- **/fellowships** — *Home-fellowship locator: map and nearest*
- **/events and /events/[slug]** — *List; detail and registration*
- **/programmes and /programmes/[slug]** — *Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again; enrolment*
- **/goye** — *Outreach stories and the born-again counter*
- **/goye/record** — *Form to record a new believer*
- **/projects and /projects/[slug]** — *Projects and the giving bar*
- **/resources** — *Hub for the six kinds of resource*
- **/resources/blog and .../[slug]** — *Blog list and post*
- **/resources/bulletins and .../[slug]** — *Weekly bulletins*
- **/resources/announcements** — *Announcements*
- **/resources/discipleship** — *Bible study and discipleship material*
- **/resources/sermons and .../[slug]** — *Sermons: video, audio, notes*
- **/resources/other** — *Everything else*
- **/contact** — *Contact form, prayer request, giving details*
- **/search** — *Search across published content*
- **/newsletter/confirm and /unsubscribe** — *Links in e-mails (one-time tokens)*
- **/certificate/[code]** — *Check that a certificate is real (shows valid or not)*
- **/privacy and /cookies** — *The two policies*
- **Overlays (no address)** — *cookie bar, new-member tour, newsletter modal: one OverlayOrchestrator on every public page*

needs a session

/admin/ : staff pages (English; Better Auth, then role and scope)

- **/admin/login** — *Sign in and six-digit code. Everyone; 5 tries then 15 minutes wait*
- **/admin** — *Dashboard: my tasks. Any staff member*
- **/admin/content** — *Blogs, sermons, bulletins, announcements, resources. Editor, Branch, Super*
- **/admin/media** — *Pictures, audio, PDFs. Editor, Department, Events*
- **/admin/comments** — *Moderation. Editor, Branch, Super*
- **/admin/translations** — *Review queue. Translator, Editor*
- **/admin/events** — *Events and registrations. Events coordinator, Department leader*
- **/admin/programmes** — *Programmes, cohorts, enrolments. Events coordinator, Discipler*
- **/admin/requests** — *Join requests and registrations. Department leader, Events coordinator*
- **/admin/goye** — *Activities, converts, cases. Discipler; Events (activities)*
- **/admin/org** — *Branches, departments, fellowships, leaders. Branch, Super; Department (own)*
- **/admin/projects** — *Projects and milestones. Branch, Super; Editor (words)*
- **/admin/giving** — *Ledger. Treasurer writes; Super and Branch read*
- **/admin/newsletter** — *Subscribers and campaigns. Editor, Super*
- **/admin/messages** — *Contact and prayer messages. Branch, Super (Treasurer, Discipler read)*
- **/admin/users** — *Staff and roles. Super, Branch*
- **/admin/audit** — *Who changed what. Super, Branch*
- **/admin/settings** — *Site settings. Super*

signature, not session

/api/ and files: doors for machines

- **/api/auth/[...all]** — *Better Auth (staff sign-in)*
- **/api/jobs/publish** — *QStash: go live at the set time*
- **/api/jobs/newsletter** — *QStash: send one batch*
- **/api/jobs/reminders** — *QStash: event and follow-up reminders*
- **/api/jobs/translate** — *QStash: ask the engine for drafts*
- **/api/jobs/rollup** — *QStash: recount, tidy old rows*
- **/api/webhooks/resend** — *delivery, bounce, complaint*
- **/api/media/sign** — *presigned upload address (staff)*
- **/api/certificate/[id]** — *certificate PDF download*
- **/api/health** — *uptime check*
- **sitemap.xml robots.txt** — *for search engines*
- **manifest opengraph image** — *install name and share picture*

Every /api/jobs/* and /api/webhooks/* checks a signature first (QStash or Resend). A request without a valid signature gets 401 and does nothing.

HOW TO READ IT

This is the **address book** of the whole website. Every row is one web address (a "route") and what lives there. Three groups: pages for **visitors** (teal), pages for **staff** (navy) and doors for **machines** such as the job scheduler (purple).

The public group lives under `/en` and `/sw`. That is how one address means one language (F04). The staff group does **not** carry a language: the admin is in English first, so volunteers do not have two copies of every screen to learn. Translating the admin is a later decision (question Q-16).

Your pop-ups are **not** routes. The cookie bar, the new-member tour and the newsletter modal have no address of their own: one small program (the OverlayOrchestrator, T09) shows them on top of any public page in the right order (F02).

Each staff row shows **who may open it**. That is only a menu hint. The real check happens again on the server for every action (F03).

MISTAKES TO AVOID

- Putting the language in a cookie only. A cookie hides the language from Google and from people who share the link.
- Giving the job doors a password. They are called by QStash, not by people, so they check a **signature** instead.
- Letting a public page show a staff-only link. The /certificate/[code] page shows only "valid" and the first name, never the phone or e-mail.
- Forgetting that dynamic routes need a "not found" page in both languages (F04).

WHAT YOU ALREADY DID RIGHT

- Your brief listed the pages. This map shows each of them has a place and a rule, and that the places you did not list (search, verify a certificate, newsletter links, job doors) are the ones that usually cause surprises later.

ATLAS · FAMILY 7

UML: object

A real example. Two filled-in examples with invented values: Grace's new-member journey (O01) and a Go Ye decision followed up and enrolled (O02). If a design cannot be filled in with sensible values, the design is wrong.

> **EXPLAINED LIKE YOU ARE FIVE**
> Like a form that has been filled in, instead of a blank one.

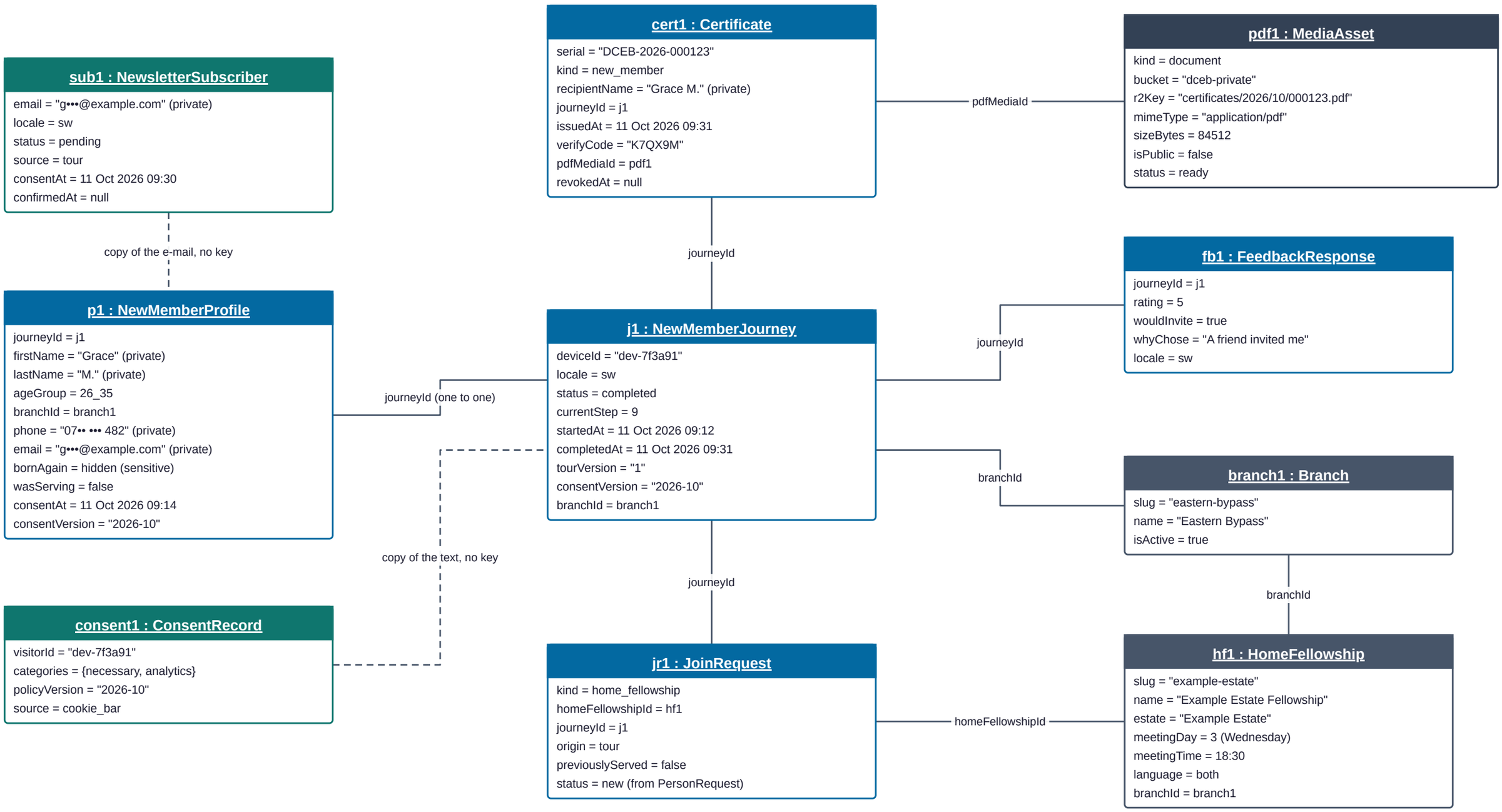
Code	Title	What it shows
O01	Object diagram: one new member's journey, filled in	One invented person after finishing the tour: which rows exist and how they point at each other
O02	Object diagram: a Go Ye decision, followed up and enrolled	One invented outreach, one believer, one discipler and one class: the rows that exist a few weeks later

001

UML: OBJECT · PHASE 1

Object diagram: one new member's journey, filled in

One invented person after finishing the tour: which rows exist and how they point at each other



Example data
Grace and every value here are invented to show the shape. Fields marked private or sensitive are masked on screen, in logs, and in the admin unless the role is allowed to see them.

HOW TO READ IT

A class diagram describes **kinds** of things ("a NewMemberJourney has a status and a step"). An **object diagram** shows **real examples** of those things at one moment ("this journey, *j1*, has status completed and step 9"). It is the best way to check that a design really works: if you cannot fill the boxes in with sensible values, the class design is wrong.

Read it from the middle. **j1** is the journey and the hub: the profile, the feedback, the certificate and the join request each point at it with `journeyId`. The certificate points at a PDF file record (**pdf1**) that tells where the file sits in the private R2 bucket. The join request points at a home fellowship that belongs to the same branch.

Look at the two **dotted** lines. **consent1** (the cookie choice) and **sub1** (the newsletter address) are connected to the journey only by a *copy of a text*, not by a database key. That is on purpose: a person can finish the tour, say no to the newsletter, or clear their cookies, and none of these rows break the others. It also means deleting one person's data (F15) never has to chase a hidden link.

MISTAKES TO AVOID

- Adding a foreign key from the journey to the newsletter list "so we can join them". Then unsubscribing becomes a change to a journey, and the two privacy rules get mixed.
- Storing the certificate PDF inside the table. Store a **file record** and keep the file in R2; the row stays small and the file can be re-made.
- Showing the sensitive row (`bornAgain`) to a role that does not need it. Only the Discipler sees it, and only as part of follow-up.

WHAT YOU ALREADY DID RIGHT

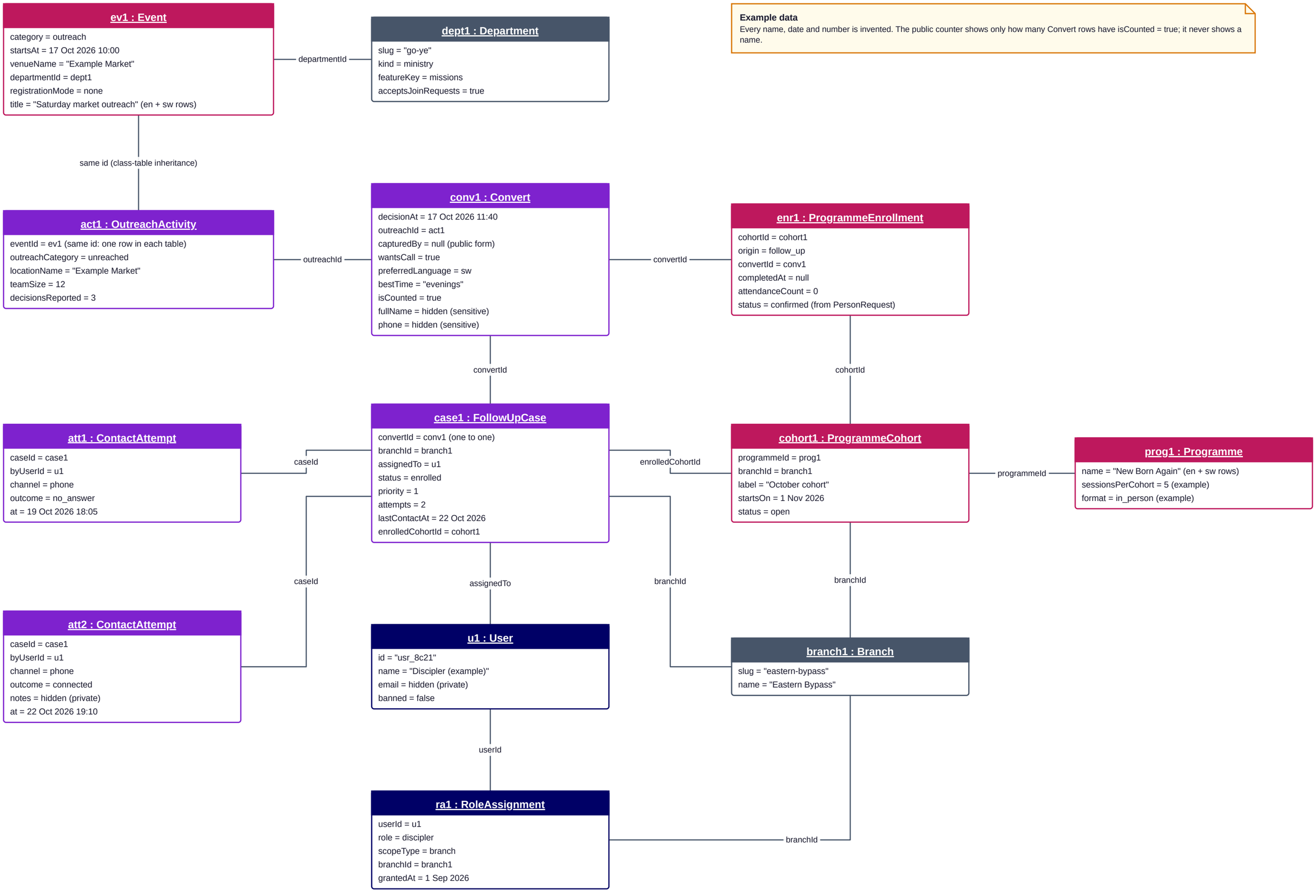
- You asked for a certificate and for a "No" that is remembered. Notice that a "No" to the newsletter creates **no row at all** here: there is nothing to delete and nothing to leak.

002

UML: OBJECT · PHASE 1

Object diagram: a Go Ye decision, followed up and enrolled

One invented outreach, one believer, one discipler and one class: the rows that exist a few weeks later



HOW TO READ IT

This second example follows the most sensitive story in the system: a person gives their life to Christ at a market outreach, and a discipler walks with them until they join the **New Born Again** class. Read it from the left.

First, an **outreach activity** is really an **event** with a few extra facts. The two boxes *ev1* and *act1* share the same id: one row in each table. That is called class-table inheritance and it is why an outreach appears on the events calendar and in the Go Ye reports without being typed twice.

Then the **convert** row (*conv1*) points at the activity. It has exactly one **case** (*case1*) for follow-up. The case has two **contact attempts**, each written by the discipler *u1*, whose power comes from the **role assignment** *ra1* (a discipler, for this one branch). Finally the case points at the **cohort** the person joined, and the **enrolment** *enr1* is the person's seat in that cohort.

MISTAKES TO AVOID

- Letting the discipler see converts from every branch. The role assignment on the right is for **one branch**; the check compares it with the case's `branchId` (F03).
- Counting people by names. The counter counts rows with `isCounted = true`, so correcting a duplicate or a test row changes the number without touching any name.
- Putting the notes of a call in a field anybody can read. `notes` is private: it is shown only to the discipler on the case, and never exported by default.

WHAT YOU ALREADY DID RIGHT

- You asked for a born-again counter and a form to add a convert. This example shows both are supported by one small row (*conv1*) and that the long story after it (calls, notes, class) lives in separate rows, so the counter stays safe to show.

ATLAS · FAMILY 8

Flow and block diagrams

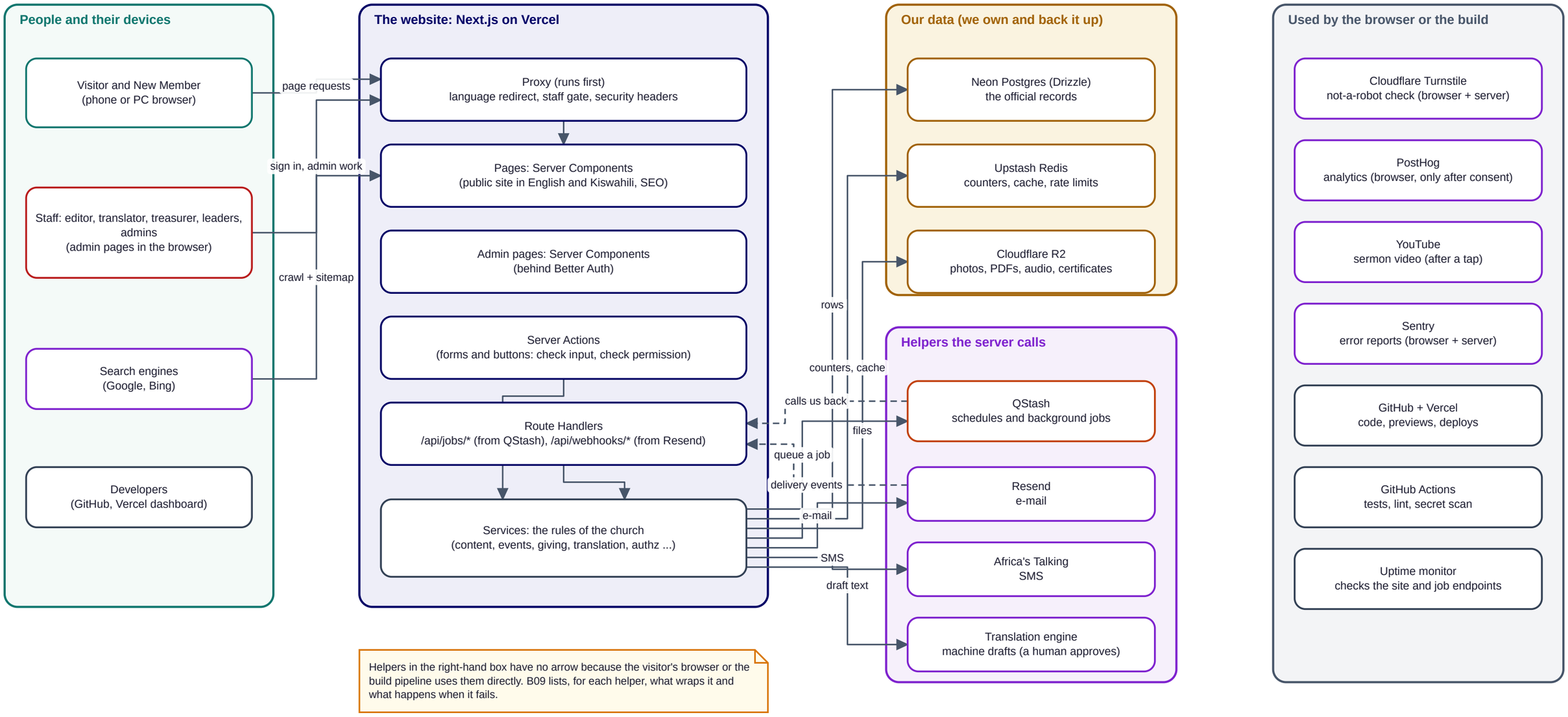
Routes and the whole system. The block diagram of the whole system (F01) and fourteen flows: first visit and the pop-up order, sign-in, a page request, translation, the tour, Go Ye, departments, publishing, events, programmes, giving, newsletter, media and privacy requests.

> **EXPLAINED LIKE YOU ARE FIVE**
> Like the signs and one-way arrows on the walkways of the compound.

Code	Title	What it shows
F01	Block diagram: the whole system on one page	People, the website on Vercel, the data stores and the outside helpers, with the direction of every arrow
F02	First visit: cookie bar and the order of the pop-ups	New-member question first, newsletter question second, and "No" remembered
F03	Staff sign-in and the three gates	Session, two-factor, role, scope: four questions before any button works
F04	One page request: language, cache and fallback	How the site chooses English or Kiswahili and what it does when a translation is missing
F05	Translating dynamic content (blogs, sermons, bulletins)	One page, two languages: write once, review once, never show an unchecked translation
F06	New-member tour: from "Yes" to the certificate	Consent first, nine short steps, hand-offs to staff, and an automatic certificate
F07	Go Ye: from a decision to a discipler's call list	Counting once, assigning fairly, never losing a person
F08	Department page and join request	How a visitor reaches a leader, and how a leader keeps their own page current
F09	Publishing: draft, review, schedule, live	One road for every kind of content, with an optional review gate
F10	Event: create, register, run the day	From a new event to a check-in list, with seats and a waiting list
F11	Programme and cohort: from a course to a completed person	Kairos, CFC, Teen 5 Days, Children 5 Days, New Born Again
F12	Giving and the project progress bar	Money moves outside the site; the site tells the truth about it
F13	Newsletter: sign up, confirm, send, unsubscribe	Double opt-in and batch sending that respects the Data Protection Act
F14	Media upload: from a file to a safe, fast picture	Direct-to-storage upload, checks, resizing and alt text
F15	Privacy request and security incident	What the church does when a person asks about their data, and when something goes wrong

Block diagram: the whole system on one page

People, the website on Vercel, the data stores and the outside helpers, with the direction of every arrow



HOW TO READ IT

This is the **one-page map** of the whole system. Read it left to right: people on the left, our website in the middle, our own data to its right, other companies' services at the far right. Every arrow is one thing that really happens.

The middle box is **one** Next.js application on Vercel. Inside it there is a clear order: the **Proxy** runs first (language and staff gate), then pages or admin pages, then **Server Actions** and **Route Handlers** as the only doors that change things, and below them the **Services** that hold the rules. Only Services talk to the data and to the helpers (this is the layer rule on B03).

The helpers are other companies. If one of them is down (for example e-mail), the website must still work: the page still loads, and the e-mail is retried later by QStash. That is why each helper is wrapped by exactly one service (see B09).

MISTAKES TO AVOID

- Letting a page talk to a helper directly "just this once". You lose the single place where keys, retries and logging live.
- Counting the helpers as part of "our data". Resend, PostHog and YouTube hold copies of some information; the privacy page must list them.
- Forgetting the people at the edge: search engines (they need sitemaps and clean titles) and developers (they need previews, tests and a way to roll back).

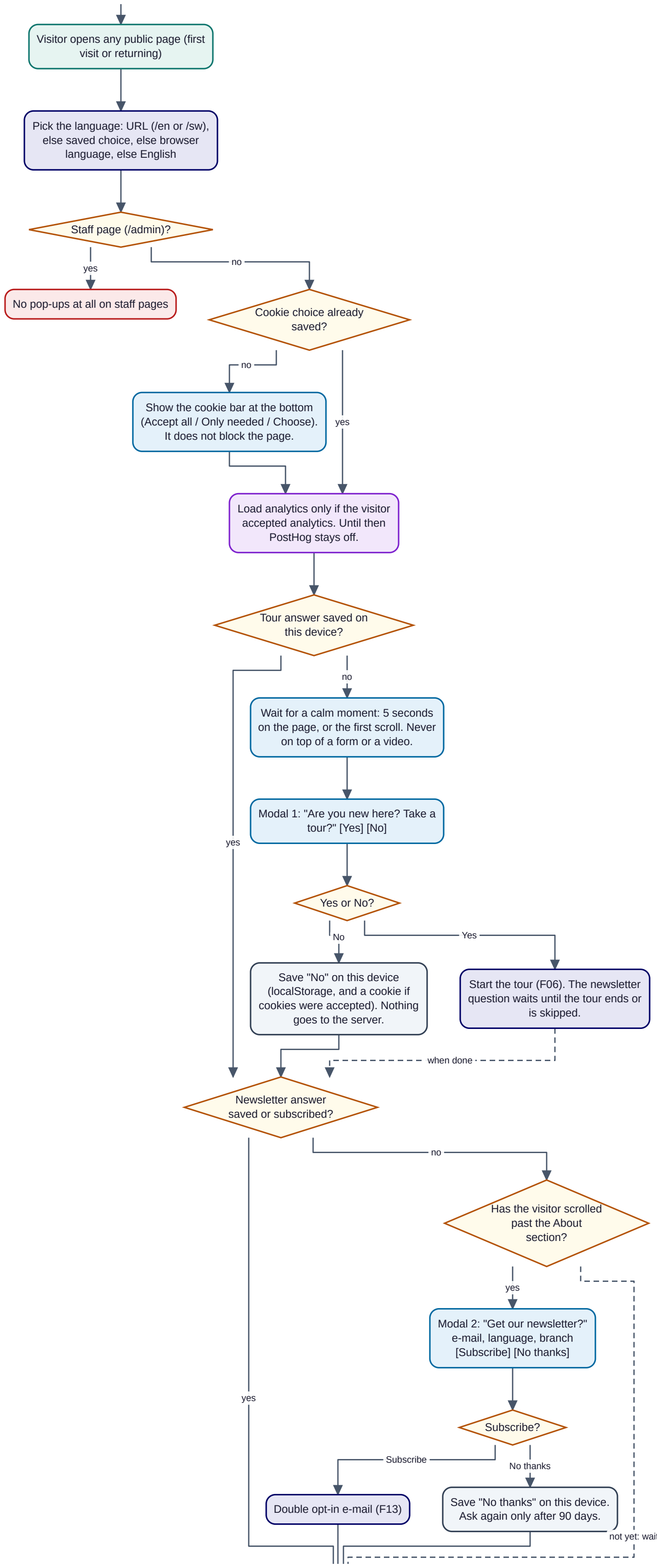
WHAT YOU ALREADY DID RIGHT

- Your stack choices (Next.js, Drizzle, Neon, Upstash Redis, QStash, Resend, PostHog, Better Auth, R2, Vercel) all appear here in the right role. Nothing in the list is idle and nothing important is missing, except small helpers we added (Turnstile, Africa's Talking, Sentry, a translation engine).

FLOW AND BLOCK DIAGRAMS · PHASE 1

First visit: cookie bar and the order of the pop-ups

New-member question first, newsletter question second, and "No" remembered



HOW TO READ IT

A website that greets every visitor with three pop-ups at once feels like a shop assistant who follows you around. This flow puts the pop-ups in a **polite order**: first the cookie bar (small, at the bottom, does not block anything), then the **new-member question**, then the **newsletter question**. Never two at once.

Each answer is remembered **separately** on the visitor's own device (localStorage; and a cookie too if cookies were accepted). The answer "No" is a real answer. For the tour it is kept on that device and not asked again; for the newsletter we ask again only after 90 days (a default the church can change, question Q-21). That is the rule you asked for, and it also means a "No" leaves no trace on our server.

The newsletter question waits until the visitor has scrolled past the **About section** (your idea) so it appears after they know who we are. If the visitor said Yes to the tour, the newsletter question waits until the tour ends, so one person never gets two questions in the same minute.

MISTAKES TO AVOID

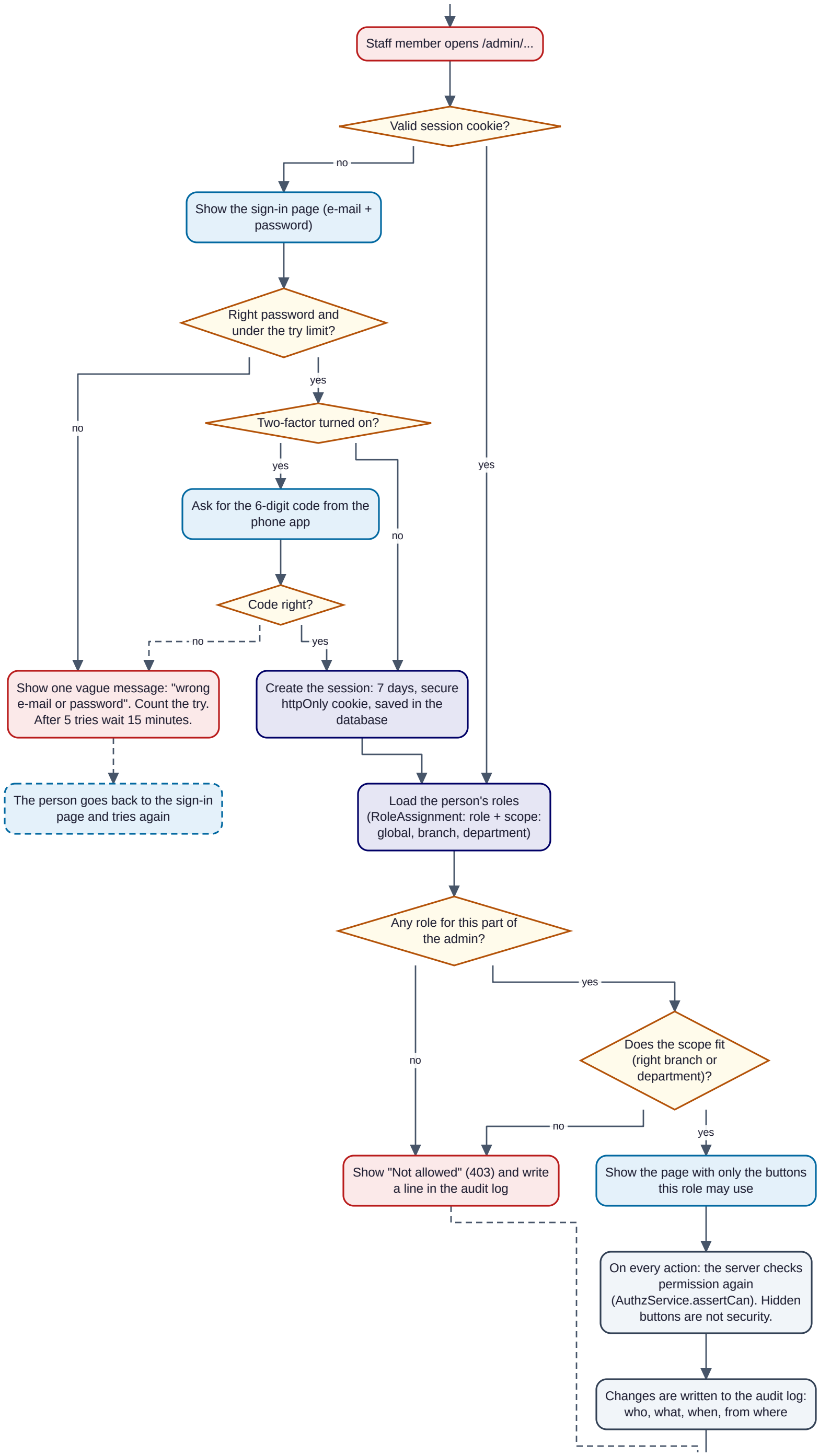
- A modal that cannot be closed with the Esc key or a visible close button. Closing counts as "Not now", not as "No": it is asked again next visit (we propose after the next visit, up to 3 times).
- Loading analytics before the cookie choice. Under the Data Protection Act 2019 non-essential tracking needs the visitor's choice first; PostHog stays off until then.
- Using the same code for visitors and for staff pages. The /admin area never shows any of this.
- Writing "No" to the server so you can "learn from it". It turns a privacy-friendly feature into data you must now protect.

WHAT YOU ALREADY DID RIGHT

- You asked for the modal order, the cookie bar and the remembered "No" in the very first brief. This diagram turns those three sentences into one testable rule set (the state machine for it is T09).

Staff sign-in and the three gates

Session, two-factor, role, scope: four questions before any button works



HOW TO READ IT

Think of a church building with a door, a corridor and locked rooms. The **session** is the key to the door (you are who you say you are). The **role** says which corridor you may walk. The **scope** says which rooms in that corridor (this branch, this department) are yours.

The first question, "valid session?", is answered by Better Auth. The remaining questions are answered by our own **AuthzService** using the RoleAssignment table. Better Auth proves **who** you are; it does not know what a branch is. That split is deliberate, and it is why Better Auth appears only for the staff side.

The check is done **again on every action** on the server. Hiding a button for a role is good for usability, but a clever person can still send the request by hand. Only the server check is security.

MISTAKES TO AVOID

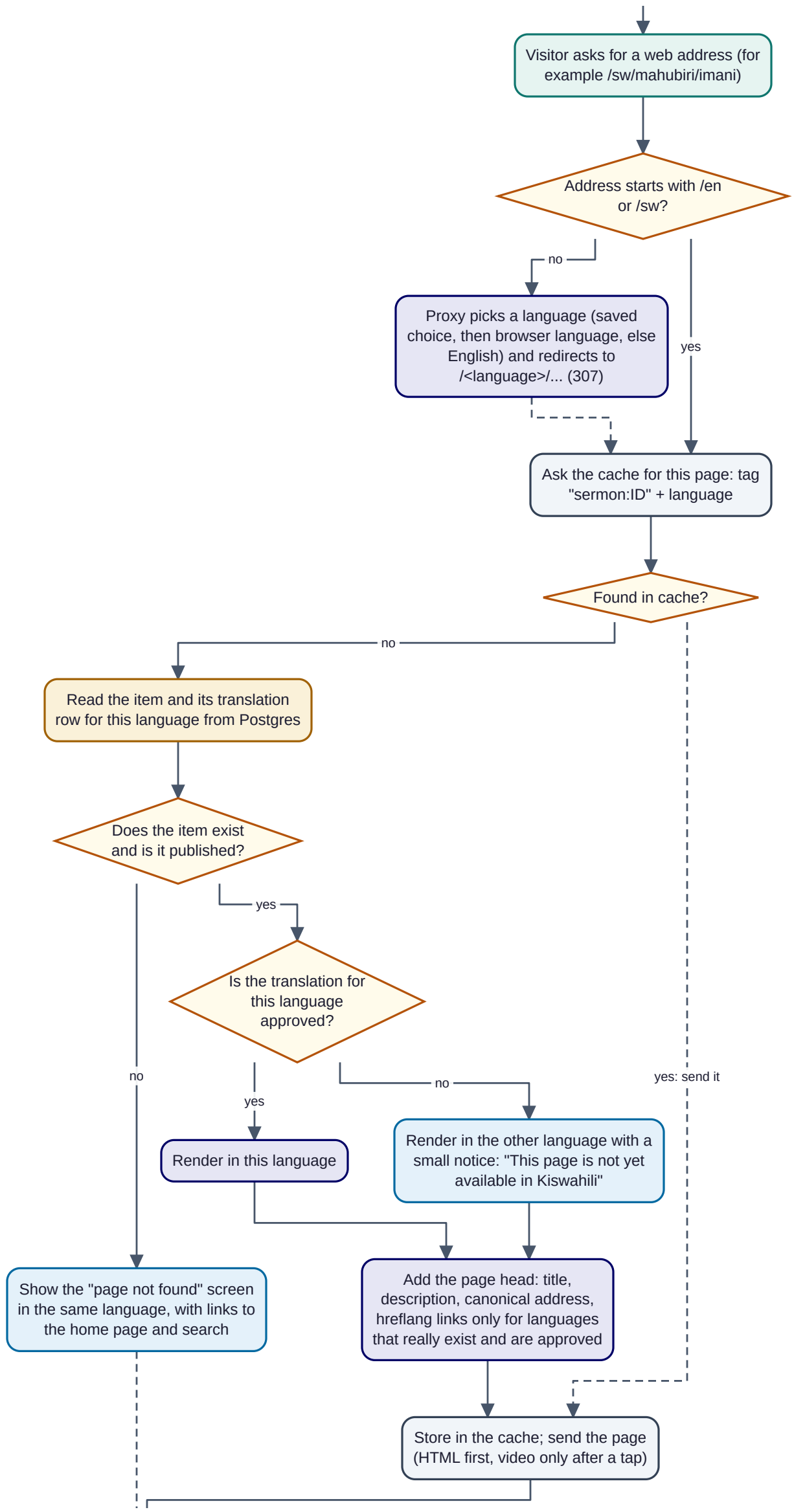
- Telling the person which half of the sign-in was wrong ("no such e-mail"). It helps a guesser. One vague message for both.
- Trusting the role stored in the browser. Roles are loaded from the database on each request (cached for that request only), so removing a role works at once.
- No audit line on a refusal. A refused attempt is the most useful line in the log when something is wrong.
- Skipping two-factor for the Super Admin and the Treasurer. These two roles can change the most; they need it first.

WHAT YOU ALREADY DID RIGHT

- You asked for Better Auth on the admin side only. Keeping it there (and out of the public site) means no passwords to guard for visitors, and a much smaller surface to attack.

One page request: language, cache and fallback

How the site chooses English or Kiswahili and what it does when a translation is missing



HOW TO READ IT

Every address on the site starts with the language: `/en/...` or `/sw/...`. That is better than a hidden cookie because **one address = one language**: a person can share it, Google can index each language, and the back button always shows the language they expect.

If the address has no language, the **Proxy** chooses one and redirects. After that, the page is looked up in the cache by **item + language**. If it is not there, it is read from the database together with its translation row.

The key rule for dynamic content such as blogs: only an **approved** translation is shown as the second language. Otherwise the page is shown in the original language with a polite notice. The visitor never sees an empty page and never sees a machine draft by accident (F05).

MISTAKES TO AVOID

- Listing hreflang alternates for a language that does not exist yet. Google then sees a link to an empty or duplicate page and trusts the whole site less. List only approved languages.
- Showing the fallback page with its own address as the "main" one. It should point (canonical) to the original-language address so search engines do not index two versions.
- Switching language by cookie only. The switch button must go to the **same page in the other language** (/en/... to /sw/...), not to the home page.

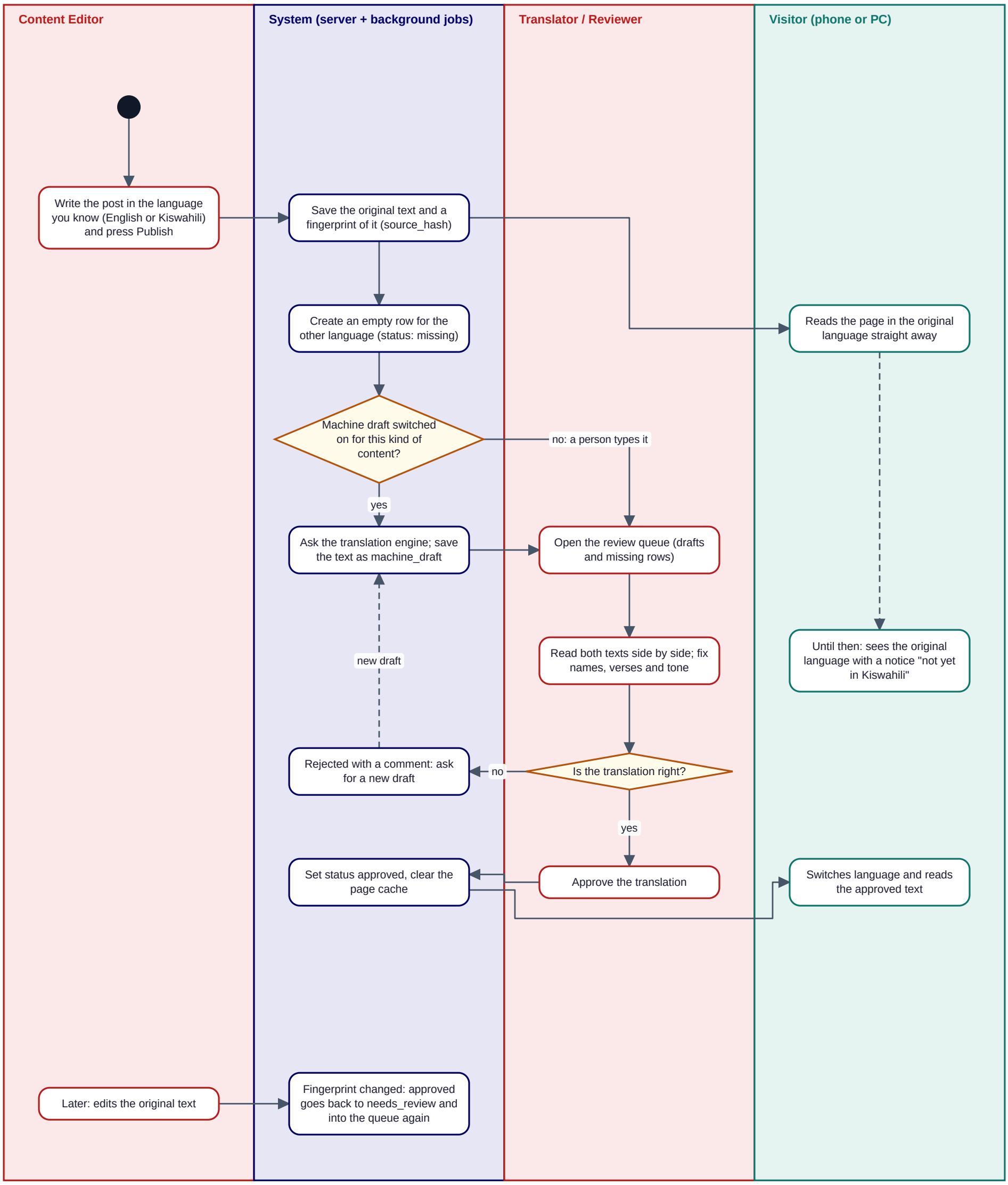
WHAT YOU ALREADY DID RIGHT

- You asked for an English and Kiswahili switch that also works for dynamic content. Putting the language in the web address makes the switch, the sharing and the SEO all work with one rule.

FLOW AND BLOCK DIAGRAMS · PHASE 1

Translating dynamic content (blogs, sermons, bulletins)

One page, two languages: write once, review once, never show an unchecked translation



Fixed words (menu, buttons, form labels) are not in this flow. They come from message files (en.json, sw.json) that developers and the translator update together.

HOW TO READ IT

This is the answer to *"I do not know how to do the Kiswahili change for all data, including blogs written in only one language"*. Think of a restaurant menu. The cook writes the menu in one language. A bilingual friend checks the other language before it is printed. Until the friend says OK, customers see the first menu.

The editor never has to **wait** or **translate**. The page goes live in its original language at once. A row for the other language is created empty, an optional machine draft fills it, and a human **approves** it. Only approved text is shown as the other language.

The **fingerprint** (source_hash) is the clever part. If the editor corrects a typo in the English a week later, the fingerprint changes and the approved Kiswahili automatically goes back to the review queue. Without that, the two languages would slowly disagree and nobody would notice.

MISTAKES TO AVOID

- Showing a machine draft to the public because "it is probably fine". Names of people, verses and church words need a human; one wrong word in a sermon title is a trust problem.
- Translating **fields** that are not text, such as slugs and dates. Only text fields are listed in the translation tables (see E11); slugs are made per language so each language has a clean web address.
- Forgetting images that contain words (posters, banners). Text inside a picture is not translated by this flow; ask for one picture per language.

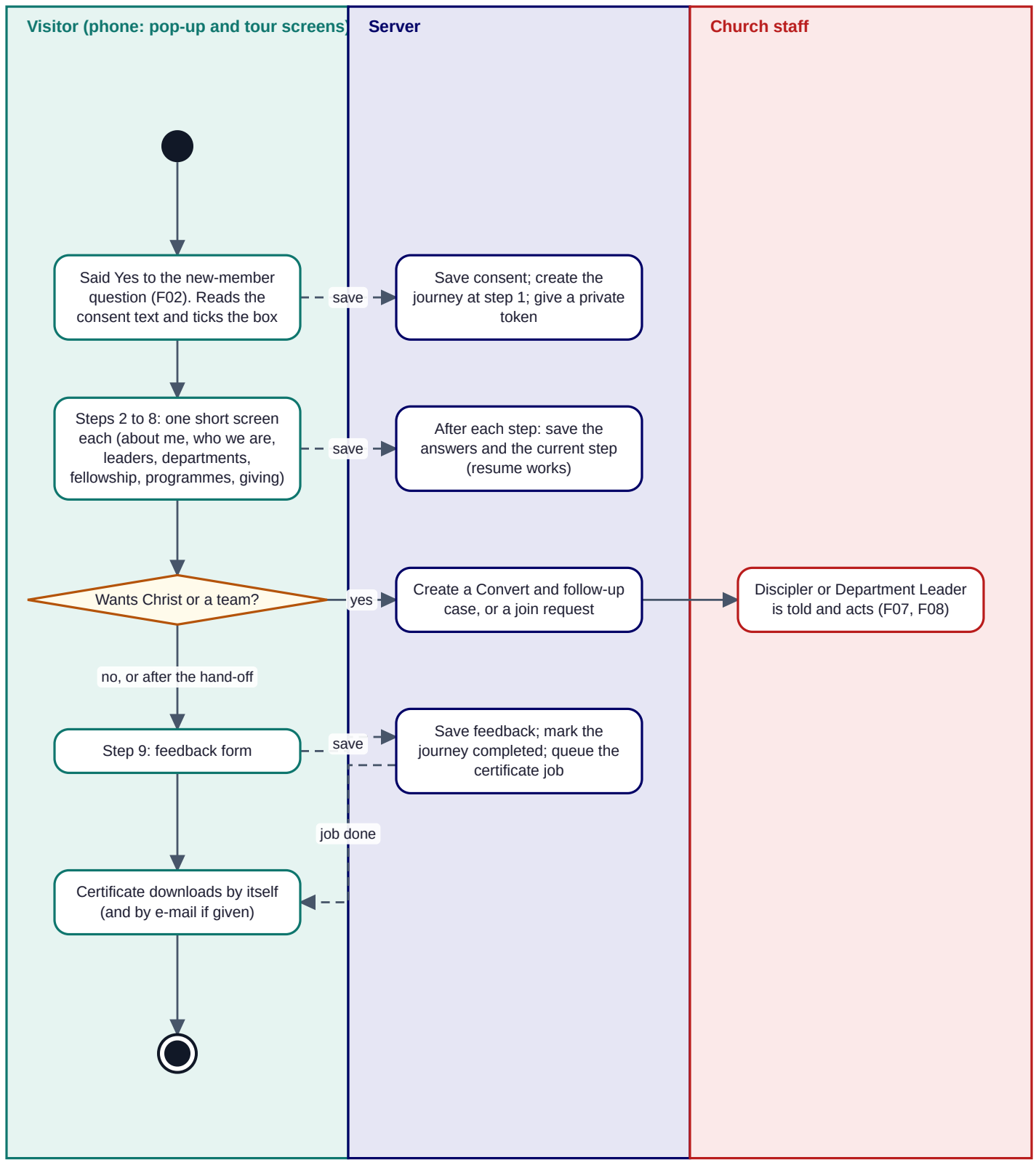
WHAT YOU ALREADY DID RIGHT

- You asked for help with dynamic data instead of only the fixed menu words. The design separates the two: fixed words live in message files (next-intl); church content lives in translation rows.

FLOW AND BLOCK DIAGRAMS · PHASE 1

New-member tour: from "Yes" to the certificate

Consent first, nine short steps, hand-offs to staff, and an automatic certificate



HOW TO READ IT

The tour is a **conversation with a rule at the front**: ask once, respect the answer, and never ask again if the answer was No. F02 shows that first question. This page starts at the moment the visitor says **Yes**.

Read the left lane from top to bottom: that is what the person sees. The dashed arrows to the server are **saves** that happen quietly after each step. The server only appears after the person says Yes **and** ticks the consent box. Because every step is saved, a dropped signal does not lose progress.

Proposed nine steps (to be confirmed, question Q-20): 1 consent, 2 about me, 3 who we are, 4 the leaders, 5 departments, 6 home fellowships, 7 programmes (including New Born Again), 8 projects and giving, 9 feedback and certificate. Steps 2 to 8 are one short screen each.

MISTAKES TO AVOID

- Asking for a phone number at step 2. Name, age group and branch are enough to start; contact details only when a person asks to be called (a decision for Christ or a join request).
- A certificate that downloads before the journey is complete. The certificate job checks status = completed, so an unfinished tour never gets one.
- Using the same pop-up logic on the staff pages. The admin area never shows visitor pop-ups.

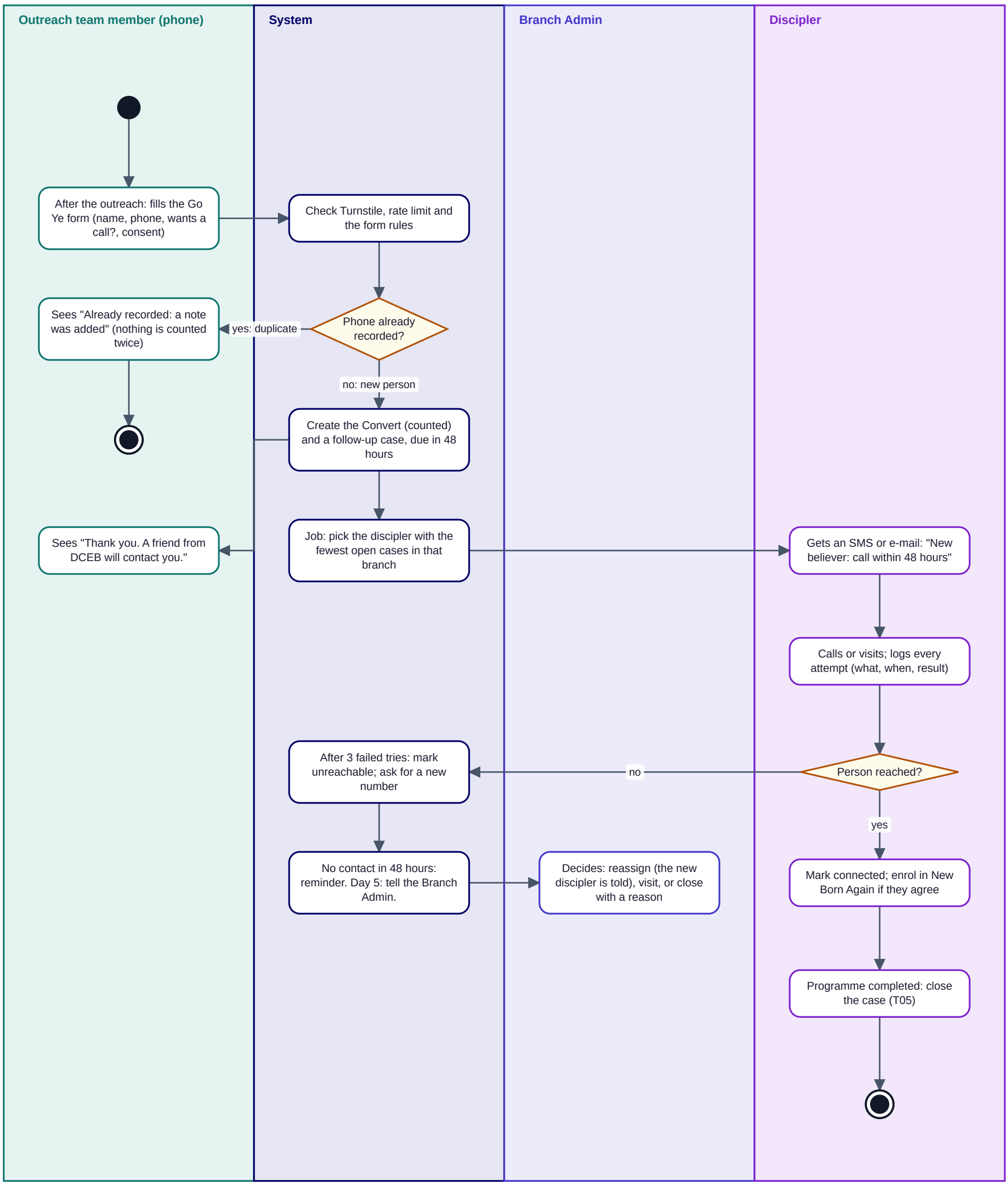
WHAT YOU ALREADY DID RIGHT

- You decided the new-member modal comes **before** the newsletter modal and that "No" is remembered. Both are built in (F02 shows the full order).

FLOW AND BLOCK DIAGRAMS · PHASE 1

Go Ye: from a decision to a discipler's call list

Counting once, assigning fairly, never losing a person



HOW TO READ IT

A person says yes to Christ during an outreach. Someone from the team types their details in the Go Ye form. From that moment the church has made a **promise**: someone will contact them. This diagram is that promise as a path.

Two details matter. **Count once**: the phone number is cleaned (+2547…) and fingerprinted, so two team members entering the same person do not make the counter go up twice. **Assign fairly**: the job picks the discipler with the fewest open cases, so the list does not pile up on one person.

The bottom half is the **safety net**: reminders at 48 hours, escalation to the Branch Admin at day 5, and an "unreachable" state that is not a dead end because a new number reopens the case.

MISTAKES TO AVOID

- Putting the converts list on a shared spreadsheet or WhatsApp group. Belief and phone numbers are sensitive; only the assigned discipler, the Branch Admin and the Super Admin see names.
- A counter that shows only a number like "3,482" with nothing behind it. Keep the counted rows; the total on the website is a count of unique counted converts.
- No owner for stale cases. Without the day-5 escalation a case can sit untouched for months.

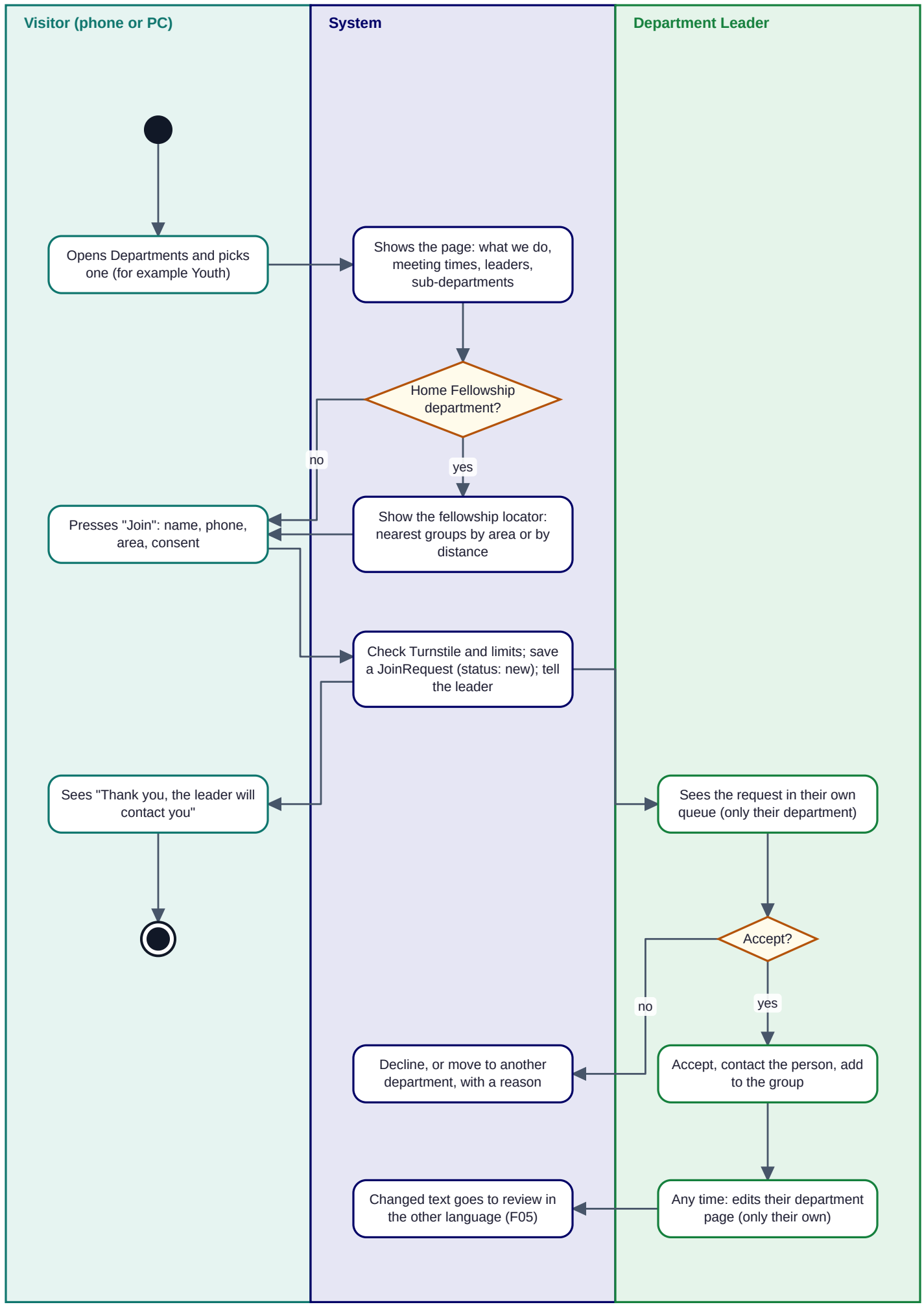
WHAT YOU ALREADY DID RIGHT

- You asked for a Go Ye page with a "born again so far" total and a form to add a convert. This is the part that makes that number honest.

FLOW AND BLOCK DIAGRAMS · PHASE 1

Department page and join request

How a visitor reaches a leader, and how a leader keeps their own page current



HOW TO READ IT

Departments are the church's shape: Youth, Children, Home Fellowship, Missions and more, each possibly with sub-departments. A visitor looks for **their place**; a leader looks after **their page**. This flow shows both ends of one conversation.

The **Home Fellowship** department has one extra step: the locator. It sorts groups by how close they are to the visitor. The visitor can type an area (privacy friendly) or allow the phone to share its location (more accurate). The site works with either.

A leader only sees **their department's** requests and edits only their department. That rule comes from the role scope on the access diagrams (B13 and B16).

MISTAKES TO AVOID

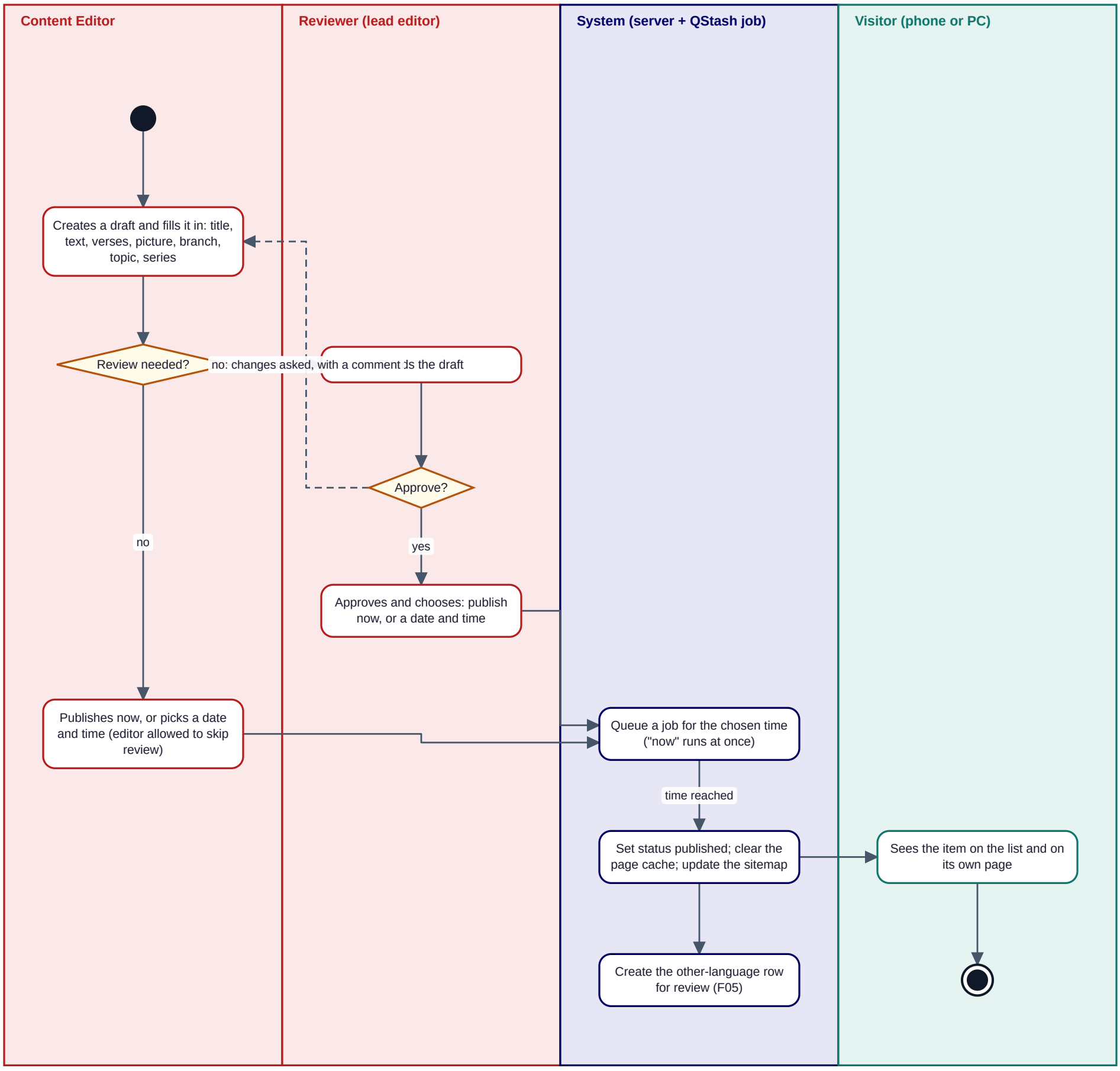
- Showing a leader's personal phone number on a public page without their agreement. Leaders choose what contact detail is shown (LeaderPosition).
- Treating "Join" as an e-mail with no owner. A request row has a status, an owner and a due date for a reason (T06).
- Using exact home addresses for fellowship locations. Show the area and meeting day; give the exact place only after contact.

WHAT YOU ALREADY DID RIGHT

- You asked for departments with a fellowship locator. Keeping the locator as a special case of the department page (not a separate system) means one editing screen for leaders.

Publishing: draft, review, schedule, live

One road for every kind of content, with an optional review gate



HOW TO READ IT

Every content type follows the same road (the **Publishable** idea): draft, review, scheduled, published, archived (T01). So an editor learns it once. The review gate is optional per type and per person: it is a church decision, not a technical one.

Scheduling lets the media team prepare Sunday's sermon on Friday and have it appear by itself at 1 pm. A small background job (QStash) wakes the server at the time. If that job fails the admin list shows **"overdue"**, so nothing quietly stays hidden.

Publishing also **clears the cache** for just that item and refreshes the sitemap, so visitors and Google see the new page quickly without rebuilding the whole site.

MISTAKES TO AVOID

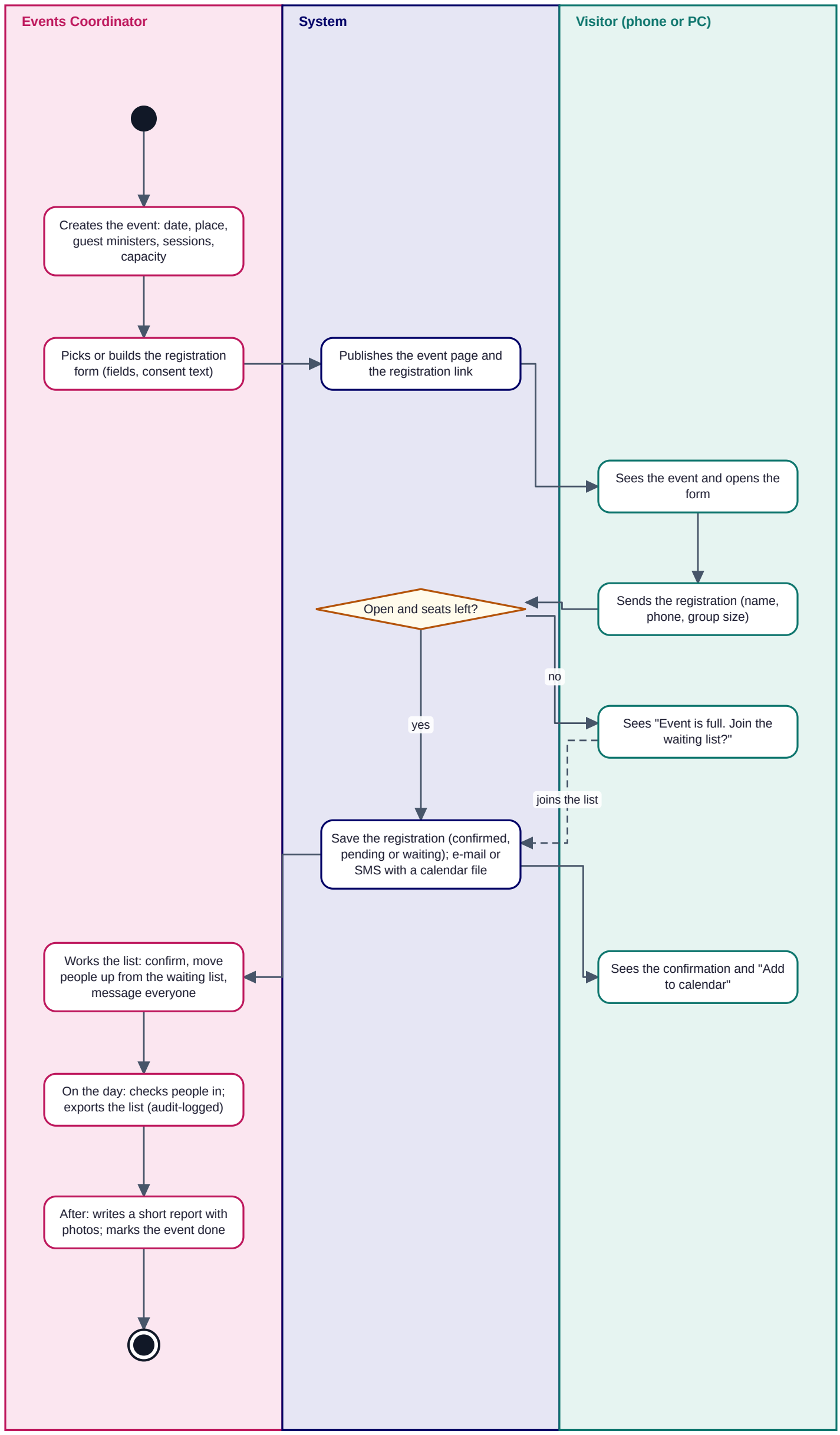
- Skipping review for sermons "because it is only a recording". A wrong date or title on a sermon is still public.
- Scheduling in the wrong time zone. Store the time as UTC and show it in Nairobi time (EAT, UTC+3) everywhere in the admin.
- Publishing a post without a Kiswahili summary and thinking it is finished. F05 starts at publish: the other-language row is created and waits for a reviewer.

WHAT YOU ALREADY DID RIGHT

- You listed many content types (blogs, bulletins, announcements, resources, sermons). Giving them one life cycle is what keeps the admin area learnable by volunteers.

Event: create, register, run the day

From a new event to a check-in list, with seats and a waiting list



HOW TO READ IT

An event is the church's most common promise: "come on this day". The coordinator makes the event once; the system does the repeating work (publish, confirm, remind, count) so the coordinator can spend time with people.

Capacity is checked **on the server**, at the moment of registration, not on the page. Two people pressing the button at the same second must not both get the last seat; the database decides, not the browser.

Payment is deliberately **not** in this phase. If an event needs a fee, the registration stays **pending** until the coordinator confirms, and a payment gateway (M-Pesa Daraja) can later be added as a new step.

MISTAKES TO AVOID

- Collecting more than you need. For a conference you need name, phone and group size; ask about allergies or health only if you really must, and label it clearly as optional.
- Sending the exported list by WhatsApp. Exports are audit-logged and should be used on the day and then deleted.
- Forgetting to close registration. The event has an explicit "registration open until" time, and the form says "Registration closed" after it.

WHAT YOU ALREADY DID RIGHT

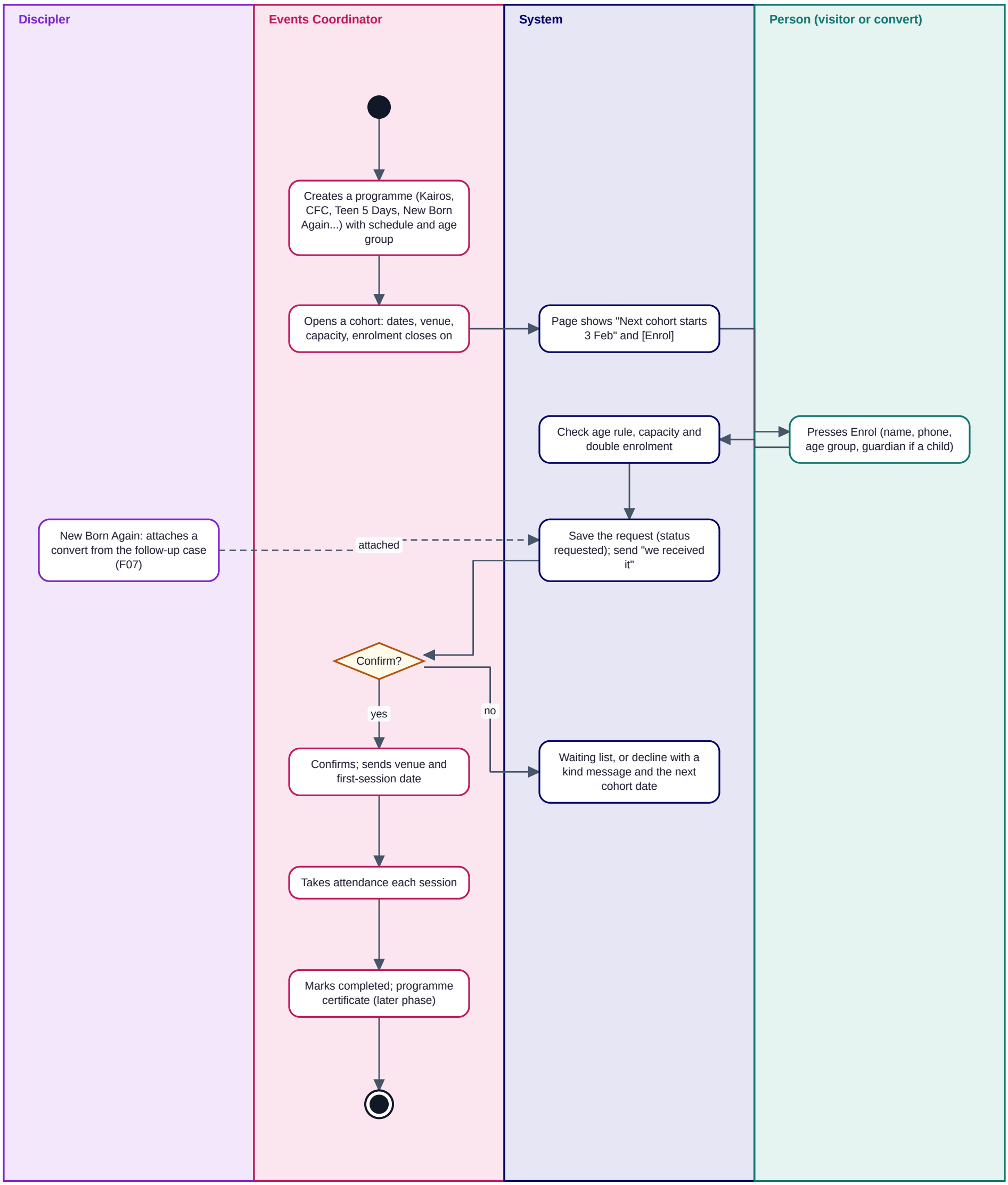
- You listed events with dates, guest ministers and registration. Treating sessions and guests as their own rows (B34) lets one event show a full programme without stuffing text into one field.

F11

FLOW AND BLOCK DIAGRAMS · PHASE 1

Programme and cohort: from a course to a completed person

Kairos, CFC, Teen 5 Days, Children 5 Days, New Born Again



HOW TO READ IT

A **programme** is the course (for example Kairos). A **cohort** is one run of it with real dates. People enrol in a **cohort**, not in a programme, because "Kairos" has no start date but "Kairos, February" does.

Programmes that serve children and teenagers need extra care: the form asks for a guardian's name and phone, and those details are treated as sensitive. They are visible only to the coordinator of that programme.

The **New Born Again** programme joins the Go Ye path: a Discipler can attach a convert to the next cohort straight from the follow-up case (F07), so the person does not have to find the page themselves.

MISTAKES TO AVOID

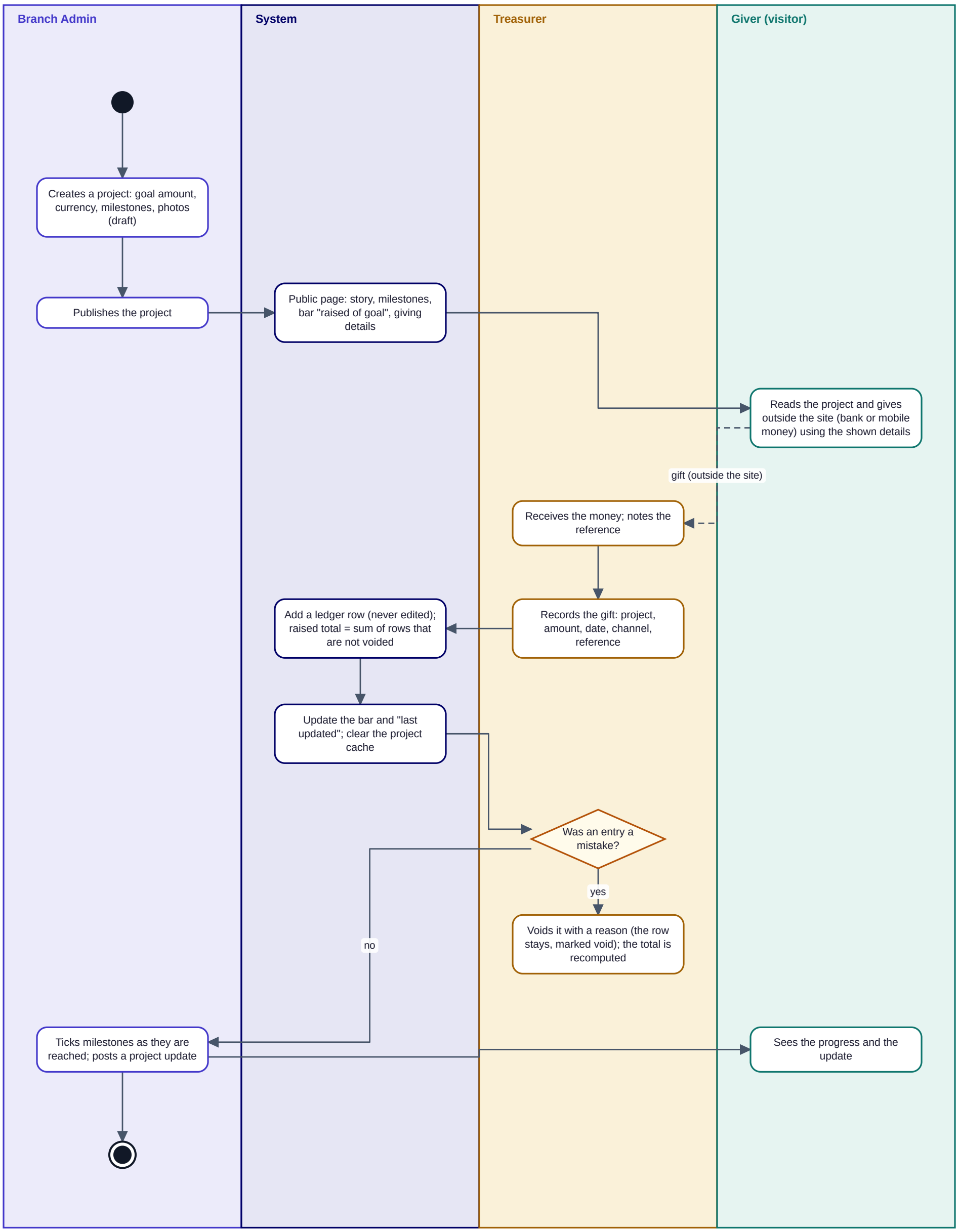
- Putting a child's details on a list that many volunteers can see. Use the minimum, with guardian consent, and show the list only to the programme team.
- Mixing up programme and cohort in the tables. Enrolment points at the cohort (B37, B38).
- Closing a cohort without telling the people on the waiting list. The system proposes the next cohort date in the decline message.

WHAT YOU ALREADY DID RIGHT

- You named the exact programmes (Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again). They are data, not code: adding a new programme is a form, not a developer task.

Giving and the project progress bar

Money moves outside the site; the site tells the truth about it



HOW TO READ IT

The brief asks for a bar that shows how much has been given and how much is left. The honest way to build that, without a payment company, is a **ledger**: a list of lines, one per gift, written by the treasurer. The bar is just the sum of the lines.

A **mistake** is never erased. The treasurer **voids** the line with a reason; the line stays visible to staff. This is how a church (or any shop) keeps books that an auditor will trust. The public bar recomputes without the voided line.

The page says "Last updated 6 Oct". It does not claim to be live bank data. If the church later adds M-Pesa Daraja, the same ledger gets lines made by the gateway instead of the treasurer and nothing else changes.

MISTAKES TO AVOID

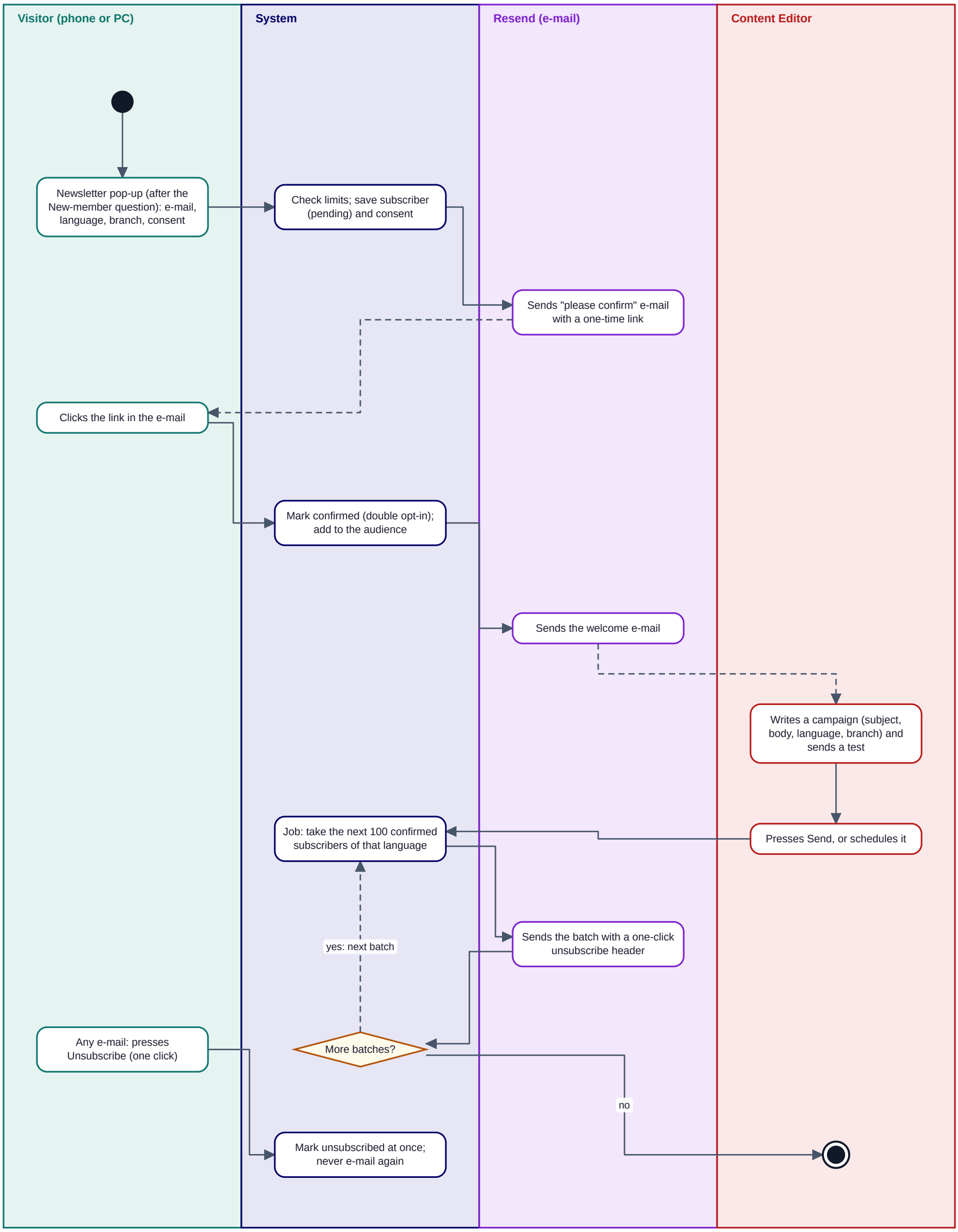
- A treasurer who edits an amount to fix a typo. Void and re-enter; the log shows both. The database blocks UPDATE on ledger amounts.
- Showing donor names on a public page without their agreement. Default to "anonymous"; show names only if the giver asked.
- Mixing currencies. Amounts are stored as whole minor units (KES cents) in one project currency; there is no float anywhere.

WHAT YOU ALREADY DID RIGHT

- You asked for a giving bar and for contact and giving details on the site. This design delivers both without storing card or bank data on the website, which removes a whole category of risk.

Newsletter: sign up, confirm, send, unsubscribe

Double opt-in and batch sending that respects the Data Protection Act



HOW TO READ IT

A newsletter list is built on **trust**. Double opt-in means a person is added only after they click a link in their own inbox. It stops people from signing up someone else, and it keeps the list clean so e-mail providers do not mark the church as spam.

Sending is done in **batches of 100** by a background job. A single function that sends 5,000 e-mails would be stopped by the hosting time limit; a job that sends 100 and schedules the next one is slow but cannot time out.

Unsubscribe is **one click and immediate**. That is a legal expectation and also the single best way to protect the sender reputation of the church's domain.

MISTAKES TO AVOID

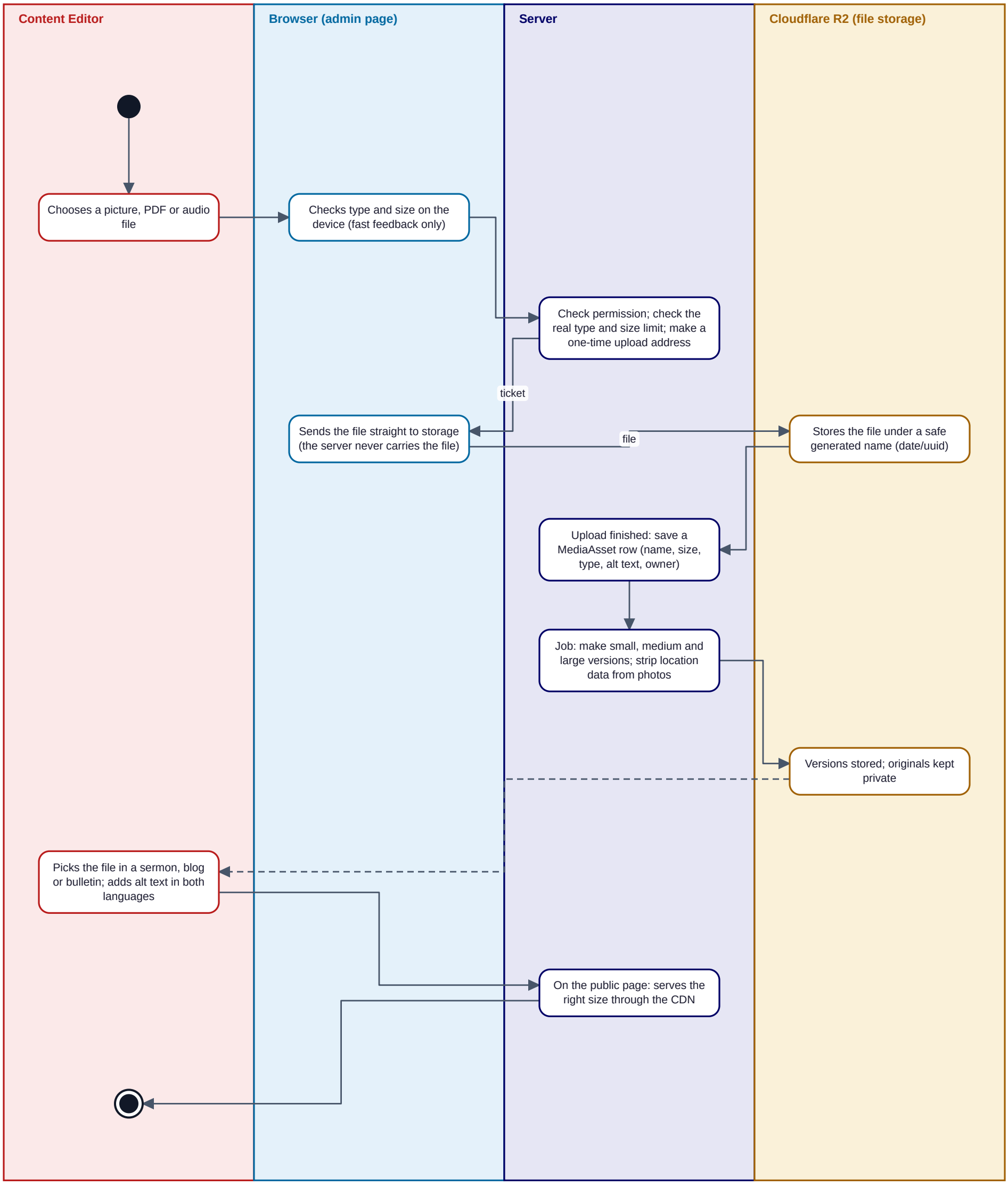
- Importing an old e-mail list from a spreadsheet. Those people never gave consent in this system, so they cannot be e-mailed from it. Ask them to sign up again.
- Sending from a personal address such as a Gmail account. Use a domain you control (for example mail.deliveranceb.org) with SPF, DKIM and DMARC set up; otherwise messages land in Spam.
- Forgetting that the modal order is fixed: the New-member question comes first, the newsletter question second, never both at once.

WHAT YOU ALREADY DID RIGHT

- You asked for a newsletter modal after the New-member modal. F02 puts the two in the right order and remembers each answer separately.

Media upload: from a file to a safe, fast picture

Direct-to-storage upload, checks, resizing and alt text



HOW TO READ IT

Photos and PDFs are heavy. If every upload went through our server, a few big sermon files could slow the whole site. So the server only **hands out a one-time upload ticket**; the browser sends the file straight to Cloudflare R2 (a presigned upload).

Checks happen **twice**: once on the device so the editor gets a quick message, and once on the server because anything from the browser can be faked. The server also checks the **real** file type, not just the name.

A background job makes several sizes and removes hidden **location data** from photos (privacy), so a youth photo does not reveal where a child lives. Alt text in both languages is requested at use time; it helps screen-reader users and search.

MISTAKES TO AVOID

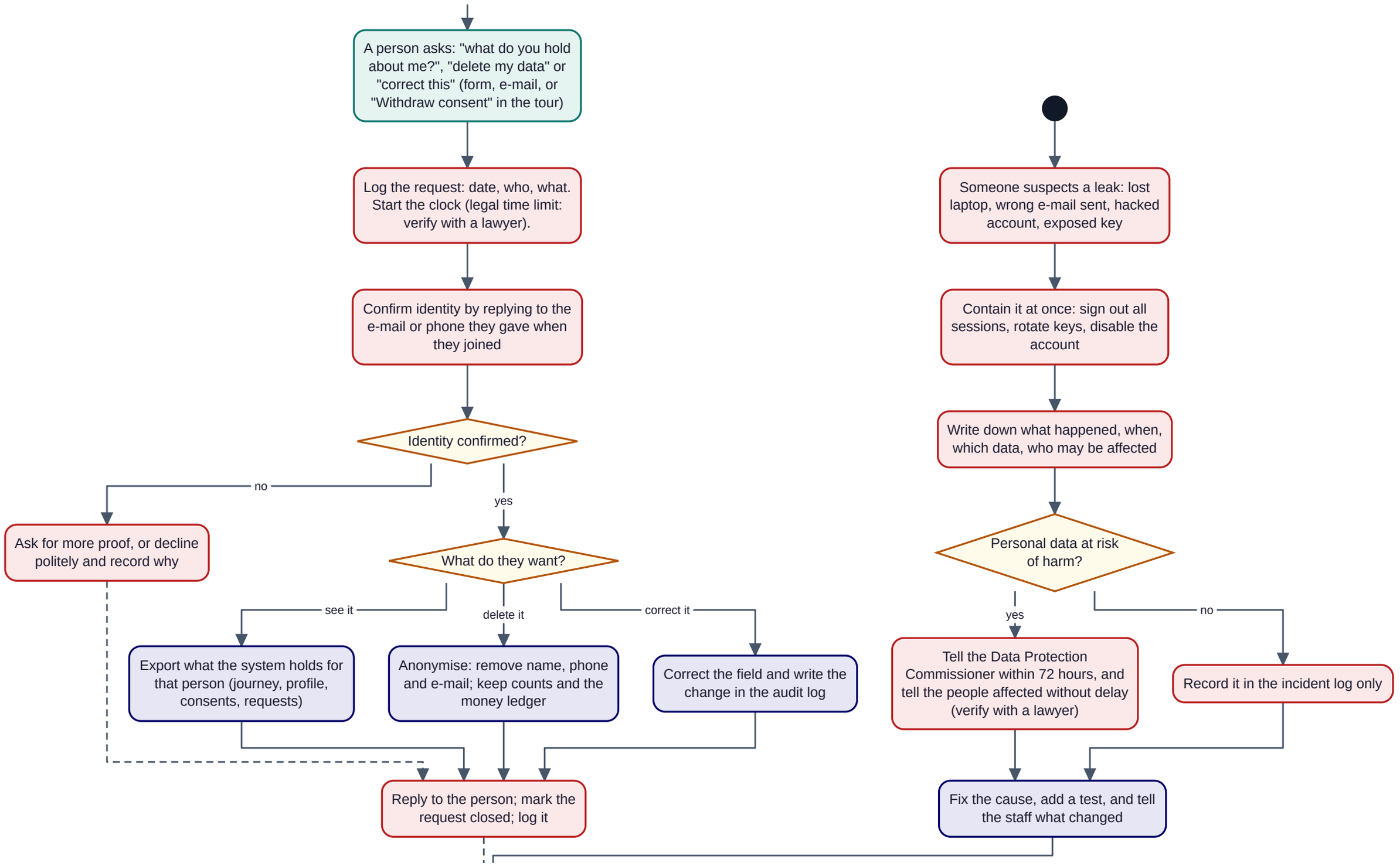
- Uploading original 12 MB phone photos to a mobile site. Always serve resized versions (WebP or AVIF) and set image sizes in the page.
- Trusting the file extension. A file named photo.jpg might be a script. Check the real content type and allow a short list of types.
- Public buckets with guessable names. Keep originals private; give the public only the resized versions through the CDN.

WHAT YOU ALREADY DID RIGHT

- You chose Cloudflare R2 for files. It has no download fees, which suits a church video and bulletin library; this flow is the safe way to use it.

Privacy request and security incident

What the church does when a person asks about their data, and when something goes wrong



HOW TO READ IT

Two stories live on this page because they are the two moments when a church website must show it can be trusted. **Left story**: a person asks what the church holds, wants it corrected, or wants it removed. **Right story**: something goes wrong (a lost laptop, a mistaken e-mail, an exposed password).

Kenya's **Data Protection Act 2019** gives people rights over their personal data and expects an organisation to report certain breaches to the Data Protection Commissioner within **72 hours**. The page uses those two ideas. **Please verify the exact duties and time limits with a Kenyan lawyer**; this diagram is a design, not legal advice.

"Anonymise" is the practical answer to deletion in a system that also keeps a money ledger and a "born again so far" counter: the person's name, phone and e-mail are removed, but the anonymous count and the accounting line stay, so totals do not change.

MISTAKES TO AVOID

- No named person owning the privacy mailbox. Put one role on it (Super Admin) and one backup.
- Deleting a person's ledger line to "honour deletion". Gifts are financial records; the donor's name is removed instead.
- Waiting until the cause is fully understood before telling anyone. Contain first, write it down, then decide who must be told.
- Keeping passwords or keys in chat messages or e-mail. After an incident, rotate anything that was ever pasted anywhere.

WHAT YOU ALREADY DID RIGHT

- In the brief you separated personal data (name, phone) from belief data (a decision for Christ). That is exactly what the Act calls sensitive data, and this page is where that distinction pays off.

ATLAS · FAMILY 9

Data flow

Where information goes. Gane-Sarson data-flow diagrams: the context diagram (D01), the nine processes (D02), eight detailed level-2 pages (D03 to D10) and a process-by-store table (D11).

> **EXPLAINED LIKE YOU ARE FIVE**
> Like the plumbing: where water enters, where it is stored, who turns the tap.

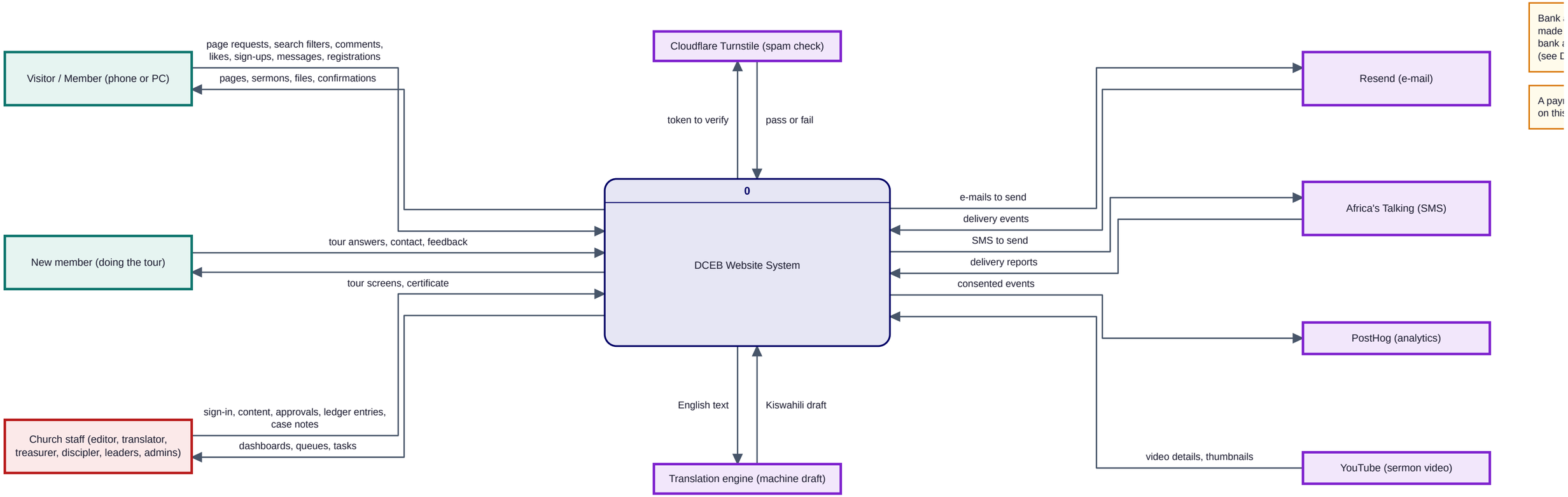
Code	Title	What it shows
D01	Context diagram (level 0)	The whole system as one bubble: who talks to it and what they exchange
D02	Level 1: the system split into nine processes	Process 0 opened up: nine numbered processes, nine data stores, and who talks to whom
D03	Level 2: manage content and church information (process 1)	How an item travels from a draft to a live page, and who keeps the church facts up to date
D04	Level 2: serve pages and take part (process 2)	What a visitor can read and do, and what each action stores
D05	Level 2: events and programmes (process 3)	Planning, registering, enrolling, deciding and running the day
D06	Level 2: Go Ye and follow-up (process 4)	From a decision for Christ to a discipler's call list and the public counter
D07	Level 2: the new-member tour and certificate (process 5)	The nine steps, where each answer is saved, and how the certificate is made
D08	Level 2: giving and projects (process 6)	Who may see money data, who may write it, and how a mistake is corrected
D09	Level 2: language and translation (process 7)	Original text, machine draft, human review, approval and the fallback
D10	Level 2: identity, access and audit (process 8)	Sign-in, the role check every other process depends on, and the audit log
D11	Process by store table: who reads and who writes	The C-R-U-D table that shows what depends on what

D01

DATA FLOW · PHASE 1

Context diagram (level 0)

The whole system as one bubble: who talks to it and what they exchange



HOW TO READ IT

A **data-flow diagram (DFD)** follows **data**, not clicks. At level 0 the entire website is one rounded box (process 0). Around it are the **externals**: people and outside services that send data in or take data out.

Read each arrow as a sentence: "Visitor sends page requests to the system" and "the system sends pages back". The diagram says nothing yet about *how* the system works inside; that comes in the next pages (level 1 and level 2).

Notice what is **not** here: the database, Redis and R2. They are inside the system, so they appear from level 1 downwards as data stores.

MISTAKES TO AVOID

- Drawing the database as an external. If the church owns and controls it, it is inside the boundary.
- Showing PostHog as a store of our data. It is a third party; we only **send** it events, and only after consent.

WHAT YOU ALREADY DID RIGHT

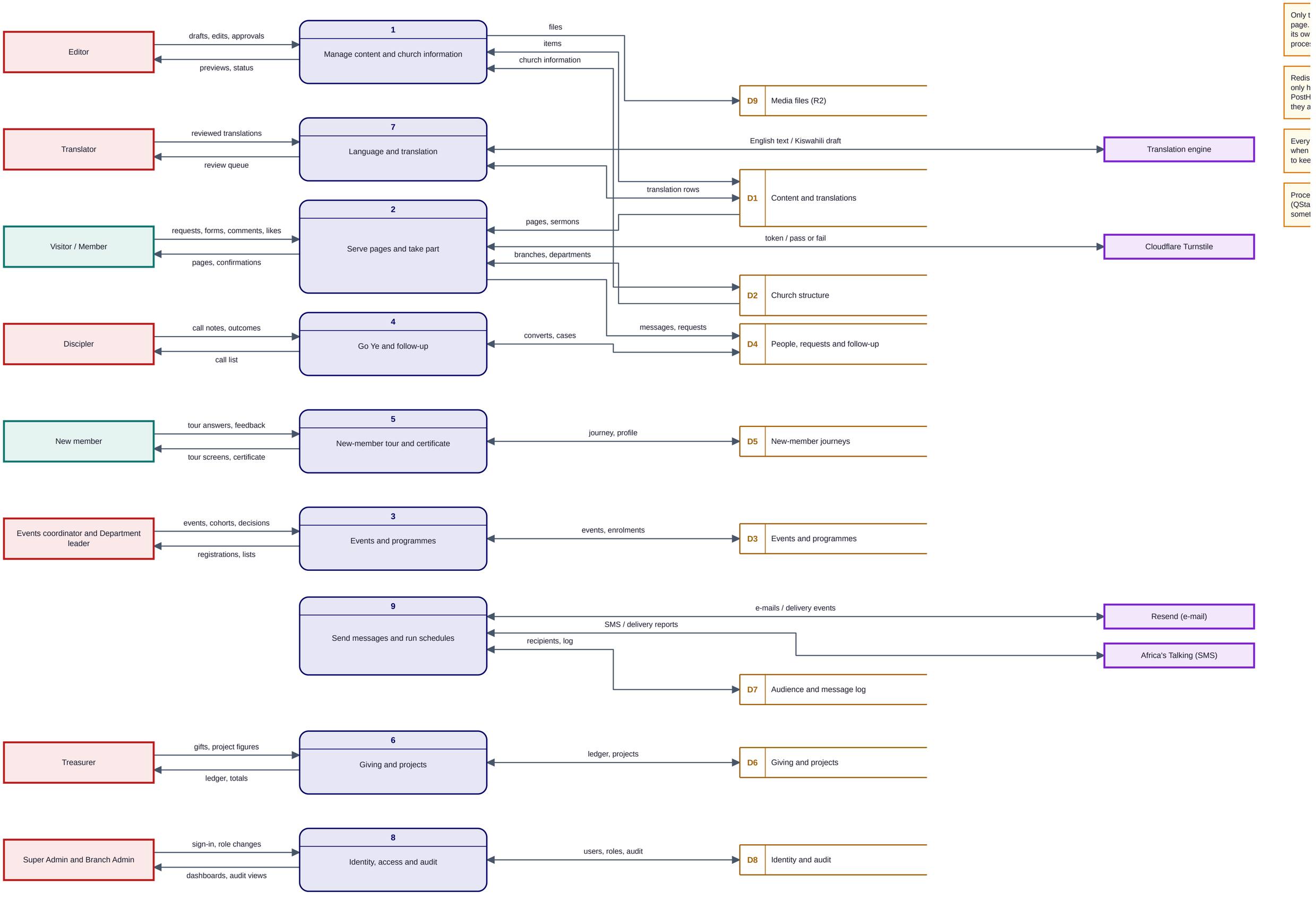
- You asked for roles to be separate from the start. Staff is one box here only because the context level hides detail; level 1 splits it into the real roles.

D02

DATA FLOW · PHASE 1

Level 1: the system split into nine processes

Process 0 opened up: nine numbered processes, nine data stores, and who talks to whom



HOW TO READ IT

Level 0 (D01) showed one bubble. Level 1 **opens the bubble** and shows nine smaller bubbles (processes), each with a number. A process is a verb: it takes data in, does something, and sends data out. The open-ended boxes with a code (D1 to D9) are **data stores**: places where data rests.

Read it in three columns. Left: **who** uses the system (people). Middle: **what** the system does (nine processes). Right: **where** the data rests (nine stores). The outside services that only one process uses (translation engine, Turnstile, Resend, SMS) sit at the far right.

Rule that keeps a DFD honest: **data never moves from a store to another store, or from a person straight to a store, without a process in between**. Every arrow here starts or ends at a numbered bubble. An arrow with two heads means the process both reads and writes that store.

MISTAKES TO AVOID

- Showing a store with no arrow into it (an unused table) or no arrow out of it (a table nobody reads). Every store here is both written and read somewhere in D03 to D10.
- Letting two processes write to the same store with different rules. D1 is written by Process 1 (items) and Process 7 (translation rows) and those two writes touch different tables on purpose.
- Mixing up a **process** with a **screen**. "Process 2: serve pages and take part" is not a page; it is every page a visitor can read plus every small form they can use.

WHAT YOU ALREADY DID RIGHT

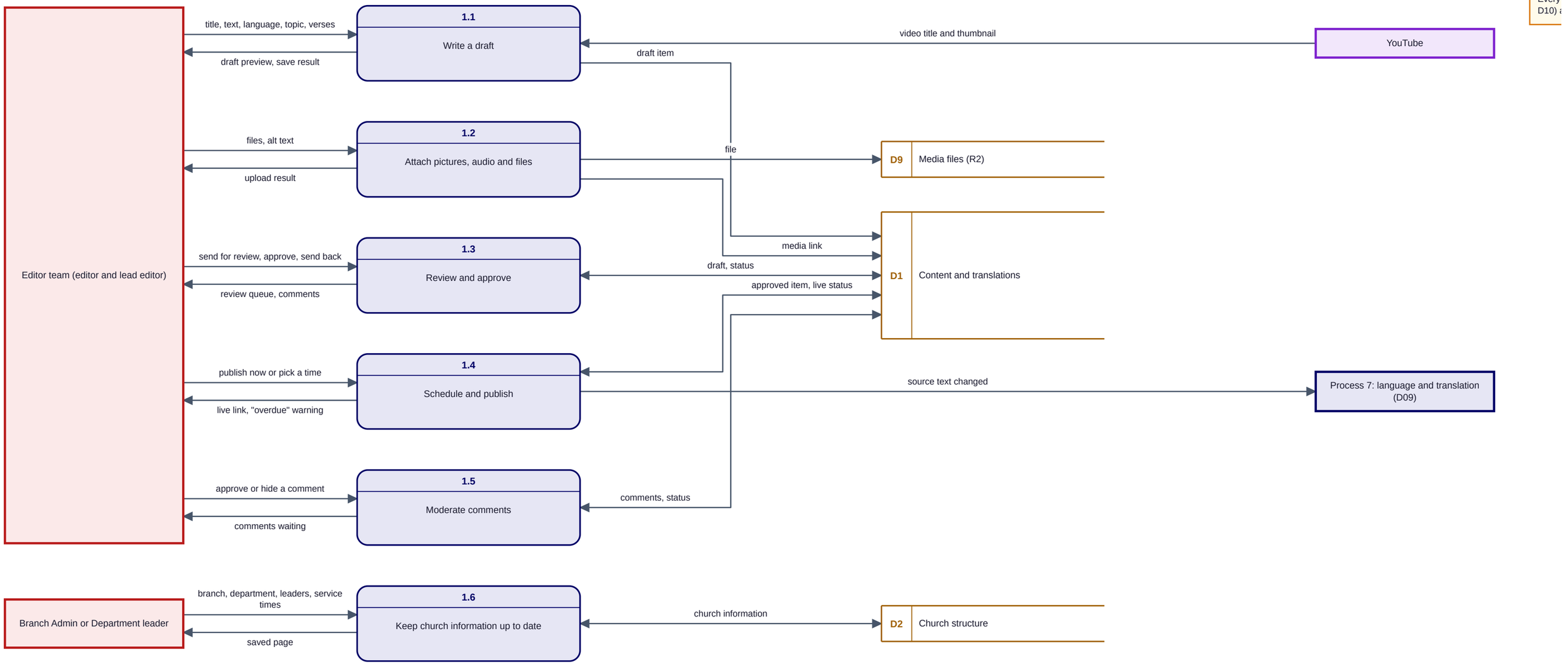
- You asked for the system to be understood as "what depends on what". The right-hand column is exactly that: each process shows the store it owns, and the stores that many processes read (D1, D2, D3) are visibly shared dependencies.

D03

DATA FLOW · PHASE 1

Level 2: manage content and church information (process 1)

How an item travels from a draft to a live page, and who keeps the church facts up to date



HOW TO READ IT

Process 1 is the **newsroom** of the church website. Everything a visitor reads (a sermon, a blog, a bulletin, an announcement, a resource) starts here as a **draft** and reaches the public only after the steps 1.1 to 1.4. The store D1 in the middle is the shelf where every item lives; each step changes the item's **status** on that shelf.

Step 1.4 does one more thing that is easy to miss: it tells **Process 7** that the original text changed or is new. That is how a Kiswahili row is created and how an old translation is marked "needs a fresh look" (D09).

Process 1.6 is different: it keeps the **church facts** (branches, departments, leaders, service times, home fellowships) in D2. These are not blog-like items; they are the map that the fellowship locator, the tour and the department pages read.

MISTAKES TO AVOID

- Letting the editor write directly into the published version. Edits always go to a draft first, so a typing mistake never goes live.
- Forgetting that comments are content too (1.5). A comment is public text under your name; it needs the same care as a blog post.
- Giving church facts (D2) to the editor role only. A Department leader should be able to fix *their own* department page, and nothing else (scope, see B11 and F03).

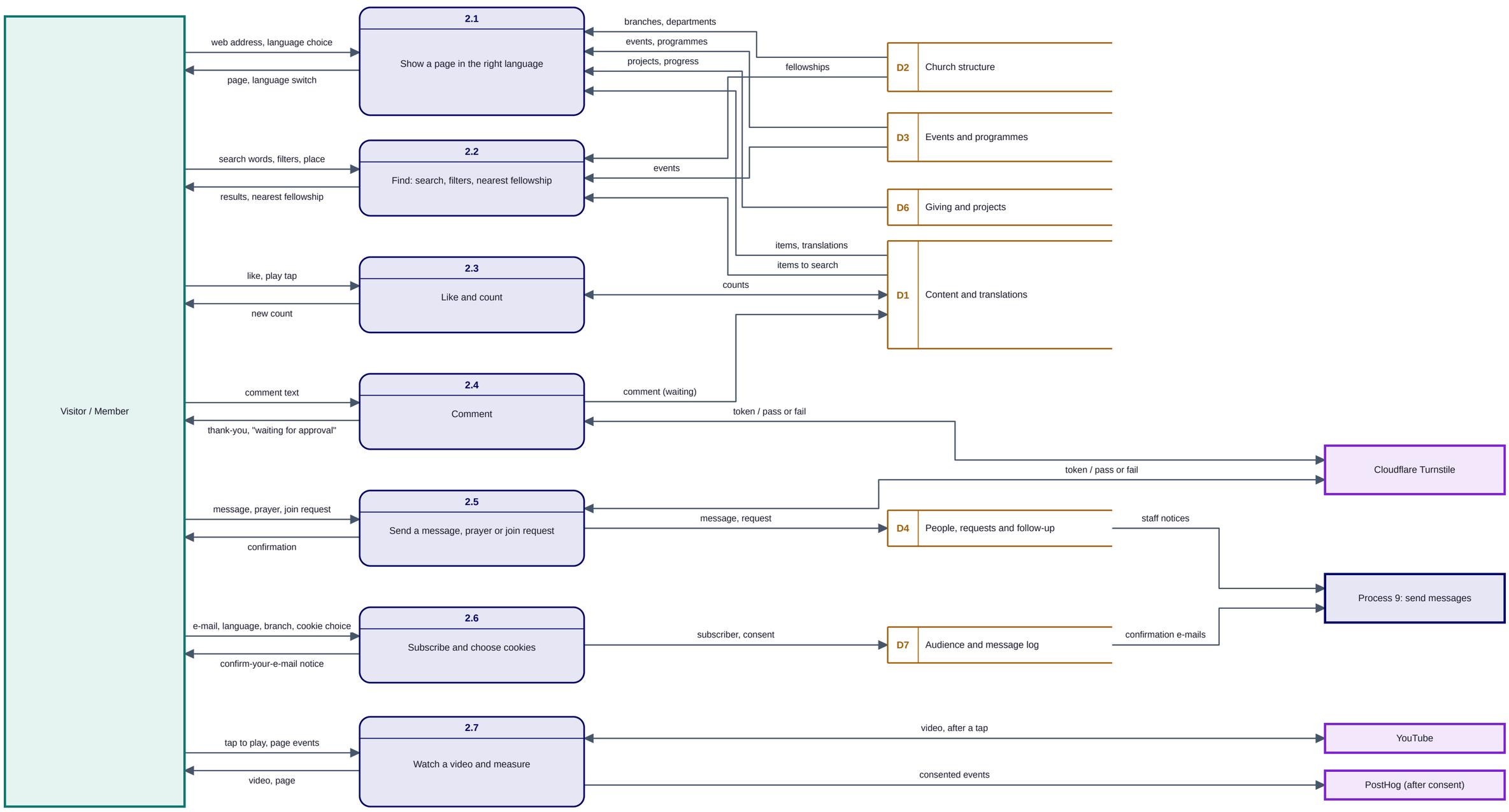
WHAT YOU ALREADY DID RIGHT

- You listed many content kinds (blogs, bulletins, announcements, discipleship material, sermons). Because they all share steps 1.1 to 1.4, the editor learns one road, not five.

DATA FLOW · PHASE 1

Level 2: serve pages and take part (process 2)

What a visitor can read and do, and what each action stores



The n
(D07).

HOW TO READ IT

Process 2 is the **front door**. Steps 2.1 and 2.2 are **read only**: the visitor asks, the system answers from the shelves D1, D2, D3 and D6. Steps 2.3 to 2.6 **write** something small: a like, a comment, a message, a sign-up. Step 2.7 is the video and the counting of visits.

Two forms (a comment and a message) first pass through **Cloudflare Turnstile**, the not-a-robot check. The **safety frame** of U02 is exactly these arrows plus the rate limiter. A signed-in account is never needed: visitors do not have passwords.

PostHog gets only events the visitor **agreed to** in the cookie bar (F02), and YouTube is loaded only **after a tap**. That is why both sit on the browser side and why the video does not slow the page.

MISTAKES TO AVOID

- Saving a comment straight to "published". It always starts as "waiting" and a person approves it (T03).
- Showing the nearest fellowship using a visitor's exact location. Use only the area they type or the town they choose; never store their position.
- Putting analytics in the page head so it runs before the cookie choice. The consent check must come first.
- Sending the message to the pastor's personal e-mail. Messages go to D4; staff get a notice (Process 9) and read them in the admin, where access is recorded.

WHAT YOU ALREADY DID RIGHT

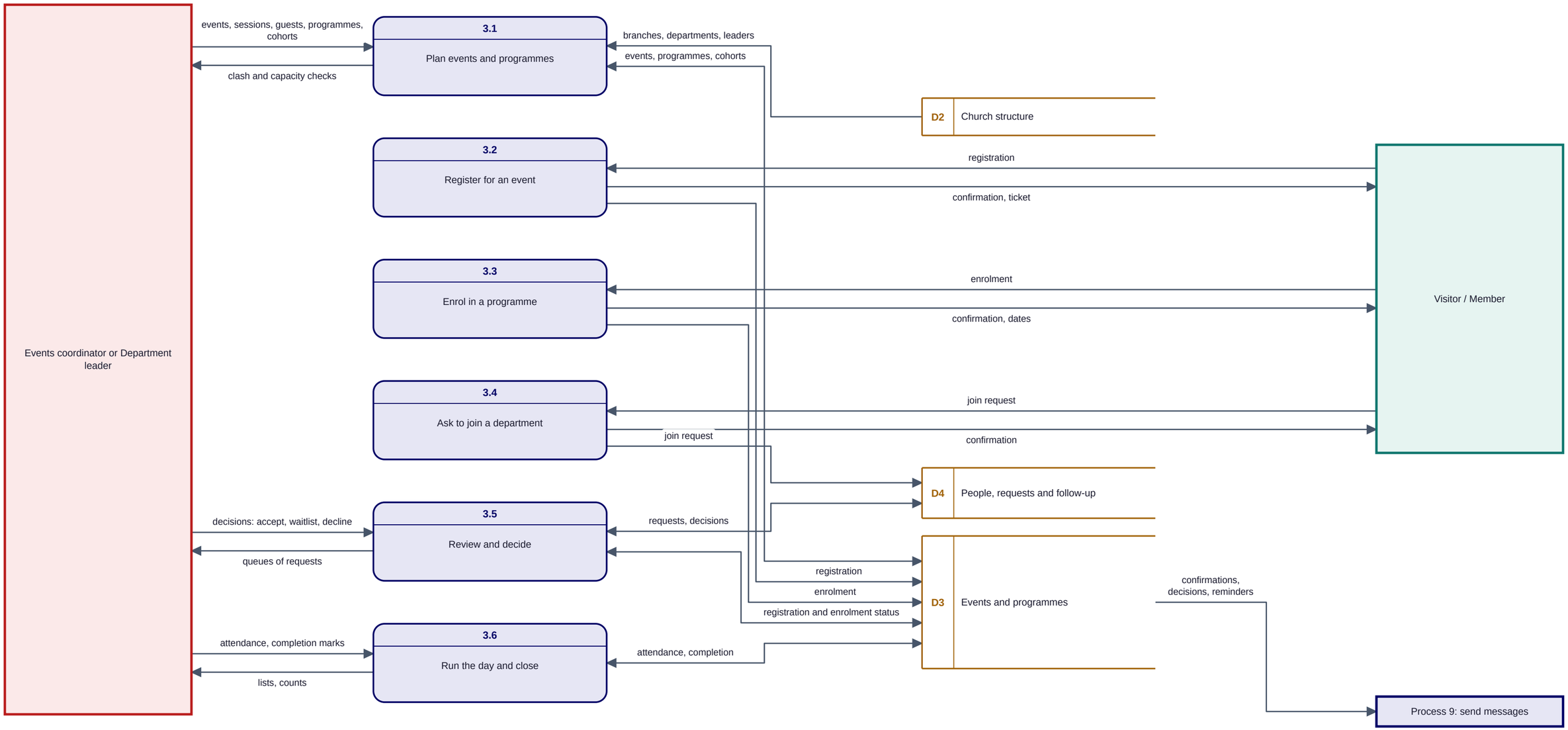
- You asked for a cookie bar with a remembered "No" and a clear pop-up order. They are step 2.6 here, and the arrow "cookie choice" ends in D7 only when the visitor chooses; a plain "No" is stored on the device alone.

D05

DATA FLOW · PHASE 1

Level 2: events and programmes (process 3)

Planning, registering, enrolling, deciding and running the day



HOW TO READ IT

This page puts **two kinds of people** on opposite sides on purpose. Staff are on the left: they plan (3.1), decide (3.5) and run the day (3.6). Visitors are on the right: they register (3.2), enrol (3.3) and ask to join a department (3.4). The three visitor steps never talk to staff directly; they leave a **request** in a store and a staff step picks it up.

Events and programmes share one store (D3) because they share the same idea: *a place and a time where people come*. A programme such as Kairos, CFC, Children 5 Days, Teen 5 Days or New Born Again is a series of sessions repeated in **cohorts**; an event is a single one. The same registration rules (capacity, waiting list, status in T06) apply to both.

Confirmations and reminders are not sent from this page. The stores hold the status and **Process 9** reads D3 and sends the e-mail or SMS. That keeps a slow mail service from slowing a registration.

MISTAKES TO AVOID

- Counting registrations without a capacity check at the moment of saving. Two people registering at once can both get the last place unless the database enforces it (E14).
- Treating "join a department" as a form that mails the leader. It is a request with a status the leader must answer (T06), otherwise it is forgotten.
- Letting every Department leader see every department's requests. Scope is the department (F03).

WHAT YOU ALREADY DID RIGHT

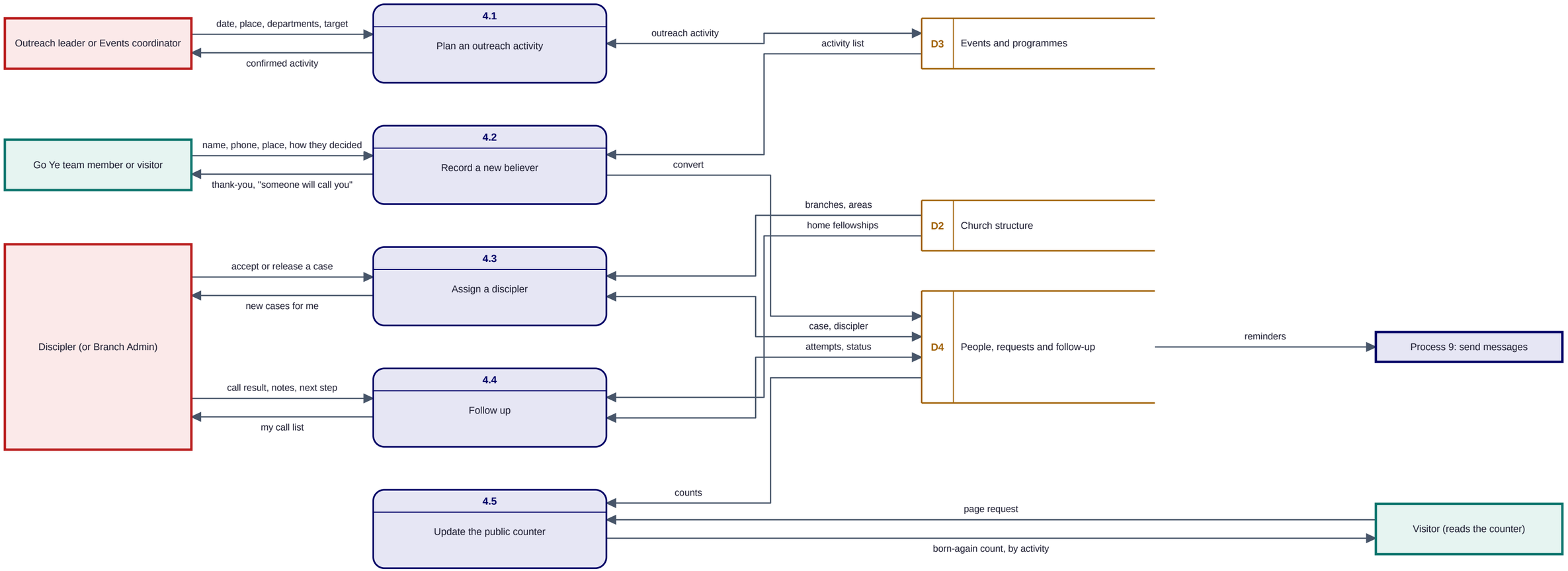
- You named the five programmes (Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again). Modelling them as **one programme idea with cohorts** avoids five copies of the same screens.

D06

DATA FLOW · PHASE 1

Level 2: Go Ye and follow-up (process 4)

From a decision for Christ to a discipler's call list and the public counter



A rem
or e-m
only if

HOW TO READ IT

Go Ye is the outreach arm. Step 4.1 plans an outreach *activity* (it is a special kind of event, which is why D3 appears here and on D05). Step 4.2 is the form you asked for: a person in the field, or the visitor themselves, records that someone gave their life to Christ. That one record is the most sensitive in the whole system, because a belief is **sensitive personal data** under Kenya's Data Protection Act.

The record then becomes a **case** (4.3) assigned to a discipler, who makes **attempts** (4.4): calls, visits, notes, an outcome (T05). Steps 4.3 and 4.4 read D2 so the case can be matched to the nearest **home fellowship** by branch and area.

Step 4.5 is the **born-again counter**. It reads counts only, never names. Visitors see a number; staff see people. That separation is the privacy design of this page.

MISTAKES TO AVOID

- Showing a list of recent converts on a public page. Show counts only; the names stay behind the Discipler role (B11).
- Recording a person without telling them. The form must say who will contact them and why; consent is stored with the record (F15).
- Letting one discipler hold hundreds of cases. Assign by branch and area, and show the load so a lead can rebalance.

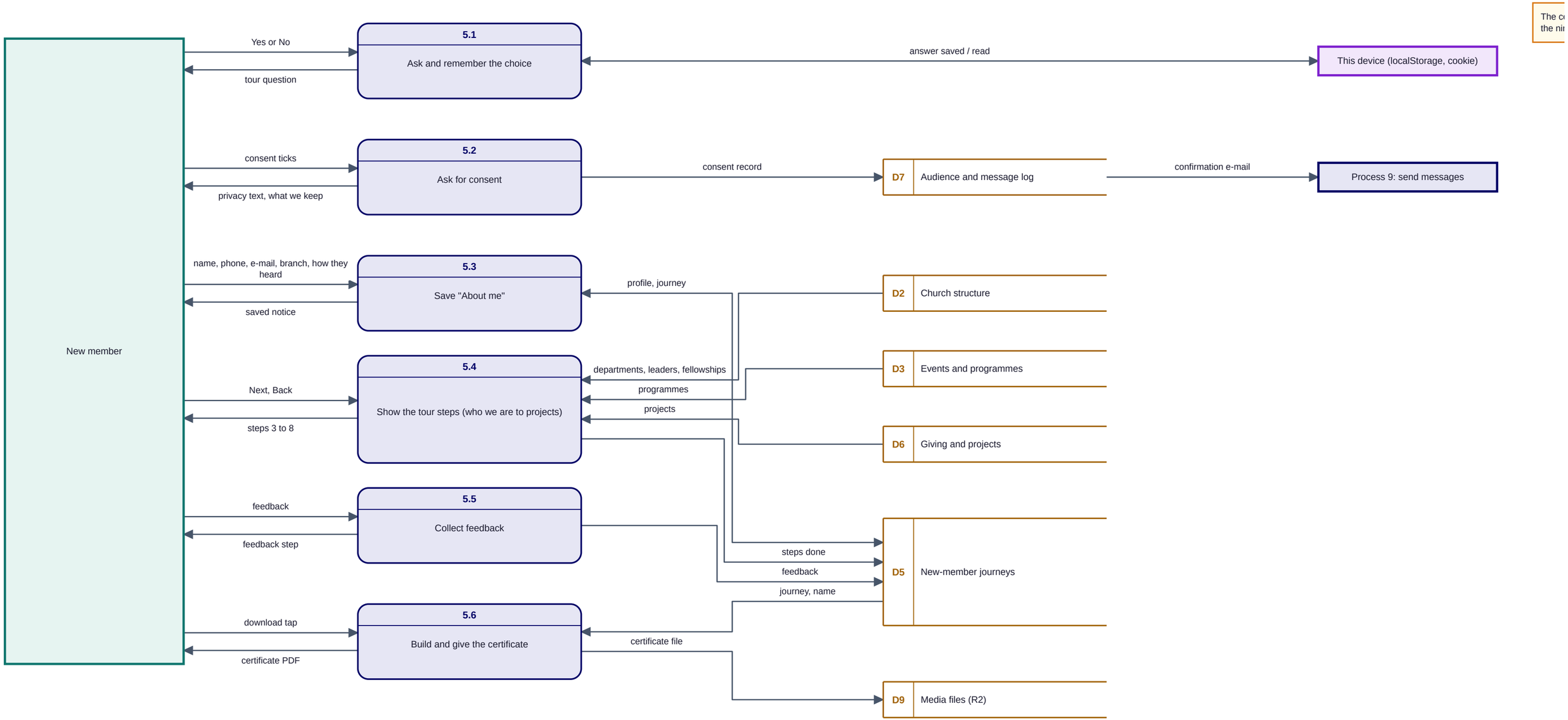
WHAT YOU ALREADY DID RIGHT

- You asked for a counter and a form to add a convert. Splitting the **private case** from the **public count** gives the church the encouraging number without exposing any person.

DATA FLOW · PHASE 1

Level 2: the new-member tour and certificate (process 5)

The nine steps, where each answer is saved, and how the certificate is made



HOW TO READ IT

This page follows one new member through your nine-step tour. Step 5.1 is the pop-up question and where the answer is remembered: only on **this device**. Step 5.2 is step 1 of the tour (consent). Step 5.3 is step 2 ("About me"). Step 5.4 covers tour steps 3 to 8: who we are, leaders, departments, home fellowships, programmes, projects and giving. Step 5.5 is step 9 (feedback) and 5.6 makes the **certificate**.

Notice the three small stores on the left of D5. Step 5.4 does not *own* departments, programmes or projects: it only **reads** them (from D2, D3, D6). So when the church changes a project, the tour is up to date with no extra work. This is the dependency idea: the tour depends on those stores, they do not depend on the tour.

The certificate is built on the server (5.6), saved in the file store D9 and offered for download. The journey and the profile live in D5, apart from the newsletter list (D7), because a person can finish the tour and still say no to the newsletter.

MISTAKES TO AVOID

- Asking for consent *after* collecting the name and phone. Consent comes first, so step 5.2 is before 5.3.
- Saving every tick-by-tick answer in the browser only. If the person closes the tab on step 6 they must be able to continue; the journey (D5) remembers the step reached.
- Making the certificate from a client-side picture. Build it on the server from saved data so it cannot be edited and the church can re-issue it.

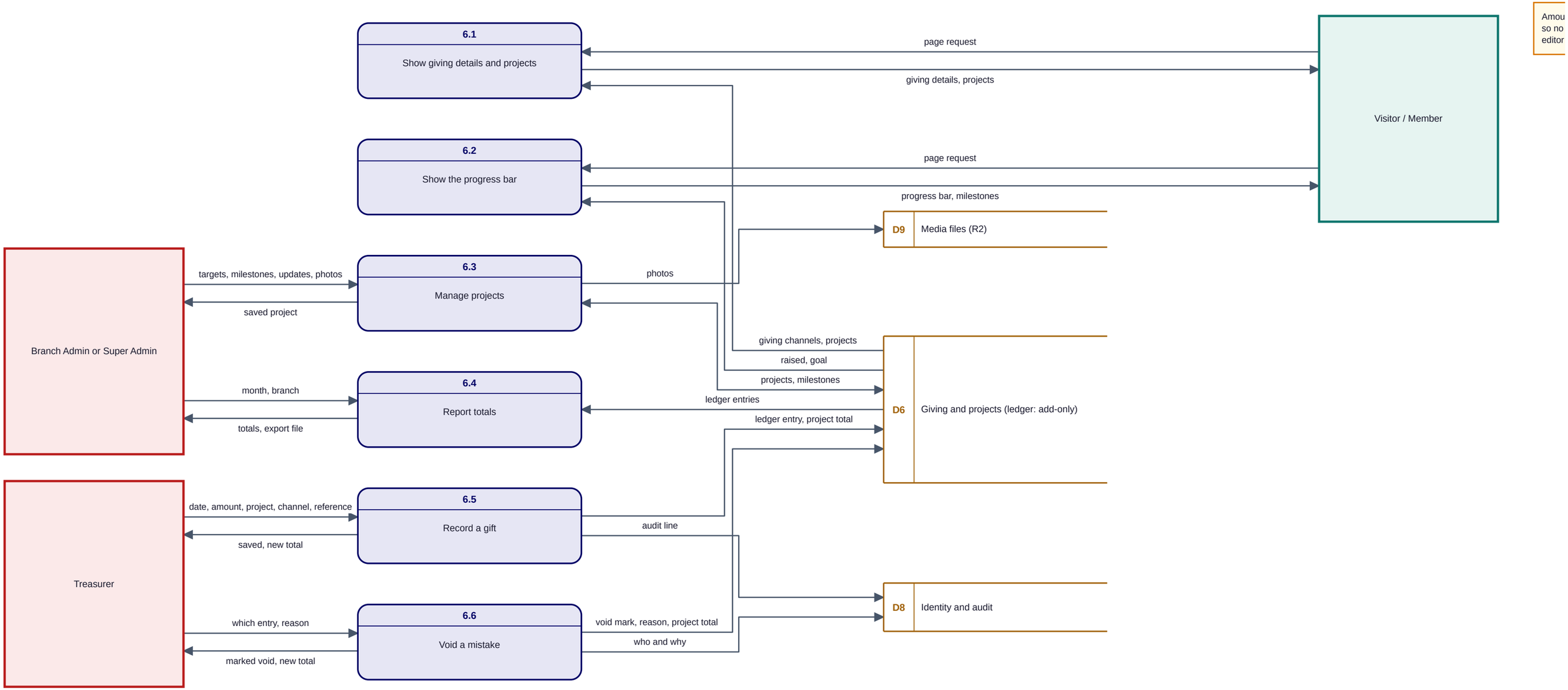
WHAT YOU ALREADY DID RIGHT

- Your brief put the new-member modal **before** the newsletter modal and made "No" a real answer. Step 5.1 stores "No" on the device only, and D7 receives nothing until the person ticks consent.

DATA FLOW · PHASE 1

Level 2: giving and projects (process 6)

Who may see money data, who may write it, and how a mistake is corrected



Amou
so no
editor

HOW TO READ IT

Money has the **strictest page** in the whole set. Visitors (right) can only *read* two things: how to give (6.1) and how far a project has come (6.2). Admins (left, upper) manage the **projects** and read the totals. Only the **Treasurer** (left, lower) can **write** a gift (6.5) or void one (6.6).

The ledger in D6 is **add-only**. A gift is a line that is never edited and never deleted. A mistake is corrected by **voiding** the line with a reason: the line stays visible to auditors and the total skips it. The project total is a saved copy, recalculated in the same step, so the bar can never disagree with the lines.

No bank or M-Pesa arrow touches this system. In your brief giving happens outside the website and the Treasurer types in what the statement shows. If the church later takes payments online, a payment provider becomes a new outside box next to the visitor.

MISTAKES TO AVOID

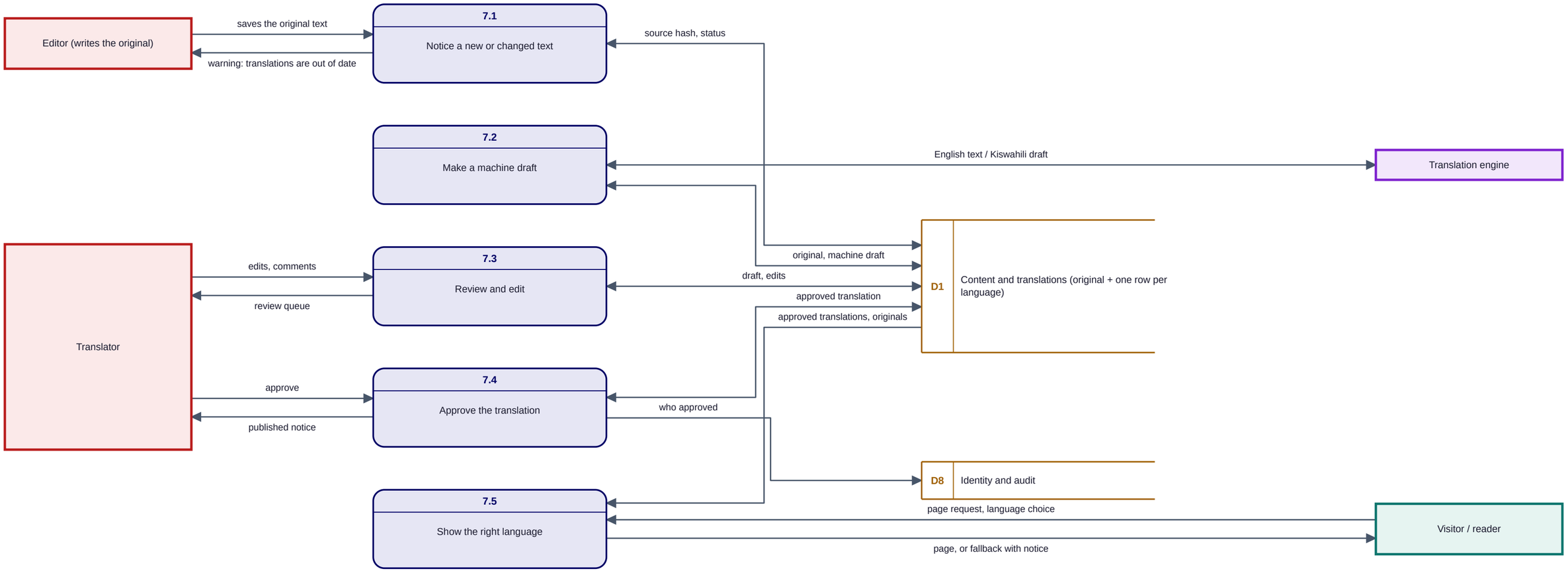
- Using a normal "edit" button on a gift. Edits destroy history; void it and add a new line instead.
- Showing donors' names on the progress bar. The bar needs only totals per project.
- Letting the Treasurer role approve its own voids without a trace. Every gift and every void writes an audit line (D8), and a reason is required.

WHAT YOU ALREADY DID RIGHT

- You asked for the project **giving bar**. Computing it from the ledger (6.2) instead of typing a number by hand means the bar can never disagree with the books.

Level 2: language and translation (process 7)

Original text, machine draft, human review, approval and the fallback



HOW TO READ IT

A blog is written in one language, but the site has two. This page is how the **second language** is made without losing control. Step 7.1 notices the original text changed (it compares a fingerprint, the **source hash**). Step 7.2 asks the translation engine for a **machine draft**. Steps 7.3 and 7.4 are done by a **human translator**. Only after approval does the visitor see the other language (7.5).

The word "machine" matters. The draft is a *starting point*. A machine can turn "grace" into a word that sounds right but means something else. In a church, doctrine words must be checked by a person, which is why the status goes **machine draft, needs review, approved** (T02) and why only "approved" is public.

When the editor changes the original after approval, 7.1 sees the fingerprint no longer matches and moves the translation back to **needs review**. Without that step the site would quietly show an old translation of new text.

MISTAKES TO AVOID

- Publishing a machine draft because "it looks fine". Reviewers must be named and approval is logged.
- Translating a whole page as one block. Translate per field (title, summary, body) so a partial change does not redo everything.
- Using the machine draft for verses. Verses come from a chosen Bible version in each language, never from the engine.

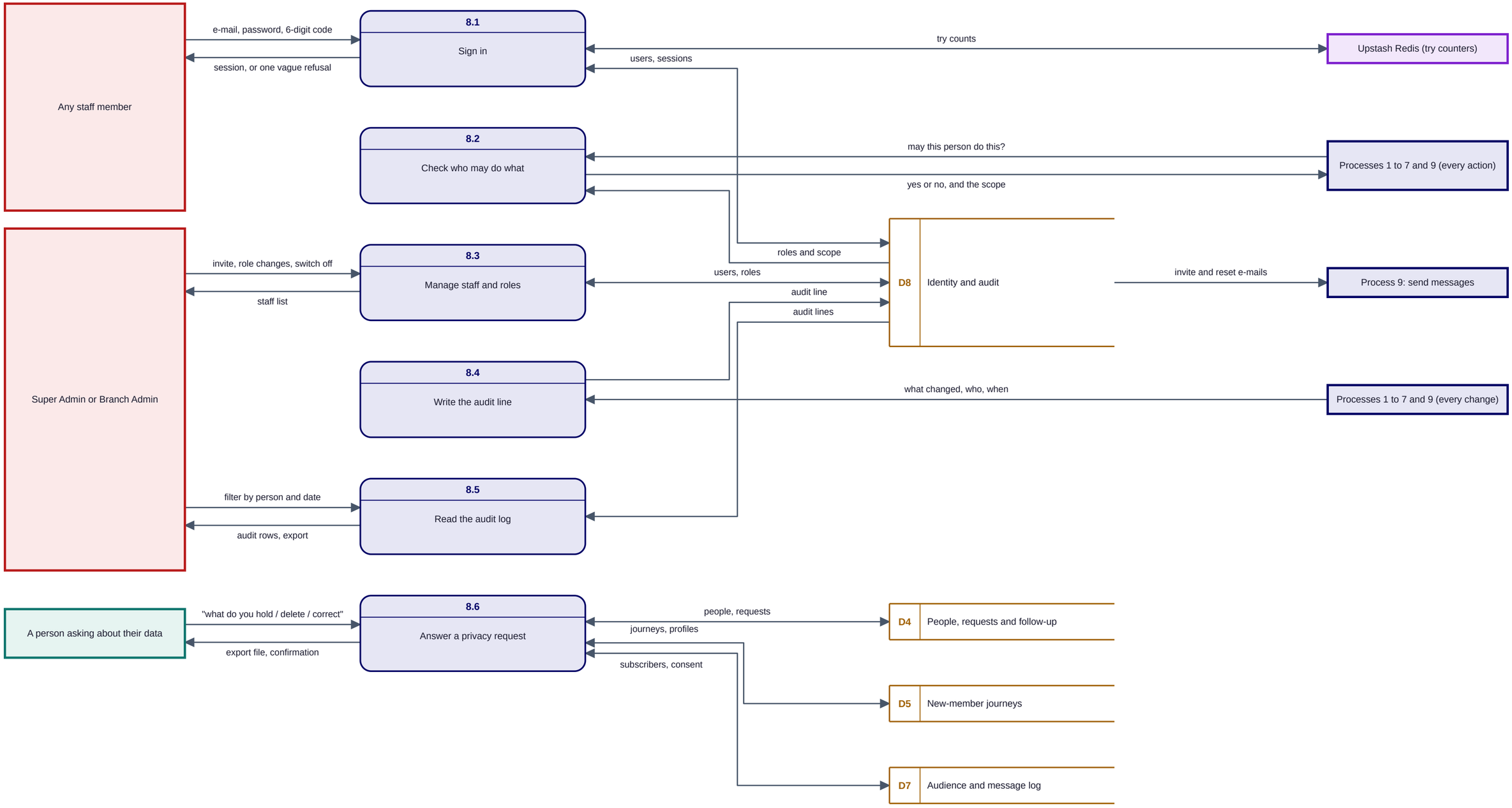
WHAT YOU ALREADY DID RIGHT

- You specifically asked how to help translate dynamic content written in one language. This diagram is that answer: automatic draft for speed, human approval for trust, fallback for no empty pages.

DATA FLOW · PHASE 1

Level 2: identity, access and audit (process 8)

Sign-in, the role check every other process depends on, and the audit log



HOW TO READ IT

Process 8 is the **doorman** of the church building. Everyone else on the right of this page depends on it: before any staff action, a process asks 8.2 "may this person do this?", and after any change it sends 8.4 "this is what changed". That is why it is a **process of its own** and not code scattered through the other eight.

Step 8.1 is Better Auth: e-mail, password and a six-digit code. It proves **who** you are. Step 8.2 is our own check of **what you may do**, from your roles and their scope (global, branch or department). The two are separate on purpose: Better Auth knows nothing about branches, and we do not want to teach it.

Step 8.6 handles the legal duty on F15. It is the one step that reaches into the personal-data stores (D4, D5, D7), which is why a Super Admin does it and every use is logged.

MISTAKES TO AVOID

- Writing the audit line from the browser. It is written by the server, in the same transaction as the change; if the change fails, the line is not written either.
- Letting the audit log be edited or cleaned up. It is add-only like the ledger.
- Putting role checks only in the menus. The server check (8.2) runs on every action.

WHAT YOU ALREADY DID RIGHT

- You asked for Better Auth on the admin side only. This page shows why that is enough: visitors never reach 8.1, so a visitor can never have a password to lose.

D11

DATA FLOW · PHASE 1

Process by store table: who reads and who writes

The C-R-U-D table that shows what depends on what

Store	1 Content and church info	2 Serve pages, take part	3 Events and programmes	4 Go Ye and follow-up	5 New-member tour	6 Giving and projects	7 Language	8 Identity and audit	9 Messages and jobs	The one owner
D1 Content and translations	CRU	CR					CRU		R	Process 1 writes items; Process 7 writes translation rows only
D2 Church structure	CRU	R	R	R	R			R		Process 1 (church information)
D3 Events and programmes		R	CRUD	CRU	R				R	Process 3 (Process 4 for outreach activities)
D4 People, requests and follow-up		C	RU	CRU				RUD	R	Process 4 (Process 2 only adds new requests)
D5 New-member journeys					CRU			RUD		Process 5
D6 Giving and projects		R			R	CRU				Process 6
D7 Audience and message log		C			C			RUD	CRU	Process 9 (Processes 2 and 5 only add)
D8 Identity and audit	C	C	C	C	C	C	C	CRUD	C	Process 8 (every process only adds audit lines)
D9 Media files (R2)	CRD	R			C	C				Process 1 (Processes 5 and 6 add their own files)

Redis is not in the table because it holds copies that can be rebuilt (E13). Letters in this table are about what the process may do; the role-by-class permission table (B11) says which person may do it.

HOW TO READ IT

This is the data-flow diagrams in **table form**. Each row is a store (a shelf of data). Each column is a process. The letters say what that process may do to that shelf: **C**reate, **R**ead, **U**pdate, **D**elete. Green = it can change the shelf. Blue = it can only read.

Read a **column** to see what one process *depends on*: Process 5 (the tour) reads D2, D3, D6 and writes D5 and D7. Read a **row** to see who *depends on a store*: D2 (church structure) is read by five processes, so a mistake there reaches the tour, the locator and the follow-up at once. That row is the biggest **shared dependency** in the system.

The last column states the **one owner rule**: every store has one process that is allowed to change its meaning; the others only add (C) or read. The rule is what keeps a system understandable when volunteers and new developers join later.

MISTAKES TO AVOID

- Two processes with U on the same store and no owner. When both change a row differently, nobody knows which is right.
- A store with only R in its row (nobody writes it) or only C (nobody reads it). Both are signs of a forgotten feature or a table that should not exist.
- Treating D8 (audit) as just another store. Every process may add a line, but only Process 8 may read, search or purge it.

WHAT YOU ALREADY DID RIGHT

- You asked what depends on what and what is "the main". This table answers it with evidence: the most-read shelves are D2 and D1, the most-written is D4 (people), and the only shelf every process touches is the audit log D8.

ATLAS · FAMILY 10

ER (database)

The database design. The whole church in 33 boxes (E01), then each area in detail, the 25 translation tables, the 27 fixed lists, the Redis keys and the rules the database itself enforces.

> **EXPLAINED LIKE YOU ARE FIVE**
> Like the filing plan: which cabinets, which folders, which cross-references.

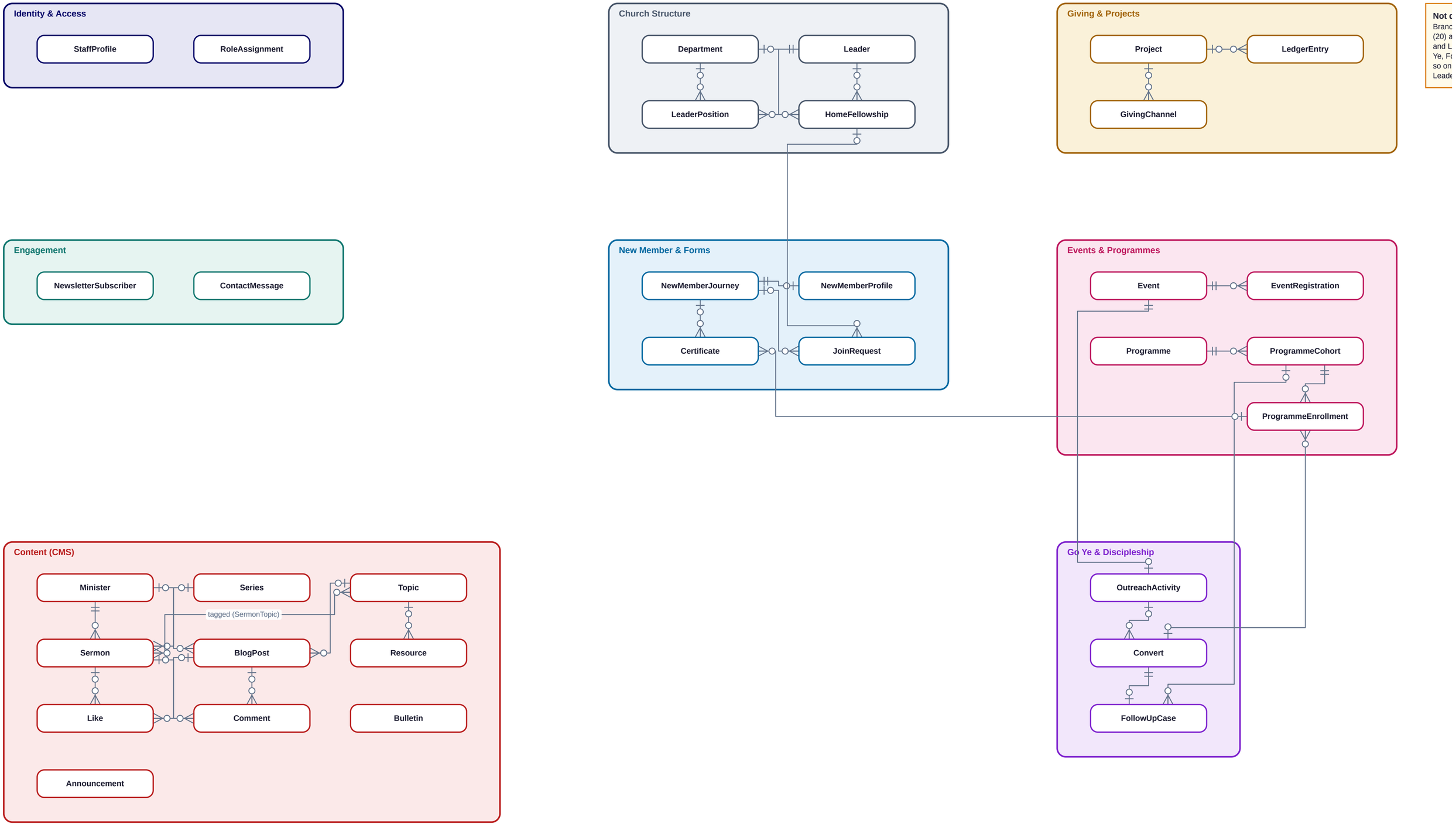
Code	Title	What it shows
E01	Conceptual ER: the whole church in 33 boxes	Things and how they relate, without columns
E02	ER: Identity and access	Staff accounts, roles, scope, audit
E03	ER: Church structure	Branches, departments, leaders, fellowships
E04	ER: Content (sermons, blogs, library)	The CMS side of the site
E05	ER: Events and programmes	Events, guests, sessions, registrations, programmes, cohorts
E06	ER: Go Ye and follow-up	Outreach, converts, cases, contact attempts
E07	ER: Giving and projects	Projects, milestones, updates, ledger, giving channels
E08	ER: New member and forms	Form builder, journey, profile, feedback, certificate, join requests
E09	ER: Engagement	Newsletter, contact, consent, notification log
E10	ER: Platform tables and who points at them	MediaAsset and SiteSetting
E11	ER: the 25 translation tables	One pattern, repeated: entity + locale = one row of words
E12	ER: the 27 enumerations (fixed lists of allowed values)	Every status and category word in one place
E13	Redis key space (Upstash)	Keys, types, lifetimes and purpose
E14	Database rules that the database itself enforces	CHECK constraints, unique indexes and partial indexes

E01

ER (DATABASE) · PHASE 1

Conceptual ER: the whole church in 33 boxes

Things and how they relate, without columns



HOW TO READ IT

This is the **map of the territory**. No columns, only things and their relationships. Read a line like a sentence: "**one** Sermon is preached by **one** Minister; a Minister preaches **many** Sermons".

The "crow's foot" (three little lines) means **many**. A bar means **exactly one**. A small circle means **optional** (zero or one).

The coloured containers are the eight **modules**. A line that leaves a container shows a dependency between modules (see K02).

MISTAKES TO AVOID

- Starting a database with tables and no map. Draw this page first with the church leaders; they will spot the missing box (for example "ministry teams") long before code exists.

WHAT YOU ALREADY DID RIGHT

- You asked for ER diagrams in the first message. This is the one to show non-technical leaders.

E02

ER (DATABASE) · PHASE 1

ER: Identity and access

Staff accounts, roles, scope, audit

user		
PK	**id**	text
	name	text
UQ	email	text
	email_verified	bool
	image	text
	role	text
	banned	bool
	ban_reason	text
	ban_expires	timestamptz
	created_at	timestamptz
	updated_at	timestamptz

session		
PK	**id**	text
FK	user_id	text
UQ	token	text
	expires_at	timestamptz
	ip_address	text
	user_agent	text
FK	impersonated_by	text
	created_at	timestamptz
	updated_at	timestamptz

account		
PK	**id**	text
FK	user_id	text
	provider_id	text
	account_id	text
	password	text
	access_token	text
	refresh_token	text
	created_at	timestamptz
	updated_at	timestamptz

verification		
PK	**id**	text
	identifier	text
	value	text
	expires_at	timestamptz
	created_at	timestamptz
	updated_at	timestamptz

Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional

audit_log		
PK	**id**	uuid
FK	actor_id	text
	action	text
	entity_type	text
	entity_id	text
FK	branch_id	uuid
	diff	jsonb
	ip	text
	user_agent	text
	at	timestamptz

leader		
PK	**id**	uuid
	… other columns on its own page	

leader_id

staff_profile		
PF	**user_id**	text
	display_name	text
	phone	text
FK	home_branch_id	uuid
FK	leader_id	uuid
FK	avatar_media_id	uuid
	ui_locale	Locale
	is_active	bool
	last_login_at	timestamptz

department		
PK	**id**	uuid
	… other columns on its own page	

department_id

role_assignment		
PK	**id**	uuid
FK	user_id	text
	role	StaffRole
	scope_type	ScopeType
FK	branch_id	uuid
FK	department_id	uuid
FK	granted_by	text
	granted_at	timestamptz
	expires_at	timestamptz

HOW TO READ IT

Better Auth owns **user, session, account, verification**. Their ids are **text**, not uuid, because Better Auth generates text ids. Every other table in the system that points at a user therefore uses text for that column.

role_assignment is the heart of access control: one row = one person holds one role in one scope (global, branch or department).

audit_log is append-only and records who changed what and when.

MISTAKES TO AVOID

- Using uuid columns to point at Better Auth user ids. The types differ and Postgres will refuse the foreign key.

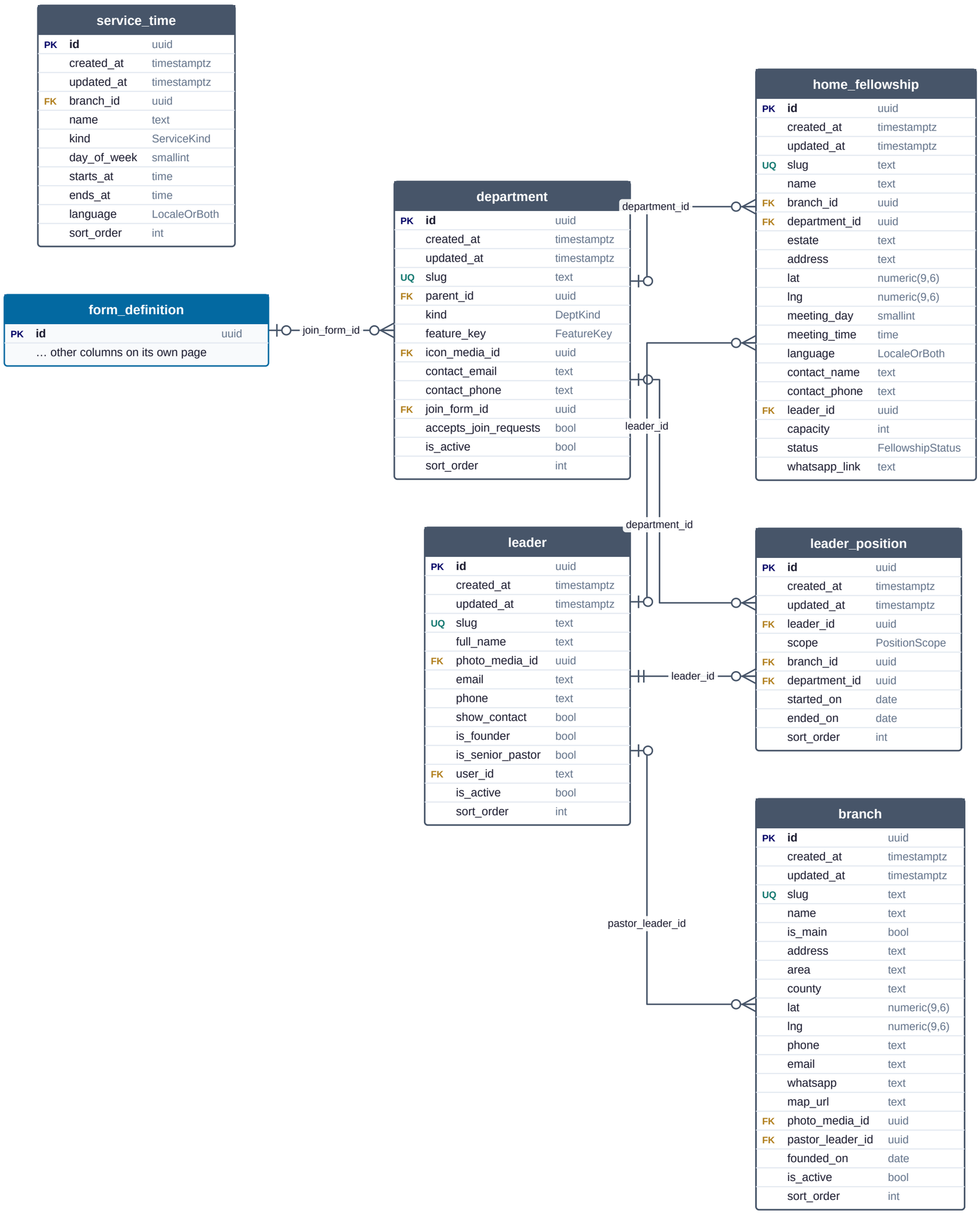
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-19** An admin area for staff only, signed in with Better Auth

ER: Church structure

Branches, departments, leaders, fellowships



Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional

HOW TO READ IT

branch is the most referenced table in the whole database. **department** points at itself (parent_id) to make sub-departments. **leader_position** keeps history: the same leader can hold different titles over time.

home_fellowship belongs to a branch and to the Home Fellowship department, and stores latitude and longitude for the nearest-fellowship finder.

MISTAKES TO AVOID

- Storing a leader's title on the leader. When they are promoted you lose the old title and the date they held it.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-03** Departments with sub-departments, leaders and a join form

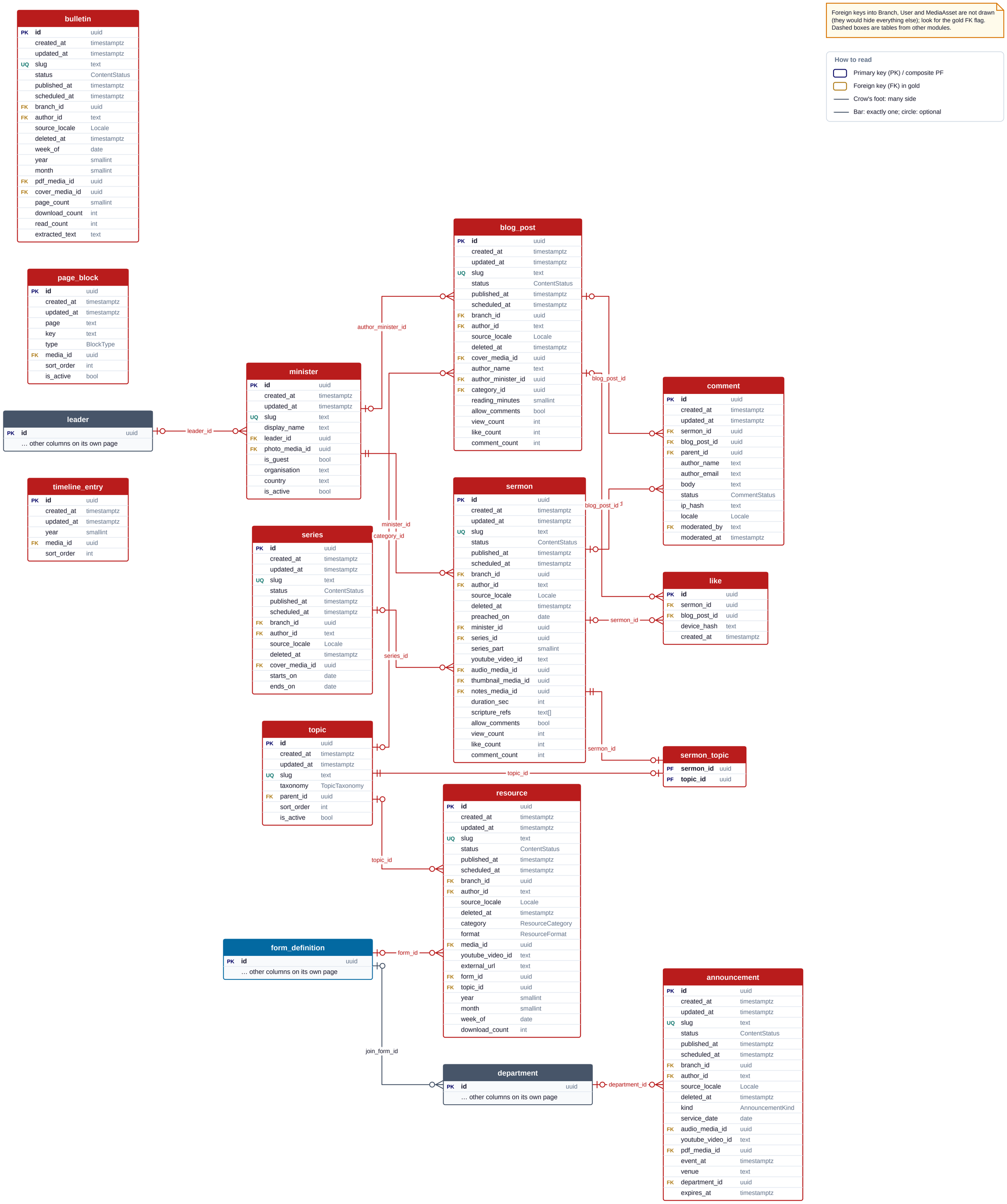
ER: Content (sermons, blogs, library)

The CMS side of the site

Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional



HOW TO READ IT

The Sermon table is wide on purpose: everything the sermon card needs is on one row, with translated text in **sermon_translation**. **sermon_topic** is a pure link table (many-to-many).

Comments and Likes both point at a sermon **or** a blog post; a CHECK constraint says exactly one of the two is set.

MISTAKES TO AVOID

- Using "polymorphic" columns (target_type + target_id) for comments and likes. Postgres cannot enforce those; two real foreign keys plus a CHECK can.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-10** Resources: blogs, bulletins, announcements (Sunday and midweek), Bible study and discipleship, other
- **R-11** Sermons with video, audio, summary, verses, filters, views, comments and likes

ER (DATABASE) · PHASE 1

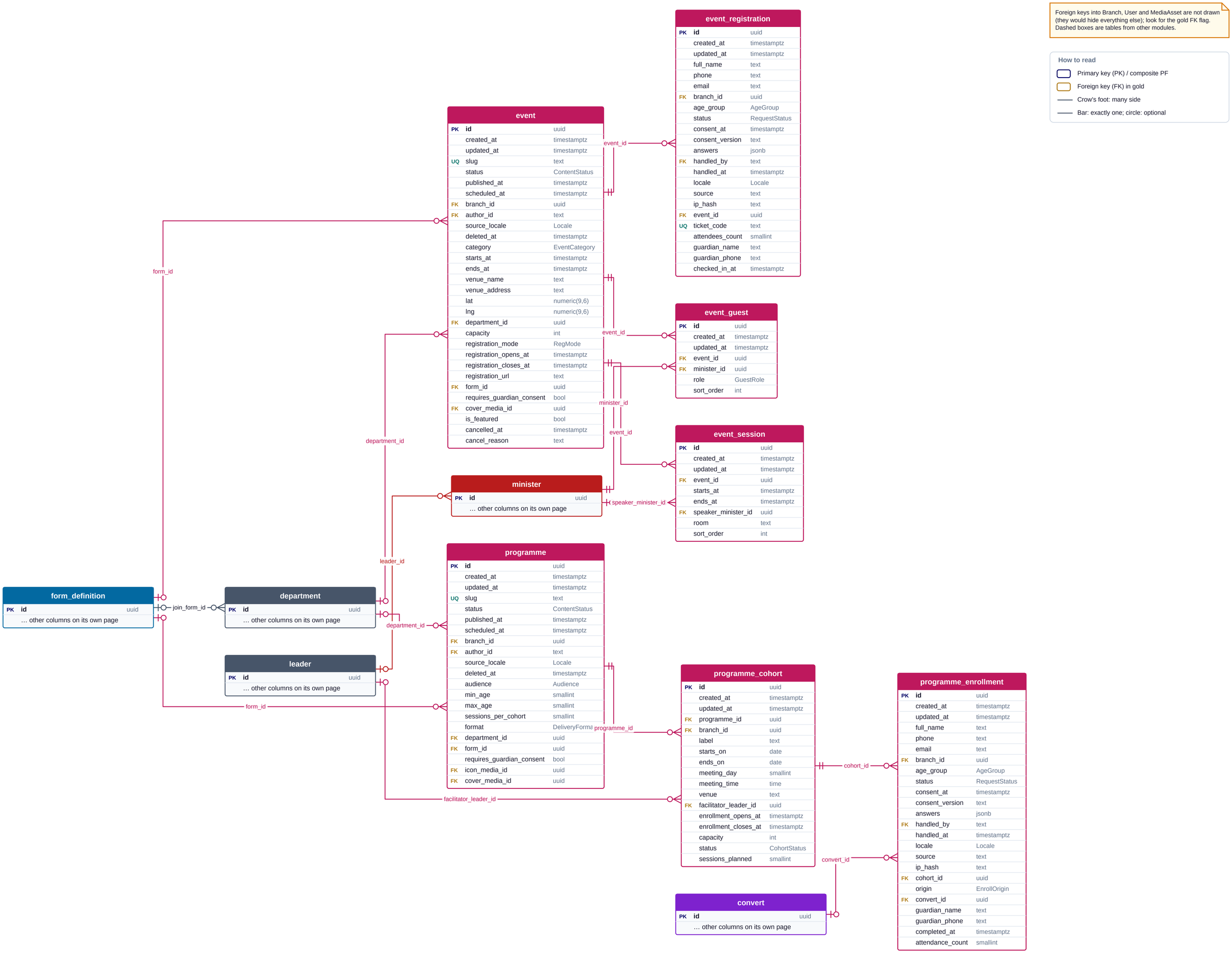
ER: Events and programmes

Events, guests, sessions, registrations, programmes, cohorts

Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional



HOW TO READ IT

event_registration and **programme_enrollment** share the PersonRequest columns, but are separate tables so each can have its own foreign key and its own indexes.

MISTAKES TO AVOID

- One giant "registrations" table with a type column. It looks tidy and becomes slow and hard to secure.

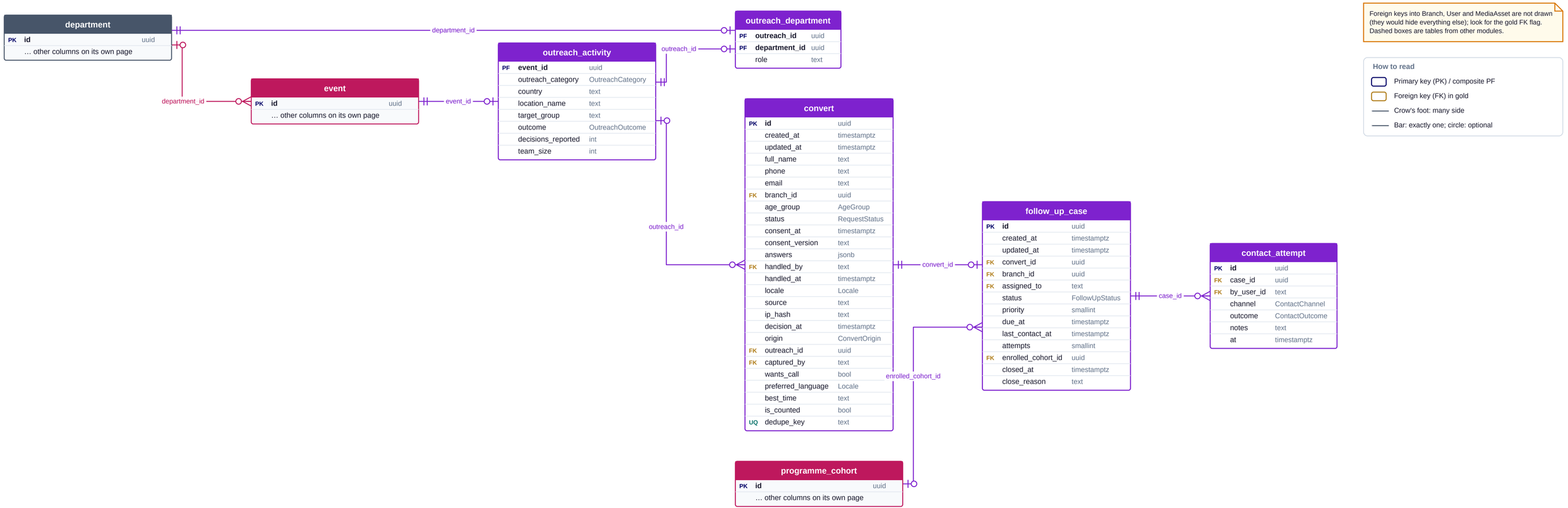
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-05** Events with dates, guest ministers, sessions and registration
- **R-06** Programmes: Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again

ER: Go Ye and follow-up

Outreach, converts, cases, contact attempts



Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional

HOW TO READ IT

outreach_activity uses **class-table inheritance**: its primary key is also a foreign key to event.id. One outreach = one row in event + one row in outreach_activity.

convert holds the sensitive details. **follow_up_case** is 1:1 with a convert (unique on convert_id). **contact_attempt** is the append-only diary.

MISTAKES TO AVOID

- Copying the convert's phone number into the follow-up case. One source of truth; join when you need it.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

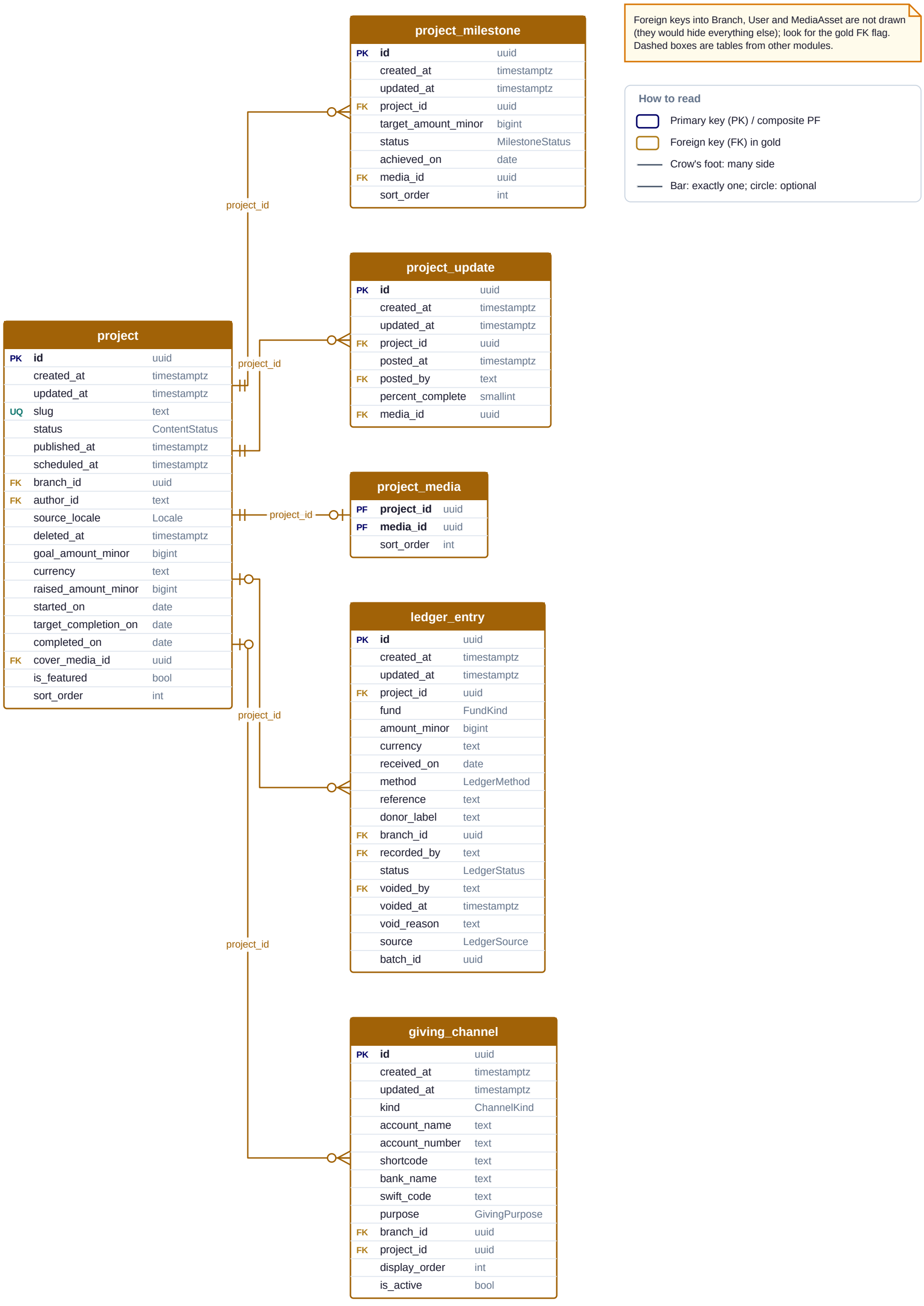
- **R-07** Go Ye: outreach, born-again counter, a form to add a convert

E07

ER (DATABASE) · PHASE 1

ER: Giving and projects

Projects, milestones, updates, ledger, giving channels



Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional

HOW TO READ IT

ledger_entry is append-only; **project.raised_amount_minor** is a cache recomputed from it in the same transaction. Amounts are **bigint minor units** (cents), never floats.

MISTAKES TO AVOID

- Storing the raised amount as an input field on the project. Someone will type it wrong and nobody will notice.

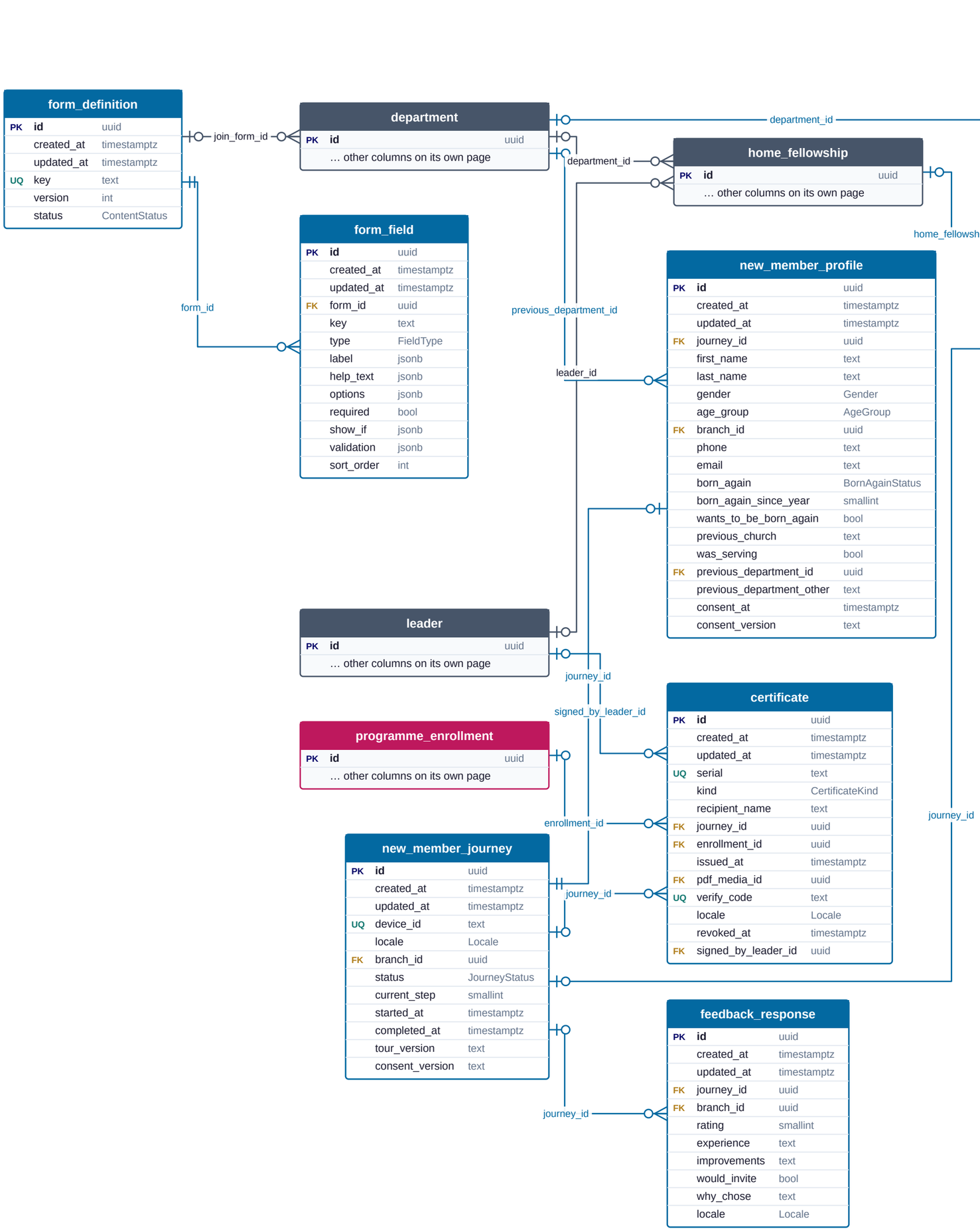
WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-08** Projects with milestones and a given / remaining bar
- **R-24** Treasurer's record of gifts that drives the project bars

ER: New member and forms

Form builder, journey, profile, feedback, certificate, join requests



Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional

HOW TO READ IT

Forms are **data**: form_definition + form_field. Answers are stored as JSON keyed by field id, and each answer row remembers the **form version** it was collected under.

certificate points at the journey (not the other way round); this removes a circular reference that would otherwise block creating the tables.

MISTAKES TO AVOID

- Hard-coding the tour questions in React components. The next change request becomes a code deployment.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-14** New-member tour of nine steps, ending in a certificate

E09

ER (DATABASE) · PHASE 1

ER: Engagement

Newsletter, contact, consent, notification log

newsletter_subscriber

PK	**id**	uuid
	created_at	timestamptz
	updated_at	timestamptz
UQ	email	text
	name	text
	locale	Locale
FK	branch_id	uuid
	status	SubscriberStatus
	source	SubSource
	confirm_token_hash	text
	confirmed_at	timestamptz
	consent_at	timestamptz
	consent_version	text
	unsubscribed_at	timestamptz
	resend_contact_id	text
	topics	text[]

newsletter_campaign

PK	**id**	uuid
	created_at	timestamptz
	updated_at	timestamptz
	kind	CampaignKind
	status	CampaignStatus
	audience_locale	Locale
	scheduled_at	timestamptz
	sent_at	timestamptz
	recipient_count	int
	resend_broadcast_id	text
FK	created_by	text
	stats	jsonb

contact_message

PK	**id**	uuid
	created_at	timestamptz
	updated_at	timestamptz
	name	text
	email	text
	phone	text
	topic	ContactTopic
FK	branch_id	uuid
	message	text
	status	MessageStatus
FK	assigned_to	text
	ip_hash	text
	replied_at	timestamptz
	locale	Locale

consent_record

PK	**id**	uuid
	created_at	timestamptz
	updated_at	timestamptz
	visitor_id	text
	categories	jsonb
	policy_version	text
	source	text
	user_agent_hash	text

notification_log

PK	**id**	uuid
	created_at	timestamptz
	updated_at	timestamptz
	channel	NotifyChannel
	template	text
	to_masked	text
	status	NotifyStatus
	provider_message_id	text
	error	text
	related_type	text
	related_id	text
	attempts	smallint
	sent_at	timestamptz

Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional

HOW TO READ IT

These tables are the **paper trail**: what people sent us, what they agreed to, and what we sent back. Consent rows are never edited, only added.

MISTAKES TO AVOID

- Letting a "delete my data" request skip the consent log. Keep the proof that consent existed, anonymise the person.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-15** Modal order (new-member question before the newsletter), a cookie bar, and "No" remembered in localStorage or cookies
- **R-16** Newsletter with a confirmed sign-up

ER: Platform tables and who points at them

MediaAsset and SiteSetting

media_asset		
PK	**id**	uuid
	created_at	timestamptz
	updated_at	timestamptz
	kind	MediaKind
	bucket	text
UQ	r2_key	text
	mime_type	text
	size_bytes	bigint
	checksum	text
	original_name	text
	width	int
	height	int
	duration_sec	int
	alt_text	jsonb
	is_public	bool
	status	MediaStatus
FK	uploaded_by	text
FK	branch_id	uuid

site_setting		
PK	**key**	text
	value	jsonb
FK	updated_by	text
	updated_at	timestamptz

Foreign keys into Branch, User and MediaAsset are not drawn (they would hide everything else); look for the gold FK flag. Dashed boxes are tables from other modules.

How to read

- Primary key (PK) / composite PF
- Foreign key (FK) in gold
- Crow's foot: many side
- Bar: exactly one; circle: optional

HOW TO READ IT

media_asset is the single registry of every file. Twenty tables point at it. The file itself lives in Cloudflare R2; the row stores its key, type, size and status (pending, ready, rejected).

site_setting holds small editable values such as the church name, social links and the converts counter baseline.

MISTAKES TO AVOID

- Saving file URLs as text in each table. When you move storage you must edit every table.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

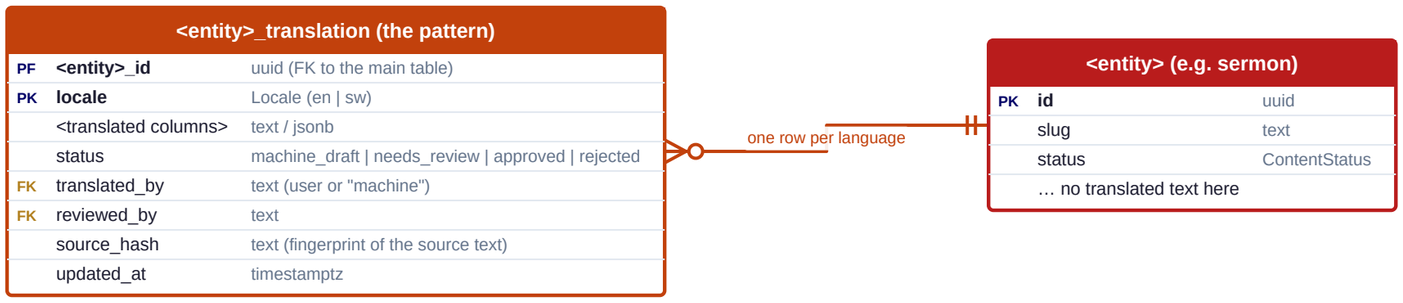
- **R-26** Next.js, Drizzle, Neon Postgres, Resend, Upstash Redis, QStash, PostHog, Better Auth (admin), Tailwind, Cloudflare R2, Vercel

E11

ER (DATABASE) · PHASE 1

ER: the 25 translation tables

One pattern, repeated: entity + locale = one row of words



<entity>_translation (the pattern)	
PF **<entity>_id**	uuid (FK to the main table)
PK **locale**	Locale (en | sw)
<translated columns>	text / jsonb
status	machine_draft | needs_review | approved | rejected
FK translated_by	text (user or "machine")
FK reviewed_by	text
source_hash	text (fingerprint of the source text)
updated_at	timestamptz

<entity> (e.g. sermon)	
PK **id**	uuid
slug	text
status	ContentStatus
… no translated text here	

Adding a third language later
Add the code to the Locale enum and insert rows. No table changes, no code changes in the modules.

branch_translation	
PF **branch_id**	uuid
PK **locale**	Locale
description	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

department_translation	
PF **department_id**	uuid
PK **locale**	Locale
name	text
summary	text
what_we_do	text
meeting_times	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

leader_translation	
PF **leader_id**	uuid
PK **locale**	Locale
bio	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

leader_position_translation	
PF **leader_position_id**	uuid
PK **locale**	Locale
title	text
responsibilities	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

home_fellowship_translation	
PF **home_fellowship_id**	uuid
PK **locale**	Locale
notes	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

minister_translation	
PF **minister_id**	uuid
PK **locale**	Locale
bio	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

bulletin_translation	
PF **bulletin_id**	uuid
PK **locale**	Locale
title	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

event_translation	
PF **event_id**	uuid
PK **locale**	Locale
title	text
description	text
fee_info	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

project_milestone_translation	
PF **project_milestone_id**	uuid
PK **locale**	Locale
title	text
description	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

series_translation	
PF **series_id**	uuid
PK **locale**	Locale
title	text
description	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

announcement_translation	
PF **announcement_id**	uuid
PK **locale**	Locale
title	text
body	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

event_session_translation	
PF **event_session_id**	uuid
PK **locale**	Locale
title	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

project_update_translation	
PF **project_update_id**	uuid
PK **locale**	Locale
title	text
body	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

topic_translation	
PF **topic_id**	uuid
PK **locale**	Locale
name	text
description	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

resource_translation	
PF **resource_id**	uuid
PK **locale**	Locale
title	text
description	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

outreach_activity_translation	
PF **outreach_activity_id**	uuid
PK **locale**	Locale
report_summary	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

giving_channel_translation	
PF **giving_channel_id**	uuid
PK **locale**	Locale
label	text
reference_instructions	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

sermon_translation	
PF **sermon_id**	uuid
PK **locale**	Locale
title	text
summary	text
key_points	jsonb
highlights	jsonb
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

page_block_translation	
PF **page_block_id**	uuid
PK **locale**	Locale
data	jsonb
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

programme_translation	
PF **programme_id**	uuid
PK **locale**	Locale
name	text
description	text
schedule_summary	text
duration_label	text
fee_info	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

form_definition_translation	
PF **form_definition_id**	uuid
PK **locale**	Locale
title	text
intro	text
consent_text	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

blog_post_translation	
PF **blog_post_id**	uuid
PK **locale**	Locale
title	text
excerpt	text
body	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

timeline_entry_translation	
PF **timeline_entry_id**	uuid
PK **locale**	Locale
title	text
body	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

project_translation	
PF **project_id**	uuid
PK **locale**	Locale
title	text
summary	text
body	text
impact	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

newsletter_campaign_translation	
PF **newsletter_campaign_id**	uuid
PK **locale**	Locale
subject	text
body	text
status	TranslationStatus
FK translated_by	text
FK reviewed_by	text
source_hash	text
updated_at	timestamptz

HOW TO READ IT

Every piece of words an editor writes (titles, summaries, blog bodies, department descriptions) lives in a **translation table**, never on the main table. The main table keeps only facts that do not change with language (dates, ids, status).

The primary key is **(entity id + locale)**: exactly one row per language per thing. The same 25 tables follow the same shape, so one TranslationService handles them all.

This answers your question about translating dynamic content such as blogs: **draft by machine, approve by person**, stored once per language.

MISTAKES TO AVOID

- Putting a language column on the main table (title_en, title_sw). It works for two languages and breaks for a third.
- Translating long HTML. Store the body as structured JSON (paragraph by paragraph) so the machine can translate text without breaking markup.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-12** English and Kiswahili switch

E12

ER (DATABASE) · PHASE 1

ER: the 27 enumerations (fixed lists of allowed values)

Every status and category word in one place

Locale
en
sw

RequestStatus
new
in_progress
confirmed
waitlisted
declined
cancelled
completed

ScopeType
global
branch
department

ResourceFormat
pdf
audio
video
link
form

OutreachCategory
school
unreached
international
mission_week

LedgerMethod
cash
mpesa
bank
cheque
other

DeptKind
ministry
fellowship
committee
service_team

LocaleOrBoth
en
sw
both

AgeGroup
under_13
13_17
18_25
26_35
36_50
51_plus

CommentStatus
pending
approved
rejected
spam

TopicTaxonomy
theme
resource_topic
blog_category

FollowUpStatus
new
assigned
attempting
connected
enrolled
completed
unreachable
declined

ChannelKind
mpesa_paybill
mpesa_till
bank
cheque
cash
international
link

FeatureKey
home_fellowship
missions
discipleship
children
youth
men
women
media

ContentStatus
draft
in_review
scheduled
published
archived

Gender
male
female
prefer_not

AnnouncementKind
sunday
midweek

EventCategory
conference
worship
marriage
youth
trip
retreat
outreach
prayer
fellowship
other

ProjectStatus
planned
fundraising
in_progress
completed
paused

JourneyStatus
started
in_progress
completed
abandoned

CohortStatus
planned
open
full
running
completed
cancelled

TranslationStatus
missing
machine_draft
needs_review
approved

StaffRole
super_admin
branch_admin
content_editor
translator
department_leader
discipler
treasurer
events_coordinator

ResourceCategory
discipleship
bible_study
missions
prayer_points
forms
other

RegMode
none
form
external
walk_in

LedgerStatus
recorded
voided

SubscriberStatus
pending
subscribed
unsubscribed
bounced
complained

HOW TO READ IT

An **enumeration** (enum) is a short, fixed list of allowed words, for example ContentStatus = draft, in_review, scheduled, published, archived. The database refuses any other word, so a typo like "publishd" cannot get in.

These lists are also the vocabulary of the state machines (T01 to T09) and of the permission letters.

MISTAKES TO AVOID

- Using free text for statuses. Reports will count "Pending", "pending" and "pending " as three different things.
- Changing an enum value after launch without a migration. Add values; do not rename them.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-26** Next.js, Drizzle, Neon Postgres, Resend, Upstash Redis, QStash, PostHog, Better Auth (admin), Tailwind, Cloudflare R2, Vercel

E13

ER (DATABASE) · PHASE 1

Redis key space (Upstash)

Keys, types, lifetimes and purpose

Keys we use (and nothing else)	
KEY sermon:{id}:views	counter · no TTL · flushed every 5 min
KEY sermon:{id}:likes	counter · no TTL · flushed every 5 min
KEY blog:{id}:views	counter · no TTL
KEY view-seen:{deviceHash}:{id}	string · TTL 30 min (count once per visit)
KEY rl:{route}:{ipHash}	sliding window · TTL = window
KEY turnstile-fail:{ipHash}	counter · TTL 1 h
KEY lock:{jobName}	string · TTL 2 min (stop two runs overlapping)
KEY locale-map:{entity}:{id}	hash · TTL 1 day (slug per language)
KEY login-fail:{emailHash}	counter · TTL 15 min
KEY tour-draft:{token}	string (JSON) · TTL 24 h (resume after reload)

Rules for Redis

1. Redis may lose data; nothing here is the only copy of anything important.
2. Every key has a prefix and a TTL, except counters that a job flushes.
3. Never store personal data (names, phones) in Redis; store hashes.
4. Keys are written only through CacheService and RateLimiter.

HOW TO READ IT

Redis is the **fast scratch pad**. It is where we count things quickly and remember things briefly. Anything that must never be lost (likes, ledger, converts) is also written to Postgres.

Upstash is used over HTTP, which suits Vercel functions that start and stop quickly; a normal Redis connection would run out of connections.

MISTAKES TO AVOID

- Using Redis as the only store for a number people care about (likes, donations). A restart could erase it.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-26** Next.js, Drizzle, Neon Postgres, Resend, Upstash Redis, QStash, PostHog, Better Auth (admin), Tailwind, Cloudflare R2, Vercel

E14

ER (DATABASE) · PHASE 1

Database rules that the database itself enforces

CHECK constraints, unique indexes and partial indexes

Constraints (CK), unique rules (UQ) and indexes (IX)		
CK	sermon: youtube_video_id OR audio_media_id is set	never an empty sermon
CK	comment: exactly one of sermon_id / blog_post_id	no orphan comments
CK	like: exactly one of sermon_id / blog_post_id	no orphan likes
UQ	like (sermon_id, device_hash) and (blog_post_id, device_hash)	one like per device
UQ	branch: only one row with is_main = true (partial unique)	exactly one main branch
UQ	convert.dedupe_key (per branch)	never count a person twice
UQ	follow_up_case.convert_id	1:1 with convert
UQ	programme_enrollment (cohort_id, phone_hash)	one place per person
UQ	event_registration (event_id, phone_hash)	one registration per phone
UQ	role_assignment (user_id, role, scope_type, scope_id)	no duplicate roles
UQ	slug per table (and per branch where branch-scoped)	stable URLs
UQ	certificate.serial	verifiable certificates
CK	ledger_entry.amount_minor > 0 and kind in (gift, adjustment)	no negative gifts; voids are flags
CK	ledger_entry: voided_at set ⇔ void_reason set	every void explains itself
IX	sermon (status, preached_on desc) where deleted_at is null	fast sermon list
IX	home_fellowship (lat, lng) where is_active	fast nearest search
IX	follow_up_case (assigned_to, status, due_at)	the discipler's worklist
IX	audit_log (entity_type, entity_id, created_at desc)	history of one record

HOW TO READ IT

Rules written in the **database** hold even when a bug or a quick script forgets them. The application checks the same rules to give friendly messages, but the database is the last line of defence.

CK = a check on one row. **UQ** = no duplicates allowed. **IX** = an index, a shortcut that makes a common search fast.

MISTAKES TO AVOID

- Relying only on form validation. A second form, an import script or an API client will bypass it.
- Adding an index "just in case". Each index slows every write. Add one only for a query you can name.

WHAT YOU ALREADY DID RIGHT

This page serves your own requirements:

- **R-25** Audit log, privacy requests and a security-incident routine

PART 11

Data dictionary

Every one of the 74 classes, printed from the same model that draws the diagrams. For each: what it is in plain words, its fields with flags, its rules, the mistakes to avoid, and which roles may do what.

Letter	Means	Letter	Means
C	create	P	publish
R	read	A	approve or moderate
U	update	X	export
D	delete or archive	S	submit through a public form (write-only)

Flags. private = personal data (name, phone, e-mail), hidden in logs. **sensitive** = belief, children's and follow-up data, the strictest handling. **translated** = lives in a translation table, one row per language. **cached** = a copy that can be rebuilt (for example a project's raised total). Fields inherited from BaseEntity (id, createdAt, updatedAt), Publishable or PersonRequest are listed on those parents and not repeated.

> **EXPLAINED LIKE YOU ARE FIVE**
>
> A dictionary entry is the full "ID card" of one kind of thing. Read the plain sentence first. If it makes sense, read the fields. If you are about to *build* that class, read the rules and the mistakes last, because they are the lessons paid for by someone else.

PART 11 · MODULE

Identity & Access (8 classes)

User

table · module Identity & Access · table `user`

The login account of a staff member. A staff login. Managed by Better Auth (its "user" table). Only staff have accounts at launch.

Field	Type	Flags
`id`	text	PK
`name`	text	required
`email`	text	unique required private
`emailVerified`	bool	required
`image`	text	
`role`	text	
`banned`	bool	
`banReason`	text	
`banExpires`	timestamptz	
`createdAt`	timestamptz	required
`updatedAt`	timestamptz	required

Does: `signIn(): Session`, `signOut(): void`, `can(action, resource): bool`

Rules

- Better Auth owns this table and its id type is text, so every "createdBy / handledBy" column in our own tables is also text.
- Email must be verified before a user can receive any role.
- Banning a user (admin plugin) kills all their sessions immediately.
- Public visitors never get a row here at launch.

Mistakes to avoid

- Adding church columns (phone, branch) straight onto the Better Auth table. Put them in StaffProfile so auth upgrades never fight your schema.
- Using the plugin "role" field as the whole permission system. It only says user/admin; real scoped roles live in RoleAssignment.

Who may: Super CRUD Branch R

Session

table · module Identity & Access · table `session`

A "you are logged in" wristband. One row per signed-in browser. Better Auth creates and expires these.

Field	Type	Flags
`id`	text	PK
`userId`	text	required, FK User, computed
`token`	text	unique, required, private
`expiresAt`	timestamptz	required
`ipAddress`	text	
`userAgent`	text	
`impersonatedBy`	text	FK User
`createdAt`	timestamptz	required
`updatedAt`	timestamptz	required

Rules

- Short expiry for admin (e.g. 7 days sliding) and revoke-all on password change.
- Cookie is httpOnly, secure, sameSite=lax.
- Created and destroyed by Better Auth only. Application code never inserts here.
- Cookie is httpOnly + secure + sameSite=lax; a session lasts 7 days and refreshes on use.

Mistakes to avoid

- Writing your own session code next to Better Auth. Two systems = two bugs.

Who may: Super RD

Account

table · module Identity & Access · table `account`

The key that opens the login. Stores HOW a user proves who they are (password hash, or Google later).

Field	Type	Flags
`id`	text	PK
`userId`	text	required, FK User, computed
`providerId`	text	required
`accountId`	text	required
`password`	text	private
`accessToken`	text	private
`refreshToken`	text	private
`createdAt`	timestamptz	required
`updatedAt`	timestamptz	required

Rules

- One user may have several accounts (email+password now, Google later). Better Auth owns this table.
- Password hash never leaves the server and is never logged.

Mistakes to avoid

- Storing plain passwords or logging login requests with bodies.

Who may: Super RD

Verification

table · module Identity & Access · table `verification`

One-time codes that expire. Short-lived codes for email verification and password reset.

Field	Type	Flags
`id`	text	PK
`identifier`	text	required
`value`	text	required private
`expiresAt`	timestamptz	required
`createdAt`	timestamptz	required
`updatedAt`	timestamptz	required

Rules

- Short-lived tokens for e-mail verification and password reset (minutes, single use).
- A cron/QStash job deletes expired rows daily.

Mistakes to avoid

- Letting tokens live for days. Short life = small damage if an email is forwarded.

Who may: Super R

StaffProfile

table · module Identity & Access · table `staff_profile`

The staff member's name-badge details. Church-specific facts about a staff member, kept apart from the auth table.

Field	Type	Flags
`userId`	text	PK required FK User computed
`displayName`	text	required
`phone`	text	private
`homeBranchId`	uuid	FK Branch
`leaderId`	uuid	FK Leader
`avatarMediaId`	uuid	FK MediaAsset
`uiLocale`	Locale	required
`isActive`	bool	required
`lastLoginAt`	timestamptz	

Rules

- 1:1 with User.
- leaderId links a staff member to their public leader profile when they have one.
- One profile per staff User: display name, phone, photo, bio shown on leader pages (only if showPublic = true).
- Deactivating a staff member sets active=false; the User row stays so the audit trail still has a name.

Mistakes to avoid

- Showing staff phone numbers publicly by default.

Who may: Super CRUD Branch CRU

RoleAssignment

table · module Identity & Access · table `role_assignment`

A sticker that says "Grace may edit sermons at Eastern Bypass". "Grace is a Content Editor for Eastern Bypass." One row = one role in one scope. A person can hold several.

Field	Type	Flags
`id`	uuid	PK
`userId`	text	required · FK User · computed
`role`	StaffRole	required
`scopeType`	ScopeType	required
`branchId`	uuid	FK Branch
`departmentId`	uuid	FK Department
`grantedBy`	text	FK User
`grantedAt`	timestamptz	required
`expiresAt`	timestamptz	

Does: `grant(): void`, `revoke(): void`, `covers(resource): bool`

Rules

- scopeType = global | branch | department; branchId / departmentId must match the scope (CHECK constraint).
- Unique on (user, role, scope, branch, department) so nobody gets the same role twice.
- Branch Admins can only grant roles inside their own branch; only Super Admin grants global roles.
- Every grant/revoke writes an AuditLog row.

Mistakes to avoid

- Putting a single "role" column on the user. A department leader who also edits blogs needs two assignments.
- Forgetting expiresAt for temporary helpers (e.g. conference volunteers).

Who may: Super CRUD · Branch CRUD

AuditLog

table · module Identity & Access · table `audit_log`

The church's security-camera footage for the database. A tamper-evident diary: who did what, to which record, when. Essential for money and personal data.

Field	Type	Flags
`id`	uuid	PK
`actorId`	text	FK User
`action`	text	required
`entityType`	text	required
`entityId`	text	
`branchId`	uuid	FK Branch
`diff`	jsonb	
`ip`	text	private
`userAgent`	text	
`at`	timestamptz	required indexed

Rules

- Append-only: no UPDATE or DELETE grants for the app role.
- Written for: grants, publishes, deletes, ledger entries and voids, exports, PII views.
- Kept 24 months; older rows archived to R2.

Mistakes to avoid

- Logging only creates. Deletes, exports and "viewed phone number" are the ones you need when something goes wrong.

Who may: Super R Branch R

MemberAccount reserved for later

table · module Identity & Access · table `member_account`

Reserved seat for member logins. PHASE 2 slot. When member logins arrive they hang off User without changing any other table.

Field	Type	Flags
`id`	uuid	PK
`userId`	text	required FK User computed
`phone`	text	private
`branchId`	uuid	FK Branch
`notifyPrefs`	jsonb	
`joinedOn`	date	

Rules

- PHASE 2. Reserved so the schema does not need a rewrite when members get logins.
- Will link a Better Auth User of type "member" to NewMemberProfile; staff and member users never share role tables.

Mistakes to avoid

- Building member login now. It doubles the security surface before the church has asked for it.

PART 11 · MODULE

Church Structure (6 classes)

Branch

table · extends **BaseEntity** · module Church Structure · table `branch`

One church building and its community. A physical church location. Almost every other table carries a branchId, so Branch is the most-referenced class in the system.

Field	Type	Flags
`slug`	text	unique required
`name`	text	required
`isMain`	bool	required
`address`	text	
`area`	text	
`county`	text	
`lat`	numeric(9,6)	
`lng`	numeric(9,6)	
`phone`	text	
`email`	text	
`whatsapp`	text	
`mapUrl`	text	
`photoMediaId`	uuid	FK MediaAsset
`pastorLeaderId`	uuid	FK Leader
`foundedOn`	date	
`isActive`	bool	required
`sortOrder`	int	required
`description`	text	translated

Does: `distanceTo(lat, lng): km`, `nextService(now): ServiceTime`

Rules

- Exactly one branch has isMain = true (partial unique index).
- Never hard-delete a branch; set isActive = false so history keeps its meaning.
- Branch scope drives who may edit what (RoleAssignment.branchId).

Mistakes to avoid

- Storing the branch NAME in other tables instead of its id. Rename once and half your data is orphaned.

Who may: Visitor R · New member R · Super CRUD · Branch RU · Editor R · Translator R · Dept leader R · Discipler R · Treasurer R · Events R

ServiceTime

table · extends **BaseEntity** · module Church Structure · table `service_time`

The timetable on the church door. When each branch meets (Sunday services, midweek prayer, youth night).

Field	Type	Flags
`branchId`	uuid	required, FK Branch, computed
`name`	text	required
`kind`	ServiceKind	required
`dayOfWeek`	smallint	required
`startsAt`	time	required
`endsAt`	time	
`language`	LocaleOrBoth	required
`sortOrder`	int	required

Rules

- Times are stored as local Nairobi time (no date), displayed with the viewer's language.
- Many per branch (Sunday 1st/2nd service, Wednesday midweek, Friday prayer).
- weekday 0-6 + local time + timezone Africa/Nairobi; the site never stores "9am" as text.

Mistakes to avoid

- Typing service times as free text on the page: they go stale in 3 months. Keep them as data.

Who may: Visitor R · New member R · Super CRUD · Branch CRUD · Editor R · Dept leader R · Events R

Department

table · extends **BaseEntity** · module Church Structure · table `department`

A team in the church. A team can have smaller teams inside it. Ministries (Youth, Men, Women, Missions…) and their sub-departments. It points at itself (parentId) to build a tree of any depth.

Field	Type	Flags
`slug`	text	unique, required
`parentId`	uuid	FK Department
`kind`	DeptKind	required
`featureKey`	FeatureKey	
`iconMediaId`	uuid	FK MediaAsset
`contactEmail`	text	
`contactPhone`	text	
`joinFormId`	uuid	FK FormDefinition
`acceptsJoinRequests`	bool	required
`isActive`	bool	required
`sortOrder`	int	required
`name`	text	translated
`summary`	text	translated
`whatWeDo`	text	translated
`meetingTimes`	text	translated

Does: `children(): Department[]`, `ancestors(): Department[]`, `leadersAt(branch): Leader[]`

Rules

- parentId must not create a loop (checked in code and by a recursive CTE test).
- featureKey wires special behaviour: home_fellowship turns on the fellowship finder, missions powers the Go Ye "join the team" link, discipleship receives converts.
- joinFormId decides which follow-up questions the new-member tour asks about this department.
- Leaders belong to a department through LeaderPosition, not through a column here, so each branch can have different leaders.

Mistakes to avoid

- Making "SubDepartment" a separate table. A department with a parent IS a sub-department; one table, one query.
- Hard-coding "if department == Missions" in the UI instead of using featureKey.

Who may: Visitor R · New member R · Super CRUDP · Branch CRUDP · Editor RU · Translator RU · Dept leader RU · Discipler R · Events R

Leader

table · extends **BaseEntity** · module Church Structure · table `leader`

A leader's public profile card. A person shown publicly as a pastor, elder or leader. NOT a login (that is User) and NOT a sermon speaker record (that is Minister).

Field	Type	Flags
`slug`	text	unique, required
`fullName`	text	required
`photoMediaId`	uuid	FK MediaAsset
`email`	text	private
`phone`	text	private
`showContact`	bool	required
`isFounder`	bool	required
`isSeniorPastor`	bool	required
`userId`	text	FK User
`isActive`	bool	required
`sortOrder`	int	required
`bio`	text	translated

Does: `currentPositions(): LeaderPosition[]`, `positionAt(date): LeaderPosition`

Rules

- Email and phone are private unless showContact = true.
- userId is optional: most leaders never log in.

Mistakes to avoid

- Merging Leader, Minister and User into one "person" table. They overlap but have different privacy and life-cycles.

Who may: Visitor R · New member R · Super CRUD · Branch CRUD · Editor RU · Translator RU · Dept leader R

LeaderPosition

table · extends **BaseEntity** · module Church Structure · table `leader_position`

A job on a leader's CV, with a start date. "When they became what, and what they hold." One leader has many positions over time; history is kept, not overwritten.

Field	Type	Flags
`leaderId`	uuid	required · FK Leader · computed
`scope`	PositionScope	required
`branchId`	uuid	FK Branch
`departmentId`	uuid	FK Department
`startedOn`	date	required
`endedOn`	date	
`sortOrder`	int	required
`title`	text	translated
`responsibilities`	text	translated

Does: `isCurrent(on): bool`

Rules

- endedOn null means current.
- A position is scoped to church, one branch, or one department (CHECK).
- Never edit a past position to record a promotion; end it and add a new one.

Mistakes to avoid

- A single "title" text column on Leader. You lose the history and the "since when".

Who may: Visitor R · New member R · Super CRUD · Branch CRUD · Dept leader CRU

HomeFellowship

table · extends **BaseEntity** · module Church Structure · table `home_fellowship`

A small church that meets in somebody's sitting room. One small group that meets in a home. Its coordinates power the "find my nearest fellowship" feature.

Field	Type	Flags
`slug`	text	unique, required
`name`	text	required
`branchId`	uuid	required, FK Branch
`departmentId`	uuid	FK Department
`estate`	text	required
`address`	text	
`lat`	numeric(9,6)	required
`lng`	numeric(9,6)	required
`meetingDay`	smallint	required
`meetingTime`	time	required
`language`	LocaleOrBoth	required
`contactName`	text	required
`contactPhone`	text	required
`leaderId`	uuid	FK Leader
`capacity`	int	
`status`	FellowshipStatus	required
`whatsappLink`	text	
`notes`	text	translated

Does: `distanceTo(lat, lng): km`, `isAcceptingMembers(): bool`

Rules

- lat/lng required: no coordinates, no "nearest" result.
- Public contact is the fellowship contact, never the host's private number unless they agreed.
- status = full or paused removes it from "nearest" results but keeps the page.

Mistakes to avoid

- Searching by typing the estate name only. Typos break it; coordinates + a typed fallback is robust.

Who may: Visitor R · New member R · Super CRUD · Branch CRUD · Dept leader CRUD

PART 11 · MODULE

Content (CMS) (13 classes)

Minister

table · extends **BaseEntity** · module Content (CMS) · table `minister`

A preacher who can have many sermons. The speaker. Clicking a minister's name filters every sermon by that person, so Minister must be its own record.

Field	Type	Flags
`slug`	text	unique required
`displayName`	text	required
`leaderId`	uuid	FK Leader
`photoMediaId`	uuid	FK MediaAsset
`isGuest`	bool	required
`organisation`	text	
`country`	text	
`isActive`	bool	required
`bio`	text	translated

Does: `sermons(filter): Sermon[]`

Rules

- leaderId is optional because guest ministers are not church leaders.
- Merge duplicates ("Pr. John" vs "Pastor John Kamau") before launch, never after.

Mistakes to avoid

- Free-text speaker names on sermons. You cannot filter or link what you did not model.

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

Series

table · extends **Publishable** · module Content (CMS) · table `series`

A playlist of sermons. A run of connected messages (e.g. "Faith that Moves").

Field	Type	Flags
`coverMediaId`	uuid	FK MediaAsset
`startsOn`	date	
`endsOn`	date	
`title`	text	translated
`description`	text	translated

Does: `parts(): Sermon[]`

Rules

- Sermon.seriesPart orders the sermons inside a series.
- A series has a start and end date; sermons attach to it with seriesPart (1, 2, 3...).
- Series with no sermons is hidden from filters.

Mistakes to avoid

- Creating a series per sermon. A series is a run of 2 or more.

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

Topic

table · extends **BaseEntity** · module Content (CMS) · table `topic`

A label that can be stuck on many things. The reusable "category dictionary". Resource topics, sermon themes and blog categories are all Topics, so a topic typed once can be reused forever.

Field	Type	Flags
`slug`	text	unique, required
`taxonomy`	TopicTaxonomy	required
`parentId`	uuid	FK Topic
`sortOrder`	int	required
`isActive`	bool	required
`name`	text	translated
`description`	text	translated

Rules

- taxonomy separates theme / resource_topic / blog_category so lists stay tidy.
- Editors pick from existing topics first; creating a new one is one click but needs a Swahili name before publish.

Mistakes to avoid

- Typing categories as free text on every upload ("Prayer", "prayer", "Prayers"). That is why you wanted a topic table, and you were right.

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

Sermon

table · extends **Publishable** · module Content (CMS) · table `sermon`

One recorded message with everything people want to know about it. The heart of the website: video (YouTube), audio (R2), summary, verses, key points, highlights, views, likes, comments.

Field	Type	Flags
`preachedOn`	date	required indexed
`ministerId`	uuid	required FK Minister
`seriesId`	uuid	FK Series
`seriesPart`	smallint	
`youtubeVideoId`	text	
`audioMediaId`	uuid	FK MediaAsset
`thumbnailMediaId`	uuid	FK MediaAsset
`notesMediaId`	uuid	FK MediaAsset
`durationSec`	int	
`scriptureRefs`	text[]	
`allowComments`	bool	required
`viewCount`	int	required cached
`likeCount`	int	required cached
`commentCount`	int	required cached
`title`	text	translated
`summary`	text	translated
`keyPoints`	jsonb	translated
`highlights`	jsonb	translated

Does: `publish(): void`, `registerView(deviceId): void`, `toggleLike(deviceId): void`, `nextInSeries(): Sermon`, `localized(locale): SermonView`

Rules

- A sermon needs a YouTube id OR an audio file OR both; never neither (CHECK).
- preachedOn is the sermon date; publishedAt is when it appeared on the site.
- branchId = branch it was uploaded from, shown on the card.
- viewCount/likeCount are CACHES of Redis counters flushed every few minutes by QStash; the truth for likes is the Like table.
- Filters: series, month, theme (Topic), time, minister, branch.

Mistakes to avoid

- Letting the browser bump viewCount directly in Postgres on every play. Count in Redis, flush in batches.
- Embedding the YouTube player on page load. Show a thumbnail and load the iframe on click (faster on mobile data).

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

SermonTopic

link table · module Content (CMS) · table `sermon_topic`

The string tying a sermon to its topics. Many-to-many link: a sermon can have several themes and a theme many sermons.

Field	Type	Flags
`sermonId`	uuid	PK · required · FK Sermon · computed
`topicId`	uuid	PK · required · FK Topic · computed

Rules

- Pure link table: (sermonId, topicId) is the primary key, so the same topic cannot be attached twice.
- Deleting a Sermon removes its links (cascade); deleting a Topic that is in use is refused.

Mistakes to avoid

- Using a comma-separated "topics" text column. You cannot filter or count it.

Who may: Visitor R · Super CRUD · Branch CRUD · Editor CRUD

BlogPost

table · extends **Publishable** · module Content (CMS) · table `blog_post`

A written article. Written articles. Blogs are written in ONE language first; a Translation row adds the other.

Field	Type	Flags
`coverMediaId`	uuid	FK MediaAsset
`authorName`	text	required
`authorMinisterId`	uuid	FK Minister
`categoryId`	uuid	FK Topic
`readingMinutes`	smallint	
`allowComments`	bool	required
`viewCount`	int	required · cached
`likeCount`	int	required · cached
`commentCount`	int	required · cached
`title`	text	translated
`excerpt`	text	translated
`body`	text	translated

Does: `publish(): void`, `estimateReadingTime(): int`

Rules

- body is rich text (Tiptap JSON or MDX) stored in the translation row.
- Reading time is computed on save, not on every view.
- Translation is per field (title, excerpt, body) in blog_post_translation; the English row is the source of truth for slug.
- body is stored as Tiptap JSON, rendered to HTML on the server and sanitised.
- allowComments can be switched off per post.

Mistakes to avoid

- Pasting raw HTML from Word. It brings hidden styles and scripts; use the editor and sanitise.

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

Comment

table · extends **BaseEntity** · module Content (CMS) · table `comment`

A message left under a sermon. A visitor's comment under a sermon or blog. Starts as "pending" until a moderator approves.

Field	Type	Flags
`sermonId`	uuid	FK Sermon, computed
`blogPostId`	uuid	FK BlogPost, computed
`parentId`	uuid	FK Comment
`authorName`	text	required
`authorEmail`	text	private
`body`	text	required
`status`	CommentStatus	required
`ipHash`	text	private
`locale`	Locale	
`moderatedBy`	text	FK User
`moderatedAt`	timestamptz	

Does: `approve(): void`, `reject(): void`, `markSpam(): void`

Rules

- Exactly one of sermonId / blogPostId is set (CHECK).
- Honeypot field + Turnstile + Upstash rate limit before the row is created.
- Emails are never shown publicly.
- Replies one level deep (parentId) to keep the page readable.

Mistakes to avoid

- Publishing comments instantly. The first spam wave will land on a church page.

Who may: Visitor S, New member S, Super RUDA, Branch RUDA, Editor RUDA

Like

table · module Content (CMS) · table `like`

A tick from one phone. One thumbs-up per device per item. No login needed.

Field	Type	Flags
`id`	uuid	PK
`sermonId`	uuid	FK Sermon, computed
`blogPostId`	uuid	FK BlogPost, computed
`deviceHash`	text	required, private
`createdAt`	timestamptz	required

Does: `toggle(deviceId): bool`

Rules

- Unique (sermonId, deviceHash) and (blogPostId, deviceHash).
- deviceHash = HMAC of a random browser id; never the IP.
- Fast path is a Redis SET; the table is the durable truth.

Mistakes to avoid

- Counting likes with a plain number. People refresh and like 40 times.

Who may: Visitor S, New member S, Super R

Bulletin

table · extends **Publishable** · module Content (CMS) · table `bulletin`

This week's church newspaper. The Sunday bulletin PDF. All years, all months; read in the browser or download.

Field	Type	Flags
`weekOf`	date	required indexed
`year`	smallint	required indexed
`month`	smallint	required
`pdfMediaId`	uuid	required FK MediaAsset
`coverMediaId`	uuid	FK MediaAsset
`pageCount`	smallint	
`downloadCount`	int	required cached
`readCount`	int	required cached
`extractedText`	text	
`title`	text	translated

Does: `download(): Url`, `readInline(): Url`

Rules

- Unique (branchId, weekOf).
- year and month are derived from weekOf and stored for fast filtering.
- A QStash job renders page 1 as a cover image and extracts text for search.

Mistakes to avoid

- Uploading every bulletin as a differently named file. Name = date, and let the system derive the rest.

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

Announcement

table · extends **Publishable** · module Content (CMS) · table `announcement`

A notice read out in church or sent during the week. Two kinds in one table: SUNDAY announcements (audio + readable + video, one per Sunday) and MIDWEEK ones (Friday youth fellowship, men's retreat…).

Field	Type	Flags
`kind`	AnnouncementKind	required
`serviceDate`	date	
`audioMediaId`	uuid	FK MediaAsset
`youtubeVideoId`	text	
`pdfMediaId`	uuid	FK MediaAsset
`eventAt`	timestamptz	
`venue`	text	
`departmentId`	uuid	FK Department
`expiresAt`	timestamptz	
`title`	text	translated
`body`	text	translated

Does: `isExpired(now): bool`

Rules

- kind = sunday requires serviceDate; kind = midweek requires eventAt (CHECK).
- One sunday announcement per (branch, serviceDate).
- Midweek announcements auto-hide after expiresAt.
- Department leaders may draft midweek announcements; an Editor approves.

Mistakes to avoid

- Two separate tables for sunday and midweek. They share 90% of their columns; use one table with a kind column (single-table inheritance).

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader CU · Discipler R · Treasurer R · Events R

Resource

table · extends **Publishable** · module Content (CMS) · table `resource`

A downloadable or watchable learning item. Discipleship material, Bible study notes, mission-week PDFs, weekly prayer points, registration forms: one table, filtered by category, topic, year and format.

Field	Type	Flags
`category`	ResourceCategory	required indexed
`format`	ResourceFormat	required
`mediaId`	uuid	FK MediaAsset
`youtubeVideoId`	text	
`externalUrl`	text	
`formId`	uuid	FK FormDefinition
`topicId`	uuid	FK Topic
`year`	smallint	required indexed
`month`	smallint	
`weekOf`	date	
`downloadCount`	int	required cached
`title`	text	translated
`description`	text	translated

Does: `download(): Url`

Rules

- Main topic = Topic row; year/month for the filters you described.
- format = form means formId is set and the page renders the registration form inline.

Mistakes to avoid

- One table per resource page (discipleship, bible_study, other…). Add a category value instead.

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

PageBlock

table · extends **BaseEntity** · module Content (CMS) · table `page_block`

A piece of the home page you can edit. Editable landing-page sections (hero, about blurb, short history) so staff can change words without a developer.

Field	Type	Flags
`page`	text	required
`key`	text	required
`type`	BlockType	required
`mediaId`	uuid	FK MediaAsset
`sortOrder`	int	required
`isActive`	bool	required
`data`	jsonb	translated

Rules

- Unique (page, key).
- Only fixed block types are allowed; this is a mini-CMS, not a page builder.
- A block = (page, key) such as home/hero_title; the code asks for the key, the CMS decides the words.
- If a block is missing the code falls back to the message file so the page never shows an empty hole.

Mistakes to avoid

- Making every sentence on the site editable. Editable text = more admin work and more typos. Make only what changes.

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

TimelineEntry

table · extends **BaseEntity** · module Content (CMS) · table `timeline_entry`

One dot on the church history line. The church history timeline used on About and in the new-member tour.

Field	Type	Flags
`year`	smallint	required
`mediaId`	uuid	FK MediaAsset
`sortOrder`	int	required
`title`	text	translated
`body`	text	translated

Rules

- Used by "short history" on Home and the full timeline on About.
- year + sortOrder decide the order; text comes from timeline_entry_translation.

Mistakes to avoid

- Storing the year inside the translated text; you cannot sort by it.

Who may: Visitor R · New member R · Super CRUDPA · Branch CRUDPA · Editor CRUDP · Translator RU · Dept leader R · Discipler R · Treasurer R · Events R

PART 11 · MODULE

Events & Programmes (7 classes)

Event

table · extends **Publishable** · module Events & Programmes · table `event`

A date on the church calendar. Anything with a date: youth conference, marriage conference, worship conference, youth trip, mission week.

Field	Type	Flags
`category`	EventCategory	required indexed
`startsAt`	timestamptz	required indexed
`endsAt`	timestamptz	
`venueName`	text	
`venueAddress`	text	
`lat`	numeric(9,6)	
`lng`	numeric(9,6)	
`departmentId`	uuid	FK Department
`capacity`	int	
`registrationMode`	RegMode	required
`registrationOpensAt`	timestamptz	
`registrationClosesAt`	timestamptz	
`registrationUrl`	text	
`formId`	uuid	FK FormDefinition
`requiresGuardianConsent`	bool	required
`coverMediaId`	uuid	FK MediaAsset
`isFeatured`	bool	required
`cancelledAt`	timestamptz	
`cancelReason`	text	
`title`	text	translated
`description`	text	translated
`feeInfo`	text	translated

Does: `isRegistrationOpen(now): bool`, `seatsLeft(): int`, `addGuest(minister, role): void`, `cancel(reason): void`

Rules

- registrationMode: none | form (our form) | external (link) | walk_in.
- seatsLeft = capacity - confirmed attendees (computed, not stored).
- Guest ministers come from EventGuest, so the same Minister page works for sermons and events.
- Fees are INFORMATION only (giving details); no payment at launch.

Mistakes to avoid

- A separate table for every kind of event. category + optional subtype (OutreachActivity) is enough.

Who may: Visitor R · New member R · Super CRUDP · Branch CRUDP · Editor RU · Translator RU · Dept leader CU · Events CRUDP

EventGuest

table · extends **BaseEntity** · module Events & Programmes · table `event_guest`

Speaker list for an event. Which ministers speak at which event.

Field	Type	Flags
`eventId`	uuid	required, FK Event, computed
`ministerId`	uuid	required, FK Minister
`role`	GuestRole	required
`sortOrder`	int	required

Rules

- A guest minister attached to an event; may be an internal Minister or an external name + photo.
- Shown on the event card as "Guest: Pastor ...".

Mistakes to avoid

- Making guest ministers free text on Event: you lose "all events with this minister".

Who may: Visitor R, Super CRUD, Branch CRUD, Events CRUD

EventSession

table · extends **BaseEntity** · module Events & Programmes · table `event_session`

One slot in the conference timetable. The day-by-day programme of a multi-day conference.

Field	Type	Flags
`eventId`	uuid	required, FK Event, computed
`startsAt`	timestamptz	required
`endsAt`	timestamptz	
`speakerMinisterId`	uuid	FK Minister
`room`	text	
`sortOrder`	int	required
`title`	text	translated

Rules

- One event can have many sessions (Day 1 morning, Day 1 evening).
- startsAt/endsAt are timestamptz; the page renders them in Africa/Nairobi.

Mistakes to avoid

- Storing local time without timezone; a phone abroad will show the wrong hour.

Who may: Visitor R, Super CRUD, Branch CRUD, Events CRUD

EventRegistration

table · extends **PersonRequest** · module Events & Programmes · table `event_registration`

A seat reserved in someone's name. Someone registering for an event. A "PersonRequest" with an event attached.

Field	Type	Flags
`eventId`	uuid	required, FK Event, computed
`ticketCode`	text	unique, required
`attendeesCount`	smallint	required
`guardianName`	text	private
`guardianPhone`	text	private
`checkedInAt`	timestamptz	

Does: `checkIn(): void`, `cancel(): void`

Rules

- Unique (eventId, phone) stops duplicate sign-ups.
- If event.requiresGuardianConsent and ageGroup is under 18, guardian fields are required.
- Confirmation by email (Resend) and optional SMS; ticketCode is shown and emailed.

Mistakes to avoid

- Letting minors register without a guardian contact.

Who may: Visitor S · New member S · Super RUX · Branch RUX · Dept leader R · Events RUDX

Programme

table · extends **Publishable** · module Events & Programmes · table `programme`

The recipe. A cohort is one time you cook it. A repeating course: Children 5 Days, CFC, Kairos, Teen 5 Days, New Born Again. Says WHAT it is and HOW OFTEN.

Field	Type	Flags
`audience`	Audience	required
`minAge`	smallint	
`maxAge`	smallint	
`sessionsPerCohort`	smallint	
`format`	DeliveryFormat	required
`departmentId`	uuid	FK Department
`formId`	uuid	FK FormDefinition
`requiresGuardianConsent`	bool	required
`iconMediaId`	uuid	FK MediaAsset
`coverMediaId`	uuid	FK MediaAsset
`name`	text	translated
`description`	text	translated
`scheduleSummary`	text	translated
`durationLabel`	text	translated
`feeInfo`	text	translated

Does: `nextCohort(branch): ProgrammeCohort`, `openCohorts(): ProgrammeCohort[]`

Rules

- A programme has many cohorts over the years.
- A programme is the course (Kairos); a ProgrammeCohort is one run of it (Kairos Jan-Mar 2027).
- Frequency text ("every Tuesday 6pm") is translatable.

Mistakes to avoid

- Mixing the course and the run. "Kairos 2027 Term 1" is a cohort, not a programme.

Who may: Visitor R · New member R · Super CRUDP · Branch CRUDP · Dept leader RU · Discipler R · Events CRUDP

ProgrammeCohort

table · extends **BaseEntity** · module Events & Programmes · table `programme_cohort`

One class intake. One run of a programme at one branch: "CFC - Eastern Bypass - Oct 2026". Enrolment attaches to the cohort, not the programme.

Field	Type	Flags
`programmeId`	uuid	required · FK Programme · computed
`branchId`	uuid	required · FK Branch
`label`	text	required
`startsOn`	date	required · indexed
`endsOn`	date	
`meetingDay`	smallint	
`meetingTime`	time	
`venue`	text	
`facilitatorLeaderId`	uuid	FK Leader
`enrollmentOpensAt`	timestamptz	
`enrollmentClosesAt`	timestamptz	
`capacity`	int	
`status`	CohortStatus	required
`sessionsPlanned`	smallint	

Does: `isOpen(now): bool`, `seatsLeft(): int`, `close(): void`

Rules

- Visitors enrol in the NEXT OPEN cohort; the programme page shows it automatically.
- status flow: planned → open → full/running → completed (or cancelled).

Who may: Visitor R · New member R · Super CRUD · Branch CRUD · Discipler RU · Events CRUD

ProgrammeEnrollment

table · extends **PersonRequest** · module Events & Programmes · table `programme_enrollment`

A name on the class register. A person joining a cohort. The same class serves the web form, the Go Ye hand-off and walk-ins.

Field	Type	Flags
`cohortId`	uuid	required · FK ProgrammeCohort · computed
`origin`	EnrollOrigin	required
`convertId`	uuid	FK Convert
`guardianName`	text	private
`guardianPhone`	text	private
`completedAt`	timestamptz	
`attendanceCount`	smallint	

Does: `complete(): void`, `withdraw(): void`

Rules

- origin = go_ye or new_member links back to the Convert or journey that caused it.
- Under-18 programmes require guardian fields.
- One person per cohort (unique on cohortId + contact).
- Status flow: requested → confirmed → attending → completed | dropped.
- No certificate column here: a Certificate points back to the enrollment, not the other way round (breaks a circular dependency).

Mistakes to avoid

- Letting the same phone number enrol twice: it inflates numbers and wastes a place.

Who may: Visitor S · New member S · Super RUX · Branch RUX · Discipler CRU · Events RUDX

PART 11 · MODULE

Go Ye & Discipleship (5 classes)

OutreachActivity

table · extends **Event** · module Go Ye & Discipleship · table `outreach_activity`

An event with a mission and a result. An Event that is also a Go Ye outreach (school, unreached community, international mission, mission week). Inherits everything from Event and adds results.

Field	Type	Flags
`eventId`	uuid	PK · required · FK Event · computed
`outreachCategory`	OutreachCategory	required · indexed
`country`	text	
`locationName`	text	
`targetGroup`	text	
`outcome`	OutreachOutcome	required
`decisionsReported`	int	
`teamSize`	int	
`reportSummary`	text	translated

Does: `complete(report): void`

Rules

- Shares its primary key with Event (class-table inheritance), so it appears on the main calendar for free.
- decisionsReported is what the team says; the counter on the website uses Convert records only.

Who may: Visitor R · Super CRUDP · Branch CRUDP · Dept leader CRU · Events CRUDP

OutreachDepartment

link table · module Go Ye & Discipleship · table `outreach_department`

Which teams helped. Which departments take part in an outreach.

Field	Type	Flags
`outreachId`	uuid	PK · required · FK OutreachActivity · computed
`departmentId`	uuid	PK · required · FK Department · computed
`role`	text	

Rules

- Link table: which departments take part in an outreach (Youth, Missions, Children...).
- Used to build the "Departments involved" list on the Go Ye page.

Mistakes to avoid

- Duplicating department names as text.

Who may: Super CRUD · Branch CRUD · Events CRUD

Convert

table · extends **PersonRequest** · module Go Ye & Discipleship · table `convert`

A person who just said yes to Jesus and wants someone to call. A decision to follow Christ, recorded with a name and phone so the discipleship team can reach out. This is the data behind the "born again" counter.

Field	Type	Flags
`decisionAt`	timestamptz	required
`origin`	ConvertOrigin	required
`outreachId`	uuid	FK OutreachActivity
`capturedBy`	text	FK User
`wantsCall`	bool	required
`preferredLanguage`	Locale	
`bestTime`	text	
`isCounted`	bool	required
`dedupeKey`	text	unique private
`fullName`	text	required sensitive
`phone`	text	sensitive

Does: `openFollowUp(): FollowUpCase`, `markDuplicate(): void`

Rules

- Belief is SENSITIVE personal data in Kenya: explicit consent text must be shown and stored (consentAt + consentVersion).
- dedupeKey = hash(normalised phone) stops double counting.
- The public counter = count(isCounted) + SiteSetting baseline.
- Visible only to Discipler(s) of that branch, Branch Admin and Super Admin.
- A FollowUpCase is opened automatically.

Mistakes to avoid

- Counting every "I am already born again" tick from the new-member tour. That is a profile fact, not a new decision.
- Showing the converts list to every editor. Least privilege: only disciplers.

Who may: Visitor S · New member S · Super RX · Branch RX · Discipler RUX

FollowUpCase

table · extends **BaseEntity** · module Go Ye & Discipleship · table `follow_up_case`

The folder a discipler keeps for each new believer. The care file for one convert: who is responsible, how many attempts, where they are on the journey.

Field	Type	Flags
`convertId`	uuid	unique, required, FK Convert, computed
`branchId`	uuid	required, FK Branch
`assignedTo`	text	FK User
`status`	FollowUpStatus	required, indexed
`priority`	smallint	required
`dueAt`	timestamptz	
`lastContactAt`	timestamptz	
`attempts`	smallint	required
`enrolledCohortId`	uuid	FK ProgrammeCohort
`closedAt`	timestamptz	
`closeReason`	text	

Does: `assign(user): void`, `logAttempt(a): void`, `enrol(cohort): void`, `close(reason): void`

Rules

- Target: first contact within 48 hours (dueAt); QStash reminds the discipler and escalates to the Branch Admin if overdue.
- 1:1 with Convert.
- Closing needs a reason (enrolled, completed, declined, unreachable).
- Created automatically when a Convert row is created; assigned to a Discipler of that branch.
- Status: new → contacting → connected → in_programme -> closed. A case can reopen.
- Reminder jobs run through QStash (day 1, day 3, day 7).

Who may: Super RU, Branch RUD, Discipler CRUD

Mistakes to avoid

- Assigning one Discipler to everything: when they are away, nobody calls the new believer.

ContactAttempt

table · module Go Ye & Discipleship · table `contact_attempt`

A line in the care diary. Each call, SMS or visit, with the result.

Field	Type	Flags
`id`	uuid	PK
`caseId`	uuid	required, FK FollowUpCase, computed
`byUserId`	text	required, FK User
`channel`	ContactChannel	required
`outcome`	ContactOutcome	required
`notes`	text	private
`at`	timestamptz	required

Rules

- Notes are private to disciplers and above; never exported by default.
- Append-only log: channel (call, SMS, WhatsApp, visit), outcome, note, who, when.
- Only the assigned Discipler, Branch Admin and Super Admin can read notes.

Mistakes to avoid

- Editing past notes. A log you can rewrite is not a log.

Who may: Super R · Branch R · Discipler CRU

PART 11 · MODULE

Giving & Projects (6 classes)

Project

table · extends **Publishable** · module Giving & Projects · table `project`

A big thing the church is building, with a thermometer. A church building or mission project with a goal, milestones, impact story and a progress bar.

Field	Type	Flags
`goalAmountMinor`	bigint	required
`currency`	text	required
`raisedAmountMinor`	bigint	required cached
`status`	ProjectStatus	required
`startedOn`	date	
`targetCompletionOn`	date	
`completedOn`	date	
`coverMediaId`	uuid	FK MediaAsset
`isFeatured`	bool	required
`sortOrder`	int	required
`title`	text	translated
`summary`	text	translated
`body`	text	translated
`impact`	text	translated

Does: `progressPercent(): number`, `remainingMinor(): bigint`, `recompute(): void`, `lastUpdatedAt(): Date`

Rules

- Money is stored in the smallest unit (cents) as bigint: KES 1,500.00 = 150000. Never floats.
- raisedAmountMinor is a CACHE = SUM(ledger where status = recorded); recompute() runs inside the same transaction as every ledger write.
- Percent is capped at 100 for display; "remaining" never goes below 0.
- "Last updated" = newest of ProjectUpdate.postedAt and LedgerEntry.recordedAt.

Mistakes to avoid

- Using decimals/floats for money. 0.1 + 0.2 is not 0.3 in a computer.
- Letting editors change goalAmount silently. It is audited and shown in the update log.

Who may: Visitor R · New member R · Super CRUDP · Branch CRUDP · Editor RU · Translator RU · Treasurer R

ProjectMilestone

table · extends **BaseEntity** · module Giving & Projects · table `project_milestone`

A step on the staircase. Stages of a project (land bought, foundation, roof…). Completed ones are ticked on the project page.

Field	Type	Flags
`projectId`	uuid	required · FK Project · computed
`targetAmountMinor`	bigint	
`status`	MilestoneStatus	required
`achievedOn`	date	
`mediaId`	uuid	FK MediaAsset
`sortOrder`	int	required
`title`	text	translated
`description`	text	translated

Rules

- Ordered steps with a target amount or date; "done" is set by Branch Admin.
- Percent complete = done milestones / all, shown beside the money bar.

Mistakes to avoid

- Calculating "percent complete" from money only. A project can be 80% funded and 10% built.

Who may: Visitor R · Super CRUD · Branch CRUD · Editor RU · Treasurer R

ProjectUpdate

table · extends **BaseEntity** · module Giving & Projects · table `project_update`

A progress note with a date. The "latest news" log at the bottom of a project page; also gives the "last updated" date.

Field	Type	Flags
`projectId`	uuid	required · FK Project · computed
`postedAt`	timestamptz	required
`postedBy`	text	FK User
`percentComplete`	smallint	
`mediaId`	uuid	FK MediaAsset
`title`	text	translated
`body`	text	translated

Rules

- A dated post ("Roofing completed") with optional photos; its createdAt feeds the "last updated" label.
- Public. Written by Branch Admin or Content Editor.

Mistakes to avoid

- Forgetting to update: a stale "last updated" date kills donor trust faster than no page at all.

Who may: Visitor R · Super CRUD · Branch CRUD · Editor CRU · Treasurer CR

ProjectMedia

link table · module Giving & Projects · table `project_media`

Photos for the project page. Photo gallery of a project.

Field	Type	Flags
`projectId`	uuid	PK · required · FK Project · computed
`mediaId`	uuid	PK · required · FK MediaAsset · computed
`sortOrder`	int	required

Rules

- Link table between Project and MediaAsset with sortOrder and caption (translatable).

Mistakes to avoid

- Uploading full 6MB camera photos. Resize on upload (StorageService).

Who may: Visitor R · Super CRUD · Branch CRUD · Editor CRUD

LedgerEntry

table · extends **BaseEntity** · module Giving & Projects · table `ledger_entry`

A line in the church's cash book. One gift recorded by the treasurer. The ONLY source of truth for the progress bars.

Field	Type	Flags
`projectId`	uuid	FK Project
`fund`	FundKind	required
`amountMinor`	bigint	required
`currency`	text	required
`receivedOn`	date	required indexed
`method`	LedgerMethod	required
`reference`	text	
`donorLabel`	text	private
`branchId`	uuid	required FK Branch
`recordedBy`	text	required FK User
`status`	LedgerStatus	required
`voidedBy`	text	FK User
`voidedAt`	timestamptz	
`voidReason`	text	
`source`	LedgerSource	required
`batchId`	uuid	

Does: `void(reason): void`

Rules

- Append-only: mistakes are VOIDED with a reason, never edited or deleted.
- amountMinor > 0 (CHECK).
- donorLabel defaults to "Anonymous"; the church sees only what it needs.
- source = manual today; "gateway" is the reserved slot for M-Pesa/Paystack later, so no schema change is needed.
- Every insert and void writes AuditLog and recomputes Project.raisedAmountMinor.

Mistakes to avoid

- Editing the total on the project instead of adding a ledger row. Then nobody can explain the number.

Who may: Super RX · Branch R · Treasurer CRUX

GivingChannel

table · extends **BaseEntity** · module Giving & Projects · table `giving_channel`

The church's payment addresses. The "how to give" details shown on the contact page and on each project: paybill, till, bank, international.

Field	Type	Flags
`kind`	ChannelKind	required
`accountName`	text	
`accountNumber`	text	
`shortcode`	text	
`bankName`	text	
`swiftCode`	text	
`purpose`	GivingPurpose	required
`branchId`	uuid	FK Branch
`projectId`	uuid	FK Project
`displayOrder`	int	required
`isActive`	bool	required
`label`	text	translated
`referenceInstructions`	text	translated

Rules

- Changing a paybill number or bank account is a high-risk edit: two-person approval (Treasurer proposes, Branch Admin confirms) and an email alert.
- projectId set = shown on that project page too.

Mistakes to avoid

- Letting any editor change account numbers. This is the #1 church-website fraud target.

Who may: Visitor R · New member R · Super CRUD · Branch RA · Treasurer CRU

PART 11 · MODULE

New Member & Forms (7 classes)

FormDefinition

table · extends **BaseEntity** · module New Member & Forms · table `form_definition`

A blank form that staff can design. A small form builder. Event registrations, programme enrolment, department follow-up questions and mission applications all use it.

Field	Type	Flags
`key`	text	unique, required
`version`	int	required
`status`	ContentStatus	required
`title`	text	translated
`intro`	text	translated
`consentText`	text	translated

Does: `toZodSchema(): ZodObject`, `render(locale): FormUI`

Rules

- A published form is immutable: edits create version + 1, old submissions keep their version.
- Fields support conditional visibility (showIf).
- Describes a configurable form (new-member tour questions, programme enrolment questions).
- Version number increases on every publish; old answers keep the version they were collected under.

Mistakes to avoid

- Editing a published form in place: old answers no longer match the questions. Always publish a new version.

Who may: Super CRUDP · Branch R · Editor RU · Dept leader RU · Events RU

FormField

table · extends **BaseEntity** · module New Member & Forms · table `form_field`

One question on the form. One question on a form. Labels and options are small JSON objects with an English and a Kiswahili text.

Field	Type	Flags
`formId`	uuid	required · FK FormDefinition · computed
`key`	text	required
`type`	FieldType	required
`label`	jsonb	required
`helpText`	jsonb	
`options`	jsonb	
`required`	bool	required
`showIf`	jsonb	
`validation`	jsonb	
`sortOrder`	int	required

Rules

- Field types: text, textarea, select, radio, checkbox, date, phone, email, consent.
- Labels and help texts are JSONB per locale: {"en": "...", "sw": "..."}.
- Conditional rule "showIf" is stored as a small JSON expression, evaluated by one shared function on server and client.

Mistakes to avoid

- Storing the label as the answer key. Use the field id, so renaming a label does not orphan answers.

Who may: Super CRUD · Branch R · Dept leader RU · Events RU

NewMemberJourney

table · extends **BaseEntity** · module New Member & Forms · table `new_member_journey`

A bookmark that remembers where the newcomer is in the tour. The server-side memory of one person's tour, created ONLY after they say Yes and give consent. "No" is remembered on their device only.

Field	Type	Flags
`deviceId`	text	unique, required, private
`locale`	Locale	required
`branchId`	uuid	FK Branch
`status`	JourneyStatus	required
`currentStep`	smallint	required
`startedAt`	timestamptz	required
`completedAt`	timestamptz	
`tourVersion`	text	required
`consentVersion`	text	

Does: `advance(step): void`, `complete(): Certificate`, `resumeLink(): Url`

Rules

- No server record for "No, thanks": it lives in localStorage (+ cookie if cookies accepted).
- deviceId is a random id from the browser, stored hashed; progress survives a refresh.
- Abandoned journeys older than 90 days are anonymised.
- Created ONLY after the visitor says Yes and accepts consent. "No" never reaches the server.
- Stores progress as currentStep + completedSteps so a refresh resumes where they left.
- Anonymous token in an httpOnly cookie links the browser to the journey until contact details are given.

Mistakes to avoid

- Saving data before consent. A "No" must leave no trace on the server.
- Asking all nine steps at once on one long page: people leave. One short step at a time.

Who may: New member CRU · Super R · Discipler R

NewMemberProfile

table · extends **BaseEntity** · module New Member & Forms · table `new_member_profile`

The newcomer's "about me" card. What the newcomer tells us about themselves in the tour.

Field	Type	Flags
`journeyId`	uuid	unique · required · FK NewMemberJourney · computed
`firstName`	text	required · private
`lastName`	text	required · private
`gender`	Gender	
`ageGroup`	AgeGroup	required
`branchId`	uuid	required · FK Branch
`phone`	text	private
`email`	text	private
`bornAgain`	BornAgainStatus	required · sensitive
`bornAgainSinceYear`	smallint	sensitive
`wantsToBeBornAgain`	bool	sensitive
`previousChurch`	text	
`wasServing`	bool	required
`previousDepartmentId`	uuid	FK Department
`previousDepartmentOther`	text	
`consentAt`	timestamptz	required
`consentVersion`	text	required

Rules

- born-again fields are sensitive belief data: collected only with explicit consent and shown only to Disciplers.
- If wantsToBeBornAgain is true, a Convert + FollowUpCase are created in the same transaction.

Mistakes to avoid

- Making every field mandatory. Only name, age group and branch are required; everything else is optional.

Who may: New member CRU · Super RX · Branch R · Discipler R

FeedbackResponse

table · extends **BaseEntity** · module New Member & Forms · table `feedback_response`

The suggestion box. The end-of-tour questionnaire: what they experienced, what to improve, would they invite a friend and why DCEB.

Field	Type	Flags
`journeyId`	uuid	FK NewMemberJourney
`branchId`	uuid	FK Branch
`rating`	smallint	
`experience`	text	
`improvements`	text	
`wouldInvite`	bool	
`whyChose`	text	
`locale`	Locale	

Rules

- One per completed journey; answers in JSONB keyed by FormField id.
- Read by Super Admin, Branch Admin and Content Editor for improvement only.

Mistakes to avoid

- Showing feedback with names to the whole team. Show it grouped, names only to the Branch Admin.

Who may: New member C · Super RX · Branch R · Editor R

Certificate

table · extends **BaseEntity** · module New Member & Forms · table `certificate`

A printable "well done" paper with a serial number. The "Completion certificate" generated when the tour ends (later also programme completion).

Field	Type	Flags
`serial`	text	unique, required
`kind`	CertificateKind	required
`recipientName`	text	required, private
`journeyId`	uuid	FK NewMemberJourney
`enrollmentId`	uuid	FK ProgrammeEnrollment
`issuedAt`	timestamptz	required
`pdfMediaId`	uuid	FK MediaAsset
`verifyCode`	text	unique, required
`locale`	Locale	required
`revokedAt`	timestamptz	
`signedByLeaderId`	uuid	FK Leader

Does: `render(): Pdf`, `verify(code): bool`

Rules

- serial like DCEB-2026-000123 (year and a running number); verifyCode lets anyone check authenticity at /verify/CODE.
- PDF is stored PRIVATE in R2 and downloaded with a short-lived signed URL.
- Rendered once by a QStash job, then cached.
- Generated once from a PDF template, stored in a PRIVATE R2 bucket and served by a short-lived signed URL.
- serial number is unique and printed on the PDF so it can be verified.
- Name on the certificate is exactly what the member typed (they confirm spelling before the PDF is made).

Mistakes to avoid

- Storing certificates in a public bucket with a guessable URL.
- Rendering the PDF in the browser only: a phone may fail; build it on the server.

Who may: New member CR · Super RUD · Branch R

JoinRequest

table · extends **PersonRequest** · module New Member & Forms · table `join_request`

A hand raised: "count me in". "I want to join the Youth fellowship / this home fellowship / the missions team." One class for every kind of join.

Field	Type	Flags
`kind`	JoinKind	required
`departmentId`	uuid	FK Department
`homeFellowshipId`	uuid	FK HomeFellowship
`journeyId`	uuid	FK NewMemberJourney
`origin`	JoinOrigin	required
`gender`	Gender	
`previouslyServed`	bool	

Does: `routeToLeader(): void`

Rules

- kind = department | home_fellowship | mission_team | fellowship_group; exactly one target column is set (CHECK).
- Notifies the matching Department Leader or fellowship contact by email/SMS.
- Men / Women / Youth suggestions use gender + ageGroup from the tour.
- Types: department, fellowship, home_fellowship, mission_team.
- Routed to the Department Leader of that department, or the Branch Admin when no leader is set.

Mistakes to avoid

- Sending every request to the Super Admin. Route to the right leader or nobody will answer.

Who may: Visitor S · New member S · Super RUX · Branch RUX · Dept leader RUX · Discipler R

PART 11 · MODULE

Engagement (5 classes)

NewsletterSubscriber

table · extends **BaseEntity** · module Engagement · table `newsletter_subscriber`

A name on the mailing list. Someone who gave their email in the popup or footer.

Field	Type	Flags
`email`	text	unique required private
`name`	text	private
`locale`	Locale	required
`branchId`	uuid	FK Branch
`status`	SubscriberStatus	required
`source`	SubSource	required
`confirmTokenHash`	text	private
`confirmedAt`	timestamptz	
`consentAt`	timestamptz	required
`consentVersion`	text	required
`unsubscribedAt`	timestamptz	
`resendContactId`	text	
`topics`	text[]	

Does: `confirm(token): void`, `unsubscribe(): void`

Rules

- Double opt-in: status stays pending until they click the email link.
- Every email carries a one-click unsubscribe; bounces and complaints from Resend update status via webhook.
- Never email minors flagged in a form.
- Double opt-in: status pending until the link in the confirmation e-mail is clicked.
- Every e-mail carries a one-click unsubscribe link (token).
- Subscribers are per branch or "all branches".

Mistakes to avoid

- Adding people from the contact form to the newsletter without asking. It is illegal under the Kenya Data Protection Act consent rule and gets your domain blocked.

Who may: Visitor S · New member S · Super RUDX · Branch R · Editor RUDX

NewsletterCampaign

table · extends **BaseEntity** · module Engagement · table `newsletter_campaign`

One newsletter issue. One email sent to the list (weekly digest or special message).

Field	Type	Flags
`kind`	CampaignKind	required
`status`	CampaignStatus	required
`audienceLocale`	Locale	
`scheduledAt`	timestamptz	
`sentAt`	timestamptz	
`recipientCount`	int	
`resendBroadcastId`	text	
`createdBy`	text	FK User
`stats`	jsonb	
`subject`	text	translated
`body`	text	translated

Rules

- A campaign is a draft → scheduled → sending → sent record; sending is done in batches by QStash so Resend limits are respected.
- Test-send to yourself is mandatory before "schedule".

Mistakes to avoid

- Sending 2000 emails in one loop inside a request: it times out half way. Use batches.

Who may: Super CRUDP · Editor CRUDP

ContactMessage

table · extends **BaseEntity** · module Engagement · table `contact_message`

A letter in the church mailbox. The contact form submissions.

Field	Type	Flags
`name`	text	required private
`email`	text	private
`phone`	text	private
`topic`	ContactTopic	required
`branchId`	uuid	FK Branch
`message`	text	required
`status`	MessageStatus	required
`assignedTo`	text	FK User
`ipHash`	text	private
`repliedAt`	timestamptz	
`locale`	Locale	

Rules

- Prayer and counselling topics route to a pastor, giving questions to the Treasurer, "visit us" to the branch admin.
- Public form (name, e-mail/phone, topic, message). Rate-limited + Turnstile.
- Routed to the Branch Admin inbox by chosen branch and topic; a copy goes to e-mail.

Mistakes to avoid

- Only e-mailing the message and not saving it: if the e-mail bounces the message is gone.

Who may: Visitor S · Super RUD · Branch RUD · Discipler R · Treasurer R

ConsentRecord

table · extends **BaseEntity** · module Engagement · table `consent_record`

A receipt that says "yes, you may use cookies". Proof that a visitor accepted (or declined) cookie categories on a given policy version.

Field	Type	Flags
`visitorId`	text	required private indexed
`categories`	jsonb	required
`policyVersion`	text	required
`source`	text	required
`userAgentHash`	text	private

Rules

- Necessary cookies need no consent; analytics (PostHog) only after consent.
- Re-ask when policyVersion changes.
- Stores WHAT was agreed (cookies analytics, newsletter, new-member data), WHEN, which policy version and the anonymised IP hash.
- Withdrawing consent adds a new row; old rows are never edited.

Mistakes to avoid

- Overwriting the old consent row when someone changes their mind. You lose the proof.

Who may: Visitor S · New member S · Super R

NotificationLog

table · extends **BaseEntity** · module Engagement · table `notification_log`

The post-office receipt book. Every email/SMS the system tries to send, with the result. The first place to look when "I never got the email".

Field	Type	Flags
`channel`	NotifyChannel	required
`template`	text	required
`toMasked`	text	
`status`	NotifyStatus	required
`providerMessageId`	text	
`error`	text	
`relatedType`	text	
`relatedId`	text	
`attempts`	smallint	required
`sentAt`	timestamptz	

Rules

- One row per e-mail/SMS attempt: template, to (hashed), provider id, status, error.
- Used to answer "did the member get the certificate e-mail?".

Mistakes to avoid

- Logging the whole e-mail body with personal data. Log template + id only.

Who may: Super R · Branch R

Platform (17 classes)

BaseEntity

abstract parent · module Platform

The "birth certificate" every record carries: who it is (id), when it was born (createdAt) and when it last changed (updatedAt). Every table starts with the same three columns. Writing them once avoids 60 copies and 60 chances for a typo.

Field	Type	Flags
`id`	uuid	PK
`createdAt`	timestamptz	required
`updatedAt`	timestamptz	required

Does: `touch(): void`

Rules

- Every concrete table has id (uuid v7 so rows sort by time), createdAt and updatedAt.
- updatedAt is set by the database trigger/ORM hook, never by the browser.

Mistakes to avoid

- Using auto-increment integers as ids: they leak how many members/converts exist and clash when branches merge data.

Publishable

abstract parent · extends **BaseEntity** · module Platform

Anything with a "Publish" button. One shared set of columns + one shared life-cycle so every content type behaves the same. Anything staff write and the public reads (sermon, blog, bulletin…) shares one life-cycle: draft, review, scheduled, published, archived.

Field	Type	Flags
`slug`	text	unique required
`status`	ContentStatus	required
`publishedAt`	timestamptz	
`scheduledAt`	timestamptz	
`branchId`	uuid	FK Branch
`authorId`	text	FK User
`sourceLocale`	Locale	required
`deletedAt`	timestamptz	

Does: `publish(): void`, `schedule(at): void`, `archive(): void`, `isVisible(now): bool`

Rules

- Life-cycle: draft → in_review → scheduled → published → archived. Only "published" and publishedAt <= now() is public.
- slug is unique per table and per sourceLocale; a changed slug leaves a 301 redirect behind.
- Soft delete only (deletedAt). Hard delete is a Super Admin script, never a button.

Mistakes to avoid

- Treating "published" as a boolean. You will soon need scheduled and archived too; a status column costs nothing extra.
- Showing drafts because a query forgot the status filter. Put the filter in ONE repository function.

PersonRequest

abstract parent · extends **BaseEntity** · module Platform

A person knocking on the church door and saying "please add me". Same form of knock whatever door they use. Event registrations, programme enrolments, join requests and new converts are all "a person asking the church for something". One parent class gives them one inbox, one status flow and one consent rule.

Field	Type	Flags
`fullName`	text	required private
`phone`	text	private
`email`	text	private
`branchId`	uuid	FK Branch
`ageGroup`	AgeGroup	
`status`	RequestStatus	required
`consentAt`	timestamptz	required
`consentVersion`	text	required
`answers`	jsonb	
`handledBy`	text	FK User
`handledAt`	timestamptz	
`locale`	Locale	
`source`	text	
`ipHash`	text	private

Does: `confirm(): void`, `decline(reason): void`, `assignTo(user): void`

Rules

- Holds a person's contact details + consent flag + a status (new, contacted, done, rejected).
- Phone and email are PII: encrypted at rest by Neon, hidden in logs, shown only to the role that needs them.
- A request is created by a public form, so it ALWAYS passes through RateLimiter + Turnstile first.

Mistakes to avoid

- Copy-pasting name/phone/status into every request table. When the rule changes you must edit four tables.

MediaAsset

table · extends **BaseEntity** · module Platform · table `media_asset`

The library card for one file. Every file (image, audio, PDF, certificate) is one row pointing to a Cloudflare R2 object. Other tables store the id, never the URL.

Field	Type	Flags
`kind`	MediaKind	required
`bucket`	text	required
`r2Key`	text	unique required
`mimeType`	text	required
`sizeBytes`	bigint	required
`checksum`	text	
`originalName`	text	
`width`	int	
`height`	int	
`durationSec`	int	
`altText`	jsonb	
`isPublic`	bool	required
`status`	MediaStatus	required
`uploadedBy`	text	FK User
`branchId`	uuid	FK Branch

Does: `presignUpload(): Url`, `publicUrl(): Url`, `signedUrl(ttl): Url`

Rules

- Browser uploads straight to R2 with a presigned URL; the app only stores the row.
- isPublic = false for certificates and anything with personal data; served with short-lived signed URLs.
- checksum prevents duplicate uploads.
- status goes uploading → ready (after a verification job) or failed.

Mistakes to avoid

- Saving the full URL. If you ever change CDN or bucket, every row breaks. Save the key.

Who may: Super CRUD · Branch CRUD · Editor CRUD · Translator R · Dept leader CR · Discipler R · Treasurer R · Events CR

SiteSetting

table · module Platform · table `site_setting`

The control-panel switches. Small global facts: born-again baseline, social links, policy versions, feature flags.

Field	Type	Flags
`key`	text	PK
`value`	jsonb	required
`updatedBy`	text	FK User
`updatedAt`	timestamptz	required

Rules

- Key/value config editable by Super Admin: church name, social links, default meta tags, bank/M-Pesa display flags.
- Secrets (API keys) NEVER live here. They live in Vercel environment variables.

Mistakes to avoid

- Putting API secrets in this table because "it is easy to edit".

Who may: Super CRU · Branch R

AuthzService

service (code, not a table) · module Platform

The security guard at the door of every action: "are you allowed to do this?" One function every server action calls before touching data.

Does: `can(user, action, resource): bool`, `scopesFor(user): Scope[]`, `assertCan(...): void`

Rules

- Every Server Action and Route Handler starts with assertCan(...). No exception.
- Decisions use RoleAssignment (role + scope) loaded once per request and cached for that request only.

Mistakes to avoid

- Checking permissions only in the UI (hiding buttons). The server must check too.

TranslationService

service (code, not a table) · module Platform

The language helper: finds the Kiswahili or English words and says so when one is missing. Reads the right language, falls back safely, and drives the AI-draft to human-review flow.

Does: `localized(entity, locale): View`, `requestDraft(entity, locale): void`, `approve(row): void`, `fallback(chain): Row`

Rules

- Fallback chain: requested locale → other locale → marked "not yet translated" badge.
- Machine drafts are stored as status machine_draft and never shown publicly until approved (or flagged "auto-translated" if Super Admin allows).

Mistakes to avoid

- Translating on every page view. Translate once, store, approve.

StorageService

service (code, not a table) · module Platform

The delivery person for photos, PDFs and audio: takes them to the cloud store and brings back a safe link. Only class allowed to talk to Cloudflare R2.

Does: `presignUpload(kind): Url`, `signedUrl(key, ttl): Url`, `remove(key): void`

Rules

- Uploads go browser → R2 by presigned URL (the file never passes through Vercel).
- Files are checked for type, size and (images) re-encoded to strip hidden data.

Mistakes to avoid

- Trusting the file extension. Check the real type.

RateLimiter

service (code, not a table) · module Platform

The queue marshal: lets people in at a sensible speed so robots cannot flood the forms. Upstash Ratelimit wrapper: per IP, per device, per form.

Does: `limit(key, rule): Result`

Rules

- Sliding-window limits per IP-hash and per route in Upstash Redis.
- Limits: comment 5/10min, newsletter 3/hour, contact 3/hour, converts 20/hour per staff device.

Mistakes to avoid

- One global limit for the whole site. A single noisy visitor blocks everyone.

JobQueue

service (code, not a table) · module Platform

The to-do list for slow jobs; a worker does them in the background and tries again if they fail. QStash wrapper for emails, reminders, digests, counter flush.

Does: `enqueue(job, delay): void`, `schedule(cron, job): void`, `verify(signature): bool`

Rules

- All slow or retryable work (emails, counters flush, translation drafts) goes through QStash with signed requests.
- Jobs are idempotent: running twice must not double-send.

Mistakes to avoid

- Running jobs that are not idempotent. A retry then sends the same email twice.

Mailer

service (code, not a table) · module Platform

The post office: sends the e-mails and writes down what was sent. Resend wrapper; every send is logged in NotificationLog.

Does: `send(template, to, data): void`, `syncContact(sub): void`

Rules

- Templates are React Email components, one per message, bilingual.
- Sending domain has SPF, DKIM and DMARC set in DNS before the first campaign.

Mistakes to avoid

- Sending from a no-reply address with no SPF/DKIM: it lands in spam.

CacheService

service (code, not a table) · module Platform

The memory shelf: keeps recent answers close so pages load fast. Next.js cache tags + Upstash Redis counters.

Does: `tag(name): void`, `revalidate(tags): void`, `counter(key): Counter`

Rules

- Next.js cache tags for pages, Redis for counters and shared lookups.
- After a publish, call revalidateTag(tag) so the public sees the change within seconds.

Mistakes to avoid

- Caching pages that show a logged-in user's data. Only cache public pages.

Analytics

service (code, not a table) · module Platform

The people counter: only counts visitors who said yes to counting. PostHog wrapper that respects cookie consent.

Does: `capture(event, props): void` , `flag(key): bool`

Rules

- PostHog loads ONLY after the visitor accepts analytics cookies.
- Event names follow the taxonomy page (object_action, snake_case).

Mistakes to avoid

- Loading PostHog before consent. Under Kenyan and EU rules that is a breach.

ModerationService

service (code, not a table) · module Platform

The doorkeeper for comments and messages: checks for spam before they reach a human. First-line spam checks for comments and contact messages.

Does: `score(text): Risk` , `isHoneypotTripped(form): bool`

Rules

- Checks: honeypot, link count, banned words list (EN + SW), duplicate body, rate limit result.
- A suspicious item becomes status "pending", never silently dropped.

Mistakes to avoid

- Relying only on word lists. Add rate limits and the human review queue.

CertificateService

service (code, not a table) · module Platform

The certificate printer: fills the template with the name and saves the PDF safely. Makes the completion PDF.

Does: `render(journey): Pdf` , `nextSerial(): string`

Rules

- Builds the PDF (react-pdf or pdf-lib) from the template, uploads to R2, stores Certificate row, returns a signed URL.
- Idempotent: calling twice for the same journey returns the same certificate.

Mistakes to avoid

- Generating the PDF in a request that can time out. Queue it and poll.

FellowshipLocator

service (code, not a table) · module Platform

The "which fellowship is nearest me?" calculator. Distance maths for the closest home fellowship.

Does: `nearest(lat, lng, n): HomeFellowship[]` , `search(estate): HomeFellowship[]`

Rules

- Primary: browser geolocation (asks permission). Fallback: typed estate/area matched against HomeFellowship.areaText.
- Distance = Haversine on lat/lng; returns the 3 nearest with their meeting day and leader contact.

Mistakes to avoid

- Silently using the IP address for location. Ask permission or let them type.

OverlayOrchestrator

service (code, not a table) · module Platform

The traffic light for pop-ups so only one shows at a time. Client-side traffic cop: cookie bar, new-member modal, newsletter modal never fight.

Does: `next(): Overlay`, `canShow(o): bool`, `dismiss(o, days): void`

Rules

- Order of overlays: 1 cookie bar (bottom), 2 new-member modal, 3 newsletter modal (after About section).
- Only ONE modal at a time; dismissals are remembered (localStorage, and cookie if accepted).

Mistakes to avoid

- Letting three popups open together on a phone. The visitor leaves.

PART 11

The 27 fixed lists (enumerations)

A fixed list of allowed values makes a mistake impossible to save: the database refuses "publsihed". Drawn on E12.

List	Allowed values
Locale	en, sw
LocaleOrBoth	en, sw, both
ContentStatus	draft, in_review, scheduled, published, archived
TranslationStatus	missing, machine_draft, needs_review, approved
RequestStatus	new, in_progress, confirmed, waitlisted, declined, cancelled, completed
AgeGroup	under_13, 13_17, 18_25, 26_35, 36_50, 51_plus
Gender	male, female, prefer_not
StaffRole	super_admin, branch_admin, content_editor, translator, department_leader, discipler, treasurer, events_coordinator
ScopeType	global, branch, department
CommentStatus	pending, approved, rejected, spam
AnnouncementKind	sunday, midweek
ResourceCategory	discipleship, bible_study, missions, prayer_points, forms, other
ResourceFormat	pdf, audio, video, link, form
TopicTaxonomy	theme, resource_topic, blog_category
EventCategory	conference, worship, marriage, youth, trip, retreat, outreach, prayer, fellowship, other
RegMode	none, form, external, walk_in
OutreachCategory	school, unreached, international, mission_week
FollowUpStatus	new, assigned, attempting, connected, enrolled, completed, unreachable, declined
ProjectStatus	planned, fundraising, in_progress, completed, paused
LedgerStatus	recorded, voided
LedgerMethod	cash, mpesa, bank, cheque, other
ChannelKind	mpesa_paybill, mpesa_till, bank, cheque, cash, international, link
JourneyStatus	started, in_progress, completed, abandoned
SubscriberStatus	pending, subscribed, unsubscribed, bounced, complained
DeptKind	ministry, fellowship, committee, service_team
FeatureKey	home_fellowship, missions, discipleship, children, youth, men, women, media
CohortStatus	planned, open, full, running, completed, cancelled

The 25 translation tables

One row per language per item. Each holds the words that must exist in both languages, plus a status and the fingerprint of the English source (E11, B08).

Table	Translated fields
branch_translation	description
department_translation	name, summary, whatWeDo, meetingTimes
leader_translation	bio
leader_position_translation	title, responsibilities

Table	Translated fields
home_fellowship_translation	notes
minister_translation	bio
series_translation	title, description
topic_translation	name, description
sermon_translation	title, summary, keyPoints, highlights
blog_post_translation	title, excerpt, body
bulletin_translation	title
announcement_translation	title, body
resource_translation	title, description
page_block_translation	data
timeline_entry_translation	title, body
event_translation	title, description, feeInfo
event_session_translation	title
outreach_activity_translation	reportSummary
programme_translation	name, description, scheduleSummary, durationLabel, feeInfo
project_translation	title, summary, body, impact
project_milestone_translation	title, description
project_update_translation	title, body
giving_channel_translation	label, referenceInstructions
form_definition_translation	title, intro, consentText
newsletter_campaign_translation	subject, body

PART 12

Roadmap and next phases

You asked me to build in phases: finish this diagram set first, then choose another family, and so on to the end. Here is where we are, what I recommend next and why, and what is still imperfect in Phase 1.

12.1 Phase 1 is complete: foundations

Family	Pages	What it settles
Class architecture and tabs	52	What the things are, who owns them, what depends on what, who may do what
Use case, sequence, state	29	What people want, the order of the conversations, the stages of each thing
Flow, block, package, route, object	19	The whole system on a page, the routes, the folders, two worked examples
Data flow	11	Where information enters, rests and leaves
ER (database)	14	Tables, links, translation tables, fixed lists, Redis keys, database rules
Total	**125**	

12.2 What I recommend for Phase 2

You said to choose another diagram family for later phases. I recommend this order, because each one removes a risk that would otherwise surprise you:

Order	Family (planned codes)	What it contains	Why this order
1	**C: architecture, hosting and delivery** (C01 to C15)	System context and containers, deployment topology, environments and branching, the CI/CD pipeline, secrets and configuration map, request life-cycle with caching, performance budget, SEO architecture, observability, backup and restore, the 12-week roadmap, agent gates, the test strategy	Q-02, Q-49 (accounts and costs) cannot be answered without it, and it fixes the shape of the pipeline before any code is written
2	**X: security and privacy** (X01 to X09)	Threat model, trust boundaries, data classification, consent, access-control tests, input and upload safety, headers and CSP, the secrets register, incident response	The law and the lawyer (Q-47, Q-48) take time; start them early
3	**W: website structure and SEO** (W01 to W14) and **G: wireframes and UI** (G01 to G25)	Sitemap, URL scheme, metadata, structured data, hreflang, navigation, landing order; design tokens, components, every page as a wireframe with empty, loading and error states	Content writers and the church leaders need to **see** pages to approve them
4	**Q: templates and plan** (Q01 to Q07)	Requirement template, acceptance tests, security review, pull-request template, bug report, decision record, weekly report; Gantt chart	Makes the build repeatable
5	Operations runbooks	Restore from a backup, roll back a release, rotate a key, handle a breach, onboard a new staff member	After launch, when the system is real

> **EXPLAINED LIKE YOU ARE FIVE**
>
> Phase 1 was the **floor plan and the wiring plan**. Phase 2 is the **construction plan** (where it stands, how it is delivered) and the **safety plan** (what could go wrong). Only after that do we draw the **interior** (the wireframes), because you cannot paint a room whose walls may move.

12.3 A 12-week build plan (to be confirmed in Phase 2)

This is the order the prompts file follows (P01 to P28). Each week ends with a **gate**: a written checklist that must pass before the next week starts.

Week	Work	Prompts	Gate
1	Repository, tools, rules, database foundation, environments	P01, P02	G3 (start)
2	Sign-in, roles, audit log	P03	
3	Church structure, media and storage	P04, P05	
4	Content life-cycle, sermons, blogs, bulletins, resources	P06 to P09	G3 (half)
5	Translation and the language switch	P10, P11	
6	Events, programmes, forms	P12, P13, P16	
7	Go Ye and follow-up	P14	
8	Giving and projects	P15	G3 (end)
9	New-member tour, certificate, pop-ups, newsletter, contact	P17 to P20	
10	Platform services, admin dashboards, public page assembly, privacy tools	P21 to P24	
11	SEO, security review, acceptance tests	P25 to P27	G4
12	Content load, staff training, release rehearsal, go-live	P28	G5

MISTAKES TO AVOID

- **Starting week 1 before the blocking questions are answered.** A wrong default found in week 7 costs seven weeks.
- **Skipping a gate because of a date.** A gate that can be skipped is a suggestion.
- **Letting the same agent write and approve.** The prompts always name a different reviewer.

12.4 Known limits of Phase 1 (honest list)

Limit	What it means	What I will do
Some class tabs show arrows that pass behind a box	Eleven tab pages (B20, B22, B24, B27, B28, B34, B37, B39, B40, B43, B44) have one or two connector lines that pass behind another box. Nothing is wrong in the data; it is a drawing flaw.	Re-route them by hand in the first correction round. In draw.io you can drag the line's handle.
A few labels are crowded	On three data-flow pages (D04, D06, D08) two labels sit close together.	Move them in draw.io or ask me to adjust.
Wireframes are not drawn	You have the structure, not the look of each page.	Phase 2 (family G).
Costs are not estimated	I did not quote prices, because plans change. Part 6 lists what grows with use (translation, SMS, video, analytics).	Phase 2 (C-series) with a cost table.
The law is a default, not advice	Every legal item is marked "verify with a lawyer".	Questions Q-47 and Q-48.
The logo and intro videos were not used inside the diagrams	They matter for the page designs, not the system design.	Used in Phase 2 wireframes.

12.5 How to ask for a change

1. Open **DCEB_Diagrams.drawio**, find the page by its code, and drag, rename or add. The arrows follow the boxes.
2. If the change affects the **meaning** (a new field, a new role, a rule), tell me the code and the change in one sentence ("B42: Treasurer may also export"). I change the model once and every dependent page changes with it.
3. If you change the draw.io file by hand and send it back, I will read it, tell you which other pages are now out of date, and fix them.

> **WHAT YOU ALREADY DID RIGHT**
>
> You asked for the pages to be "able to shift". They are: every shape is a real shape, every arrow is a real connector, pages are linked by the Index, and the role layers can be switched on and off on B01.